EPHEC
ÉCOLE SUPÉRIEURE
ENSEIGNEMENT POUR ADULTES

Année académique **2026-2027**

Programmation orientée objet — Java

Support de cours

Tous les modules, dans l'ordre des séances

Du module INI au module FIC

Ce document a été élaboré avec l'aide de Claude, un assistant d'intelligence artificielle développé par Anthropic. L'enseignant en a conçu le contenu et la progression, l'a relu, corrigé et validé, et en assume la responsabilité.

EPHEC — Bachelier en informatique, niveau 2

Table des matières

INI — Introduction au cours 6
INI §1 — Le cours 7
INI §2 — Ma façon de travailler 9
INI §3 — Les documents de référence 10
En résumé 11
POO — Classes et objets 12
POO §1 — De C# à Java 13
POO §2 — Classe et objet 14
POO §3 — Encapsulation 16
POO §4 — Membres de classe : `static` 18
POO §5 — Entrée de données au clavier 21
POO §6 — Primitifs et références 22
POO §7 — Les wrappers 24
Ce qu'il faut retenir 25
TMP — Dates et heures 26
TMP §1 — Les classes de `java.time` 27
TMP §2 — Mesurer un écart 29
TMP §3 — Une date ne change pas 29
TMP §4 — Un détour par l'affichage 30
TMP §5 — Date ou instant ? 32
TMP §6 — L'arithmétique du calendrier 32
TMP §7 — `Period` ou `Duration` 33
TMP §8 — Une date et son texte 33
Ce qu'il faut retenir 34
TAB — Tableaux, boucles et opérateur ternaire 35
TAB §1 — Un tableau, c'est une taille fixe 36
TAB §2 — Parcourir un tableau 36
TAB §3 — Des lignes de tailles différentes 38
TAB §4 — L'opérateur ternaire 38
TAB §5 — Des objets qui se connaissent 40
Ce qu'il faut retenir 41
ENU — Énumérations et records 42
ENU §1 — Des valeurs qui se comptent 43
ENU §2 — Déclarer une énumération 43
ENU §3 — L'ordre porte l'information 44
ENU §4 — Comparer et aiguiller : `switch` 45
ENU §5 — Une `enum` est une classe 47
ENU §6 — Un détour par deux outils 48

ENU §7 — Les records 49
Ce qu'il faut retenir 51
ABS — Héritage et classes abstraites 52
ABS §1 — L'héritage en Java 53
ABS §2 — Le même code, trois fois 55
ABS §3 — La classe abstraite 56
ABS §4 — Ce qui monte, ce qui descend 57
ABS §5 — Le polymorphisme 59
ABS §6 — La mère appelle la fille 60
ABS §7 — Deux étages 62
ABS §8 — Décider soi-même 63
Ce qu'il faut retenir 64
ITF — Interfaces 65
ITF §1 — Ce qu'une interface déclare 66
ITF §2 — Plusieurs interfaces à la fois 67
ITF §3 — Interface ou classe abstraite ? 68
ITF §4 — Dépendre d'un contrat 69
ITF §5 — Le patron Observateur 71
ITF §6 — Ce qu'une interface contient aussi 73
Ce qu'il faut retenir 74
PKG — Packages et visibilité 75
PKG §1 — Un package, un dossier 76
PKG §2 — Le niveau sans mot-clé 77
PKG §3 — Champ public ou accesseur 78
PKG §4 — La surface publique 79
PKG §5 — Découper en packages 80
PKG §6 — Le quatrième niveau : `protected` 81
Ce qu'il faut retenir 82
COL — Collections 83
COL §1 — Une structure qui grandit 84
COL §2 — Une liste, des positions 85
COL §3 — Un ensemble, sans doublons 87
COL §4 — Une table, des clés 89
COL §5 — Passer un comportement 90
COL §6 — Comparer et trier 92
COL §7 — Des collections emboîtées 93
COL §8 — Une file, deux bouts 94
Aide-mémoire 95
Ce qu'il faut retenir 96
EXC — Exceptions 98
EXC §1 — Un objet qui interrompt 99

EXC §2 — Lever, et déclarer ... 100
EXC §3 — Attraper là où l'on peut agir ... 102
EXC §4 — Celles que le compilateur ne voit pas ... 104
Ce qu'il faut retenir ... 105
JUN — Tests unitaires avec JUnit ... 107
JUN §1 — À vous d'écrire les tests ... 108
JUN §2 — Ce qu'un test contient ... 109
JUN §3 — Vérifier tout le contrat ... 111
JUN §4 — Choisir ses cas, et les justifier ... 112
JUN §5 — Du code qui se laisse tester ... 115
Ce qu'il faut retenir ... 116
GEN — Types génériques ... 117
GEN §1 — Un type en paramètre ... 118
GEN §2 — Autant de types qu'il en faut ... 119
GEN §3 — Ce que le compilateur efface ... 120
GEN §4 — Une famille de types ... 121
GEN §5 — Une méthode générique ... 123
GEN §6 — Hériter d'une classe générique ... 124
Ce qu'il faut retenir ... 125
LAM — Lambdas et références de méthodes ... 126
LAM §1 — Une classe qui perd son nom ... 127
LAM §2 — Ce que le compilateur sait déjà ... 128
LAM §3 — Réutiliser ce qui existe ... 129
Ce qu'il faut retenir ... 131
FX — Interfaces graphiques avec JavaFX ... 132
FX §1 — Lancer une application JavaFX ... 133
FX §2 — Une fenêtre et ses composants ... 134
FX §3 — Brancher un bouton ... 136
FX §4 — Ce que dit un événement ... 137
FX §5 — Un événement voyage ... 139
FX §6 — Une valeur qui prévient ... 140
FX §7 — Lier plutôt qu'écouter ... 142
FX §8 — Séparer l'écran du calcul ... 143
FX §9 — Décrire l'écran en FXML ... 143
FX §10 — La calculatrice, pas à pas ... 145
Aide-mémoire ... 147
Ce qu'il faut retenir ... 148
FIC — Fichiers ... 150
FIC §1 — Ce qui entre, ce qui sort ... 151
FIC §2 — Fermer quoi qu'il arrive ... 152
FIC §3 — Écrire et relire du texte ... 153

FIC §4 — Écrire l'objet tel quel 155
Aide-mémoire 156
Ce qu'il faut retenir 158
Annexe A — Guide IntelliJ IDEA 159
À quoi sert ce guide 159
Sommaire 159
A — Avant de commencer 161
B — Ouvrir le projet du cours 161
C — L'écran 164
D — Écrire du code 167
E — Exécuter 170
F — Transformer du code sans le casser 174
G — Naviguer 175
H — Raccourcis et réglages 176
Annexe — Erreurs et solutions 180
Annexe B — Mémento technique 185
À quoi sert ce mémento 185
Sommaire 185
La plateforme Java 186
Les interfaces graphiques 188
De votre code au programme qui tourne 190
Ce qui se passe pendant l'exécution 192
Maven 193
Recevoir les fichiers du cours 195
Git, en deux mots 196
Annexe C — Mémento Clean Code 198
À quoi sert ce mémento 198
Les règles en un coup d'œil 198
Nommer 200
Écrire une méthode 204
Commenter 208
Objets et encapsulation 209
Traiter les erreurs 210
Écrire des tests 213
Structurer une classe 215
La règle du boy-scout 218
Pour aller plus loin 218
Index 219

Module INI

INI — Introduction au cours

Ce que couvre le cours, comment il s'organise, et comment travailler

Cette introduction présente le cours : ce qu'il couvre et ce que vous devrez savoir faire au terme de l'année, comment il s'organise et comment il est évalué, puis la façon dont s'articulent les différents supports, et les documents de référence qui vous accompagneront toute l'année.

INI §1 — Le cours

Qui suis-je ?

Avant d'entrer dans le cours, quelques mots sur celui qui le donne.

Mon parcours professionnel a commencé au helpdesk d'ING, qui s'appelait encore la BBL à l'époque. Je suis ensuite devenu testeur, puis développeur : c'est à ce moment-là que j'ai suivi, à l'EPHEC, le même parcours que vous. Je suis devenu ensuite analyste fonctionnel, puis Scrum Master, et j'évolue aujourd'hui vers un rôle de coach, d'équipe ou d'organisation.

Ce qui m'a d'abord attiré vers l'informatique, c'est sa capacité à rendre visibles certains objets mathématiques, comme l'ensemble de Mandelbrot ou le jeu de la vie de Conway.

J'aime les années 80, qui ont bercé mon adolescence : Simple Minds, Depeche Mode, le film *Blade Runner*. J'aime aussi le café.

Enfin, je suis attentif, sans doute davantage depuis que j'ai trois enfants, aux problématiques systémiques dans lesquelles notre monde est plongé : les ressources, l'énergie, la biodiversité. C'est ce qui m'a poussé à m'inscrire à une formation en développement durable.

Contenu du cours

Ce cours approfondit et met en pratique ce que vous avez vu en C#. Il aborde des concepts plus avancés de la programmation orientée objet, comme l'héritage, les types génériques et les expressions lambda, quelques classes de la bibliothèque standard de Java, et quelques outils, dont `JUnit` pour les tests et `JavaFX` pour les interfaces graphiques.

Chaque notion est expliquée, puis pratiquée au travers d'exercices. Le travail personnel entre les séances est indispensable : c'est lui qui permet d'assimiler ce qui a été vu au cours. Si vous rencontrez une difficulté, signalez-la au plus vite plutôt que de la laisser s'installer.

Les modules du cours

Le cours est découpé en modules. Chacun porte un code de trois lettres, que vous retrouverez partout : sur les slides, dans le nom des exercices, dans les packages Java et sur Moodle. Le code désigne le sujet, pas la séance : un module peut occuper moins d'une séance, ou en déborder.

Code	Sujet
`POO`	Programmation orientée objet : classes et objets
`TMP`	Temps : dates et durées
`TAB`	Tableaux
`ENU`	Énumérations et records
`ABS`	Héritage, classes abstraites
`ITF`	Interfaces
`PKG`	Packages et visibilité
`COL`	Collections
`EXC`	Exceptions
`JUN`	Tests unitaires (`JUnit`)
`GEN`	Types génériques
`LAM`	Classes anonymes et expressions lambda
`FX`	Interfaces graphiques (`JavaFX`)
`FIC`	Fichiers
`REC`	Exercices récapitulatifs

L'ordre est indicatif. Il peut évoluer en cours d'année ; Moodle indique, au fil des séances, les modules abordés.

Les acquis d'apprentissage

Les acquis d'apprentissage, repris du dossier pédagogique du cours, décrivent ce que vous devez être capable de faire au terme du cours. Ils se regroupent en deux familles.

La première porte sur les objets : l'étudiant sera capable de concevoir, d'installer et d'utiliser des objets appropriés à la solution. Concrètement, il pourra :

- définir et implémenter des classes et des objets Java simples ;
- compléter des classes Java implémentées par un tiers ;
- utiliser des classes et des objets Java existants, et interagir avec eux ;
- dériver de nouvelles classes par héritage d'une classe Java existante ;
- utiliser les classes du JDK pour accéder à des fichiers, et pour réaliser une application disposant d'une interface graphique simple (`JavaFX`) ;
- démontrer, au travers d'un exercice simple, sa compréhension du concept et du fonctionnement des expressions lambda.

La seconde porte sur les tests : l'étudiant sera capable de concevoir et de mettre en œuvre une procédure de test partiel et intégré. Il pourra :

- implémenter des classes de test simples (`JUnit`), à partir d'une classe implémentée par un tiers ou de spécifications techniques ;
- implémenter des classes et des objets Java simples, et les classes de test correspondantes ;
- repérer et corriger des problèmes simples dans des classes écrites par un tiers, en exécutant une batterie de tests `JUnit`.

En d'autres termes, vous devez pouvoir concevoir des classes et les utiliser, employer l'héritage à bon escient, écrire et exécuter des tests qui vérifient qu'une classe fonctionne, et utiliser les classes du JDK vues au cours : fichiers, collections, exceptions et événements. Les deux premiers points prolongent ce que vous connaissez déjà en C#.

L'organisation pratique

Le cours compte quatorze séances, du 19 janvier au 18 mai 2027. Chaque séance commence à 18 h 00 et se termine au plus tard à 21 h 35, avec une pause de quinze minutes, et alterne théorie, démonstration et exercices.

Vous travaillerez avec Java et l'environnement de développement IntelliJ IDEA. Maven gère la structure des projets et les bibliothèques, `JUnit` sert aux tests et `JavaFX` aux interfaces graphiques. Les versions à utiliser sont annoncées sur Moodle : travaillez avec celles-là toute l'année, ce sont celles de l'examen.

> **Attention** - Présence aux séances
>
> Message officiel de la direction de l'EPHEC : « En cas de dépassement des 40 % d'absences injustifiées dans une unité d'enseignement, l'étudiant se verra automatiquement désinscrit du cours et non-admis à l'examen. »

L'examen

L'examen a lieu le 8 juin 2027. Il se déroule à cahier ouvert : il ne demande pas de par cœur, mais la capacité à écrire du code qui fonctionne. Il se passe sur machine et dure deux heures trente.

Il compte cinq ou six questions, dont la forme reste stable d'une session à l'autre :

- écrire une classe simple, qui lève une exception ;
- écrire un type générique ;
- écrire et exécuter une classe de tests pour une classe donnée ;

- écrire une hiérarchie d'interfaces et de classes ;
- compléter une application graphique pour qu'elle fonctionne ;
- et peut-être une sixième question, dont l'existence et le sujet vous seront confirmés en cours d'année.

Chaque question correspond à un ou plusieurs modules. Les exercices récapitulatifs de fin d'année en reprennent la structure.

INI §2 — Ma façon de travailler

Le déroulé d'un module

Un module suit toujours le même chemin :

Étape	Ce qu'elle apporte
Les slides	les notions, présentées en séance
La démonstration	les notions mises en pratique devant vous, en direct dans IntelliJ
Les exercices	votre pratique, à partir du fascicule d'exercices du module
GameHub	le fil rouge, un projet plus grand que vous faites évoluer à chaque séance

Les slides ne contiennent pas les énoncés : une slide de renvoi indique le code des exercices à faire, par exemple `POO.1`, et l'énoncé complet se trouve dans le fascicule d'exercices du module.

Les fascicules d'exercices

Chaque module a son fascicule d'exercices, distribué en PDF ; le recueil des exercices les rassemble en un seul document. Les exercices y sont numérotés par sujet : `TAB.3` est le troisième exercice sur les tableaux, quelle que soit la séance où on le fait. Le même code donne le nom du package dans le projet IntelliJ : `TAB.3` se trouve dans `be.ephec.java.tab.e03`.

Tous les énoncés suivent la même structure : l'objectif, le contexte, l'énoncé, les spécifications à respecter, la manière de vérifier votre code, et les questions à se poser. Cette dernière rubrique est la plus importante : elle signale ce que l'exercice cherche à vous faire découvrir.

Le projet ExercicesJava

Les exercices sont livrés dans un projet Maven, ExercicesJava, à ouvrir dans IntelliJ. À chaque séance, une archive zip ajoute les exercices du jour ; les solutions sont distribuées après le cours. Le guide IntelliJ et le mémento technique expliquent comment ouvrir le projet.

Les classes sont déjà écrites, avec leurs méthodes. Le corps de chaque méthode se contente de lever une `UnsupportedOperationException`, qui signifie « pas encore écrit » :

```
public int computeAge(int annee) {
    throw new UnsupportedOperationException(
            "computeAge n'est pas encore implementee");
}
```

Le projet compile donc dès le départ : c'est à vous de remplacer ces lignes par le vrai code.

À l'examen, en revanche, certaines questions vous demanderont de créer vous-même une classe qui n'existe pas encore : le fichier, la déclaration de la classe et ses méthodes (voir `IDE.8` du guide IntelliJ). Profitez des exercices pour vous y entraîner, en écrivant parfois une classe de toutes pièces plutôt qu'en complétant le squelette fourni.

Les tests fournis

La plupart des exercices sont accompagnés d'une classe de test `JUnit`. Vous la lancez vous-même, aussi souvent que vous le voulez : c'est votre moyen de savoir si votre réponse est correcte, sans attendre la séance suivante. Un test rouge signale que la réponse n'est pas encore correcte ; quand tous les tests sont verts, elle l'est.

Un test vert ne dit pas que votre code est bien écrit, seulement qu'il fait ce qui est demandé. Et les tests ne vérifient jamais un point que l'énoncé laisse à votre choix.

Vous n'avez pas besoin de connaître `JUnit` pour lancer ces tests : le mémento Exercices explique comment faire, et le module `JUN` vous apprendra à écrire les vôtres.

Le fil rouge GameHub

GameHub est une plateforme de jeux rudimentaire, construite petit à petit au fil de l'année : chaque module y ajoute une pièce, qui s'appuie sur ce qui a été construit précédemment. Ses énoncés sont numérotés `G.1`, `G.2`, et ainsi de suite, dans l'ordre imposé par leurs dépendances. GameHub vit dans un projet séparé des exercices.

Là où un exercice isole une notion, GameHub vous oblige à la faire cohabiter avec tout ce que vous avez déjà écrit. C'est la situation que vous retrouverez à l'examen, et surtout dans tout projet réel.

Les tests de module et leurs badges

Chaque module a, sur Moodle, un test : huit questions tirées au hasard parmi quinze. Après chaque réponse, vous voyez si elle est juste et, sinon, la partie du support à revoir. Le test est réussi à 6 sur 8, et vous pouvez le refaire autant de fois que vous le voulez : les questions changent d'une tentative à l'autre.

Chaque test réussi vous donne un badge. Quatre badges d'étape marquent les grandes parties du cours, et un dernier, « Java, de bout en bout », la réussite des quatorze tests. Un tout premier badge, « Pied à l'étrier », vous accueille dès que vous avez téléchargé le projet de départ.

Ces badges ne comptent pas dans la note. Ce sont des repères, pour voir où vous en êtes et ce qu'il vous reste à revoir : prenez-les comme un encouragement, à votre rythme.

Ce support de cours

Les slides accompagnent l'explication orale. Ce support reprend chaque notion en prose, avec le code formaté, module par module. C'est lui qu'il faut relire entre les séances.

INI §3 — Les documents de référence

Quatre documents

À côté des fascicules et de ce support, quatre documents de référence vous accompagnent toute l'année. On ne les lit pas d'une traite, à part le mémento Exercices : on les ouvre quand un énoncé ou une slide y renvoie. Vous les trouverez en annexe de ce support, et sur Moodle, dans la section Documents.

Document	Préfixe	À quoi il sert
Mémento Exercices		lancer les tests, lire un résultat
Guide IntelliJ	`IDE`, `ERR`	se servir de l'outil, comprendre une erreur
Mémento technique	`TEC`	comprendre ce qui se passe sous le capot
Mémento Clean Code	`CC`	écrire du code lisible

Le **mémento Exercices** rassemble ce qui vaut pour tous les fascicules, et qu'il serait inutile de répéter dans chacun. Lisez-le une fois en début d'année. Il explique la numérotation des exercices, montre comment lancer les classes de test fournies et lire leur résultat, et pose une règle :

> **À retenir** - gardez toujours un code qui compile
>
> Tout le dossier `src/main/java` se compile d'un seul bloc : un exercice laissé à moitié écrit empêche de lancer tous les autres, y compris ceux des séances précédentes. Avant de passer à autre chose, vérifiez que votre code compile. Sinon, deux issues : le corriger, ou mettre le morceau en chantier entre commentaires avec `Ctrl+/` (⌘/ sur Mac). C'est aussi ce que l'examen demande : on y rend du code qui compile.

Le **guide IntelliJ** couvre ce qu'il faut savoir d'IntelliJ pour ce cours, captures d'écran à l'appui : l'installation, la création et l'ouverture d'un projet, l'exécution, le débogage, le paramétrage. Ses entrées `ERR` partent du message affiché et expliquent sa cause la plus probable : c'est le premier réflexe à avoir quand quelque chose ne fonctionne pas.

Le **mémento technique** explique ce qu'IntelliJ fait à votre place : ce que sont la JVM, le JRE et le JDK, comment un fichier `.java` devient un programme, comment Maven récupère les bibliothèques, comment se lance une application `JavaFX`. Vous pouvez écrire vos premiers programmes sans le lire, mais il répond à la plupart des « pourquoi ça marche ? ».

Le **mémento Clean Code** rassemble 37 règles simples, en sept sections (noms, méthodes, commentaires, objets, erreurs, tests, classes), pour écrire du code qu'un autre développeur, ou vous-même dans six mois, comprendra sans effort. Un programme qui fonctionne n'est pas forcément un programme lisible. Ces règles s'inspirent du livre *Clean Code* de Robert C. Martin. Elles ne font pas l'objet d'un module : elles arrivent par petites touches, au fil des exercices, quand elles deviennent utiles. Elles s'appliquent aussi à l'examen : un code qui fonctionne mais ne les respecte pas peut y perdre des points.

Comment lire un renvoi

Les fascicules, les slides et ce support renvoient aux mémentos par des codes :

Renvoi	Où chercher
`IDE.14`	la section 14 du guide IntelliJ
`CC-MET.8`	la règle 8 de la section « méthodes » du mémento Clean Code
`TEC-EXE.4`	la section 4 de la partie « exécution » du mémento technique
`POO §6`	la section 6 du module `POO`, dans ce support

Un renvoi vous dit où chercher, au moment où vous en avez besoin. Par exemple, l'exercice `POO.5` vous demande de lire une Javadoc : il vous renvoie à `IDE.14`, qui montre comment l'afficher sans quitter IntelliJ. Le premier renvoi arrive dès le module `POO` : prenez l'habitude de le suivre.

Quelques liens utiles

La documentation officielle de Java, et notamment la Javadoc qui décrit chaque classe de la bibliothèque standard, se trouve sur le site d'Oracle : `docs.oracle.com/en/java/javase/`. Choisissez la version de Java annoncée sur Moodle : les classes évoluent d'une version à l'autre.

Le site de Jean-Michel Doudoux, `jmdoudoux.fr/java/dej/`, propose en français un tour très complet du langage et de sa bibliothèque.

En résumé

- Un module, c'est un sujet, avec son code de trois lettres.
- Chaque module suit le même chemin : slides, démonstration, exercices du fascicule, GameHub.
- Des tests vous permettent de vérifier vous-même vos réponses.
- Les mémentos s'ouvrent quand un renvoi vous y envoie.

Le module suivant, `POO`, entre dans le vif du sujet : il commence par ce qui change quand on passe de C# à Java.

Module POO

POO — Classes et objets

Passer de C# à Java, et poser les bases de tout le cours

Vous connaissez déjà la programmation orientée objet en C#. Java en est très proche : la plupart de ce que vous savez se transpose directement. Ce module fait l'inventaire des différences, puis reprend les notions de base telles que Java les écrit : la classe et l'objet, l'encapsulation, les membres de classe, la lecture au clavier, et ce que contient vraiment une variable.

C'est le module fondateur du cours : tous les suivants s'appuient sur lui.

> **À la fin de ce module, vous saurez**
> - écrire une classe Java, avec ses champs, son constructeur et sa méthode `toString` ;
> - protéger l'état d'un objet, et juger quels accesseurs il mérite ;
> - distinguer ce qui appartient à la classe (`static`) de ce qui appartient à l'objet ;
> - lire au clavier avec `Scanner`, et trouver seul ce qu'offre une classe du JDK dans sa Javadoc ;
> - expliquer ce que contient une variable d'objet, et ce qu'une méthode reçoit en paramètre ;
> - séparer le calcul du dialogue, pour rendre le calcul testable.

POO §1 — De C# à Java

Le même programme, dans les deux langages

Les deux programmes suivants font exactement la même chose.

```csharp
// C#
namespace ProgOO
{
    class Personne
    {
        public string Nom;
        public string Prenom;
        public int AnneeNaissance;
        public int Age;

        public void CalculAge(int Annee)
        {
            Age = Annee - AnneeNaissance;
        }

        public void Afficher()
        {
            Console.WriteLine(Prenom + " "
            + Nom + " (" + Age + " ans)");
        }
    }

    class Program
    {
        static void Main(string[] args) {
            /* ... */
        }
    }
}
```

```java
// Java
package progoo;

class Personne {
    public String nom;
    public String prenom;
    public int anneeNaissance;
    public int age;

    public void calculAge(int annee) {
        age = annee - anneeNaissance;
    }

    public void afficher() {
        System.out.println(prenom + " "
        + nom + " (" + age + " ans)");
    }
}

public class Program {
    public static void main(String[] args)
{
        /* ... */
    }
}
```

Cinq différences les séparent :

En C#	En Java
`namespace ProgOO`	`package progoo;` : un package correspond aussi à un dossier sur le disque
`Nom`, `CalculAge`	`nom`, `calculAge` : la majuscule initiale est réservée aux classes, énumérations et interfaces
plusieurs classes publiques par fichier	au plus une classe `public` par fichier, qui lui donne son nom : ici, `Program.java`
`Console.WriteLine`	`System.out.println` : les classes du framework ne sont pas les mêmes
`Main` peut rester privée	`main` doit être `public static`

La classe `Personne`, déclarée sans modificateur, n'est visible que dans son package : c'est ce qui lui permet de partager le fichier `Program.java` avec une classe `public`.

N'importe quelle classe peut être exécutable

En Java, une classe devient exécutable dès qu'elle contient une méthode de signature exacte :

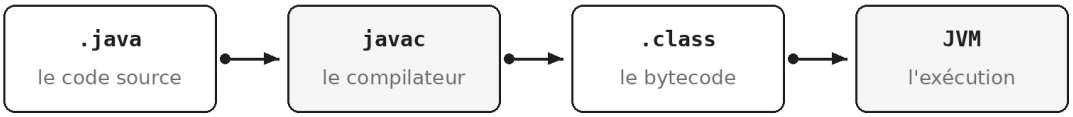
```java
public static void main(String[] args)
```

IntelliJ affiche alors une flèche verte dans la marge, à côté de la classe (voir `IDE.16` du guide IntelliJ). Pour essayer un ensemble de classes, écrivez plutôt une classe `Programme` séparée, qui les utilise : vos classes métier restent ainsi débarrassées de tout code d'essai. Le module `JUN` présentera une manière bien plus fiable de vérifier qu'une classe fonctionne : les classes de test.

Une précision si vous installez un JDK récent chez vous : depuis Java 25, `main` peut s'écrire plus simplement (sans `public`, sans `static`, et même sans paramètre). Le cours et l'examen utilisent Java 21, où la signature `public static void main(String[] args)` reste la seule valide.

Du code source au programme

Le code que vous écrivez n'est pas exécuté tel quel.



Du code source à l'exécution

Le compilateur `javac` traduit le code source en *bytecode*, rangé dans des fichiers `.class`, et c'est ce bytecode que la machine virtuelle Java (la JVM) exécute, sur n'importe quel système. Le mémento technique détaille ces trois états (`TEC-CMP.1`) et explique pourquoi un même `.class` tourne partout (`TEC-CMP.2`).

> **À retenir**
>
> - `namespace` devient `package`.
> - Une majuscule pour les types, une minuscule pour tout le reste.
> - Un fichier `.java` contient au plus une classe `public`, qui lui donne son nom.
> - `Console.WriteLine` devient `System.out.println`.
> - `main` doit être `public static`.

POO §2 — Classe et objet

Les quatre principes de la POO

La programmation orientée objet repose sur quatre principes :

Principe	En une phrase	Où dans le cours
Encapsulation	l'objet protège son état	ce module, voir `POO §3`
Abstraction	ne modéliser que ce qui compte pour le problème	partout
Héritage	une classe reprend et étend une autre	module `ABS`
Polymorphisme	traiter uniformément des objets différents	modules `ABS` et `ITF`

L'encapsulation regroupe dans une même classe les données et les méthodes qui les manipulent, et cache les détails internes : on agit sur un objet par ses méthodes, pas en touchant directement à ses champs.

L'abstraction consiste à ne retenir, d'un objet du monde réel, que les caractéristiques utiles au problème : une personne n'a pas la même forme dans un logiciel de paie et dans un logiciel médical.

Une classe, des objets

Une classe est un moule : elle décrit des champs et des méthodes, mais ne fait rien par elle-même. Un objet est un exemplaire construit à partir de ce moule, par `new`.

```
public class Livre {
    public String titre;
    public String auteur;
    public int anneeDeParution;
}
```

```
Livre dune = new Livre();
dune.titre = "Dune";
```

Chaque appel à `new` fabrique un nouvel objet, avec ses propres valeurs de champs. La variable à laquelle on assigne cet objet avec `=` ne le contient pas : elle permet de le retrouver (voir `POO §6`). Deux livres créés par deux `new` sont deux objets distincts, même s'ils portent le même titre.

> **Démonstration** - `Demo_POO` : construire une classe, puis l'encapsuler
>
> La démonstration part d'une classe vide et lui ajoute des champs publics. Elle montre qu'un champ public accepte n'importe quelle valeur, même absurde, sans que rien ne proteste. C'est le problème que `POO §3` résout.

Le constructeur

Le constructeur est la méthode appelée par `new`. Il porte le nom exact de la classe, ne déclare aucun type de retour (pas même `void`), et met l'objet dans un état correct dès sa création.

```
public class Livre {
    private String titre;
    private int anneeDeParution;

    public Livre(String titre, int anneeDeParution) {
        this.titre = titre;
        this.anneeDeParution = anneeDeParution;
    }
}
```

```
Livre dune = new Livre("Dune", 1965);
Livre vide = new Livre();   // refuse : ce constructeur n'existe plus
```

Une classe qui ne déclare aucun constructeur en reçoit un d'office, le constructeur *par défaut* : il ne prend aucun paramètre et laisse chaque champ à sa valeur par défaut (0 pour un nombre, `false` pour un booléen, `null` pour un objet). Ce constructeur disparaît dès que vous écrivez le vôtre : dans l'exemple, `new Livre()` ne compile plus.

IntelliJ peut générer un constructeur à partir des champs (voir `IDE.12`). Le mot-clé `private`, que vous connaissez depuis C#, est expliqué en `POO §3`.

Le mot-clé `this`

Dans une méthode ou un constructeur, `this` désigne l'objet sur lequel on travaille. On s'en sert surtout quand un paramètre porte le même nom qu'un champ : le nom seul désigne alors le paramètre, qui masque le champ, et `this.titre` désigne le champ.

```
public Livre(String titre, int anneeDeParution) {
    this.titre = titre;   // le champ recoit le parametre
    titre = titre;        // le parametre recoit... lui-meme
}
```

> **Attention**
>
> L'oubli de `this` est une erreur silencieuse. `titre = titre;` est parfaitement valide : le paramètre est affecté à lui-même, et le champ reste à `null`. Le compilateur ne dit rien ; IntelliJ, en revanche, souligne la ligne (voir `IDE.15`).

Un champ `final`

Certaines données ne doivent jamais changer une fois l'objet créé : le titre d'un livre, le pseudo d'un joueur. Déclarer le champ `final` confie cette règle au compilateur : le champ reçoit sa valeur exactement une fois, au plus tard à la fin du constructeur, et toute affectation ultérieure est refusée.

```
public class Livre {
    private final String titre;

    public Livre(String titre) {
        this.titre = titre;
    }

    public void renommer(String nouveauTitre) {
        titre = nouveauTitre;
        // refuse : cannot assign a value to final variable titre
    }
}
```

L'immuabilité n'est plus une affaire de vigilance : elle est vérifiée par le compilateur. Un champ `final` ne s'écrit pas en majuscules pour autant : les majuscules sont réservées aux constantes, à la fois `static` et `final` (voir `POO §4`).

La méthode `toString`

Tout objet sait se transformer en texte, grâce à sa méthode `toString()`. Celle qu'on reçoit par défaut n'est pas très utile : elle retourne le nom de la classe suivi d'un `@` et d'un numéro, par exemple `Livre@1b6d3586`. On la redéfinit donc presque toujours :

```
@Override
public String toString() {
    return titre + " (" + anneeDeParution + ")";
}
```

```
System.out.println(dune);         // Dune (1965)
String texte = "Lu : " + dune;    // Lu : Dune (1965)
```

`toString()` est rarement appelée explicitement : `System.out.println(dune)` l'appelle pour vous, et la concaténation d'une chaîne et d'un objet aussi. L'annotation `@Override` indique au compilateur qu'on remplace une méthode existante, celle que toute classe reçoit de la classe `Object` ; le module `ABS` explique ce mécanisme d'héritage.

> **Exercices** - `POO.1` Une première classe, puis `G.1` Le joueur et `G.2` La partie (fil rouge GameHub)
>
> Pour choisir vos noms, lisez les règles `CC-NOM.1` (le nom communique l'intention) et `CC-NOM.5` (des noms pour les choses, des verbes pour les actions) du mémento Clean Code.

POO §3 — Encapsulation

Un objet dont les champs sont publics ne contrôle pas son propre état. Cette section montre comment reprendre ce contrôle, et pourquoi il ne suffit pas de générer des accesseurs pour y parvenir.

Le problème d'un champ `public`

Avec un champ `public`, n'importe quel code peut modifier l'objet sans passer par lui :

```
Livre dune = new Livre("Dune", 1965);
dune.anneeDeParution = -3000;   // champ public : accepte sans broncher
```

Le programme compile, s'exécute, et produit des résultats faux ou incohérents sans que rien ne le signale.

Fermer les champs, ouvrir des accesseurs

Encapsuler une classe, c'est rendre ses champs `private` et n'y donner accès que par des méthodes, les *accesseurs*, qui jouent le rôle des propriétés de C# :

- le **getter** lit la valeur : `getTitre()` ;
- le **setter** la modifie : `setTitre(String)` ;
- pour un booléen, le getter commence par `is` : `isDisponible()`.

```
public class Livre {
    private String titre;

    public String getTitre() {
        return titre;
    }

    public void setTitre(String titre) {
        this.titre = titre;
    }
}
```

IntelliJ sait générer les accesseurs (voir `IDE.12`).

Ce que le détour permet

Un getter et un setter qui se contentent de lire et d'écrire le champ ne protègent rien : ils ne font que déplacer l'accès. Leur intérêt apparaît quand on y place une règle (règle qui dépend du contexte dans lequel évoluent nos objets), ce qu'un champ public ne permettra jamais :

```
public void setAnneeDeParution(int annee) {
    if (annee < 0) {
        throw new IllegalArgumentException("Annee invalide : " + annee);
    }
    this.anneeDeParution = annee;
}
```

Ici, le setter refuse une année négative, et l'objet ne peut plus se retrouver dans un état absurde. Le refus prend la forme d'une exception, que le module `EXC` étudie en détail : retenez pour l'instant que `throw` interrompt la méthode et signale l'erreur à l'appelant. `IllegalArgumentException` est une exception dite *non contrôlée* : le compilateur n'exige pas qu'on l'attrape, et le code appelant reste débarrassé de tout `try/catch`.

Un accesseur n'est pas une obligation

Le générateur d'IntelliJ produit en un clic un getter et un setter pour chaque champ. C'est commode, mais le résultat ne vaut guère mieux que des champs publics : n'importe qui peut encore tout lire et tout modifier.

Posez-vous la question pour chaque champ. Doit-il être lu de l'extérieur ? Doit-il être modifié, et selon quelle règle ? Chaque accesseur doit avoir une raison d'être, et un champ `final` n'a par définition aucun setter. La règle `CC-OBJ.2` du mémento Clean Code développe ce point.

La donnée qui n'existe pas

L'âge d'un livre change chaque année, alors que son année de parution ne change jamais. Stocker l'âge dans un champ, c'est accepter qu'il devienne faux au premier janvier suivant, sans que rien ne le mette à jour.

Ce qu'on peut calculer n'a pas à être stocké : une méthode le calcule à la demande, à partir de ce qui est stable.

```
public int getAge() {
    return LocalDate.now().getYear() - anneeDeParution;
}
```

Vu de l'extérieur, `getAge()` ressemble à un getter ordinaire ; aucun champ ne se cache pourtant derrière. `LocalDate.now().getYear()` donne l'année en cours : les classes de dates font l'objet du module `TMP`.

Exercices - `POO.2` La même classe, encapsulée, puis `POO.3` L'âge n'est pas une donnée à stocker

La section `IDE.12` du guide IntelliJ montre comment générer accesseurs et constructeurs, et ce qu'il faut éviter quand le squelette les déclare déjà.

POO §4 — Membres de classe : `static`

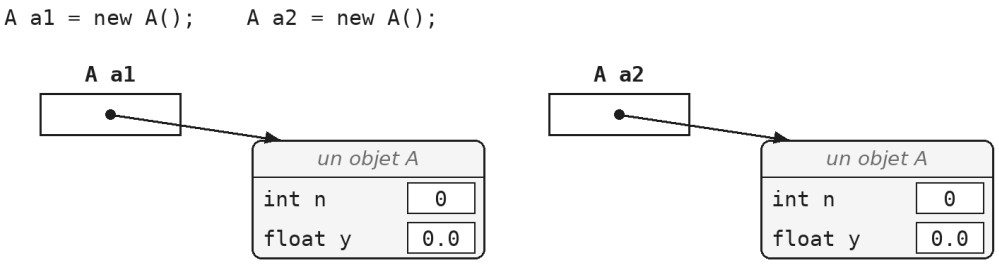
Jusqu'ici, chaque champ appartenait à un objet : chaque livre a son titre. Certaines informations n'appartiennent pourtant à aucun objet en particulier. Le mot-clé `static` les rattache à la classe elle-même.

Champs d'objet et champ de classe

Sans `static`, chaque objet reçoit ses propres exemplaires des champs. Soit la classe `A` suivante :

```
class A { int n; float y; }
```

Une fois cette classe définie, `a1.n` et `a2.n` sont deux cases mémoire distinctes : modifier l'une ne change rien à l'autre.

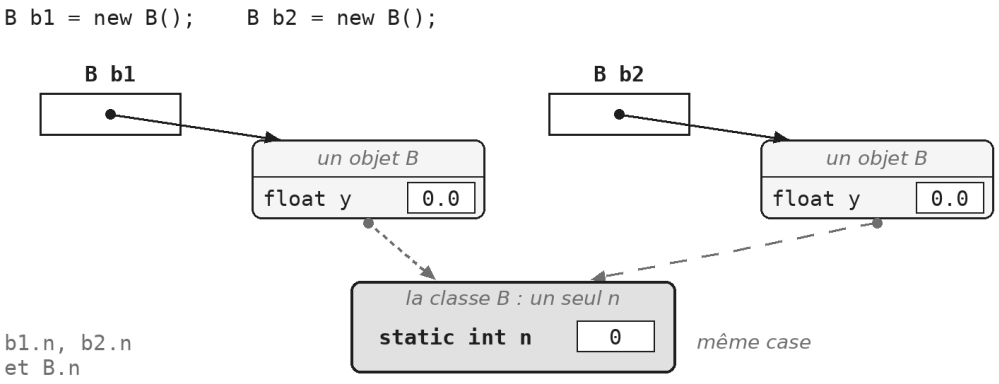


Sans `static` : chaque objet a ses propres champs

Déclaré `static`, un champ n'existe plus qu'une fois, pour toute la classe. Prenons maintenant la classe `B` :

```
class B { static int n; float y; }
```

Ici, `b1.n` et `b2.n` désignent la même case : ce que l'un y écrit, l'autre le lit. Le champ `y`, lui, reste propre à chaque objet.



Avec `static` : un seul n, partagé par tous les objets

Puisqu'un champ statique appartient à la classe, on le désigne de préférence par le nom de la classe : `B.n` dit clairement qu'il s'agit d'un champ partagé, là où `b1.n` laisse croire qu'il appartient à `b1`. À l'intérieur de la classe `B`, le nom `n` suffit. Un champ statique peut aussi être `private` : il reste alors accessible de l'extérieur par des méthodes, comme n'importe quel champ encapsulé.

Démonstration - `Demo_STA` : ce qui appartient à la classe, ce qui appartient à l'objet

La démonstration numérote des voitures, d'abord à la main puis grâce à un compteur partagé. Le débogueur montre ensuite que chaque voiture a son propre numéro, alors que le compteur n'existe qu'une fois (voir `IDE.19`).

Un compteur d'instances

Un champ statique peut compter les objets créés. Qu'affiche ce programme ?

```
class Ticket {
    private static int crees = 0;

    public Ticket() {
        crees++;
        System.out.println("Ticket " + crees + " cree");
    }
}
```

```
public class Programme {
    public static void main(String[] args) {
        Ticket a;
        System.out.println("main 1");
        a = new Ticket();
        System.out.println("main 2");
        Ticket b;
        System.out.println("main 3");
        b = new Ticket();
        Ticket c = new Ticket();
        System.out.println("main 4");
    }
}
```

Le compteur `crees` est partagé par tous les tickets : chaque constructeur l'incrémente, et tous voient la même valeur. Le programme affiche, dans l'ordre : `main 1`, `Ticket 1 cree`, `main 2`, `main 3`, `Ticket 2 cree`, `Ticket 3 cree`, `main 4`.

Remarquez que la déclaration `Ticket a;` ne crée aucun objet : seul `new` appelle le constructeur. Remarquez aussi que ce compteur ne diminue jamais : Java ne prévient pas un objet de sa disparition (voir `TEC-EXE.4` du mémento technique).

Méthodes de classe

Une méthode `static` ne travaille sur aucun objet particulier : on l'appelle sur la classe.

```
double racine = Math.sqrt(16);
int annee = LocalDate.now().getYear();
```

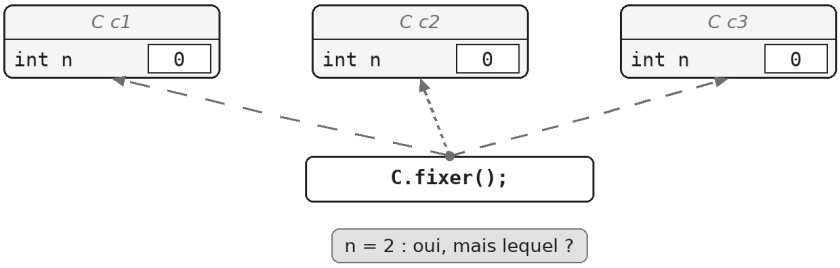
`Math.sqrt(16)` ne demande pas de créer un objet `Math` ; la classe `Math` ne permet d'ailleurs pas d'en créer. Vous en utilisez depuis le début : `main` est une méthode de classe. Java accepte qu'on appelle une méthode statique à travers une variable d'objet, mais l'écriture laisse croire que la méthode dépend de cet objet : c'est trompeur, et IntelliJ le signale par un avertissement.

Une méthode statique n'a pas de `this`

C'est l'erreur la plus fréquente du module. Une méthode statique ne peut utiliser ni les champs ni les méthodes non statiques de sa classe. Voici un contre-exemple qui peut vous faire comprendre pourquoi :

```
class C {
    int n = 0;

    public static void fixer() {
        n = 2;   // refuse : non-static variable n cannot be
                 //          referenced from a static context
    }
}
```



`C.fixer()` est appelée sur la classe : quel n modifier ?

La méthode `fixer()` est appelée sur la classe, pas sur un objet : s'il était possible de l'appeler, elle n'aurait aucun moyen de savoir de quel `n` on parle, puisqu'il en existe un par objet. Le même raisonnement interdit `this`, qui n'a rien à désigner. C'est aussi ce qui explique que `main` soit statique : il s'exécute avant qu'aucun objet n'existe.

Les constantes

`static` et `final` répondent à deux questions différentes. `static` : la valeur est-elle propre à chaque objet, ou partagée par toute la classe ? `final` : peut-elle changer après avoir été fixée ? Les deux réponses se combinent, ce qui donne quatre sortes de champs :

	Peut changer	Ne change plus (`final`)
Propre à chaque objet	`private int anneeDeParution;` : l'année d'un livre, qu'un setter peut corriger	`private final String titre;` : le titre d'un livre, initialisé une fois pour toutes par le constructeur
Partagé par la classe (`static`)	`private static int crees;` : un compteur d'instances	`public static final double TVA = 0.21;` : une constante

Une constante a donc besoin des deux mots ; un compteur d'instances n'a besoin que de `static`, et un champ immuable d'objet que de `final`.

```
public class Facture {
    public static final double TVA = 0.21;
    public static final int JOURS_DE_PAIEMENT = 30;
}
```

```
double prix = 100 * (1 + Facture.TVA);
```

Par convention, une constante s'écrit en majuscules, avec des soulignés entre les mots. Nommer une valeur plutôt que l'écrire en dur rend le code lisible et évite de la répéter : c'est la règle `CC-NOM.7` du mémento Clean Code.

À quoi servent les méthodes de classe

Elles servent dans trois situations :

- **répondre sur l'ensemble des objets** plutôt que sur l'un d'eux : combien d'objets ont été créés ?
- **offrir un service sans enregistrer d'état** : `Math.sqrt`, `Integer.parseInt` ;
- **fabriquer des objets** : `LocalDate.now()`, `LocalDate.of(...)` retournent une date sans qu'on ait écrit `new`. Ces méthodes s'appellent des *fabriques* (et ces méthodes-là contiennent bien un `new`, puisqu'un nouvel objet est créé).

Les fabriques

Un point du plan peut se décrire par ses coordonnées ou par une distance et un angle : dans les deux cas, deux nombres réels. Deux constructeurs `Point(double, double)` ne peuvent pas coexister, puisqu'ils auraient la même signature. Deux méthodes statiques, elles, se distinguent par leur nom, et ce nom dit en plus comment lire les paramètres.

```
public class Point {
    private final double x;
    private final double y;

    private Point(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public static Point cartesien(double x, double y) {
        return new Point(x, y);
    }

    public static Point polaire(double rayon, double angle) {
        double x = rayon * Math.cos(angle);
        double y = rayon * Math.sin(angle);
        return new Point(x, y);
    }
}
```

```
Point a = Point.cartesien(3, 4);
Point b = Point.polaire(5, 0.93);
```

Remarquez le constructeur `private` qui oblige tout le code extérieur à passer par les fabriques : personne ne peut plus écrire `new Point(3, 4)` en se demandant ce que représentent les deux nombres.

> **Exercices** - `STA.1` Une constante et un compteur d'instances, puis `STA.2` Un constructeur privé et des fabriques
>
> `STA.1` demande d'observer les champs dans le débogueur : la section `IDE.19` du guide IntelliJ explique comment poser un point d'arrêt et lire le panneau Variables.

POO §5 — Entrée de données au clavier

Un programme qui dialogue avec son utilisateur doit pouvoir lire ce qu'il tape. Cette section présente la classe `Scanner`, et profite de l'occasion pour apprendre à lire la documentation d'une classe du JDK.

La classe `Scanner`

La classe `Scanner`, du package `java.util`, lit ce que l'utilisateur tape dans la console. On la construit une seule fois, sur `System.in`, puis on appelle ses méthodes autant de fois que nécessaire :

```
Scanner clavier = new Scanner(System.in);
String nom = clavier.nextLine();
int age = clavier.nextInt();
```

Ouvrir plusieurs `Scanner` sur la même entrée provoque des comportements difficiles à comprendre : un seul suffit pour tout le programme.

La Javadoc

Pour savoir ce qu'offre une classe du JDK, on consulte sa Javadoc. Chaque page présente la classe, puis l'inventaire de ses constructeurs et de ses méthodes. Pour chaque méthode, elle indique les paramètres attendus, la valeur retournée et les exceptions possibles.

Inutile d'ouvrir un navigateur : IntelliJ affiche cette documentation au survol d'un nom, ou avec `Ctrl+Q` / `F1` (voir `IDE.14`). La version en ligne, pour Java 21, se trouve à l'adresse `docs.oracle.com/en/java/javase/21/docs/api/`.

Pour `Scanner`, la documentation indique le package, `java.util`, puis le principe : l'entrée est découpée en *jetons*, séparés par défaut par des espaces ou des passages à la ligne, et chaque jeton peut être converti dans le type voulu grâce aux différentes méthodes `next`. Pour ce cours, quelques méthodes suffisent :

Méthode	Ce qu'elle lit
`nextLine()`	le reste de la ligne courante, sans le passage à la ligne final
`next()`	le mot suivant
`nextInt()`	le jeton suivant, converti en `int` ; `InputMismatchException` si ce n'est pas un entier
`nextDouble()`	le jeton suivant, converti en `double`

La documentation de `nextInt()` signale aussi la variante `nextInt(radix)`, qui interprète le nombre dans la base indiquée.

Un piège : `nextInt` suivi de `nextLine`

```
System.out.print("Annee : ");
int annee = clavier.nextInt();       // lit 1965, laisse la fin de ligne
System.out.print("Titre : ");
String titre = clavier.nextLine();   // retourne "" sans attendre
```

Quand l'utilisateur tape 1965 puis Entrée, `nextInt()` consomme les chiffres mais laisse le passage à la ligne dans le tampon. Le `nextLine()` suivant le trouve immédiatement et retourne une chaîne vide : pour l'utilisateur, la question du titre a été sautée.

On peut ordonner les questions pour éviter qu'un `nextLine()` suive un `nextInt()`, ou consommer la fin de ligne par un `nextLine()` supplémentaire. L'exercice `POO.5` vous demandera de chercher une manière de ne jamais tomber dans le piège.

> **Exercices** - `POO.4` Saisie au clavier (première partie seulement), puis `G.3` Le point d'entrée (fil rouge GameHub)
>
> La seconde partie de `POO.4` vient après `POO §6`.

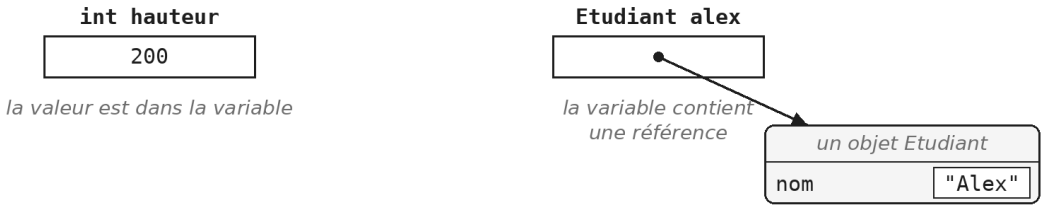
POO §6 — Primitifs et références

Une variable Java ne contient pas la même chose selon son type. Cette différence explique plusieurs comportements surprenants, dont celui que l'exercice `POO.4` vous fait constater.

Ce que contient une variable

Java compte huit types primitifs : `byte`, `short`, `int`, `long`, `float`, `double`, `char` et `boolean`. Une variable de l'un de ces types contient directement sa valeur.

Une variable d'objet, elle, ne contient pas l'objet. Elle contient une *référence*, c'est-à-dire le moyen de retrouver l'objet, qui vit ailleurs en mémoire. Le mémento technique décrit où vivent l'un et l'autre (`TEC-EXE.1` à `TEC-EXE.3`).



Un primitif contient sa valeur ; une variable d'objet, une référence

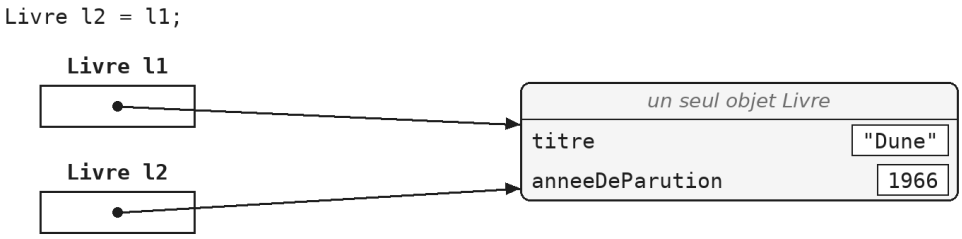
Deux noms, un seul objet

L'affectation copie toujours le contenu de la variable. Pour un `int`, ce contenu est le nombre ; pour un objet, c'est la référence.

```
int a = 5;
int b = a;
b = 7;                                      // a vaut toujours 5

Livre l1 = new Livre("Dune", 1965);
Livre l2 = l1;                              // l2 designe le MEME objet
l2.setAnneeDeParution(1966);
System.out.println(l1.getAnneeDeParution()); // 1966
```

Après `l2 = l1`, les deux variables désignent le même livre, et une modification faite par l'une se voit par l'autre.



Après `l2 = l1` : deux variables, un seul objet

Le débogueur rend ce partage visible : il affiche à côté de chaque objet un numéro, et deux variables qui portent le même numéro désignent le même objet.

> **Exercices** - `POO.4` Saisie au clavier, seconde partie
>
> Observez, puis expliquez par écrit ce que vous constatez, avant de lire la suite.

Ce que vous avez observé

Java passe toujours les arguments **par valeur** : une méthode reçoit une copie de ses arguments.

```
static void modifier(int n, Livre livre) {
    n = 99;                          // change la copie
    livre.setAnneeDeParution(2000);  // change l'objet partage
    livre = new Livre("Autre", 1);   // la copie designe un autre objet
}
```

- Pour un `int`, la méthode reçoit une copie du nombre : modifier le paramètre ne change rien chez l'appelant.

- Pour un objet, elle reçoit une copie de la référence, qui désigne le même objet que la variable de l'appelant : ce qu'on fait à l'objet à travers elle se voit de partout.
- Ce n'est pas pour autant un passage par référence : la troisième ligne fait désigner un autre objet à la copie, et la variable de l'appelant n'en est pas affectée. À la sortie de la méthode ci-dessus, le paramètre disparaît : plus rien ne désigne le livre « Autre », et le ramasse-miettes pourra récupérer sa place (voir `TEC-EXE.4` du mémento technique).

`TEC-EXE.3` reprend ce raisonnement dans le mémento technique.

POO §7 — Les wrappers

Chaque type primitif a une classe jumelle, qui l'enveloppe et lui ajoute des méthodes utiles. Ces classes sont l'occasion de retrouver les membres statiques de `POO §4`.

Un objet pour chaque type primitif

À chacun des huit types primitifs correspond une classe du package `java.lang`, appelée *wrapper* ou classe enveloppe. Le nom reprend celui du type avec une majuscule, sauf pour `int` et `char` :

Type primitif	Wrapper
`byte`	`Byte`
`short`	`Short`
`int`	`Integer`
`long`	`Long`
`float`	`Float`
`double`	`Double`
`char`	`Character`
`boolean`	`Boolean`

Ce qu'offrent les wrappers

Les wrappers offrent surtout des membres statiques, qu'on appelle donc sur la classe :

```
int n = Integer.parseInt("42");             // une conversion
int plusGrand = Integer.MAX_VALUE;          // une constante
boolean chiffre = Character.isDigit('7');   // un test

Integer boite = 5;     // autoboxing
int valeur = boite;    // unboxing
```

Java convertit automatiquement un primitif en wrapper et inversement : c'est l'*autoboxing* et l'*unboxing*. Cela permet d'écrire `Integer boite = 5;` sans conversion explicite.

Méfiez-vous de `Character.isSpace`, que vous croiserez dans d'anciens exemples : elle est dépréciée depuis Java 1.1 au profit de `Character.isWhitespace`. Elle ne reconnaît que cinq caractères ASCII (l'espace, la tabulation, le passage à la ligne, le saut de page et le retour chariot), là où `isWhitespace` suit la définition Unicode des blancs. Les espaces insécables n'en font pas partie : `TMP §4` montre où ce détail peut piéger un test.

Séparer le calcul du dialogue

Un calcul écrit directement dans le `main`, mêlé aux lectures et aux affichages, ne peut être vérifié qu'à la main. Placé dans une méthode qui reçoit ses données en paramètres et retourne un résultat, il peut être appelé et vérifié automatiquement, avec autant de valeurs qu'on veut.

```
public class Disque {
    public static double aire(double rayon) {
        return Math.PI * rayon * rayon;
    }

    public static void main(String[] args) {
        Scanner clavier = new Scanner(System.in);
        System.out.print("Rayon : ");
        double rayon = clavier.nextDouble();
        System.out.println("Aire : " + aire(rayon));
    }
}
```

La méthode `aire` calcule et retourne un résultat : elle ne lit ni n'affiche rien. Le `main` dialogue avec l'utilisateur : il ne calcule rien. Le calcul devient testable.

> **À retenir**
>
> Cette séparation est une règle du cours à partir de maintenant. Le mémento Clean Code la formule en `CC-MET.8` : une méthode agit ou répond, jamais les deux.

> **Exercices** - `POO.5` Changement de base, et logique testable
>
> L'exercice demande de lire la Javadoc de `Integer.parseInt(String, int)` : la section `IDE.14` du guide montre comment l'afficher sans quitter IntelliJ.

Ce qu'il faut retenir

- Une classe décrit, `new` construit, le constructeur initialise.
- `this` désigne l'objet en cours ; `final` fige un champ.
- Des champs `private`, et des accesseurs seulement s'ils se justifient.
- `static` : ce qui appartient à la classe, pas à l'objet.
- Une variable d'objet contient une référence ; les arguments sont toujours passés par valeur.
- Le calcul dans une méthode, le dialogue dans le `main`.

Ce module a posé les bases sur lesquelles reposent tous les suivants : des classes qui protègent leur état, des membres de classe quand une information n'appartient à aucun objet, et une distinction nette entre ce que contient une variable primitive et ce que contient une variable d'objet. La séparation entre calcul et dialogue sera appliquée dans tous les exercices à venir.

Module TMP

TMP — Dates et heures

Fabriquer, comparer, décaler, mesurer et afficher des dates

Manipuler une date paraît simple tant qu'on ne s'y met pas. Combien de jours dans un mois ? Ça dépend. Combien de jours dans une année ? Ça dépend aussi. Le 31 février existe-t-il ? Non, mais rien n'empêche de l'écrire. Les programmes des années 90 stockaient souvent une date dans trois `int` et faisaient l'arithmétique eux-mêmes : c'est la source d'une quantité impressionnante de bogues, entre les mois de longueur variable, les années bissextiles et les passages d'année.

Le JDK fait ce travail pour vous, et le fait correctement, dans le package `java.time`. Votre part consiste à savoir quelle classe demander, et à connaître les quelques pièges que ce module passe en revue.

> **À la fin de ce module, vous saurez**
>
> - choisir entre `LocalDate`, `LocalTime` et `LocalDateTime`, et les fabriquer, les lire et les comparer ;
> - mesurer un écart, et choisir entre `ChronoUnit`, `Period` et `Duration` ;
> - expliquer pourquoi une date ne change jamais, et ce que cela implique ;
> - afficher des nombres et des dates proprement, d'une façon qu'un test peut vérifier ;
> - passer d'une date à son texte et inversement, sans laisser passer une date impossible.

TMP §1 — Les classes de `java.time`

Les trois classes du quotidien

Trois classes couvrent l'essentiel des besoins :

Classe	Ce qu'elle représente	Exemple
`LocalDate`	une date, sans heure	`2027-01-19`
`LocalTime`	une heure, sans date	`18:00`
`LocalDateTime`	les deux	`2027-01-19T18:00`

Le mot « `Local` » signifie qu'elles ignorent les fuseaux horaires : elles décrivent une date ou une heure telle qu'on la lit sur un calendrier ou une horloge murale, sans dire où. Pour un anniversaire ou un horaire de cours, c'est exactement ce qu'il faut. Pour un vol Bruxelles-Tokyo, il faudrait `ZonedDateTime`, qui dépasse ce cours.

Le fuseau de la machine n'intervient qu'une seule fois, dans `now()`, pour décider quelle date il est à cet instant. L'objet produit n'en garde aucune trace : un `LocalDateTime` déplacé d'un serveur à l'autre désigne toujours la même heure de calendrier.

Le package `java.time` fait partie de la bibliothèque standard : il n'y a rien à installer ni à déclarer dans le `pom.xml`. C'est ce qui le distingue des bibliothèques tierces, que Maven doit aller chercher (voir `TEC-PLA.1` du mémento technique).

Laquelle choisir ?

Le tableau suivant est la carte du module. Les deux dernières lignes font l'objet de `TMP §7` : retenez pour l'instant qu'il existe deux façons de mesurer un écart, et qu'elles ne se valent pas.

Ce que vous représentez	La classe
une date de naissance, une échéance	`LocalDate`
une heure d'ouverture	`LocalTime`
un rendez-vous, un horodatage	`LocalDateTime`
un écart en années, mois et jours	`Period`
un écart en heures, minutes et secondes	`Duration`

Fabriquer une date

La classe `LocalDate` n'a pas de constructeur public. On fabrique une date par une méthode statique, appelée sur la classe et non sur un objet : `now`, `of` ou `parse`. C'est le motif déjà rencontré au module `POO` avec les méthodes statiques des wrappers.

```
LocalDate aujourdhui = LocalDate.now();
LocalDate naissance  = LocalDate.of(1990, 5, 17);
LocalDate lue        = LocalDate.parse("1990-05-17");
```

La méthode `of` prend les nombres dans l'ordre naturel (année, mois, jour), et le mois va de 1 à 12, sans le décalage d'une case qu'on trouve dans de vieilles API. La méthode `parse` n'accepte que le format ISO, `AAAA-MM-JJ` ; tout autre format demandera un `DateTimeFormatter`, voir `TMP §8`.

Lire un morceau de date

Les accesseurs sont nombreux et leurs noms sont prévisibles : la complétion d'IntelliJ suffit à les trouver.

```
int annee          = naissance.getYear();
int jourDuMois     = naissance.getDayOfMonth();
DayOfWeek jour     = naissance.getDayOfWeek();
boolean bissextile = naissance.isLeapYear();
int joursDuMois    = naissance.lengthOfMonth();
```

La méthode `getDayOfWeek` ne retourne pas un nombre mais un `DayOfWeek`, qui est une énumération : un type dont les valeurs sont connues d'avance. Vous les étudierez au module `ENU` ; pour l'instant, il suffit de savoir qu'un `DayOfWeek` peut s'afficher et se comparer. Et `lengthOfMonth` répond à une question que vous ne voudrez jamais calculer vous-même.

Comparer deux dates

Les opérateurs `<` et `>` ne fonctionnent que sur les types primitifs. Une date est un objet : on la compare donc par des méthodes, comme on compare des chaînes avec `equals` plutôt qu'avec `==`.

```
naissance.isBefore(aujourdhui);   // true
naissance.isAfter(aujourdhui);    // false
naissance.isEqual(aujourdhui);    // false
```

Quelques exemples avec `LocalDate`

Objectif	Code
la date du jour	`LocalDate aujourdhui = LocalDate.now();`
une date donnée (15/06/2008)	`LocalDate.parse("2008-06-15");` ou `LocalDate.of(2008, 6, 15);`
la date de demain	`LocalDate demain = LocalDate.now().plusDays(1);`
aujourd'hui moins un mois	`LocalDate.now().minus(1, ChronoUnit.MONTHS);`
le jour de la semaine	`DayOfWeek jeudi = LocalDate.parse("2010-07-15").getDayOfWeek();`
année bissextile ?	`boolean bissextile = LocalDate.now().isLeapYear();`
nombre de jours du mois, de l'année	`aujourdhui.lengthOfMonth();` et `aujourdhui.lengthOfYear();`
nombre de jours entre deux dates	`long jours = ChronoUnit.DAYS.between(naissance, aujourdhui);`

Dans ce tableau, `now`, `parse` et `of` s'appellent sur la classe elle-même ; `MONTHS` et `DAYS` sont des constantes de `ChronoUnit`, qui désigne une unité de mesure et se trouve dans le package `java.time.temporal`. Toutes les autres méthodes s'appellent sur une date. Aucune ne modifie la date d'origine : `plusDays` ou `minus` retournent une nouvelle date (voir `TMP §3`).

L'heure seule, puis les deux à la fois

La classe `LocalTime` suit exactement les mêmes conventions : pas de constructeur, des méthodes statiques, des accesseurs prévisibles, `isBefore` et `isAfter` pour comparer.

```
LocalTime debut = LocalTime.of(18, 0);
LocalTime fin   = LocalTime.parse("21:35");
int heure       = debut.getHour();
```

Quelques constantes désignent des heures remarquables : `LocalTime.MIN` et `LocalTime.MIDNIGHT` valent toutes deux minuit, `LocalTime.NOON` midi, et `LocalTime.MAX` la dernière nanoseconde de la journée.

La classe `LocalDateTime` combine les deux, et sait retourner chacune de ses moitiés :

```
LocalDateTime rdv = LocalDateTime.of(2027, 1, 19, 18, 0);

rdv.toLocalDate();   // 2027-01-19
rdv.toLocalTime();   // 18:00
```

Retenez cette dernière possibilité : passer d'une classe à l'autre est souvent la réponse à une méthode qui refuse vos arguments.

> **Démonstration** - `Demo_TMP` : explorer une classe du JDK sans quitter l'éditeur
>
> La démonstration ne présente aucune API nouvelle. Elle montre comment trouver seul, dans l'éditeur, ce qu'on ne connaît pas encore : la complétion comme table des matières, la bulle de documentation comme réseau de liens, et le piège d'un nom qui ressemble à ce qu'on cherche sans en être. C'est la méthode que les énoncés supposent acquise chaque fois qu'ils écrivent « cherchez dans la Javadoc ». Voir `IDE.11` et `IDE.14` du guide IntelliJ.

TMP §2 — Mesurer un écart

Combien de jours entre deux dates ? La question a l'air anodine ; elle demande de choisir une unité, et de savoir dans quel sens compter.

`ChronoUnit`, l'unité de mesure

```
import java.time.temporal.ChronoUnit;

long jours = ChronoUnit.DAYS.between(naissance, aujourdhui);
long mois  = ChronoUnit.MONTHS.between(naissance, aujourdhui);
```

La tournure surprend : c'est l'unité qui contient la méthode `between`, et les deux dates sont ses arguments. On écrit `ChronoUnit.DAYS.between(...)`, et non `ChronoUnit.between.DAYS`. Lisez-la de gauche à droite : « en jours, l'écart entre ceci et cela ». `ChronoUnit` propose `DAYS`, `WEEKS`, `MONTHS`, `YEARS`, et une dizaine d'autres unités.

Deux pièges de comptage

- **`between` compte des unités entières.** Entre le 1er et le 30 janvier, il n'y a pas un mois entier : `MONTHS.between` retourne 0. Ce n'est pas un défaut, c'est une définition, et il faut la connaître avant de s'en étonner.
- **L'ordre des arguments décide du signe.** Inverser les deux dates inverse le signe du résultat, qui peut donc être négatif.

Le résultat est un `long`, pas un `int` : un écart exprimé en secondes déborde très vite la capacité d'un `int`. La question revient plus loin (voir `TMP §5`).

> **Exercices** - `TMP.1` Mesurer un écart entre deux dates
>
> Quatre méthodes statiques mesurent le même écart dans quatre unités. Elles ne lisent rien au clavier et n'affichent rien : elles reçoivent deux dates et retournent un nombre. C'est la séparation entre le calcul et le dialogue, posée au module `POO` et appliquée partout ensuite.

TMP §3 — Une date ne change pas

Décaler une date

Les méthodes qui déplacent une date existent par dizaines, et leurs noms ne posent aucune difficulté : `plusDays`, `minusMonths`, `withDayOfMonth`. Leur comportement, en revanche, mérite qu'on regarde de près. Avant de lire la suite, devinez ce qu'affiche la dernière ligne, puis exécutez le code pour vérifier.

```
LocalDate rendezVous = LocalDate.of(2027, 1, 19);

rendezVous.plusWeeks(1);
System.out.println(rendezVous);   // ?
```

Exercices - `TMP.2` Les dates sont immuables

Exécutez d'abord le programme fourni, et lisez sa sortie. Il contient une erreur qui ne provoque ni refus du compilateur, ni plantage, ni message d'aucune sorte, et donne pourtant un résultat faux.

Une date est immuable

La ligne affiche `2027-01-19` : la date n'a pas bougé. Comme un `String`, un `LocalDate` est **immuable** : sa valeur est fixée à la construction et ne changera jamais. La méthode `plusWeeks` ne modifie rien ; elle fabrique un nouvel objet et le retourne. Si ce nouvel objet n'est pas affecté à une variable, il est perdu.

```
LocalDate rendezVous = LocalDate.of(2027, 1, 19);
LocalDate uneSemainePlusTard = rendezVous.plusWeeks(1);   // 2027-01-26
```

Vous aviez déjà rencontré cette mécanique sans la nommer : `texte.toUpperCase()` ne met pas la chaîne en majuscules, elle en retourne une autre.

Ce n'est pas une gêne, c'est une garantie. Vous pouvez confier un objet immuable à n'importe quelle méthode sans craindre qu'elle le modifie dans votre dos : c'est ce que réclame `CC-MET.7` du mémento Clean Code, qui met en garde contre les effets de bord.

À retenir

Les méthodes d'une date retournent une nouvelle date ; elles ne modifient jamais celle sur laquelle on les appelle. Affectez toujours leur résultat.

TMP §4 — Un détour par l'affichage

Les exercices qui suivent produisent des nombres longs et des durées, et `println` ne suffira plus. Cette section est le seul moment de l'année consacré à l'affichage formaté ; tout ce qui vient après s'en sert, y compris le module `TAB`.

`println` et `print`

```
System.out.println("Bonjour");   // affiche et passe a la ligne
System.out.print("Bonjour");     // affiche, sans passer a la ligne

System.out.print("A");
System.out.print("B");           // => AB, sans passage a la ligne après
```

La méthode `print` fait la même chose que `println`, sans le retour à la ligne. Elle sert dès qu'on construit une ligne morceau par morceau : l'affichage d'une matrice, par exemple, au module `TAB`.

`printf` : un gabarit et des valeurs

La méthode `printf` prend un gabarit contenant des « trous à remplir », marqués par `%`, puis les valeurs qui viennent remplir ces trous, dans l'ordre : le premier trou reçoit la première valeur.

```
System.out.printf("%s a %d ans%n", "Max", 21);
// Max a 21 ans
```

Pour un affichage simple, ce n'est pas plus court qu'une concaténation ; dès qu'il y a trois valeurs ou une question de présentation, c'est beaucoup plus lisible, et souvent plus court. Employez `%n` plutôt que `\n` : `%n` produit le retour à la ligne de la machine sur laquelle le programme tourne.

Gabarit	Résultat	Effet
`%d`	`1234567`	un entier
`%,d`	`1,234,567`	séparateur de milliers

Gabarit	Résultat	Effet
`%s`	`Max`	du texte
`%f`	`3.141590`	six décimales par défaut
`%.2f`	`3.14`	deux décimales
`%8.2f`	`    3.14`	deux décimales, sur 8 caractères
`%n`	(saut de ligne)	le retour à la ligne de la machine
`\t`	(tabulation)	pas un gabarit : un caractère de tabulation, utilisable dans toute chaîne

Le gabarit `%8.2f` sert à aligner une colonne de nombres : chaque valeur occupe la même largeur, et les séparateurs décimaux se superposent.

La locale décide du séparateur

Sans précision, le séparateur des milliers et celui des décimales dépendent de la langue de la machine :

```
System.out.printf("%,d%n", 1157479200);
// machine francaise : 1 157 479 200  (espace INSECABLE etroite)
// machine anglaise  : 1,157,479,200

System.out.printf(Locale.US, "%,d%n", 1157479200);
// partout           : 1,157,479,200
```

C'est le piège le plus coûteux de cette section, parce qu'il ne se voit pas. Le JDK français sépare les milliers par un espace insécable étroit, qui ressemble à s'y méprendre à un espace ordinaire. Un test qui compare le texte produit échouera sur une différence invisible à l'œil, et vous chercherez l'erreur ailleurs.

> **À retenir**
>
> Dès que le texte produit est comparé par un test, précisez la locale.

> **Pour aller plus loin** - une espace qui n'en est pas une
>
> Le séparateur de la locale française est l'espace fine insécable, `\u202F`. Pour Java, ce n'est pas un « blanc » : `Character.isWhitespace('\u202F')` retourne `false`, comme pour toutes les espaces insécables, qui par définition ne doivent pas couper un mot. Découper `"1 157 479 200"` produit par `String.format` sur ses espaces, avec `split(" ")` ou `isWhitespace`, ne donne donc qu'un seul morceau. Voir aussi `POO §7`, à propos de `Character.isWhitespace`.

`String.format`, la version testable

La méthode `String.format` prend exactement les mêmes gabarits que `printf`. La différence est décisive pour la suite du cours :

```
// affiche la valeur a la console
System.out.printf(Locale.US, "%,d", n);

// retourne la valeur a l'appelant
String s = String.format(Locale.US, "%,d", n);
```

- `printf` écrit dans la console et ne retourne rien : aucune classe de test ne peut vérifier son travail ;
- `String.format` retourne une chaîne, qu'un test peut comparer.

C'est la règle posée au module `POO`, reformulée pour l'affichage : le noyau retourne une valeur, le `main` se contente de dialoguer. Vos méthodes de formatage retourneront donc des `String`, et seul le `main` appellera `printf`.

TMP §5 — Date ou instant ?

Ce que `LocalDate` ne contient pas

Un `LocalDate` désigne une case du calendrier, le 17 mai 1990, et rien d'autre : pas d'heure, pas de minute, pas de seconde. Que donne alors la seconde ligne ?

```
ChronoUnit.DAYS.between(d1, d2);      // fonctionne
ChronoUnit.SECONDS.between(d1, d2);   // ?
```

> **Exercices** - `TMP.3` Compter des secondes : date ou instant ?
>
> L'exercice reprend `TMP.1` en changeant l'unité. Décommentez la ligne signalée et lisez le message : la capacité à lire un message d'erreur vaut mieux qu'une réponse apprise. Le formatage du résultat emploie ce qui vient d'être vu en `TMP §4`.

Le JDK refuse plutôt que d'inventer

La seconde ligne lève une exception : `Unsupported unit: Seconds`. Demander à deux cases de calendrier combien de secondes les séparent n'a pas de sens, et le JDK refuse plutôt que de choisir une heure à votre place.

La méthode `atStartOfDay()` fabrique le `LocalDateTime` qui correspond à minuit. C'est alors vous qui prenez la décision, et elle reste une convention, pas une vérité : votre résultat est juste à quelques heures près.

Le débordement annoncé en `TMP §2` se vérifie ici : un `int` s'arrête à 2 147 483 647, soit environ 68 ans de secondes. Au-delà, il faut un `long`.

TMP §6 — L'arithmétique du calendrier

Ajouter un mois n'est pas ajouter trente jours. Les deux dernières lignes donnent-elles la même date ? Décidez-le avant d'exécuter.

```
LocalDate d = LocalDate.of(2027, 1, 31);

d.plusDays(30);
d.plusMonths(1);
d.plusMonths(1).plusMonths(1);   // ?
d.plusMonths(2);                 // ?
```

> **Exercices** - `TMP.4` L'arithmétique des mois
>
> Deux méthodes ; la seconde compare l'ajout d'un coup et l'ajout un par un, et retourne un booléen. L'énoncé donne les quatre cas à observer, en année ordinaire et en année bissextile.

Java rabote au dernier jour du mois

Le 31 janvier 2027 plus un mois donne le 28 février 2027 : le 31 février n'existe pas, et Java rabote au dernier jour disponible du mois d'arrivée. Une fois raboté, le 31 est perdu. La seconde addition repart du 28, et non du 31 : ajouter deux fois un mois n'est donc pas ajouter deux mois.

Quelle autre réponse aurait été possible ? Le 3 mars ? Une exception ? Chacune a ses inconvénients.

> **À retenir**
>
> Pour une échéance, décidez si vous comptez en jours ou en mois, et dites-le.

TMP §7 — `Period` ou `Duration`

Deux classes mesurent un écart. Elles ne mesurent pas dans la même unité de pensée, et les confondre produit du code qui ne compile pas, ce qui, pour une fois, est une bonne nouvelle.

`Period`, les unités du calendrier

```
Period p = Period.between(naissance, aujourdhui);
p.getYears();    // 36
p.getMonths();   // 8
p.getDays();     // 2
```

La classe `Period` décompose l'écart en années, mois et jours, exactement comme on annonce un âge. Elle travaille sur des `LocalDate`. Ces unités n'ont pas de longueur fixe : un mois vaut 28, 29, 30 ou 31 jours. Un `Period` ne mémorise donc pas une quantité de temps, mais une recette d'ajustement du calendrier.

C'est la réponse promise au module `POO`. La méthode `getAge()` ne connaissait que l'année de naissance : une personne née en décembre était déclarée plus âgée qu'elle ne l'est, dès le mois de janvier. Avec la date de naissance complète, `Period.between(naissance, LocalDate.now()).getYears()` donne l'âge exact.

`Duration`, les unités de l'horloge

```
Duration d = Duration.between(debut, fin);
d.toHours();     // 3
d.toMinutes();   // 215
```

La classe `Duration` compte en heures, minutes et secondes, des unités qui ne varient jamais. Elle travaille sur des `LocalDateTime` (ou des `LocalTime`).

La règle tient en une ligne : `Period` pour des dates, `Duration` pour des heures. Le compilateur la fait respecter, puisque chacune refuse le type de l'autre.

> **Exercices** - `TMP.5 Period ou Duration`
>
> Trois méthodes. La troisième est celle qui compte : écrire naïvement `toHours() + "h" + toMinutes()` ne donne pas ce qu'on attend, et la Javadoc de `Duration` contient la méthode qui règle le problème. C'est exactement la démarche montrée en démonstration.

Le total et le reste

`toHours() + "h" + toMinutes()` donne `3h215` : `toMinutes()` retourne le total des minutes, 215. C'est `toMinutesPart()` qui retourne le reste après extraction des heures, 35. Toute la famille `toXxx` retourne un total ; toute la famille `toXxxPart` retourne un reste.

Au-delà de `Duration`, retenez la leçon de méthode : deux noms voisins dans la même liste de complétion peuvent faire des choses différentes, et rien dans la liste ne le dit. La complétion montre les noms ; la Javadoc dit ce qu'ils font.

TMP §8 — Une date et son texte

Une date affichée au format ISO se lit mal, et une date saisie par un utilisateur n'est jamais au format ISO. Cette section fait le passage dans les deux sens.

De la date vers le texte

```
DateTimeFormatter fr = DateTimeFormatter.ofPattern("dd/MM/yyyy");
String texte = date.format(fr);    // "19/01/2027"
```

Le motif est une chaîne où chaque lettre désigne un champ, et le nombre de lettres la largeur souhaitée : `dd` le jour, `MM` le mois, `yyyy` l'année.

> **Attention**
>
> `mm` est la minute, `MM` le mois. La confusion ne provoque aucune erreur : elle donne simplement une date fausse.

Du texte vers la date

Le même formateur sert dans les deux sens : `date.format(fr)` transforme une date en texte, et `LocalDate.parse(texte, fr)` fait l'inverse, en lisant le texte selon le même motif. Cette lecture s'appelle l'analyse (*parsing* en anglais).

```
DateTimeFormatter fr = DateTimeFormatter.ofPattern("dd/MM/yyyy");
LocalDate d = LocalDate.parse("19/01/2027", fr);
LocalDate e = LocalDate.parse("19-01-2027", fr);   // exception
```

Un texte qui ne correspond pas au motif lève une `DateTimeParseException` : le programme s'arrête et affiche une trace. Traiter proprement ce genre d'incident est le sujet du module `EXC` ; pour l'instant, contentez-vous de constater ce qui se passe.

> **Exercices** - `TMP.6` Formater et analyser une date
>
> Trois méthodes : formater, analyser, et analyser en refusant les dates impossibles. Analysez `"31/02/2027"` et regardez ce que vous obtenez avant de lire la suite.

L'indulgence silencieuse

L'appel `parse("31/02/2027")` retourne `2027-02-28`, sans rien signaler. Par défaut, le formateur est indulgent : il rabote. Une saisie absurde vient de devenir une date parfaitement plausible, et plus rien dans le programme ne permettra de s'en apercevoir. C'est exactement ce que proscrit `CC-ERR.1` du mémento Clean Code : mieux vaut une exception qu'une valeur arrangée.

Le mode strict refuse les dates impossibles. Il s'obtient avec `withResolverStyle(ResolverStyle.STRICT)`, et réclame alors le motif `uuuu` au lieu de `yyyy` pour l'année. Le mode indulgent n'est pas un défaut du JDK : il existe des contextes où ajuster vaut mieux que refuser. Mais pour un formulaire d'inscription, le mode strict est le bon choix, et il faut l'écrire explicitement.

Ce qu'il faut retenir

- Trois classes : `LocalDate`, `LocalTime`, `LocalDateTime`.
- Une date est immuable : les méthodes retournent une nouvelle date, elles ne modifient pas.
- Une date n'est pas un instant : on ne compte pas de secondes entre deux cases du calendrier.
- `Period` pour des dates, `Duration` pour des heures.
- `String.format` se teste, `printf` non.
- Précisez la locale, et choisissez le mode strict.

La plus transversale de ces règles est la deuxième : l'immuabilité dépasse largement les dates. Vous la retrouverez sur `String`, sur les `record` du module `ENU`, et dans le mémento Clean Code.

Module TAB

TAB — Tableaux, boucles et opérateur ternaire

Ranger des valeurs, les parcourir, et choisir en une expression

Vous avez déjà manipulé des tableaux, en C# et au cours d'introduction à la programmation. Ce module ne les réapprend pas : il s'arrête sur ce qui surprend encore, en particulier le fait qu'un tableau soit une référence, puis sur les deux façons de le parcourir. Il présente ensuite l'opérateur ternaire, et termine sur des objets qui collaborent et se rangent dans un tableau.

À la fin de ce module, vous saurez

- déclarer un tableau, et expliquer pourquoi sa taille ne change plus ;
- prévoir ce que devient un tableau passé à une méthode ;
- choisir entre la boucle indexée et la boucle `for-each` ;
- construire et parcourir un tableau de tableaux, même quand ses lignes ont des longueurs différentes ;
- employer l'opérateur ternaire là où il aide, et y renoncer là où il nuit ;
- faire collaborer plusieurs classes, dont l'une range des objets dans un tableau.

TAB §1 — Un tableau, c'est une taille fixe

Déclarer un tableau

Un tableau se déclare de deux façons : avec `new` et une taille, ou avec la liste de ses valeurs entre accolades.

```
int[]    nombres = new int[5];            // 5 cases, toutes a 0
String[] mots    = { "Hello", "world" };  // 2 cases, deja remplies
```

Dans le premier cas, le tableau naît rempli de valeurs par défaut : des zéros pour des `int`, `null` pour des objets. Dans les deux cas, sa taille est fixée une fois pour toutes, au moment de sa création. C'est la limite du tableau, et c'est aussi ce qui le rend simple et rapide. Quand cette limite devient gênante, il faut une collection, qui grandit selon les besoins : c'est le sujet du module `COL`.

> **Démonstration** - `Demo_TAB` : cinq pièges, et pourquoi ils existent
>
> La démonstration ouvre le module, avant toute théorie. Elle ne présente pas les tableaux : elle tombe dans leurs pièges, un par un, et regarde ce que le langage répond. Le point à retenir tient en une phrase : un tableau n'est pas une valeur, c'est une référence. Quatre des cinq pièges en découlent.

Ce que la démonstration a montré

- **Un tableau de `int` naît à zéro, un tableau d'objets à `null`.** Accéder à une case `null` comme si elle contenait un objet lève une `NullPointerException`.
- **`System.out.println(tableau)` affiche une adresse, pas des valeurs.** Pour voir le contenu, il faut `Arrays.toString(tableau)`, de la classe `java.util.Arrays`.
- **`tableau.length` est un champ, `texte.length()` une méthode.** Il n'y a rien à comprendre : c'est une incohérence du langage, pas une règle. Autant le savoir plutôt que de chercher une justification.
- **Passer un tableau à une méthode, c'est passer son adresse.** C'est le point le plus important.

Une méthode qui reçoit un tableau ne reçoit pas une copie : elle reçoit l'adresse du vôtre, comme pour tout objet (voir `POO §6`). Elle peut donc le modifier sans rien retourner, et la modification se voit chez l'appelant. Pour travailler sur une vraie copie, il faut la demander, avec `Arrays.copyOf`. Le mémento technique décrit où vivent le tableau et la variable qui le désigne (`TEC-EXE.2` et `TEC-EXE.3`).

Un tableau de tableaux

Un tableau à deux dimensions n'existe pas vraiment en Java : c'est un tableau dont chaque case contient un autre tableau.

```
int[][] matrice = new int[3][3];   // un tableau DE TABLEAUX

matrice[1]                         // la deuxieme ligne, un int[]
matrice[1][2]                      // la troisieme case de cette ligne
```

Retenez la distinction entre les deux dernières lignes. `matrice[1]` n'est pas un nombre : c'est un tableau, qu'on peut passer à une méthode, mesurer avec `length` ou parcourir. `matrice[1][2]` est un nombre. Cette lecture explique aussi qu'il y ait deux `length` différents : `matrice.length` compte les lignes, `matrice[1].length` compte les cases de la deuxième ligne.

Elle a une conséquence que `TAB §3` exploite : rien n'oblige les lignes à avoir toutes la même longueur.

TAB §2 — Parcourir un tableau

Deux formes de boucle permettent de parcourir un tableau, et chaque parcours demande de choisir entre les deux.

La boucle indexée

La boucle `for` classique fait avancer un compteur, qui sert d'indice. Comme elle dispose de l'indice, elle peut écrire dans le tableau :

```
for (int i = 0; i < tab.length; i++) {
    tab[i] = tab[i] * 2;
}
```

Notez la condition : `i < tab.length`, avec un inférieur strict, parce que les indices vont de zéro à `length - 1`. Un inférieur ou égal provoque une `ArrayIndexOutOfBoundsException`, et c'est probablement l'erreur la plus fréquente de toute la programmation quand on utilise des tableaux.

La boucle for-each

La boucle `for-each` se lit « pour chaque entier `v` contenu dans `tab` » :

```
for (int v : tab) {
    System.out.println(v);
}
```

Elle n'expose pas l'indice, ce qui la rend plus courte, plus lisible, et supprime d'un coup le risque de sortir du tableau. En contrepartie, `v` est une copie de la valeur de la case : lui affecter quelque chose ne modifie pas le tableau. C'est une restriction, pas un défaut, et l'exercice `TAB.2` vous fera essayer pour le constater.

Laquelle choisir

Vous voulez	La boucle
lire seulement	`for-each`
écrire dans le tableau	boucle indexée
connaître la position de chaque élément	boucle indexée

Dans le doute, prenez la `for-each` : c'est celle qui se relit le mieux. La règle tient en une ligne : prenez la boucle qui montre le moins de choses. Si l'indice ne sert à rien, ne l'écrivez pas.

Deux boucles imbriquées

Pour remplir une matrice, la boucle extérieure parcourt les lignes, et la boucle intérieure les cases de chaque ligne :

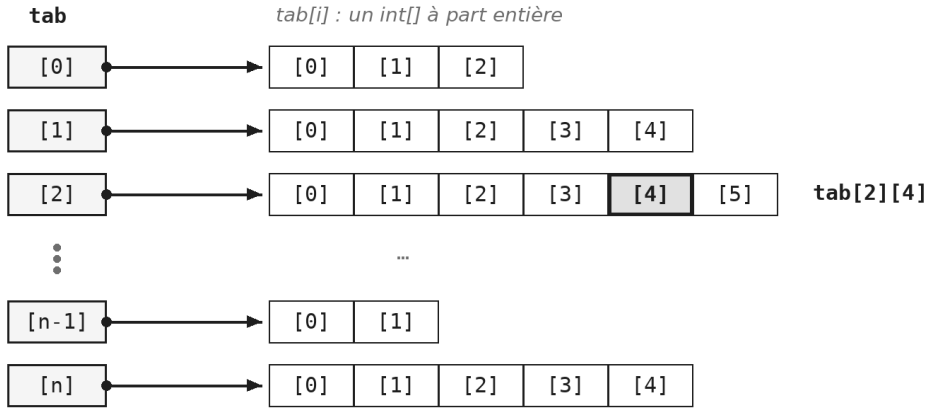
```
int valeur = 1;
for (int ligne = 0; ligne < n; ligne++) {
    for (int col = 0; col < n; col++) {
        matrice[ligne][col] = valeur;
        valeur = valeur + 1;
    }
}
```

Le compteur `valeur` doit survivre au passage d'une ligne à l'autre : il se déclare donc avant la première boucle. Déclaré à l'intérieur d'une boucle, il serait remis à sa valeur initiale à chaque tour. Avant d'écrire deux boucles imbriquées, regardez la portée de chaque variable.

> **Exercices** - `TAB.1` Une matrice carrée
>
> Deux méthodes statiques : `carree` construit la matrice, `enTexte` la met en forme. Aucune des deux n'affiche : c'est la séparation entre le calcul et le dialogue, posée au module `POO`. Le format d'`enTexte` est imposé au caractère près, parce que le test le vérifie.

TAB §3 — Des lignes de tailles différentes

Un tableau de tableaux n'a aucune raison d'être rectangulaire : chaque case de gauche contient un tableau, de la longueur qu'on veut.



Un tableau de tableaux : chaque ligne a sa propre longueur

La colonne de gauche est le tableau `tab` : ses cases ne contiennent pas des nombres, mais chacune une référence vers un autre tableau, représentée par la flèche. Chaque ligne est donc un `int[]` complet, avec sa propre longueur : `tab[0].length` vaut 3, `tab[2].length` vaut 6. Un tableau « rectangulaire » n'est que le cas particulier où ces longueurs sont égales.

Pour lire `tab[2][4]`, on suit d'abord la flèche de la case `[2]`, puis on prend la case `[4]` du tableau d'arrivée.

Déclarer les lignes une par une

Pour un tableau dont les lignes ont des longueurs différentes, la seconde paire de crochets reste vide : on annonce le nombre de lignes, pas leur longueur. Chaque ligne est ensuite créée à sa taille.

```
int[][] triangle = new int[n][];

for (int i = 0; i < n; i++) {
    triangle[i] = new int[i + 1];
}
```

Entre la déclaration et la création d'une ligne, cette ligne vaut `null`. Y accéder donne une `NullPointerException`, et non un tableau vide.

Le parcours d'un tel tableau ne peut plus supposer une largeur constante. Deux boucles `for-each` imbriquées s'en sortent naturellement : la première parcourt des `int[]`, la seconde des `int`, et chaque ligne annonce sa propre longueur.

> **Exercices** - `TAB.2` Le triangle de Pascal
>
> Chaque nombre est la somme des deux qui le surplombent, les bords valent 1, et la ligne d'indice `i` contient `i + 1` nombres. L'affichage se fait avec deux boucles `for-each` imbriquées. Vous pourriez aussi utiliser des boucles indexées, mais c'est inutile pour afficher un contenu (une action qui ne fait que lire les valeurs contenues dans le tableau).

TAB §4 — L'opérateur ternaire

Un opérateur fait en une expression ce qu'un `if` demande en cinq lignes. Cette section montre où il aide, et où il commence à nuire.

La syntaxe

```
// condition ? valeurSiVrai : valeurSiFaux
```

```
// avec un if
public int max(int a, int b) {
    if (a > b) {
        return a;
    } else {
        return b;
    }
}
```

```
// avec l'operateur ternaire
public int max(int a, int b) {
    return a > b ? a : b;
}
```

Le point d'interrogation sépare la condition des deux valeurs, et les deux-points séparent les valeurs entre elles. Lisez-le comme une question : « `a` est-il plus grand que `b` ? Si oui `a`, sinon `b`. » *Ternaire* signifie simplement « à trois opérandes » : c'est le seul opérateur de Java dans ce cas, d'où son nom.

Ce qu'un `if` ne permet pas

Le ternaire retourne une valeur, le `if` ne retourne rien. C'est la différence essentielle, et la seule bonne raison d'employer le ternaire.

```
int plusGrand = a > b ? a : b;      // possible
int plusGrand = if (a > b) a; ...   // n'existe pas
```

Un `if` est une instruction : il fait quelque chose. Le ternaire est une expression : il vaut quelque chose. On peut donc l'écrire partout où une valeur est attendue, à droite d'un `return` ou d'une affectation, ce qu'un `if` ne permet pas. Employer un ternaire uniquement pour raccourcir un `if` qui ne retourne rien, c'est en revanche le détourner de son usage.

Un booléen se retourne tel quel

Une écriture revient souvent dans les copies d'examen :

```
// trop verbeux
if (a > b) {
    return true;
} else {
    return false;
}
```

```
// direct
return a > b;
```

Une comparaison comme `a > b` vaut déjà `true` ou `false` : c'est un booléen, au même titre qu'une variable `boolean`. Le `if` de gauche ne fait que recopier ce booléen, en cinq lignes. Le ternaire n'arrange rien : `return a > b ? true : false;` est la même redondance, en une ligne. Face à l'un comme à l'autre, `return a > b;` dit la même chose, plus directement.

Le code de gauche fonctionne, mais il montre qu'on n'a pas vu qu'une comparaison est une valeur : à l'examen, c'est une faute de style (`CC-MET.9`).

Où s'arrête l'intérêt

Le ternaire s'imbrique : `a ? x : (b ? y : z)`. Rien ne l'interdit, et le compilateur ne protestera jamais. La question n'est pas ce que le langage autorise, mais ce qu'un collègue pourra relire dans six mois. L'exercice vous demande d'en écrire un à trois niveaux : faites-le vraiment, relisez-le, et décidez vous-même si vous le garderiez. C'est une décision d'écriture, pas une règle ; le mémento Clean Code se contente de demander des méthodes courtes et lisibles (`CC-MET.2`).

Exercices - `TAB.3` L'opérateur ternaire

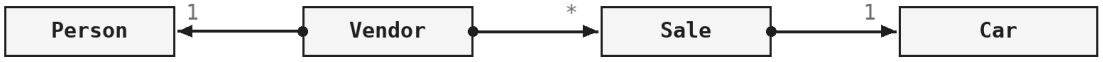
Un petit jeu de dés, fourni et fonctionnel à deux méthodes près. Chacune tient en une seule instruction avec un ternaire. Attention au sens de la comparaison : selon la façon dont vous l'écrivez, l'égalité tombe du côté du premier ou du second joueur, et le test le vérifie. Remarquez aussi que ces deux méthodes ne lancent aucun dé : elles reçoivent les résultats en paramètres. C'est ce qui les rend prévisibles, et donc testables. Vous retrouverez cette idée au module `JUN`.

TAB §5 — Des objets qui se connaissent

Jusqu'ici, les exercices manipulaient une classe à la fois. Le dernier en fait collaborer quatre, et range des objets dans un tableau.

Un objet peut en contenir un autre

Un champ peut avoir pour type une classe, comme il peut être un `int` : vous en écrivez depuis le module `POO`. Ce qui change ici, c'est le nombre de classes qui collaborent.



un vendeur a une personne, des ventes ; une vente porte sur une voiture

Les quatre classes de l'exercice `TAB.4`

Le vendeur est modélisé comme un rôle que joue une personne : un `Vendor` possède une `Person`, sous la forme d'un champ. Il enregistre des ventes (`Sale`), et chaque vente porte sur une voiture (`Car`).

Dans la solution, le champ `personne` est `final` : un vendeur ne change pas de personne. Le tableau des ventes l'est aussi, et pourtant ses cases se remplissent au fil des ventes. `final` fige la référence, pas l'objet désigné : le champ désignera toujours le même tableau, mais rien n'empêche de modifier ce tableau (voir `POO §2` et `POO §6`).

On aurait pu, tout aussi légitimement, écrire qu'un vendeur *est* une personne, ce qui se traduit par de l'héritage, que vous connaissez par le C#. Ce n'est pas faux : c'est un autre modèle. Celui de l'exercice a un avantage : la même personne pourrait jouer d'autres rôles, être cliente par exemple, sans changer de classe. Le module `ABS` compare les deux approches sur cet exemple.

Ne pas stocker ce qui se déduit

Le prix catalogue et la réduction suffisent à retrouver le prix réel d'une vente. En faire un troisième champ créerait une donnée redondante : le jour où la réduction change, plus rien ne garantit que le prix réel suive. C'est exactement le problème du champ `age` rencontré en `POO.3` (voir `POO §3`), et la règle vaut bien au-delà des voitures : ce qui se déduit ne se stocke pas, il se calcule.

Un tableau de taille fixe dans un objet

C'est ici que la taille fixe, annoncée en ouverture du module, devient une contrainte concrète. Le vendeur range ses ventes dans un tableau dimensionné à la construction. Il faut donc :

- décider d'un maximum, puisque le tableau ne grandira pas ;
- retenir soi-même combien de cases sont occupées ;
- refuser une vente quand le tableau est plein : `enregistrer` retourne alors `false` ;
- parcourir les ventes jusqu'au nombre enregistré, et non jusqu'à `ventes.length`.

Cette dernière règle mérite d'être essayée plutôt que lue : parcourir jusqu'à `ventes.length` passe aussi sur les cases vides, qui valent `null`. Écrivez d'abord cette version-là, et regardez ce qui se passe.

Exercices - `TAB.4` Des objets qui se connaissent

Quatre classes, dont trois à écrire entièrement, et un format de texte vérifié au caractère près. Deux questions du fascicule touchent au mémento Clean Code : `Sale` a besoin de la marque et du modèle de la voiture ; vaut-il mieux qu'elle lise ses champs, ou qu'elle les lui demande (`CC-OBJ.1`) ? Et faut-il un accesseur qui retourne le tableau de ventes, sachant ce que l'appelant pourrait alors en faire (`CC-OBJ.2`) ?

Ce qu'il faut retenir

- Un tableau a une taille fixe, décidée au `new`.
- Un tableau est une référence : le passer à une méthode, c'est passer son adresse.
- `matrice[1]` est une ligne, `matrice[1][2]` est un nombre.
- La `for-each` pour lire, la boucle indexée pour écrire.
- Le ternaire retourne une valeur ; un `if`, non.
- Ce qui se déduit ne se stocke pas.

La deuxième règle est celle qui coûte le plus cher quand on l'oublie : une méthode qui modifie le tableau qu'elle reçoit le modifie aussi pour l'appelant.

Module ENU

ENU — Énumérations et records

Des valeurs qui se comptent, et des données qui ne changent pas

Ce module ajoute deux formes de type à celles que vous connaissez. L'énumération (`enum`) sert quand les valeurs possibles se comptent sur les doigts : une couleur de feu, un jour de la semaine, un résultat de partie. Le `record` sert quand une donnée n'est qu'une donnée, sans comportement propre et sans état qui change.

Les deux ont un point commun : ils font dire au compilateur ce qu'un commentaire disait jusqu'ici. Une énumération interdit la quatrième couleur de feu ; un record interdit la modification après construction. Dans les deux cas, l'erreur devient impossible à écrire, plutôt que pénible à retrouver.

À la fin de ce module, vous saurez

- déclarer une énumération, et utiliser les méthodes que toute énumération possède ;
- tirer parti de l'ordre des constantes, en sachant ce qu'il engage ;
- comparer des constantes et aiguiller avec les trois formes du `switch` ;
- donner des données et des méthodes à une énumération ;
- tirer un nombre au hasard et assembler un texte avec un séparateur ;
- déclarer un record, et reconnaître quand il ne convient pas.

ENU §1 — Des valeurs qui se comptent

Trois couleurs, pas une de plus

Un feu de signalisation a exactement trois couleurs. Avant les énumérations, on les représentait par des constantes entières :

```
static final int VERT   = 0;
static final int ORANGE = 1;
static final int ROUGE  = 2;

feu.setCouleur(7);   // compile sans broncher
```

Cela fonctionne, au sens où le programme tourne, et trois constantes bien nommées valent déjà mieux que les nombres 0, 1 et 2 écrits en clair (`CC-NOM.7`). Le problème est ailleurs : le type de la variable est `int`, et un `int` accepte toutes les valeurs d'un `int`. La ligne qui demande la couleur 7 compile, et casse un jour à l'exécution. Le même défaut existe avec un `String` : rien ne distingue « VERT » de « vert », de « verd » ou d'un espace de trop.

L'énumération déplace cette vérification : ce qui était une convention à respecter devient une règle que le compilateur applique.

D'autres types que la classe

Jusqu'ici, écrire du Java voulait dire écrire des classes. Il existe d'autres formes de type, et vous en verrez cinq cette année :

Type	Ce qu'il apporte	Où
`enum`	une liste de valeurs fermée	ce module
`record`	une donnée immuable, en une ligne	ce module
`abstract class`	un comportement partagé, incomplet	module `ABS`
`interface`	un contrat sans implémentation	module `ITF`
classe anonyme	une classe écrite sur place, sans nom	module `LAM`

Il n'y a rien à retenir de ce tableau pour l'instant, sinon que le paysage est plus large que les classes, et qu'il se découvrira morceau par morceau.

ENU §2 — Déclarer une énumération

La syntaxe

Le mot-clé `enum` remplace `class`, et la liste des valeurs possibles s'écrit entre accolades :

```
public enum CouleurDeFeu { VERT, ORANGE, ROUGE }
```

```
CouleurDeFeu couleur = CouleurDeFeu.VERT;
couleur = CouleurDeFeu.ROUGE;   // d'accord
couleur = 7;                    // refus a la compilation
```

Ces valeurs sont des constantes ; comme toute constante en Java, elles s'écrivent en majuscules. Une variable déclarée `CouleurDeFeu` ne peut recevoir qu'une des trois valeurs, ou `null`. La dernière ligne ne compile pas, et c'est exactement le service rendu : l'erreur se voit au moment où elle s'écrit, pas six mois plus tard.

Une énumération est un type comme un autre. Elle sert de type de champ, de type de paramètre et de type de retour.

Ce que toute énumération sait faire

Cinq méthodes existent sur toute énumération, sans que rien soit écrit : elles viennent de la classe `java.lang.Enum`, dont toute énumération hérite.

Appel	Ce qu'il retourne	Exemple
`name()`	le nom de la constante	`"ROUGE"`
`ordinal()`	sa position, à partir de o	`2`
`values()`	un tableau de toutes les constantes	`[VERT, ORANGE, ROUGE]`
`valueOf("VERT")`	la constante qui porte ce nom	`VERT`
`compareTo(autre)`	l'écart entre les deux positions	`-2`

- `name()` retourne le nom écrit dans la déclaration ; `toString()` fait la même chose tant qu'on ne la redéfinit pas.
- `values()` est statique : elle s'appelle sur le type, pas sur une valeur, et retourne les constantes dans l'ordre de déclaration.
- `valueOf` fait le chemin inverse, du texte vers la constante, et lève une exception si le nom est inconnu.
- `compareTo` retourne un résultat négatif si la première constante est déclarée avant la seconde. C'est ce qui rend une énumération triable sans effort.

L'ordre de déclaration compte

Les méthodes `values`, `ordinal` et `compareTo` suivent l'ordre dans lequel les constantes sont écrites. Cet ordre devient donc une donnée du programme :

```
CouleurDeFeu.VERT.ordinal();      // 0
CouleurDeFeu.ROUGE.ordinal();     // 2
CouleurDeFeu.values().length;     // 3
```

Deux conséquences pratiques. D'abord, cet ordre se choisit : pour un feu de signalisation, c'est celui du cycle, et non l'ordre alphabétique. Ensuite, il ne se modifie pas à la légère : insérer une constante au milieu change le résultat d'`ordinal` pour toutes celles qui suivent. C'est sans effet si le code ne s'appuie que sur les noms, et lourd de conséquences s'il s'appuie sur les positions.

Pour la même raison, évitez d'enregistrer un ordinal dans un fichier ou une base de données. Le nom, retourné par `name()`, est stable ; la position ne l'est pas. Cela ressortira au module `FIC`.

ENU §3 — L'ordre porte l'information

Une énumération ne remplace pas seulement des constantes : elle peut contenir une partie de la logique, et faire disparaître des conditions.

> **Démonstration** - `Demo_ENU` : le cycle sans conditions
>
> La démonstration part d'un lecteur de musique dont le bouton enchaîne trois modes de lecture, avec trois conditions. Un quatrième mode est ajouté à l'énumération, et le lecteur ne l'atteint jamais : le compilateur n'a rien dit, et le comportement est faux. La méthode est ensuite réécrite en deux lignes, à partir de `values` et d'`ordinal` ; le cinquième mode fonctionne alors sans que rien d'autre soit touché.

Passer à la valeur suivante

Le tableau retourné par `values()` contient les constantes dans l'ordre de déclaration, donc dans l'ordre du cycle :

```
CouleurDeFeu[] cycle = CouleurDeFeu.values();

int suivante = (couleur.ordinal() + 1) % cycle.length;
couleur = cycle[suivante];
```

Ajouter un à la position courante donne la suivante, et le reste de la division par la longueur du tableau ramène à zéro quand on dépasse la fin. Le nombre de couleurs n'apparaît nulle part : c'est `cycle.length` qui le donne, et il suit la déclaration quoi qu'il arrive.

> **Attention**
>
> Les parenthèses autour de l'addition sont indispensables : l'opérateur `%` a priorité sur `+`. Sans elles, le modulo s'applique d'abord au 1, et le feu reste bloqué sur sa première couleur. Le code compile sans le moindre avertissement : ce genre de faute est le plus coûteux qui soit, parce que rien ne le signale. Un test l'attrape immédiatement.

Ce que ce détour fait gagner

Le gain n'est pas la longueur du code : deux lignes au lieu de six, ce serait une économie sans intérêt. Le gain est que ce qui change souvent, la liste des valeurs, ne se change plus qu'à un seul endroit. Une quatrième couleur s'ajoute sans qu'une ligne change ailleurs, et il n'y a plus de condition à lire, donc plus d'endroit où se tromper. C'est le critère de `CC-CLS.4` : une conception se juge à ce qu'elle exige quand le besoin bouge.

La contrepartie est réelle, et il faut la nommer. En s'appuyant sur l'ordre, le code dépend d'une information qui ne se lit pas dans la méthode, mais dans la déclaration de l'énumération. Quelqu'un qui réordonne les constantes par souci de propreté casse le cycle sans le savoir. La Javadoc de l'énumération doit donc dire que l'ordre est celui du cycle et qu'il ne se modifie pas : un commentaire utile dit ce que le code ne peut pas dire (`CC-COM.3`).

> **Exercices** - `ENU.1` Un feu de signalisation
>
> L'exercice demande explicitement les deux versions de `changeDeCouleur` : celle avec des conditions, et celle sans. La comparaison ne vaut que si les deux sont écrites. Il ajoute une question de conception : le constructeur doit placer le feu dans le même état que `desactive`, et il vaut mieux appeler la méthode que recopier ses instructions.

ENU §4 — Comparer et aiguiller : `switch`

Cette section installe le `switch`, dans ses trois formes. Il resservira tout au long de l'année, et une énumération est le meilleur endroit pour l'apprendre, parce que c'est là que le compilateur peut vérifier qu'aucun cas n'a été oublié.

Comparer deux valeurs

```
if (couleur == CouleurDeFeu.ROUGE) {
    System.out.println("Stop");
}
```

Sur les énumérations, `==` fait ce qu'on attend. Chaque constante est créée une seule fois, au chargement de la classe, et toute variable qui vaut `ROUGE` désigne exactement le même objet : comparer les références revient donc à comparer les valeurs.

C'est une exception notable. Sur les `String` et sur les objets en général, `==` ne compare pas les valeurs mais compare les références, ce qui peut réserver des surprises. Sur les énumérations, cet opérateur est à la fois correct et préférable à `equals`, parce qu'il ne lève pas d'exception si la variable vaut `null`.

La forme classique

```
switch (jour) {
    case SAMEDI:
    case DIMANCHE:
        System.out.println("Repos");
        break;
    default:
        System.out.println("Travail");
}
```

C'est la forme héritée du C, et celle que vous rencontrerez dans le code existant. Le `switch` évalue une expression, saute à l'étiquette correspondante, et exécute tout ce qui suit jusqu'au premier `break`. Sans `break`, l'exécution continue sur le cas suivant : c'est voulu ici, pour traiter le samedi et le dimanche ensemble, mais c'est bien plus souvent accidentel. L'oubli du `break` est la faute la plus fréquente de cette forme.

Sur une énumération, les étiquettes s'écrivent sans le nom du type : `SAMEDI` et non `Jour.SAMEDI`, parce que le type est déjà connu par l'expression du `switch` (le paramètre `jour` passé au `switch` est de type `Jour`).

La forme à flèches

```
switch (jour) {
    case SAMEDI, DIMANCHE -> System.out.println("Repos");
    default               -> System.out.println("Travail");
}
```

Cette forme est arrivée avec Java 14, et c'est celle à écrire aujourd'hui. Avec la flèche, un seul cas s'exécute, jamais celui qui suit : il n'y a plus de `break` à oublier. Les étiquettes qui partagent un traitement se regroupent sur une même ligne, séparées par des virgules. Quand une branche demande plusieurs instructions, on ouvre des accolades après la flèche.

Un `switch` qui retourne une valeur

Un `switch` peut produire une valeur au lieu d'exécuter des instructions. Il s'utilise alors comme n'importe quelle expression, et se termine par un point-virgule :

```
String message = switch (jour) {
    case SAMEDI, DIMANCHE -> "Repos";
    case VENDREDI         -> "Bientôt";
    default               -> "Au boulot !";
};
```

Chaque branche fournit la valeur, elle ne l'affiche pas. Cette forme mérite d'être préférée dès qu'il s'agit de choisir une valeur : la variable `message` reçoit sa valeur une fois pour toutes, ce qui élimine le cas où une branche oublierait de l'affecter. Quand une branche a besoin de plusieurs lignes, elle s'écrit entre accolades et fournit sa valeur avec le mot-clé `yield`.

Tous les cas, sans oubli

Sur une énumération, le compilateur connaît la liste complète des cas. Il peut donc vérifier que toutes les constantes sont traitées :

```
String consigne = switch (couleur) {
    case VERT   -> "Passez";
    case ORANGE -> "Ralentissez";
    case ROUGE  -> "Arretez";
};
```

Cet exemple n'a pas de `default`, et il compile, parce que les trois couleurs y sont. Le jour où une quatrième couleur s'ajoute à l'énumération, ce même code cesse de compiler, et le message désigne l'endroit exact à compléter. C'est l'inverse de la démonstration, où l'ajout d'un mode passait inaperçu. Écrire un `default` qui ne sert qu'à faire taire le compilateur revient à renoncer à cette vérification : quand tous les cas sont couverts, mieux vaut s'en passer.

Exercices - `ENU.2` Une température, deux unités

L'énumération sert ici de paramètre de méthode, son usage le plus courant. La vraie question est ailleurs : combien de valeurs l'objet doit-il stocker ? En garder deux, une par unité, c'est accepter qu'elles se contredisent un jour. La conversion se fait donc au moment où on la demande, à partir d'une seule valeur stockée. Pour nommer les méthodes, voir `CC-MET.4`.

ENU §5 — Une `enum` est une classe

Jusqu'ici, l'énumération n'était qu'une liste de noms. Elle peut aussi contenir des données et des méthodes, ce qui déplace la question de la conception : où placer l'information ?

Une énumération qui contient des données

```
public enum Discount {

    BASIC(0.05), FIDELITY(0.075), SPECIAL(0.10);

    private final double taux;

    Discount(double taux) {
        this.taux = taux;
    }

    public double appliquer(double prix) {
        return prix * (1 - taux);
    }
}
```

Une énumération est une classe, et le reste jusqu'au bout : elle accepte des champs, un constructeur et des méthodes. Chaque constante passe ses arguments au constructeur, et devient un objet construit une fois pour toutes, avec ses propres valeurs.

L'intérêt se voit à l'usage. Le taux d'une remise appartient à la remise, et non à la classe qui calcule un prix. Écrire `SPECIAL.appliquer(prix)` place le calcul là où vit l'information, et évite qu'une table de correspondance traîne ailleurs dans le programme.

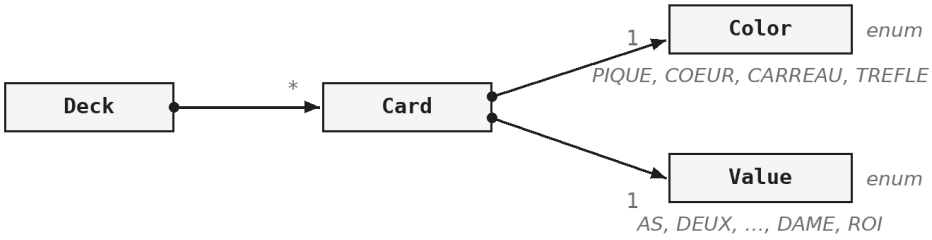
Trois détails de syntaxe

- **Le point-virgule après la dernière constante** devient obligatoire dès qu'un champ, un constructeur ou une méthode suit la liste. Son oubli est la faute de frappe la plus fréquente sur ce sujet.
- **Le constructeur est implicitement privé :** un `new` sur une énumération est impossible. Les seules instances possibles sont celles de la liste, créées au chargement de la classe ; c'est ce qui garantit qu'il n'existe qu'un objet par constante, et donc que `==` fonctionne.
- **Les champs sont déclarés `final`**, parce qu'une constante dont l'état change n'est plus une constante. Rien ne l'impose techniquement, mais s'en écarter n'a pas de bonne raison.

Un jeu de cartes, avec des énumérations

Le jeu de cartes est l'exemple type où les énumérations s'imposent, parce que rien n'y change jamais : il n'y aura pas de cinquième couleur, et le roi restera la plus haute figure. Avant de coder, il faut distinguer ce qui est fixe de ce qui se combine :

- les 4 couleurs et les 13 valeurs ne changeront jamais : ce sont deux énumérations, `Color` et `Value` ;
- chaque constante connaît son libellé : `PIQUE` porte « Pique », `AS` porte « As » ;
- une carte associe une couleur et une valeur : c'est une classe `Card`, à deux champs ;
- le jeu, `Deck`, rassemble les 52 cartes, sans que le nombre 52 apparaisse dans le code.



Le modèle du jeu de cartes

Une seule énumération de 52 constantes, `AS_DE_PIQUE`, `DEUX_DE_PIQUE` et ainsi de suite, fonctionnerait. Mais elle ne dirait rien de la structure du jeu : rien n'y indiquerait que `AS_DE_PIQUE` et `AS_DE_COEUR` ont la même valeur. Deux énumérations de 4 et 13 constantes portent cette structure, et la carte n'est plus que leur rencontre.

Toutes les combinaisons

Construire un jeu complet, c'est produire toutes les combinaisons des deux énumérations. Deux boucles `for-each` imbriquées sur `values()` y suffisent :

```
Card[] cartes = new Card[Color.values().length * Value.values().length];
int i = 0;
for (Color couleur : Color.values()) {
    for (Value valeur : Value.values()) {
        cartes[i] = new Card(couleur, valeur);
        i++;
    }
}
```

Le nombre 52 n'apparaît nulle part : il est le produit des tailles des deux énumérations (4 × 13), et se recalcule tout seul. Ajouter une couleur ou une valeur suffit à changer la taille du jeu, sans toucher à cette boucle. C'est le même raisonnement qu'en `ENU §3`, appliqué à deux dimensions.

ENU §6 — Un détour par deux outils

Deux classes de la bibliothèque standard, sans rapport avec les énumérations, mais dont l'exercice suivant a besoin.

Outil 1 : `Random` pour tirer un nombre au hasard

La classe `java.util.Random` produit des nombres au hasard[1]. Sa méthode `nextInt(n)` retourne un entier de `0` inclus à `n` exclu :

```
Random hasard = new Random();

int de = hasard.nextInt(6) + 1;            // 1 a 6
int rang = hasard.nextInt(cartes.length);  // 0 a cartes.length - 1
```

La borne exclue tombe bien : c'est exactement l'intervalle des indices d'un tableau de cette taille. Pour simuler un dé, il faut donc ajouter un au résultat.

Un seul objet `Random` suffit pour tout un programme : en créer un nouveau à chaque tirage est inutile. Un seul champ, construit une fois, règle la question.

Pour mélanger un tableau, l'exercice demande de parcourir les cartes de la fin vers le début, et d'échanger chacune avec une carte tirée au hasard parmi elle-même et celles qui la précèdent, c'est-à-dire avec l'indice

[1] En réalité, `Random` est un générateur *pseudo-aléatoire* : il calcule chaque nombre à partir du précédent, par une formule fixe. La suite a l'air due au hasard, mais elle est entièrement déterminée par sa valeur de départ, la *graine* : `new Random(42)` produit toujours la même suite, ce qui est précieux pour reproduire un test. Pour tout ce qui touche à la sécurité (mots de passe, jetons), on utilise `java.security.SecureRandom`, conçue pour être imprévisible.

`hasard.nextInt(i + 1)`. Tirer dans tout le paquet mélangerait aussi, mais pas équitablement : certaines dispositions sortiraient plus souvent que d'autres. Ce mélange porte un nom, l'algorithme de Fisher-Yates.

Outil 2 : `StringJoiner` pour assembler un texte avec un séparateur

Afficher une liste séparée par des virgules est un exercice classique, avec toujours le même piège : la virgule en trop à la fin. On s'en tire d'ordinaire par un test sur l'indice, qui encombre la boucle sans rien apporter au sens. La classe `StringJoiner` fait exactement ce travail :

```
StringJoiner joiner = new StringJoiner(", ");
for (Card carte : cartes) {
    joiner.add(carte.toString());
}
System.out.println(joiner);
```

On lui donne le séparateur, on ajoute les éléments, et elle place le séparateur entre eux. Il n'y a plus de condition à lire. Son constructeur accepte aussi un préfixe et un suffixe, pour encadrer la liste de crochets ou de parenthèses.

> **Exercices** - `ENU.3` Un jeu de cartes
>
> L'exercice réunit tout ce que `ENU §5` a montré : deux énumérations qui connaissent un libellé, un constructeur, une méthode, et la construction du jeu par double boucle. L'affichage doit tenir sur une ligne, cartes séparées par des virgules : c'est là que `StringJoiner` sert. Pour documenter l'ordre des constantes, voir `CC-COM.3`.

ENU §7 — Les records

Changement de sujet. Le `record` ne remplace pas l'énumération : il répond à un autre besoin, transporter une donnée, sans comportement et sans état qui change.

Une donnée, et rien d'autre

Le mot-clé `record` déclare une classe faite pour contenir des données. Ses *composants* se déclarent entre parenthèses, comme des paramètres :

```
public record Point(int x, int y) { }
```

```
Point origine = new Point(0, 0);
System.out.println(origine.x());   // 0
```

Le corps entre accolades reste vide dans le cas le plus simple, et il est fréquent qu'il le reste. L'accesseur porte le nom du composant, sans préfixe : on écrit `origine.x()`, et non `origine.getX()`. C'est déroutant les premières fois, et c'est la convention.

Ce que le compilateur écrit

À partir de cette seule ligne, le compilateur écrit :

- le constructeur, avec les composants dans l'ordre déclaré ;
- un accesseur par composant, nommé `x()` et non `getX()` ;
- les méthodes `toString`, `equals` et `hashCode`.

```
Point a = new Point(1, 2);
Point b = new Point(1, 2);

a.toString();   // "Point[x=1, y=2]"
a.equals(b);    // true
```

Une ligne de déclaration remplace une trentaine de lignes écrites à la main. La méthode `equals` mérite qu'on s'y arrête : deux objets `Point` construits séparément avec les mêmes valeurs sont égaux, parce que le record compare ses composants. Ce n'est pas le comportement d'une classe ordinaire, où `equals` compare les références tant qu'on ne la redéfinit pas. La question reviendra au module `COL`.

Un record ne change pas

- Un record est `final` : aucune classe n'en hérite.
- Ses composants sont `final` : il n'existe aucun setter.
- Un record construit reste tel quel jusqu'à sa disparition.

L'immuabilité n'est pas un effet de bord, c'est le principe. Un record modélise une valeur, et une valeur ne se modifie pas : on en fabrique une autre. Changer l'abscisse d'un point revient à construire un nouveau point. C'est ce qui rend l'objet sûr à partager, puisque personne ne peut le modifier dans le dos de celui qui l'a créé (`CC-OBJ.2`).

Ajouter une méthode

Le corps du record accueille des méthodes, qui accèdent directement aux composants :

```
public record Rectangle(int largeur, int hauteur) {

    public int aire() {
        return largeur * hauteur;
    }
}
```

La règle est la même que pour une classe ordinaire : ce qui se calcule ne se stocke pas. Ajouter un composant `aire` obligerait à le maintenir cohérent avec la largeur et la hauteur, et ouvrirait la porte à la contradiction.

Valider à la construction

Rien n'empêche, par défaut, de construire un rectangle de largeur nulle. Le *constructeur compact* est l'endroit prévu pour l'interdire :

```
public record Rectangle(int largeur, int hauteur) {

    public Rectangle {
        if (largeur <= 0 || hauteur <= 0) {
            throw new IllegalArgumentException("dimensions positives");
        }
    }
}
```

Il porte le nom du record, sans liste de paramètres ni parenthèses, et s'exécute avant l'affectation des composants. L'instruction `throw` interrompt la construction et signale l'erreur à l'appelant : l'objet n'est pas créé. Le mécanisme complet des exceptions est le sujet du module `EXC`.

Quand le record ne convient pas

Le record est excellent là où il convient, et inadapté partout ailleurs. Le critère est simple : un objet qui change d'état, ou qui porte une logique métier, n'est pas un record.

Ce que vous modélisez	La forme
une adresse, un point, un résumé de partie	un record
un compte dont le solde change	une classe
un feu qui change de couleur	une classe
un objet dont d'autres doivent hériter	une classe

Exercices - `ENU.4` Un record d'étudiant, puis `G.6` à `G.9` (fil rouge GameHub)

L'en-tête du record est fourni : votre part se limite à la méthode `passe`, et c'est tout l'intérêt de l'exercice. Plusieurs tests sont verts avant que vous ayez écrit une ligne : ce n'est pas une panne, c'est le compilateur qui a fait le travail. Pour la conception, voir `CC-OBJ.2`. Le fil rouge applique ensuite les deux notions du module à GameHub : les énumérations `TypeJeu` et `ResultatPartie`, puis le record `ResumePartie`.

Ce qu'il faut retenir

- Une énumération fait vérifier par le compilateur une liste de valeurs fermée.
- L'ordre de déclaration est une donnée : `values()`, `ordinal()` et `compareTo` le suivent.
- `==` compare deux constantes en toute sécurité.
- Un `switch` qui couvre toutes les constantes se passe de `default`, et le compilateur signale le cas oublié.
- Une énumération est une classe : elle peut contenir des données et des méthodes.
- Un record est une donnée immuable ; le compilateur écrit son constructeur, ses accesseurs, `toString`, `equals` et `hashCode`.

Les deux types de ce module ont le même effet : une erreur qui ne se voyait qu'à l'exécution devient une erreur de compilation.

Module ABS

ABS — Héritage et classes abstraites

Ce qui est commun monte, ce qui varie descend

Ce module répond à une question de conception : que faire quand plusieurs classes font la même chose, à un détail près ? La réponse tient en une phrase : ce qui est commun monte, ce qui varie descend.

L'outil pour y parvenir est la classe abstraite, et le mécanisme qui la rend utile est le polymorphisme. Les deux ont été annoncés au module `POO`, avec les quatre principes de la programmation orientée objet. C'est aussi la matière de la quatrième question de l'examen, et l'endroit où se concentre l'erreur la plus fréquente : recopier la même méthode dans trois sous-classes.

À la fin de ce module, vous saurez

- écrire une sous-classe en Java, appeler le constructeur de sa mère et redéfinir une méthode ;
- déclarer une classe abstraite et une méthode abstraite, et dire à quoi elles servent ;
- décider, méthode par méthode, ce qui monte dans la classe mère et ce qui descend ;
- traiter des objets de classes différentes à travers leur type commun, sans test de type ;
- écrire un traitement complet dans la classe mère, et ne laisser aux sous-classes que les détails ;
- construire une hiérarchie à deux étages, puis la concevoir seul à partir d'un énoncé.

ABS §1 — L'héritage en Java

Vous avez vu l'héritage en C#. Cette section le retrouve en Java, sur des classes ordinaires : la syntaxe, l'appel au constructeur de la classe mère, la redéfinition d'une méthode et la visibilité. Rien d'abstrait avant `ABS §3`.

Le point de départ : un compte bancaire

Les exemples de cette section partent d'une classe ordinaire, un compte bancaire :

```
public class CompteBancaire {

    private double solde;
    protected long numero;

    public CompteBancaire(long numero) {
        this.numero = numero;
    }

    public void deposer(double montant) {
        solde += montant;
    }

    public double getSolde() {
        return solde;
    }
}
```

Un compte a un numéro et un solde. On y dépose de l'argent, et on peut consulter le solde. Rien d'autre : c'est la classe mère des exemples qui suivent. Remarquez déjà le `private` du solde et le `protected` du numéro : la dernière sous-section, « Ce que la fille voit de la mère », y revient.

Une classe qui en reprend une autre

Le mot-clé `extends` place une classe sous une autre. La sous-classe reçoit les champs et les méthodes de sa classe mère, sans les réécrire, et ajoute ce qui lui est propre :

```
public class CompteEpargne extends CompteBancaire {

    private final double taux;

    public void crediterInterets() {
        deposer(getSolde() * taux);
    }
}
```

L'héritage exprime une relation : un compte d'épargne *est* un compte bancaire. Tout ce que `CompteBancaire` sait faire, `CompteEpargne` le sait aussi, sans qu'une seule ligne soit recopiée. La sous-classe appelle `deposer` et `getSolde` sans les avoir écrites, puisqu'elle en a hérité.

Une classe ne peut étendre qu'une seule classe. C'est une différence importante avec les interfaces, que le module `ITF` aborde.

Appeler le constructeur de la classe mère

Un objet `CompteEpargne` contient aussi la partie `CompteBancaire`, avec ses champs. Quelqu'un doit les initialiser, et ce quelqu'un est le constructeur de la classe mère :

```
public class CompteEpargne extends CompteBancaire {

    public CompteEpargne(long numero, double taux) {
        super(numero);
        this.taux = taux;
    }
}
```

L'appel s'écrit `super(...)`, et doit être la première instruction du constructeur. La classe mère initialise sa part, la classe fille la sienne. Si vous n'écrivez pas cet appel, le compilateur ajoute `super()` tout seul, et se plaint quand la classe mère n'a pas de constructeur sans argument. Si la classe mère a un constructeur sans argument, cela ne posera pas de problème (voir `POO §2`).

Redéfinir une méthode

Une sous-classe peut redéfinir une méthode héritée, en la réécrivant avec la même signature :

```
public class CompteEpargne extends CompteBancaire {

    @Override
    public void deposer(double montant) {
        super.deposer(montant);
        journaliser(montant);
    }
}
```

L'annotation `@Override` demande au compilateur de vérifier qu'il s'agit bien d'une redéfinition. Une faute de frappe dans le nom, un paramètre oublié, et le compilateur refuse. Sans l'annotation, le programme compilerait, créerait une nouvelle méthode à côté de l'ancienne, et appellerait la méthode de la mère, en silence. L'annotation est facultative pour le langage ; elle ne l'est pas dans votre code.

L'appel `super.deposer(montant)` ajoute un comportement sans jeter celui de la mère. Sans lui, la version de la fille remplacerait purement et simplement l'autre.

Ce que la fille voit de la mère

Hériter ne donne pas tous les droits :

```
public class CompteBancaire {

    private double solde;        // invisible dans CompteEpargne
    protected long numero;       // visible dans CompteEpargne
}
```

- **Un champ `private` de la mère reste invisible dans la fille**, même si l'objet le contient bel et bien.
- **Le modificateur `protected`** ouvre le membre aux sous-classes, et aussi au package. C'est le bon choix pour un détail de fonctionnement partagé avec les filles, mais qui ne regarde pas le reste du programme. Le module `PKG` le présente en détail (voir `PKG.5`).
- **Un accesseur `public`** reste une solution, quand l'information regarde tout le monde.

Le réflexe de tout passer en `protected` pour se simplifier la vie coûte cher : chaque champ ouvert est un champ que les sous-classes peuvent corrompre, et qu'on ne peut plus renommer sans impacter les sous-classes qui y font référence.

De C# à Java

Les concepts sont les mêmes, l'écriture change :

En C#	En Java
`class Fille : Mere`	`class Fille extends Mere`
`: base(...)`	`super(...)`, première instruction du constructeur
`override`	`@Override`, facultative mais à toujours écrire
`is`	`instanceof`
`abstract`	`abstract`

Deux différences méritent l'attention. En C#, `override` est obligatoire et fait partie de la déclaration ; en Java, `@Override` est une annotation facultative, ce qui veut dire qu'on peut redéfinir sans la mettre, et donc se

tromper sans être prévenu. En C#, l'appel au constructeur de base s'écrit dans l'en-tête ; en Java, il s'écrit dans le corps, en première instruction.

A une personne, ou est une personne ?

Au module `TAB`, le vendeur de l'exercice `TAB.4` *avait* une `Person` : c'était un champ. L'héritage permet l'autre modèle, où le vendeur *est* une personne :

```
// a une personne : un role
class Vendor {
    private Person personne;
    private Sale[] ventes;
}
```

```
// est une personne : l'heritage
class Vendor extends Person {
    private Sale[] ventes;
}
```

Les deux se défendent. L'héritage est le plus direct : le vendeur reçoit le nom, le prénom et tout ce que la personne sait faire, sans rien redéclarer. Il convient tant que le vendeur restera vendeur, toute sa vie dans le programme.

Le modèle par rôle est plus souple. La même personne peut être vendeur et cliente à la fois, ou cesser d'être vendeur sans disparaître. Avec l'héritage, c'est impossible : un objet ne change jamais de classe, et une classe Java n'a qu'une seule mère.

La question à se poser est donc moins « est-ce vrai dans la vie ? » que « ce rôle peut-il changer, ou se cumuler ? ». Si oui, le champ ; si non, l'héritage.

ABS §2 — Le même code, trois fois

Cette section pose le problème que la classe abstraite résout. Ce problème est la duplication entre classes sœurs, et c'est l'erreur la plus fréquente à l'examen.

Un exemple : prévenir un utilisateur

Une application doit prévenir ses utilisateurs, et elle a plusieurs moyens de le faire : un mail, un SMS, une notification sur le téléphone. Trois canaux, un même geste : envoyer un texte. La première idée, la plus naturelle, est d'écrire une classe par canal :

```
public class NotificationMail {
    public void envoyer(String texte) {
        System.out.println("mail : " + texte);
    }
}
```

`NotificationSms` et `NotificationPush` sont identiques, au mot `mail` près. Rien dans ce code n'est faux : il compile, il fonctionne, et chaque classe est courte et lisible. Le défaut n'apparaît qu'au moment où la demande change.

Deux défauts, dont un invisible

Le premier défaut se voit : la méthode d'envoi est écrite trois fois. Ajoutez une règle (refuser les messages vides, horodater l'envoi, compter les notifications), et il faut l'implémenter dans chaque classe. Le jour où l'une est oubliée, le programme se comporte différemment selon le canal, et personne ne saura dire pourquoi. C'est l'objet de `CC-CLS.3` du mémento Clean Code : la duplication entre sous-classes se traite en faisant remonter ce qui est commun, pas en le recopiant plus soigneusement.

Le second défaut ne se voit pas, et il est plus grave :

```
NotificationMail[] canaux = { mail, sms, push };
// refus : sms et push ne sont pas des NotificationMail
```

Ces trois classes font la même chose, et le compilateur n'en sait rien. Il n'existe aucun type qui les désigne toutes les trois, donc aucun moyen d'écrire une boucle qui les traite ensemble. Le seul type commun est

`Object`, qui ne sait rien envoyer du tout. Ajouter WhatsApp demandera de retoucher tous les endroits qui traitent les canaux un par un.

> **Démonstration** - `Demo_ABS` : laisser l'outil supprimer la duplication
>
> La démonstration part des trois notifications, et joue d'abord les deux tentatives impossibles : le tableau, puis la modification en trois endroits. Elle montre ensuite le geste inverse de la recopie : faire remonter. IntelliJ extrait la classe mère sans qu'une ligne soit tapée ; reste à décider ce qui ne peut pas remonter, le nom du canal, qui devient abstrait. Le tableau compile alors.

Le geste, pas le code

Repérer une duplication est une chose ; la supprimer sans rien casser en est une autre. IntelliJ le fait pour vous, avec deux outils de refactorisation :

- **Extract Superclass** (clic droit sur la classe, puis Refactor, Extract, Superclass) crée la classe mère et y déplace ce que vous désignez ;
- **Pull Members Up** fait remonter une méthode d'une classe existante vers sa mère.

L'intérêt n'est pas le temps gagné, qui est faible. C'est qu'un déplacement fait à la main oublie un `import`, renomme mal un champ ou laisse une copie derrière lui, alors que l'outil met à jour tout ce qui référence ce qu'il déplace (ce qui devient non négligeable dans un projet qui contient plusieurs centaines de classes, voire plus).

ABS §3 — La classe abstraite

Une classe qu'on ne peut pas instancier

La classe mère extraite à la section précédente pose une question : que voudrait dire `new Notification(...)` ? Une notification qui n'est ni un mail, ni un SMS, ni un push n'a pas de canal, donc rien à faire. Le mot-clé `abstract` transforme cette absurdité en erreur de compilation :

```
public abstract class Notification { ... }
```

```
Notification envoi;                  // d'accord
envoi = new Notification("Ada");     // refus a la compilation
envoi = new NotificationMail("Ada"); // d'accord
```

La classe existe, elle a des champs, des méthodes et un constructeur, mais elle ne produit aucun objet à elle seule. La distinction à retenir est celle entre déclarer et instancier : la variable `envoi` est de type `Notification`, ce qui est une promesse sur ce qu'elle sait faire, et non sur la classe exacte de l'objet qu'elle désigne.

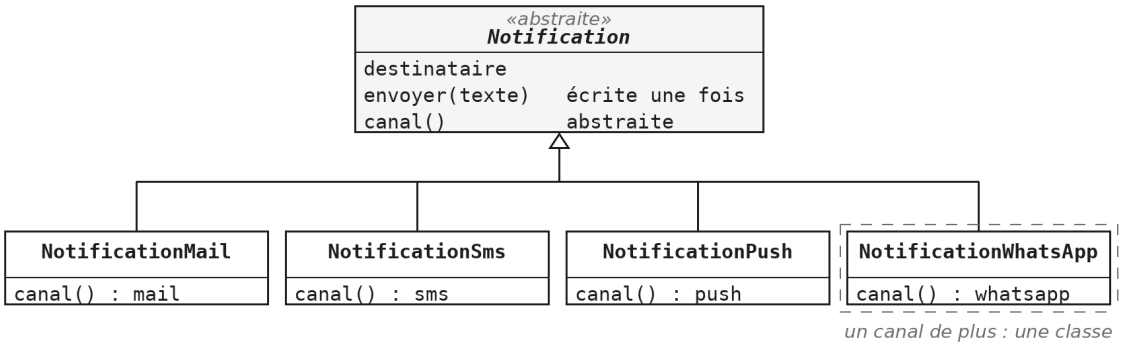
À quoi sert un type qu'on ne construit pas

Il sert de vocabulaire. On l'utilise comme type de variable, de paramètre, de valeur de retour, et surtout comme type d'élément d'un tableau (et plus tard d'une collection, voir le module `COL`) :

```
Notification[] canaux = {
    new NotificationMail("Ada"),
    new NotificationSms("0470..."),
    new NotificationPush("ada42")
};
```

Écrire `Notification[]` dit au compilateur que tous les éléments savent envoyer, sans dire par quel canal. Une méthode qui reçoit une `Notification` accepte les trois sortes, et n'a pas à les distinguer. Ce tableau ne contient d'ailleurs aucune `Notification` au sens strict : seulement des objets de sous-classes. `ABS §5` explique ce qui se passe quand on appelle une méthode dessus.

Le diagramme résume la hiérarchie obtenue : la classe mère abstraite, qui écrit l'envoi une seule fois et déclare le canal abstrait, et ses sous-classes concrètes, qui ne fournissent que leur canal. La quatrième, WhatsApp, s'ajoute sans rien modifier d'autre : une classe, et le tableau la traite comme les autres.



La classe abstraite Notification et ses sous-classes

Un constructeur, quand même

Une classe abstraite contient des champs, et ces champs doivent être initialisés. C'est le travail d'un constructeur, comme ailleurs ; la seule différence est que personne ne l'appelle par `new`, mais toujours par `super` depuis une sous-classe :

```
public abstract class Notification {

    protected final String destinataire;

    protected Notification(String destinataire) {
        this.destinataire = destinataire;
    }
}
```

Pourquoi `protected` plutôt que `public`, puisque `new` est de toute façon interdit ? Parce que le modificateur est une déclaration d'intention : il dit à celui qui lit la classe que ce constructeur existe pour les sous-classes. C'est une des questions de `ABS.1`.

ABS §4 — Ce qui monte, ce qui descend

C'est le cœur du module. Tout le reste en découle : la méthode abstraite, l'obligation faite aux sous-classes, et la façon de juger un découpage.

La méthode abstraite

Une méthode abstraite se déclare par son en-tête, suivi d'un point-virgule. Elle n'a pas de corps, parce que la classe mère ne sait pas comment faire :

```
public abstract class Notification {

    protected abstract String canal();

    public void envoyer(String texte) {
        System.out.println(canal() + " a " + destinataire + " : " + texte);
    }
}
```

La méthode `envoyer` est écrite une seule fois et appelle `canal()`, que chaque sous-classe fournit : c'est la règle de partage de la sous-section suivante, sur un premier exemple.

Déclarer une méthode abstraite, c'est nommer une obligation. La classe mère affirme que toute notification a un canal, et reconnaît dans le même mouvement qu'elle ne peut pas dire lequel. Une classe qui contient une méthode abstraite est elle-même abstraite ; l'inverse n'est pas vrai, une classe abstraite peut très bien n'avoir aucune méthode abstraite.

La règle de partage

- Une méthode dont le code serait identique partout appartient à la classe mère.
- Une méthode dont le code diffère à chaque fois est abstraite.

```
public abstract class Forme {

    // ce qui VARIE : chaque sous-classe le definit
    public abstract String nom();

    // ce qui est COMMUN : ecrit une seule fois
    public void affiche() {
        System.out.println("Je suis un " + nom());
    }
}
```

Regardez la méthode `affiche`. Elle est écrite une seule fois, dans la classe mère, et elle appelle `nom()`, qui n'existe nulle part. Cela compile, parce que le compilateur sait que toute sous-classe concrète en fournira une version. C'est le mécanisme qui porte `ABS §6`.

Le test pratique est simple. Si vos sous-classes contiennent plus d'une méthode, demandez-vous ce qui aurait dû monter.

L'obligation faite aux sous-classes

C'est la contrepartie de la promesse faite par la classe mère :

```
public class Cercle extends Forme {

    @Override
    public String nom() {
        return "cercle";
    }
}
```

Une sous-classe concrète a l'obligation de définir toutes les méthodes abstraites qu'elle hérite. Sinon, elle est abstraite à son tour, et doit le dire dans son en-tête ; ce n'est pas un accident de parcours, c'est exactement ce dont `ABS §7` a besoin. Le compilateur ne laisse pas passer l'oubli.

Remarquez que `Cercle` ne contient qu'une méthode. Tout le reste (la position, la rotation, l'affichage) vit dans la classe mère, et n'est écrit qu'une fois.

`@Override`, aussi pour une méthode abstraite

Pour Java, fournir le corps d'une méthode abstraite, c'est la redéfinir : l'annotation `@Override` s'écrit de la même façon. On pourrait la penser inutile ici, puisque le compilateur refuse déjà une classe concrète qui oublie une méthode abstraite. La différence est dans l'endroit où il le dit :

```
public class Cercle extends Forme {

    @Override
    public String Nom() {          // refus : Nom() ne redefinit rien
        return "cercle";
    }
}
```

Sans `@Override`, la faute de frappe `Nom()` crée une nouvelle méthode, et l'erreur tombe sur la ligne de la classe : `Cercle` ne définirait pas `nom()`. Avec l'annotation, l'erreur tombe sur la méthode mal écrite. On trouve la faute là où elle est. Vous n'avez d'ailleurs pas à l'écrire vous-même : quand IntelliJ génère les méthodes à fournir (Code, puis Implement Methods), il ajoute `@Override` à chacune.

Petit détour : le reste d'une division négative

Ce point n'a rien à voir avec les classes abstraites ; il est ici parce que `ABS.1` en a besoin, et qu'il piège tout le monde une fois.

```
-90 % 360               // -90, et non 270

(-90 % 360 + 360) % 360 // 270
```

La rotation d'une forme doit toujours valoir entre 0 inclus et 360 exclu. Le réflexe est d'écrire un modulo, et il fonctionne pour `setRotation(370)`, qui donne bien 10. Il échoue pour `setRotation(-90)`, parce qu'en Java le reste de la division garde le signe du dividende. La correction tient en une expression : le premier modulo ramène la valeur dans l'intervalle, l'addition la rend positive, et le second modulo efface le tour de trop.

> **Exercices** - `ABS.1` Des formes géométriques
>
> Chaque méthode demande une décision : monte-t-elle ou descend-elle ? La mesure de la réussite est donnée par l'énoncé : chacune des trois sous-classes ne devrait contenir qu'une seule méthode. Si l'une en a deux, quelque chose n'est pas remonté. Voir `CC-CLS.3`.

ABS §5 — Le polymorphisme

Un tableau du type le plus général

```
Shape[] formes = {
    new Circle(centre), new Square(centre), new Triangle(centre)
};

for (Shape forme : formes) {
    forme.affiche();
}
```

Comment un tableau de `Shape` peut-il exister alors que `Shape` ne s'instancie pas ? Parce que le type déclaré d'une variable et la classe réelle de l'objet sont deux choses différentes. Le type déclaré dit ce qu'on a le droit de demander ; la classe réelle dit qui répond. Un `Circle` *est* un `Shape`, donc il a sa place dans le tableau. La boucle ne contient aucune trace des trois formes : ajouter un losange ne la changera pas d'une ligne.

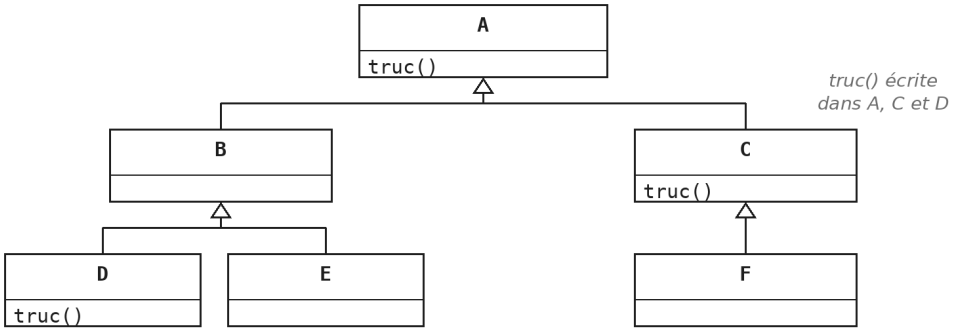
Qui répond à l'appel

```
Shape forme = new Circle(centre);

forme.affiche();   // "Je suis un cercle et je m'affiche"
```

La variable est déclarée `Shape`, et pourtant c'est la version du cercle qui s'exécute. Le compilateur vérifie seulement que `Shape` possède bien une méthode `affiche` ; le choix de la version revient à la machine virtuelle, au moment de l'appel, en regardant l'objet réel. Ce mécanisme s'appelle la *liaison dynamique*, et c'est lui qui donne tout son intérêt à la classe abstraite. C'est le polymorphisme, le quatrième des principes présentés au module `POO`.

Quelle version s'exécute, dans une chaîne d'héritage



Une méthode `truc()` écrite dans A, redéfinie dans C et dans D

Quand une méthode est redéfinie à plusieurs étages d'une hiérarchie, Java part de la classe réelle de l'objet et remonte vers ses ancêtres, jusqu'à la première classe qui écrit la méthode. C'est cette version qui s'exécute.

Objet de la classe	`truc()` exécutée	Pourquoi
`A`	celle de `A`	`A` l'écrit
`B`	celle de `A`	`B` ne l'écrit pas : on remonte à `A`
`C`	celle de `C`	`C` la redéfinit
`D`	celle de `D`	`D` la redéfinit
`E`	celle de `A`	ni `E` ni `B` ne l'écrivent
`F`	celle de `C`	`F` ne l'écrit pas, `C` oui

Le type de la variable ne change rien : avec `A x = new F();`, l'appel `x.truc()` exécute la version de `C`.

```
public class D extends B {
    @Override
    public void truc() {
        super.truc();          // la version dont dispose B : celle de A
        System.out.println("et la suite propre a D");
    }
}
```

Dans `D`, `super.truc()` ne désigne pas forcément une méthode écrite dans `B`, la mère directe : il désigne la version dont `B` dispose, c'est-à-dire la plus proche en remontant. Ici, `B` n'écrit pas `truc()`, c'est donc celle de `A` qui s'exécute.

Ni `instanceof`, ni test de type

```
for (Shape forme : formes) {
    if (forme instanceof Circle) {   // a ne pas ecrire
        ...
    }
}
```

L'opérateur `instanceof` existe et a ses usages, mais il n'a rien à faire dans une boucle comme celle-ci. Si vous avez besoin de savoir quelle forme vous tenez pour décider quoi faire, c'est que ce « quoi faire » aurait dû être une méthode de la forme. Ce que vous testez, la machine virtuelle le sait déjà.

Le test pratique est celui de `CC-CLS.4` : combien de classes faut-il modifier pour ajouter une sorte de plus ? Avec le polymorphisme, aucune : on écrit la nouvelle classe, et le programme existant la traite sans le savoir. Avec des tests de type, il faut retrouver chaque endroit où le test est écrit. Les énoncés `ABS.2` à `ABS.5` posent tous cette contrainte.

> **Exercices** - `ABS.2` Des animaux qui font du bruit
>
> Le même geste qu'`ABS.1`, sur un cas plus court, avec une nouveauté : le tableau est rempli au hasard, donc le programme ignore vraiment ce qu'il contient. Deux questions valent le détour : combien de lignes faut-il modifier pour ajouter une vache, et que devient l'exercice sans classe abstraite ? Voir `CC-CLS.4`.

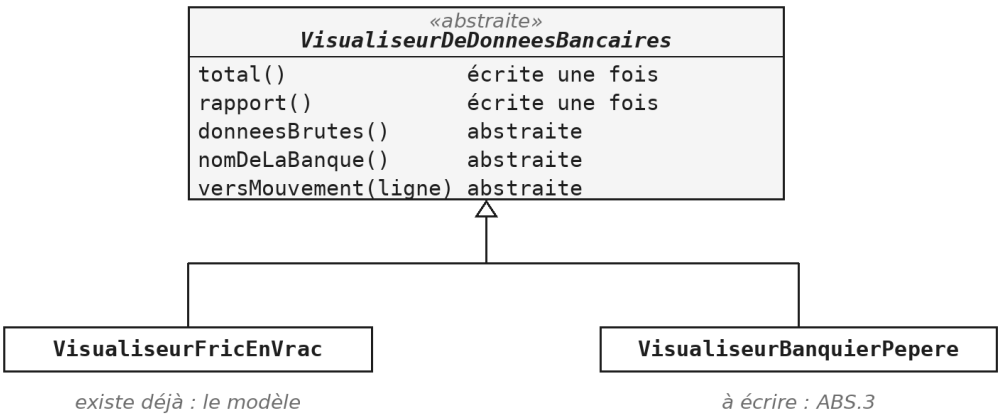
ABS §6 — La mère appelle la fille

C'est la forme la plus utile de la classe abstraite, et le cas le plus proche du travail réel : la classe mère écrit le déroulé complet d'un traitement, et laisse aux sous-classes les seuls endroits qui varient.

De quoi il s'agit

Une application affiche les mouvements bancaires de ses utilisateurs. Elle fonctionne aujourd'hui avec une seule banque, Fric en Vrac, et il faut y ajouter Banquier Pépère. Les deux banques livrent les mêmes

informations (un compte, un montant, une communication), et chaque ligne devient un `Mouvement`, un `record`. Elles ne les livrent pas dans le même ordre de colonnes, et c'est la seule chose qui les distingue.



Le dispositif d'`ABS.3` : la mère écrit le traitement, chaque banque fournit les détails

Le squelette, puis le détail

```
public abstract class VisualiseurDeDonneesBancaires {

    // a fournir par chaque banque
    protected abstract String[][] donneesBrutes();
    protected abstract String nomDeLaBanque();
    protected abstract Mouvement versMouvement(String[] ligne);

    // deja ecrit, une fois pour toutes
    public double total() { ... }
    public String rapport() { ... }
}
```

La classe mère ne délègue pas le traitement : elle le possède. Elle sait convertir l'ensemble, additionner les montants et mettre en forme le rapport. Ce qu'elle ignore, c'est d'où viennent les données, comment s'appelle la banque et comment se lit une ligne. Ces trois trous sont déclarés abstraits ; chaque banque les bouche à sa façon, et hérite de tout le reste sans rien écrire.

Le renversement est le point à comprendre : ce n'est pas la sous-classe qui appelle la mère, c'est la mère qui appelle la sous-classe. Le code commun mène la danse, et vient chercher le détail au moment où il en a besoin.

Deux banques, un seul traitement

Banque	`ligne[0]`	`ligne[1]`	`ligne[2]`
Fric en Vrac	compte	montant	communication
Banquier Pépère	communication	compte	montant

Toute la différence entre les deux visualiseurs tient donc dans `versMouvement` : l'un lit le compte en `ligne[0]`, l'autre en `ligne[1]`. Le reste est identique. C'est exactement la situation où la duplication est tentante : recopier le visualiseur existant et déplacer trois indices prend cinq minutes et fonctionne. Puis le calcul du total se révèle faux un jour, et il faut le corriger dans les deux.

Pourquoi `protected`

La méthode qui convertit une ligne est `protected`, alors que c'est précisément une méthode qu'une autre classe doit définir. Parce que « une autre classe » veut dire ici « une sous-classe », et rien de plus. Le programme qui affiche les rapports n'a aucune raison de lire une ligne lui-même : il appelle `rapport` et `total`. Tout ce qui sert à construire le résultat est un détail de fabrication. L'ouvrir en `public` ne casserait rien aujourd'hui, mais transformerait un détail interne en promesse faite au reste du programme, qu'il faudrait ensuite tenir.

La mesure de la réussite

- La nouvelle classe devrait faire une vingtaine de lignes, dont trois méthodes de deux lignes.
- Si elle est plus longue, c'est que quelque chose a été recopié de la mère.
- Le programme ne contient ni `instanceof`, ni condition sur la banque.

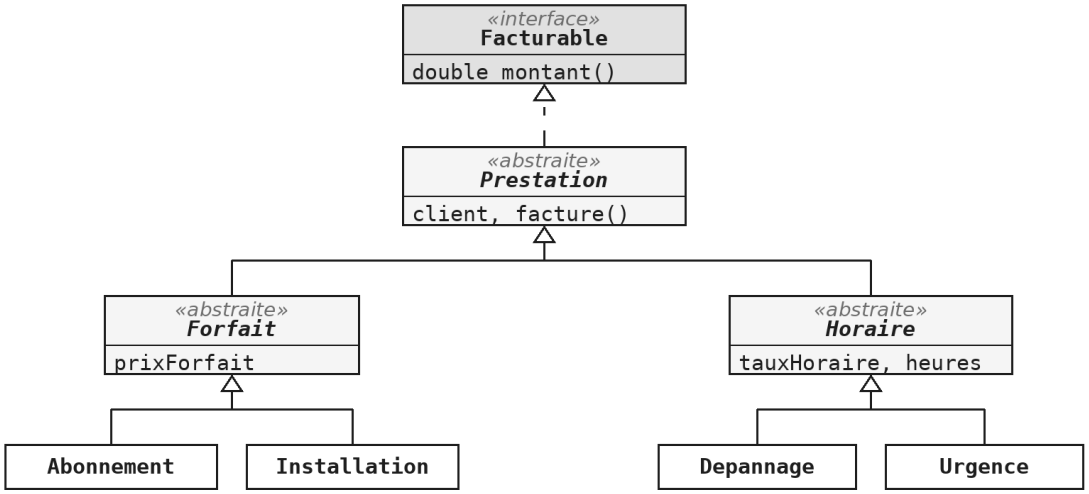
> **Exercices** - `ABS.3` Deux banques, un seul traitement
>
> L'exercice ne demande pas d'écrire un dispositif, mais d'en compléter un qui existe déjà. C'est la situation la plus fréquente en entreprise : lire ce qui est fourni avant d'écrire, et n'ajouter que ce qui manque réellement. Le visualiseur de la première banque sert de modèle.

ABS §7 — Deux étages

Les exercices précédents n'avaient qu'un seul niveau d'abstraction. Cette section en ajoute un second, et c'est la forme que prend la quatrième question de l'examen.

Une classe abstraite peut en étendre une autre

Une société facture au forfait ou à l'heure, et chaque façon connaît deux variantes. Deux niveaux de regroupement apparaissent naturellement : ce qui est vrai de toute prestation, et ce qui n'est vrai que des prestations horaires, ou que des forfaits.



La hiérarchie d'`ABS.4` : une interface, trois classes abstraites, quatre classes concrètes

`Prestation` connaît le client, parce que toute prestation en a un. `Horaire` connaît le taux et le nombre d'heures, parce que les forfaits n'en ont pas. `Urgence` connaît la majoration, parce qu'elle est seule à en avoir une. Quatre classes sont concrètes, trois ne le sont pas : une classe est abstraite tant qu'elle ne suffit pas à décrire une prestation réelle.

Où placer chaque champ

Un champ se déclare à l'étage où il devient commun. Trop haut, il encombre qui n'en a pas l'usage ; trop bas, il s'écrit deux fois.

```
public abstract class Horaire extends Prestation {

    private final double tauxHoraire;
    private final int heures;

    protected double base() {
        return heures * tauxHoraire;
    }
}
```

C'est la règle d'`ABS §4`, appliquée à plusieurs étages. Le taux horaire n'appartient pas à `Prestation`, puisqu'un abonnement n'en a pas. Il n'appartient pas non plus à `Depannage` et à `Urgence` séparément, puisque les deux en ont un : il appartient à `Horaire`. La méthode `base` suit le même raisonnement ; `Urgence` s'en servira en y appliquant sa majoration. Sa visibilité est une des décisions que l'exercice vous laisse : c'est un détail de calcul dont les sous-classes ont besoin, et personne d'autre (voir `PKG.5`).

Implémenter sans définir

```
public abstract class Prestation implements Facturable {

    public String facture() {
        return client + " : " + montant() + " EUR";
    }
}
```

Les interfaces sont le sujet du module `ITF`. Il suffit ici d'en savoir ceci : une interface énonce ce qu'une classe doit savoir faire, et `implements` s'engage à le faire. `Prestation` s'y engage sans tenir l'engagement, et c'est légal parce qu'elle est abstraite : elle reporte l'obligation sur ses descendants, exactement comme une méthode abstraite ordinaire. La méthode `facture`, elle, est écrite tout de suite, et appelle une méthode qui n'existe dans aucune classe au moment où on l'écrit. C'est le mécanisme d'`ABS §6`, poussé un cran plus loin.

> **Exercices** - `ABS.4` Une hiérarchie à deux niveaux, à faire après le module `ITF`
>
> L'exercice réunit tout le module : deux étages d'abstraction, une interface à implémenter, un champ à placer au bon niveau, un tableau du type le plus général et pas un seul test de type. Il demande l'interface `Facturable`, qui appartient au module suivant. Deux décisions vous appartiennent, qu'aucun test ne vérifie : où implémenter `montant`, et quelle visibilité donner aux accesseurs intermédiaires.

ABS §8 — Décider soi-même

Le dernier exercice du module ne fournit aucun squelette. C'est le seul du recueil dans ce cas, et c'est la situation de l'examen.

Quand il n'y a plus de squelette

Les exercices précédents donnaient les classes, les signatures et les types : il restait à remplir les corps. Ici, vous partez d'un fichier vide, et les décisions de conception sont l'exercice lui-même. Combien de classes, laquelle est abstraite, où va chaque méthode : à vous de savoir. Trois questions suffisent à démarrer :

1. **Qu'est-ce que les classes ont en commun ?** Ce qui est commun monte, et ne s'écrit qu'une fois.
2. **Que sait faire la classe « en général » ?** S'il y a une chose qu'elle ne sait pas faire alors que toutes ses sortes la savent, vous tenez votre méthode abstraite.
3. **Ce qui varie peut-il se réduire à un détail ?** Regardez ce qui change d'une sorte à l'autre, et relisez `ABS.1`.

La classe de test est fournie, mais en commentaire, parce qu'elle appelle des classes qui n'existent pas encore. Décommentez-la quand vos classes seront écrites, d'un seul coup, avec `Ctrl+/`.

> **Exercices** - `ABS.5` Tout écrire, à partir de rien, puis `G.10` La classe abstraite `Jeu` (fil rouge GameHub)
>
> `ABS.5` fait écrire un service postal à trois tarifs. Le piège n'est pas dans le calcul mais dans la borne : « jusqu'à 50 g inclus » s'écrit avec un `<=`, et une erreur d'un gramme ne se voit pas en essayant au hasard ; le test à la borne l'attrape tout de suite. Comparez ensuite le temps passé sur `ABS.5` et sur `ABS.4`, et demandez-vous où est passée la différence : à écrire, ou à décider. Côté fil rouge, `G.10` introduit la classe abstraite `Jeu`, socle des jeux concrets à venir.

Ce qu'il faut retenir

- `extends` pour hériter, `super(...)` en première instruction du constructeur, `@Override` toujours.
- Une classe abstraite ne s'instancie pas ; elle sert de type commun.
- Ce qui est commun monte, ce qui varie descend, sous forme de méthode abstraite.
- La méthode exécutée est celle de la classe réelle de l'objet : c'est le polymorphisme.
- Un test de type dans une boucle signale une hiérarchie mal découpée.
- La mère peut écrire tout le traitement, et appeler les détails que les filles fournissent.

La mesure d'un bon découpage tient en une question : combien de classes faut-il modifier pour ajouter une sorte de plus ? La bonne réponse est zéro, et une classe à écrire.

Module ITF

ITF — Interfaces

Décrire ce qu'on attend, sans dire qui le fera

Le module `ABS` a montré comment partager du code entre des classes voisines. Celui-ci répond à une autre question : comment faire travailler ensemble des classes qui n'ont rien en commun ?

La réponse tient en une phrase : décrire ce qu'on attend, sans dire qui le fera. Une interface énonce un service, et laisse à d'autres le soin de le rendre. C'est la notion qui porte le découplage, et donc la majorité des conceptions que vous rencontrerez en entreprise. Elle sert aussi dans deux questions de l'examen : la hiérarchie d'`ABS.4`, dont le sommet est une interface, et l'interface graphique, qui fonctionne entièrement sur ce principe.

> **À la fin de ce module, vous saurez**
>
> - déclarer une interface, et l'implémenter dans des classes qui n'ont aucun lien de parenté ;
> - utiliser une interface comme type de variable et de paramètre ;
> - combiner une classe mère et plusieurs interfaces dans une même classe ;
> - choisir entre interface et classe abstraite, ou les employer ensemble ;
> - faire dépendre une classe d'un contrat plutôt que d'une autre classe ;
> - reconnaître et écrire le patron Observateur.

ITF §1 — Ce qu'une interface déclare

Démonstration - `Demo_ITF` : trois mots, et le programme accepte l'inconnu

La démonstration part d'un programme qui fonctionne : un fournisseur, des clients abonnés, des produits qui arrivent. Une épicerie veut être prévenue elle aussi ; elle a la bonne méthode, avec la bonne signature, et le compilateur la refuse quand même : le fournisseur a été écrit pour une classe, pas pour une capacité. La fausse bonne idée est jouée volontairement : faire hériter le magasin de la classe client. Ça compile, et c'est un mensonge fait au compilateur, car un magasin n'est pas un client. L'interface est alors extraite, trois mots changent dans le fournisseur, et le magasin est prévenu comme les clients. Ajouter un entrepôt ne demande plus qu'une classe.

Déclarer une interface

Une interface énonce ce qu'une classe doit savoir faire, sans dire comment. Le mot-clé `interface` remplace `class`, et les méthodes s'y déclarent par leur en-tête, sans corps :

```
public interface Mesurable {

    double mesure();
}
```

Une interface ressemble à une classe abstraite qui n'aurait que des méthodes abstraites. La ressemblance est un bon point de départ, et `ITF §3` dira où elle s'arrête. Les méthodes y sont publiques sans qu'on l'écrive : une méthode d'interface sans corps est destinée à être écrite ailleurs, donc elle est toujours publique, et le préciser n'apporte rien. Le compilateur accepte le mot `public` si vous l'écrivez, mais IntelliJ le signale comme redondant.

Le nom choisi dit une capacité, pas une nature. C'est une habitude utile : beaucoup d'interfaces du JDK se terminent par *-able*, comme `Comparable`, `Runnable` ou `Serializable`.

Une classe s'engage

Le mot-clé `implements` déclare qu'une classe tient la promesse, et le compilateur vérifie que toutes les méthodes annoncées sont écrites :

```
public class Examen implements Mesurable {

    private final int points;

    @Override
    public double mesure() {
        return points;
    }
}
```

```
public class Colis implements Mesurable {

    private final double grammes;
    private final String destinataire;

    @Override
    public double mesure() {
        return grammes;
    }

    public String destinataire() {
        return destinataire;
    }
}
```

Regardez ce que ces deux classes ont en commun : rien. Un examen se mesure en points, un colis en grammes, et aucune classe mère ne les réunirait sans inventer une parenté qui n'existe pas. C'est exactement ce que la classe abstraite ne sait pas faire, et ce pour quoi l'interface existe. L'annotation `@Override` n'est pas obligatoire, mais elle vaut d'être écrite, pour la même raison qu'au module `ABS` : une faute de frappe dans le nom devient une erreur de compilation au lieu d'un silence.

Le type peut être l'interface

```
Mesurable m = new Colis(1250);
System.out.println(m.mesure());        // OK

Mesurable autre = new Mesurable();     // erreur de compilation
System.out.println(m.destinataire());  // erreur de compilation
```

Une interface n'a pas de constructeur et ne décrit aucun objet : il n'y a rien à instancier, elle ne sert que de type. La seconde erreur est plus intéressante que la première. L'objet désigné est bel et bien un colis, et il a bel et bien un destinataire, mais la variable est déclarée `Mesurable` : le compilateur ne regarde que ce que ce type promet. Ce n'est pas une gêne, c'est l'intérêt même du procédé : le code qui manipule des mesurables ne peut plus dépendre par accident d'un détail des colis. C'est le polymorphisme du module `ABS`, sans classe mère.

Recevoir un comportement en paramètre

```
public static double total(Mesurable[] choses) {
    double somme = 0;
    for (Mesurable chose : choses) {
        somme += chose.mesure();
    }
    return somme;
}
```

Cette méthode additionne des examens et des colis dans le même tableau, sans savoir lesquels. Elle fonctionnera aussi avec une classe qui n'existe pas encore, du moment que cette classe implémente `Mesurable`. Retenez la formulation : le type d'un paramètre déclare ce dont la méthode a besoin, pas qui le fournit. Écrire `Colis` au lieu de `Mesurable` reviendrait à exiger beaucoup plus que nécessaire. `ITF §4` en fait un principe.

> **Exercices** - `ITF.1` Une interface et deux implémentations, puis `ITF.2` Payer en ligne
>
> `ITF.1` isole l'idée dans le plus petit programme possible : une interface à une méthode, deux implémentations, et une méthode qui reçoit un calculateur. Le test lui passe une soustraction fabriquée sur place, que votre code n'a jamais vue. `ITF.2` reprend la même structure sur un cas réaliste : le client d'un côté, les moyens de paiement de l'autre. Les classes de paiement retournent leur compte-rendu au lieu de l'afficher (`CC-MET.7`), et le formatage du montant précise `Locale.US` (voir `TMP §4`).

ITF §2 — Plusieurs interfaces à la fois

Une seule mère, autant d'interfaces qu'on veut

```
public class Square extends Shape
        implements Translatable, Rotatable {

    // ...
}
```

Une classe Java n'hérite que d'une seule autre : il n'y a pas de double `extends`. L'héritage multiple est interdit, et pour une bonne raison : si deux classes mères fournissaient la même méthode avec deux corps différents, aucune règle simple ne dirait laquelle s'applique. Les interfaces ne posent pas ce problème, puisqu'elles ne fournissent aucun corps. Une classe peut donc en implémenter autant qu'elle veut. L'ordre des mots est imposé : `extends` d'abord, `implements` ensuite, et chacun des deux peut aussi s'utiliser seul.

Un comportement qui traverse les familles

```
public interface Translatable {
    void translate(double dx, double dy);
}
```

```
public interface Rotatable {
    void rotate(double alpha);
}
```

Une classe abstraite dit ce qu'un objet *est*, une interface ce qu'il *sait faire*. Un carré est une forme, et cette phrase justifie la classe abstraite `Shape`. Un carré sait se déplacer, et cette phrase-là justifie une interface, parce que la capacité déborde largement la famille des formes : une image ou un texte peuvent se déplacer sans être des formes. Une capacité se moque de la généalogie.

Les deux interfaces sont séparées, et ce n'est pas un hasard : une figure pourrait se déplacer sans pouvoir tourner. Réunir les deux méthodes dans une seule interface obligerait toute classe déplaçable à savoir tourner ; `ITF §4` dit pourquoi c'est une mauvaise idée.

Où écrire la méthode

```
public abstract class Shape
        implements Translatable, Rotatable {

    // abstraites, ou ecrites ici ?
}
```

`Shape` s'engage sur les deux interfaces, et deux issues sont possibles : soit elle déclare les méthodes abstraites et laisse chaque forme les écrire, soit elle les écrit une fois pour toutes et ses filles n'ont rien à faire. Le critère est celui d'`ABS §4`, et il n'a pas changé : le code serait-il identique dans les trois sous-classes ? Si oui, il monte.

> **Exercices** - `ITF.3` Des formes qui se déplacent et tournent
>
> L'exercice combine une hiérarchie de classes et deux interfaces qui la traversent ; le travail d'écriture est court, la décision est tout l'exercice. Le dernier test manipule un tableau de `Translatable` : essayez d'appeler `rotate` ou `nom` sur un de ses éléments, et lisez ce que dit le compilateur : une variable de type `Translatable` ne connaît que ce que promet l'interface, même si l'objet qu'elle désigne sait faire davantage. La dernière question de l'énoncé reste ouverte : la normalisation de l'angle, déjà écrite en `ABS.1`, réapparaît ici ; est-ce une duplication au sens de `CC-CLS.3` ?

ITF §3 — Interface ou classe abstraite ?

Deux outils, deux intentions

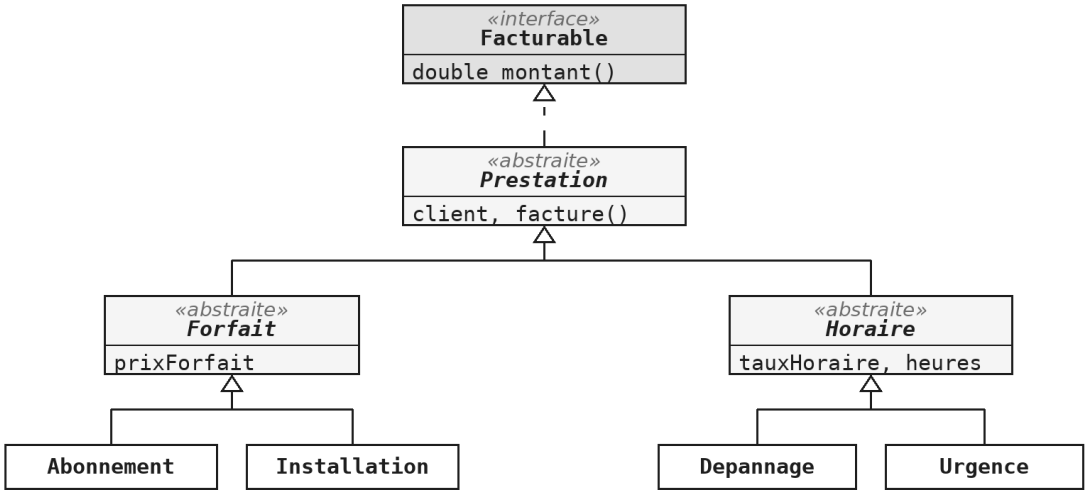
	Classe abstraite	Interface
Contient du code ?	oui, et des champs	des signatures seulement[2]
Combien par classe ?	une seule	autant qu'on veut
Ce qu'elle exprime	« est un »	« sait faire »
Ce qu'elle apporte	le partage du code	le découplage

La troisième ligne est celle qui décide presque toujours. Formulez la phrase à voix haute : un carré *est* une forme, un carré *sait* se déplacer. La bonne réponse est celle des deux phrases qui n'est pas ridicule. Les deux premières lignes en sont des conséquences : une interface ne contient pas de code, donc elle ne partage rien, donc rien n'empêche d'en implémenter plusieurs.

[2] Ce n'est plus tout à fait vrai depuis Java 8 (2014) : une interface peut aussi contenir des méthodes avec un corps, les méthodes `default` et les méthodes `static`, et depuis Java 9 des méthodes `private` qui les aident. `ITF §6` présente les deux premières. Elles restent l'exception : l'essentiel d'une interface, ce sont ses signatures.

Les deux ensemble

Les deux outils ne s'excluent pas. L'interface énonce ce que tout le monde sait faire ; la hiérarchie partage le code, étage par étage. C'est la structure d'`ABS.4`, annoncée au module précédent :



La hiérarchie d'`ABS.4` : l'interface au sommet, les classes abstraites en dessous

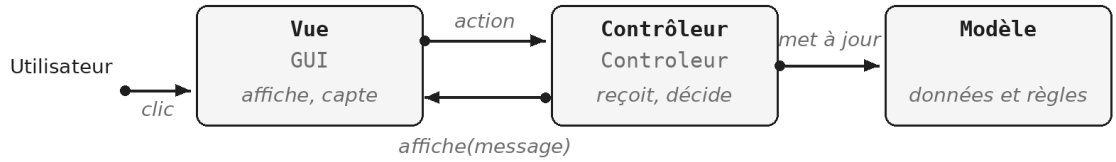
L'interface `Facturable` dit ce que toute prestation doit savoir faire : donner son montant. Les classes abstraites rangent les champs et le code commun à l'étage où ils deviennent communs. `Prestation` implémente `Facturable` sans écrire `montant`, ce qui est légal parce qu'elle est abstraite : elle reporte l'obligation sur ses descendants.

> **Exercices** - `ABS.4` Une hiérarchie à deux niveaux
>
> L'exercice mis de côté au module `ABS`, faute de connaître les interfaces, devient faisable maintenant. C'est aussi la forme exacte de la quatrième question de l'examen. Deux décisions vous appartiennent, qu'aucun test ne vérifie : où implémenter `montant`, et quelle visibilité donner aux accesseurs intermédiaires (voir `PKG.5`).

ITF §4 — Dépendre d'un contrat

Le modèle MVC : qui fait quoi

Les exemples de cette section parlent d'un contrôleur et d'une classe `GUI`. Ces deux noms viennent d'un découpage très répandu dans les applications graphiques, le modèle MVC : Modèle, Vue, Contrôleur.



Le découpage MVC

- **La vue** est ce que l'utilisateur voit : la fenêtre, ses boutons, ses zones de texte. Elle capte les gestes, mais ne décide de rien.
- **Le modèle** contient les données et les règles du métier, sans rien savoir de l'écran.
- **Le contrôleur** fait le lien : il reçoit l'action transmise par la vue, met le modèle à jour, puis demande à la vue d'afficher le résultat.

Chaque rôle peut ainsi changer sans toucher aux autres, du moins en principe. La flèche « affiche » est celle qui pose problème : si le contrôleur la tient vers une classe d'affichage précise, il ne pourra plus parler à rien d'autre. Les modules `FX` reviendront sur ce découpage avec `JavaFX`.

Deux classes qui se tiennent

```
public class Controleur {

    private GUI afficheur;

    public void setAfficheur(GUI afficheur) {
        this.afficheur = afficheur;
    }
}
```

Ce code fonctionne, et c'est ce qui le rend instructif : le défaut n'est pas un bug, il est dans la dépendance. Le contrôleur n'a besoin que d'une chose, afficher un message, et il exige pourtant une classe entière. Le jour où l'application gagne une version console, ou une version qui écrit dans un fichier de trace, le contrôleur devra être rouvert. Et pour tester le contrôleur, il faut construire une interface graphique : le module `JUN` y reviendra.

Une interface entre les deux

```
public interface Afficheur {
    void affiche(String message);
}
```

```
public class Controleur {

    private Afficheur afficheur;

    public void setAfficheur(Afficheur afficheur) { ... }
}
```

Trois mots ont changé, et la dépendance a changé de sens. Le contrôleur ne dépend plus de la fenêtre : les deux dépendent de l'interface, plus petite et plus stable que l'une comme l'autre. Affichage console, fenêtre ou trace : au choix, sans rouvrir le contrôleur. Le gain de testabilité devient concret : une classe de test peut fournir au contrôleur un afficheur de quatre lignes qui retient le dernier message, et vérifier ce qui a été signalé sans ouvrir la moindre fenêtre.

Le type le plus faible qui suffit

Le *principe de ségrégation* demande de ne réclamer que ce dont on a vraiment besoin. Il a deux conséquences pratiques.

- **Pour les types que vous écrivez** (un paramètre, un champ, une variable) : le type le plus faible qui suffit est le bon. Déclarer `Afficheur` plutôt que `Fenetre` n'est pas une coquetterie, c'est une réduction délibérée de ce que le code peut faire.
- **Pour les interfaces que vous concevez** : une interface qui déclarerait à la fois `affiche`, `efface`, `redimensionne` et `imprime` obligerait toute classe affichante à savoir imprimer. Mieux vaut plusieurs petites interfaces, qu'une classe combine si elle le souhaite. C'est la raison pour laquelle `Translatable` et `Rotatable` sont deux interfaces, et pas une.

Un comportement qui ne suit pas la hiérarchie

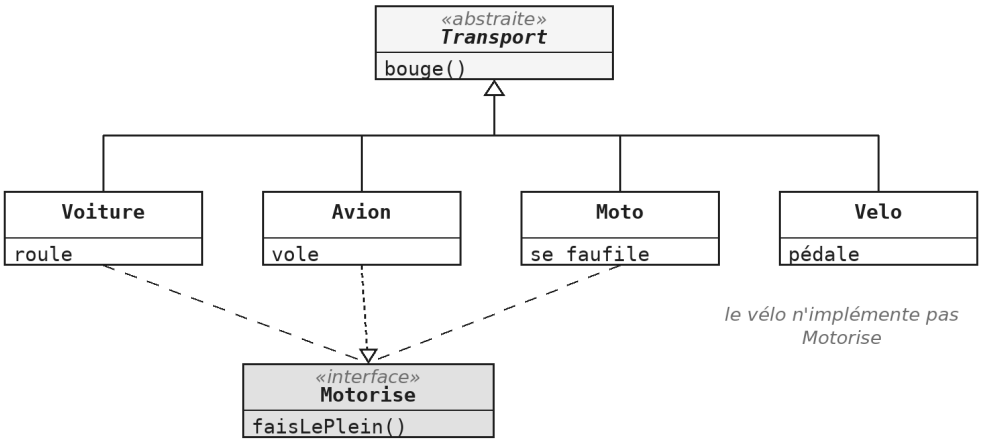
Une classe abstraite `Transport` déclare ce que tout transport sait faire, bouger, et chaque sous-classe le fait à sa façon. Une station-service doit faire le plein des transports qu'on lui présente. Tant que les transports sont une voiture, un avion et une moto, tout va bien ; puis on ajoute le vélo, qui est bien un transport, mais n'a pas de moteur.

```
public class StationService {

    // a ne pas ecrire
    public void faireLePlein(Transport transport) {
        if (transport instanceof Velo) {
            // ne rien faire
        } else {
            // faire le plein
        }
    }
}
```

Le module `ABS` a déjà condamné le test de type, mais la situation n'est pas la même. Là-bas, le test trahissait une hiérarchie mal découpée. Ici, le découpage est correct : un vélo est bien un transport. C'est le comportement qui traverse la famille au lieu de la suivre ; aucune classe mère ne sépare les transports à moteur des autres, et en inventer une mentirait sur ce qu'est un transport. Le jour où un tricycle arrive à la pompe, la station devra être modifiée, et rien dans le compilateur ne préviendra qu'on l'a oubliée.

L'interface supprime le test



Seuls les transports à moteur implémentent Motorise

```
public interface Motorise {
    void faisLePlein();
}
```

```
public class StationService {

    public void faireLePlein(Motorise motorise) {
        motorise.faisLePlein();
    }
}
```

En déclarant le paramètre `Motorise`, la station n'a plus rien à vérifier : un appel avec un vélo ne compile pas, et l'erreur est signalée à l'endroit où elle est commise. Il faut aller jusqu'au bout du raisonnement : écrire `if (transport instanceof Motorise)` dans la station ferait disparaître le nom du vélo, mais garderait le test. On aurait déplacé le problème, pas résolu. C'est la règle `CC-CLS.4`, et le principe de ségrégation, appliqués ensemble.

ITF §5 — Le patron Observateur

Un patron de conception

Un *patron de conception* est une solution éprouvée à un problème de conception qui revient souvent. Ce n'est ni une bibliothèque ni du code à copier : c'est une structure de classes, qu'on reconnaît et qu'on applique. On les range en trois familles : création, structure et comportement. Leur premier intérêt est le vocabulaire : dire

« c'est un observateur » à un collègue transmet en trois mots ce qu'il faudrait un quart d'heure à décrire. Le catalogue `refactoring.guru` en présente les principaux.

Le problème réglé par l'Observateur

Plusieurs objets veulent être prévenus quand quelque chose arrive : un produit en stock, un fichier qui change, un bouton cliqué. Les deux solutions spontanées sont mauvaises :

- **faire interroger le magasin en boucle** par les intéressés gaspille du temps, et arrive toujours un instant trop tard ;
- **faire prévenir nommément chaque intéressé** oblige le magasin à tenir une liste par type d'acteur, et à être modifié chaque fois qu'un nouveau type apparaît.

On cherche un dispositif où les intéressés se signalent eux-mêmes, et où le magasin ne sait rien d'eux.

Le dispositif

```
public class Magasin {

    private final Abonne[] abonnes = new Abonne[10];

    public void receptionne(String produit) {
        for (Abonne abonne : abonnes) {
            abonne.nouveauProduit(produit);
        }
    }
}
```

L'interface `Abonne` déclare une seule méthode, `nouveauProduit`. Le magasin tient un tableau d'abonnés, et trois méthodes suffisent : s'abonner, se désabonner, prévenir. Lisez la classe en cherchant le mot « client » : il n'y est pas, et c'est tout le patron. La méthode `receptionne` appelle `nouveauProduit` sur chaque abonné, sans savoir lequel répondra ni ce qu'il fera. Le tableau a une taille fixe de dix, une limite assumée que le module `COL` lèvera.

Ce que ça change

- **Ajouter un acteur** (un entrepôt, un prospect, un transporteur) demande une classe, et rien d'autre à modifier.
- **Le magasin ne saura jamais à qui il parle.** Il garde l'initiative de l'appel, mais pas la connaissance des destinataires.
- **On ne demande pas son état à l'abonné : on lui demande d'agir** (`CC-OBJ.1`).

Ce dispositif vous servira plus tôt que vous ne le croyez : toute la gestion des événements en `JavaFX` fonctionne ainsi. Un bouton ne connaît pas votre programme, il connaît une interface, et votre code vient s'y abonner.

> **Exercices** - `ITF.4` Prévenir sans connaître
>
> Il faut ajouter un `Prospect` à un dispositif qui existe déjà et qui fonctionne, sans modifier ni le `Magasin`, ni l'interface, ni les deux autres abonnés. Si vous vous surprenez à vouloir modifier le magasin, relisez-le : rien n'y nomme un client ou un employé, donc rien n'a besoin d'y changer.

ITF §6 — Ce qu'une interface contient aussi

Des constantes, et des méthodes utilitaires

```
public interface Mesurable {

    double SEUIL = 0.001;   // public static final

    double mesure();

    static boolean negligeable(Mesurable m) {
        return m.mesure() < SEUIL;
    }
}
```

Un champ d'interface est implicitement `public static final` : une interface ne peut déclarer que des constantes, et un champ sans valeur initiale ne compile pas. Une interface peut aussi contenir des méthodes `static`, corps compris, ajoutées par Java 8. Elles s'appellent sur l'interface, `Mesurable.negligeable(colis)`, et servent à ranger les petits utilitaires liés à l'interface là où on les cherche. Ne confondez pas les deux `static` : celui du champ est imposé, celui de la méthode est un choix.

Ajouter une méthode sans casser l'existant

```
public interface Vehicle {

    void demarre();
    void arrete();

    default void klaxonne() {
        System.out.println("Tuut");
    }
}
```

Une interface implémentée par dix classes ne peut pas gagner une méthode abstraite : les dix cessent de compiler. Une méthode `default` contient un corps, dont toutes les classes héritent ; chacune peut la redéfinir, aucune n'y est obligée. Java 8 a introduit ce mécanisme pour ajouter des méthodes à `List` et à ses voisines sans casser tout le code existant du monde. Vous n'écrirez pas de méthode par défaut cette année, mais vous en utiliserez sans le savoir dès le module `COL`.

Cette notion écorne le tableau d'`ITF §3` : depuis Java 8, une interface peut contenir du code. La distinction utile reste celle des intentions, « est un » contre « sait faire », et une interface qui se met à contenir beaucoup de code est généralement une classe abstraite qui s'ignore.

Une interface sans aucune méthode

```
public class Contact implements Serializable {

    // rien a implementer : l'interface ne demande rien
}
```

Une *interface de marquage* ne déclare rien : elle pose une étiquette. Elle ne dit pas « sait faire », mais « appartient à cette catégorie ». `Serializable`, que vous rencontrerez au module `FIC`, autorise un objet à être écrit dans un fichier : la bibliothèque vérifie à l'exécution que l'objet porte l'étiquette, et lève `NotSerializableException` sinon. Depuis Java 5, ce rôle d'étiquette est mieux tenu par les annotations (`@Override`, `@Deprecated`…) : on ne crée donc plus guère d'interfaces de marquage, et `Serializable` reste sur votre route parce qu'elle est plus ancienne.

Une interface, une seule méthode : l'interface fonctionnelle

```
@FunctionalInterface
public interface Calculateur {

    int calcule(int a, int b);
}
```

Une interface qui déclare exactement une méthode abstraite est dite *fonctionnelle*. Vous en avez écrit sans le savoir, comme `Calculateur` et `Translatable`, et le JDK en est plein : `Runnable`, `Comparator`, et `EventHandler`, que vous retrouverez à chaque bouton du module `FX`. L'annotation `@FunctionalInterface` est facultative ; comme `@Override`, elle fait refuser par le compilateur une interface qui ne respecterait pas la règle. Au module `LAM`, ces interfaces gagneront une écriture abrégée, l'expression lambda, qui tient sur une ligne au lieu d'une classe entière.

> **Exercices** - `G.11` à `G.13` (fil rouge GameHub)
>
> Les trois incréments appliquent le patron Observateur au fil rouge : l'interface `ObservateurPartie` et ses trois moments (le début, le coup, la fin), la `Partie` qui enregistre ses observateurs, puis un premier observateur, un journal qui affiche le déroulement. La liste d'observateurs vous est donnée : les collections sont le sujet du module `COL`.

Ce qu'il faut retenir

- Une interface énonce ce qu'une classe sait faire ; `implements` s'y engage.
- Une classe a une seule mère, et autant d'interfaces qu'elle veut.
- Classe abstraite : « est un », pour partager du code ; interface : « sait faire », pour découpler.
- Le type d'un paramètre déclare le besoin : le type le plus faible qui suffit est le bon.
- Quand un comportement traverse la hiérarchie, une interface supprime le test de type.
- L'Observateur : les intéressés s'abonnent, le sujet les prévient sans les connaître.

Le découplage est la leçon qui dépasse ce module : une classe qui ne dépend que de contrats peut être réutilisée, testée et remplacée sans qu'on rouvre le reste du programme.

Module PKG

PKG — Packages et visibilité

Qui a le droit de voir quoi

Les deux modules précédents ont montré comment des classes partagent du code et collaborent. Celui-ci pose une question laissée de côté jusqu'ici : qui a le droit de voir quoi.

Vous avez écrit jusqu'ici `private` devant les champs et `public` devant le reste, souvent par habitude. Ce module donne les raisons de ce choix, et y ajoute un niveau que la plupart des programmeurs emploient sans le savoir. La question n'est pas théorique : elle se pose à chaque classe de la quatrième question de l'examen, où il faut décider ce que la classe mère montre à ses filles, et ce qu'elle cache au reste du programme.

> **À la fin de ce module, vous saurez**
>
> - ranger une classe dans un package, et l'utiliser depuis un autre grâce à `import` ;
> - prévoir ce qu'un membre laisse voir selon son modificateur d'accès, y compris quand on n'en écrit aucun ;
> - choisir entre un champ public et un accesseur, en sachant ce que chacun engage ;
> - réduire la surface publique d'une classe au strict nécessaire ;
> - découper un programme en packages, et déplacer une classe sans rien casser ;
> - placer `protected` parmi les quatre niveaux d'accès.

PKG §1 — Un package, un dossier

Ranger une classe

```
// src/main/java/be/ephec/biblio/modele/Livre.java

package be.ephec.biblio.modele;

public class Livre {
    // ...
}
```

La ligne `package` ouvre toujours le fichier. Vous avez rencontré la règle au module `POO`, avec la classe `Personne` du package `progoo`. Elle se précise ici : chaque segment du nom correspond à un dossier, et un nom à plusieurs segments correspond à autant de dossiers imbriqués.

Le nom va du général au précis : l'organisation qui produit le code, le projet dans le contexte duquel il est produit, puis le module. L'usage est de commencer par le nom de domaine de l'organisation, à l'envers : `be.ephec` pour l'EPHEC. Deux équipes qui écrivent chacune une classe `Livre` ne se gênent alors jamais. Les exercices suivent la même logique, avec `be.ephec.java.pkg.e01` pour le premier exercice de ce module.

Un nom de package s'écrit entièrement en minuscules. IntelliJ crée les dossiers quand vous créez un package, et signale tout écart entre la ligne `package` d'un fichier et le dossier où il se trouve.

Le nom complet d'une classe

```
package be.ephec.biblio.service;

import be.ephec.biblio.modele.Livre;   // nom complet
import java.time.LocalDate;

public class Emprunt {
    private Livre livre;                // nom court
    private LocalDate dateRetour;
}
```

Le nom complet d'une classe commence par son package : celui de `Livre` est `be.ephec.biblio.modele.Livre`. L'instruction `import` permet ensuite d'écrire le nom court. Un `import` ne copie rien et ne charge rien : c'est un raccourci d'écriture, qui dit au compilateur où chercher quand il rencontre le nom court.

- **Une classe du même package ne nécessite pas d'`import`.**
- **Les classes de `java.lang` non plus**, comme `String`, `Math` ou `System` : le compilateur les importe d'office, ce qui explique que vous ne les ayez jamais importées.
- **`import java.util.*;`** importe toutes les classes du package `java.util`, mais pas celles de ses sous-packages.
- **Quand deux classes portent le même nom court**, `java.util.Date` et `java.sql.Date` par exemple, il faut écrire le nom complet d'au moins l'une des deux.

Un sous-package est un package à part

```
be.ephec.biblio             Main
be.ephec.biblio.modele      Livre, Auteur
be.ephec.biblio.service     Emprunt, Catalogue

// trois packages, sans lien de parente
```

Les noms de packages forment une arborescence, mais pas les droits. L'idée fausse est tenace : la hiérarchie existe dans les noms et dans les dossiers, mais pour le compilateur, `be.ephec.biblio` et `be.ephec.biblio.modele` sont deux packages aussi étrangers l'un à l'autre que `java.util` et `java.time`. Un package n'a aucun accès privilégié à ses sous-packages, ni l'inverse, et deux sous-packages voisins sont deux packages distincts.

L'arborescence est donc un rangement, pas une famille. Elle aide le lecteur à s'y retrouver, et c'est tout. Gardez-le en tête pour `PKG.2`, dont les deux classes vivent dans `e02.coffre` et `e02.controle`.

Deux dossiers, un seul package

```
src/main/java/be/ephec/biblio/modele/Livre.java
src/test/java/be/ephec/biblio/modele/LivreTest.java

// les deux fichiers commencent par la meme ligne :
package be.ephec.biblio.modele;
```

Maven range le code dans `src/main/java` et les classes de test dans `src/test/java`, pour ne pas livrer les seconds avec le premier. Mais une classe de test déclare le même package que la classe qu'elle teste, et Maven les compile dans ce même package. Le package est une notion du langage, le dossier racine une convention de Maven.

Une classe de test et la classe qu'elle teste sont donc voisines au sens strict, même si l'arborescence d'IntelliJ les affiche dans deux branches différentes. Toutes les classes de test du recueil sont construites ainsi, et la section suivante dit pourquoi ce voisinage compte.

PKG §2 — Le niveau sans mot-clé

Qui peut atteindre le membre d'une classe

```
package be.ephec.biblio.modele;

public class Livre {

    private String isbn;              // private : la classe Livre
    String cote() { ... }             // defaut : le package modele
    public String titre() { ... }     // public : tout le programme
}
```

Un mot de vocabulaire d'abord. Un *membre* d'une classe, c'est l'un de ses champs ou l'une de ses méthodes. Les mots `public`, `private` et les autres s'appellent des *modificateurs d'accès*. Ne les confondez pas avec les accesseurs, qui sont des méthodes comme `getTitre` (voir `PKG §3`).

- **Le niveau `private`** : le code de la classe seul.
- **Le niveau `public`** : tout le programme.
- **Sans modificateur** : le package, et nulle part ailleurs.

Les modificateurs s'appliquent aussi aux constructeurs et à la classe elle-même. Il en existe un quatrième, `protected`, que vous avez rencontré au module `ABS` : il attendra `PKG §6`, parce qu'il ne se comprend qu'avec l'héritage.

Retenez les mots exacts de la définition de `private` : *le code de la classe*, et non *l'objet*. `PKG.1` vous demande ce que cela implique quand deux objets de la même classe se rencontrent. Essayez avant de conclure.

Le niveau qu'on choisit sans le savoir

```
package be.ephec.biblio.modele;

class Fiche {                 // invisible hors du package
    String cote;              // idem
    Fiche(String cote) {      // idem
        this.cote = cote;
    }
}
```

Le niveau sans mot-clé s'obtient en n'écrivant rien. Il s'appelle *visibilité de package*, ou visibilité par défaut, et vaut pour les membres, les constructeurs et la classe elle-même. Le module `POO` l'avait signalé en passant : la classe `Personne`, déclarée sans modificateur, n'était visible que dans son package.

Son danger est que rien ne signale qu'on l'a choisi. Tant que toutes les classes vivent dans le même package, ce niveau ne se distingue pas de `public` : tout compile, partout. Il se comporte comme `private` pour les autres packages, et on ne le découvre que le jour où le programme est découpé.

Il a pourtant un usage délibéré, qui découle de `PKG §1` : une classe de test vit dans le même package que la classe testée. Un membre laissé à ce niveau lui est accessible, sans être ouvert au reste du programme.

> **Démonstration** - `Demo_PKG` : déménager une classe, et voir ce qui casse
>
> La démonstration part d'un programme qui fonctionne, entièrement rangé dans un seul package, et le découpe en trois avec l'outil de refactoring. La compilation se brise aussitôt. Chaque erreur désigne un endroit où le code s'appuyait sur une proximité qui vient de disparaître, et le compilateur pose à chaque fois une question de conception.

> **Exercices** - `PKG.1` Ce que voit une classe du même package
>
> L'exercice est une expérience : six lignes commentées, une par membre. Décommentez-les une à une et notez celles que le compilateur refuse. La liste attendue par le test se déduit de ce que vous avez observé, pas l'inverse. La première question de fin d'exercice porte sur le périmètre exact de `private`, et la réponse surprend souvent.

PKG §3 — Champ public ou accesseur

Ce qu'un champ public promet

```
public class Livre {

    public int exemplaires;
}
```

```
// ailleurs, sans que Livre le sache
livre.exemplaires = -3;
```

Un champ public donne deux droits à la fois, lire et écrire, à tout le monde et pour toujours. La classe perd la main sur sa propre donnée : rien ne l'empêche de valoir moins trois, et rien ne la prévient quand c'est le cas.

Le coût le plus lourd vient à long terme, parce que d'autres classes dépendent du champ. Le jour où vous voudrez le renommer, changer son type ou le remplacer par un calcul, toutes les classes qui l'emploient devront changer avec lui. Dans une équipe, ces classes ne sont pas toutes les vôtres.

Ce que l'accesseur garde

```
public class Livre {

    private int exemplaires;
    private int empruntes;

    public int getDisponibles() {       // hier : un champ
        return exemplaires - empruntes;
    }
}
```

Un accesseur public ne donne que la lecture. La classe peut calculer la valeur, la formater ou la refuser, et son corps change sans qu'aucun appelant ne le sache. La méthode `getDisponibles` retournait hier un champ ; elle fait aujourd'hui un calcul, et aucun appelant n'a eu à changer.

C'est la réponse à une question que beaucoup se posent depuis le module `POO` : à quoi bon un getter qui se contente de retourner un champ ? Le jour où il fait autre chose, personne ne doit le savoir, et c'est

précisément ce qu'il permet. Le getter et le champ public ne sont donc pas équivalents, même quand leur code se ressemble : l'un expose une donnée, l'autre un comportement, celui de la donner.

L'autre moitié de la règle `CC-OBJ.2` compte autant : un accesseur n'est pas une obligation. Écrire un getter et un setter pour chaque champ revient à rendre les champs publics avec plus de lignes. Chaque accesseur est une décision.

> **Exercices** - `PKG.2` Ce que voit une classe d'un autre package
>
> La première partie reprend l'expérience de `PKG.1`, avec une seule différence : le package. Notez lequel des six membres change de camp, et rapprochez-le de `PKG §2`. La seconde partie impose une contrainte qui est toute la consigne : le champ reste tel qu'il est. Le passer en `public` ferait compiler l'exercice en dix secondes, et l'un des deux tests est écrit pour le refuser.

PKG §4 — La surface publique

Chaque membre public est une promesse

On appelle *surface publique* l'ensemble de ce qu'une classe montre aux autres packages. C'est la partie de la classe sur laquelle d'autres s'appuient, et donc la seule qu'on ne peut plus modifier sans prévenir.

Une asymétrie décide de tout. Une méthode privée qu'on rend publique ne casse rien, puisque personne ne l'utilisait. Une méthode publique qu'on rend privée casse tous ceux qui l'appelaient, et on ne sait pas toujours qui ils sont. D'où la règle : partir du plus fermé, et n'ouvrir que sur un besoin réel. Ouvrir par défaut, c'est s'interdire de changer d'avis ; fermer par défaut laisse toutes les portes ouvertes.

Choisir, membre par membre

```
public class Catalogue {

    private Livre[] livres = new Livre[100];

    public void ajouter(Livre livre) { ... }
    public Livre chercher(String isbn) { ... }

    boolean estCoherent() { ... }       // pour les tests
    private void agrandir() { ... }
}
```

1. **Le niveau `private` d'abord** : c'est le cas le plus fréquent.
2. **La visibilité de package**, si une voisine en a besoin (par exemple pour les tests).
3. **Le niveau `public`** pour ce que les autres appellent.

Lisez la classe en partant de ce qu'un utilisateur du catalogue doit pouvoir faire : ajouter un livre, en chercher un. Ce sont les deux seules méthodes publiques. Le tableau et la façon de l'agrandir sont des détails de fabrication, et restent privés. La méthode `estCoherent` est un cas intermédiaire : elle vérifie que le catalogue n'est pas corrompu, ce qui intéresse les tests mais pas les utilisateurs. Une voisine, ici, c'est d'abord la classe de test : la visibilité de package la met à sa portée, et la cache au reste du programme.

Le choix se fait donc membre par membre, et une question suffit : qui a réellement besoin de ceci ?

Une seule classe publique par fichier

```
// fichier Catalogue.java

public class Catalogue { ... }

class Rayon { ... }             // permis : visible dans le package

public class Etagere { ... }    // erreur de compilation
```

La surface publique se mesure aussi en classes. Une classe est publique ou de package, rien d'autre : `private` et `protected` sont interdits sur une classe écrite directement dans son fichier. Une classe sans modificateur est un détail d'implémentation de son package, et les autres packages l'ignorent.

Un fichier contient au plus une classe publique, et porte son nom. Ce n'est pas une convention mais une règle du compilateur : il exige qu'une classe publique `Etagere` vive dans un fichier `Etagere.java`, et il le dit dans son message. Rien n'interdit de placer plusieurs classes non publiques dans un même fichier, mais l'usage est d'en écrire une par fichier, et IntelliJ s'y range.

> **Exercices** - `PKG.3` Réduire la surface publique d'une classe
>
> La classe fournie fonctionne, et tout y est public. Le programme fourni définit ce dont un utilisateur a réellement besoin : il n'appelle que trois opérations. Le travail consiste à fermer tout le reste sans que ce programme cesse de compiler. Comptez les promesses que faisait la version d'origine, et celles que fait la vôtre : la différence mesure le travail accompli. Voir `CC-CLS.2`.

PKG §5 — Découper en packages

Ranger par responsabilité

```
be.ephec.biblio.modele          Livre, Auteur
be.ephec.biblio.service         Emprunt, Catalogue
be.ephec.biblio.application     Main
```

Un package regroupe des classes qui changent pour les mêmes raisons. C'est le critère de la règle `CC-CLS.1`, appliqué à un groupe de classes. Un découpage courant compte trois couches :

Couche	Ce qu'elle contient	Elle change quand
le modèle	les données du domaine	le domaine change
les services	les règles de gestion	une règle change
l'application	le lancement, l'affichage	la façon de lancer ou d'afficher change

Le modèle est la couche la plus précieuse, parce qu'elle ne dépend d'aucune autre : la même classe `Livre` peut servir demain dans une application graphique sans qu'une ligne y change. Une quatrième couche, la persistance (ou *data*), écrit les données dans une base ou un fichier ; elle viendra avec le module `FIC`.

Les imports dessinent les dépendances

```
application  ->  service  ->  modele

// les classes du modele n'importent
// ni les services, ni l'application
```

Les `import` d'une classe disent de quoi elle dépend. Tant que tout vit dans un seul package, les dépendances existent mais ne se voient pas ; le découpage les rend lisibles.

Lisez la flèche comme « connaît ». L'application connaît les services, qui connaissent le modèle. Le modèle ne connaît personne, et c'est ce qui le rend réutilisable. Dans un programme bien découpé, les dépendances vont

toutes dans le même sens, et une dépendance qui remonte signale une responsabilité rangée dans la mauvaise couche.

Déplacer une classe avec l'outil

Pour déplacer une classe à l'aide d'IntelliJ : clic droit sur elle dans l'arborescence, puis Refactor, puis Move, ou directement `F6`. IntelliJ demande le package de destination, déplace le fichier, réécrit sa ligne `package` et met à jour tous les `import` du projet (voir `IDE.21`). Sur un clavier de Mac, la touche `F6` demande parfois d'appuyer en même temps sur `fn`.

C'est le même principe que les refactorings du module `ABS` : l'outil connaît toutes les références, vous non. Un déplacement à la main garde une ligne `package` fausse et oublie toujours un `import`. Un copier-coller laisse deux classes du même nom dans deux packages, ce qui compile parfaitement et ne fait jamais ce qu'on croit.

> **Exercices** - `PKG.4` Découper un programme en packages
>
> Quatre classes, un seul package, trois packages à créer. Faites-le avec `F6` : le dernier test vérifie qu'aucune classe ne subsiste à l'ancienne place, ce qui vous protège d'un copier-coller. Une fois le découpage fait, regardez les `import` de chaque classe et dessinez les flèches. Puis relisez `PKG §2` : certaines classes s'appuyaient sur la visibilité de package de leurs voisines. Le sont-elles encore ? Voir `CC-CLS.1`.

PKG §6 — Le quatrième niveau : `protected`

Le rappel du module `ABS`

```
package be.ephec.biblio.modele;

public abstract class Document {
    private final String cote;
    protected int pages;

    protected Document(String cote) {
        this.cote = cote;
    }
}
```

Vous avez employé `protected` au module `ABS`, pour un champ que les filles devaient lire, et pour les méthodes que la mère appelle et que les filles écrivent. Ce module lui donne sa place exacte parmi les quatre niveaux.

- **Le niveau `protected` ouvre un membre aux sous-classes.**
- **Il l'ouvre aussi au package**, ce qu'on oublie souvent : un membre `protected` est visible de tout le package, sous-classe ou non. Il est donc plus ouvert que la visibilité de package, jamais moins.
- **Son effet propre ne se voit que pour une sous-classe d'un autre package.** Dans le même package, `protected` ne change rien. C'est exactement la situation de `PKG.5`.

Les quatre niveaux

Niveau	Classe	Package	Sous-classe ailleurs	Reste
`private`	oui	non	non	non
(rien)	oui	oui	non	non
`protected`	oui	oui	oui	non
`public`	oui	oui	oui	oui

Le tableau se lit de haut en bas comme une échelle : chaque niveau ouvre tout ce qu'ouvre le précédent, plus une colonne. C'est ce qui permet de le reconstruire sans l'apprendre.

La colonne « sous-classe ailleurs » désigne une sous-classe rangée dans un autre package. Une sous-classe du même package est d'abord une voisine : elle relève de la deuxième colonne. Dans une interface, la question ne se pose presque pas : comme vu au module `ITF`, ses méthodes sans corps sont publiques sans qu'on l'écrive.

Choisir `protected`

```
public abstract class Document {

    private int pages;                   // la mere

    protected int getPages() {           // les filles
        return pages;
    }

    protected abstract String format();  // idem
}
```

Un membre `protected` est une promesse faite à toutes les sous-classes, présentes et futures. La règle de `PKG §4` s'applique donc sans changement : une sous-classe peut être écrite par quelqu'un d'autre, dans un autre package, des mois plus tard, et tout ce qui lui est ouvert ne peut plus changer sans la casser.

Un champ `protected` laisse les filles lire et écrire, comme le faisait le champ public de `PKG §3`, devant un public plus petit. Un champ privé et un accesseur `protected` laissent la mère maîtresse de la valeur. Le niveau `protected` trouve son meilleur usage dans les méthodes que la mère appelle et que les filles écrivent, comme au module `ABS`.

Ce n'est pas une interdiction : `PKG.5` vous fait écrire les deux versions, et la question de fin vous demande laquelle vous préférez, et pourquoi.

> **Exercices** - `PKG.5` `protected`, ou ce que l'héritage ouvre
>
> L'exercice place deux classes dans le même package, autour d'une classe mère rangée ailleurs. L'une en hérite, l'autre s'en sert seulement : les deux sont voisines et n'ont pourtant pas les mêmes droits. Le cœur de l'exercice est le tableau d'expériences : trois champs, deux classes, six essais. Notez le message exact du compilateur à chaque refus, puis écrivez en une phrase ce que `protected` ouvre. `ABS.4` vous demandait quelle visibilité donner aux accesseurs intermédiaires de la hiérarchie : vous avez maintenant tout ce qu'il faut pour répondre.

Ce qu'il faut retenir

- Un package est un dossier ; un sous-package est un package à part, sans droits sur son parent.
- Sans modificateur, un membre est visible du package, et seulement du package.
- Un champ public expose une donnée ; un accesseur, un comportement. Et chaque accesseur est une décision.
- La surface publique se réduit au strict nécessaire : partir du plus fermé, n'ouvrir que sur un besoin réel.
- Les packages se découpent par responsabilité, et les dépendances vont toutes dans le même sens.
- `protected` ouvre aux sous-classes, et aussi au package.

La leçon qui dépasse ce module tient en une asymétrie : on peut toujours ouvrir plus tard, on ne peut presque jamais refermer.

Module COL

COL — Collections

Ranger des objets dans une structure qui grandit, et choisir la structure appropriée

Un tableau a une taille fixe, choisie à sa création. Le module `TAB` l'avait annoncé : quand cette limite devient gênante, il faut une collection. Une collection grandit toute seule, et chacune tient une promesse différente.

Ce module répond à deux questions : comment ranger des objets dans une structure qui grandit, et laquelle choisir. La seconde compte plus que la première. La syntaxe s'apprend en une heure ; le bon choix évite des dizaines de lignes de vérifications. C'est le module le plus fourni du recueil, avec sept exercices, et le fil rouge en dépend aussi : le jeu de bataille de GameHub range ses cartes dans des collections.

À la fin de ce module, vous saurez

- déclarer une collection par son interface, et choisir son implémentation ;
- utiliser une liste (`List`), un ensemble (`Set`), une table associative (`Map`) et une file à deux bouts (`Deque`) ;
- choisir entre elles d'après la promesse que chacune tient ;
- expliquer comment un ensemble décide que deux éléments sont égaux ;
- passer un comportement à une méthode de collection, sous la forme d'un objet ;
- lire et construire une collection emboîtée.

COL §1 — Une structure qui grandit

> **Démonstration** - `Demo_COL` : choisir une collection en se trompant d'abord
>
> La démonstration part d'une liste d'invités où le même nom figure deux fois. Une première correction est proposée, puis une seconde, beaucoup plus courte, qui ne tient qu'en un mot changé. Le même remède sert ensuite à un deuxième problème. Retenez la méthode plutôt que la syntaxe : on ne choisit pas une collection par habitude, on la choisit sur la promesse qu'elle tient. Et quand la structure porte une règle, personne n'a plus besoin d'y penser.

Ce qu'on écrit entre chevrons

```
List<String> titres = new ArrayList<>();

titres.add("Germinal");
titres.add(42);                  // refuse a la compilation

String premier = titres.get(0);  // aucune conversion
```

`List<String>` se lit « une liste de `String` ». Le type entre chevrons (`<` et `>`) est celui des éléments de la collection. C'est un paramètre : la même interface `List` sert pour des chaînes, des nombres ou des livres. Le compilateur vérifie à chaque ajout que l'élément est du bon type, et il connaît le type de ce qui sort. L'erreur apparaît donc à la compilation, pas à l'exécution.

À droite du `new`, les chevrons vides, appelés *diamant*, évitent de répéter le type : le compilateur le déduit de la déclaration. Vous apprenez ici à lire cette notation ; l'écriture d'une classe qui reçoit un type en paramètre, comme `List` le fait, est le sujet du module `GEN`.

Des objets, pas des types primitifs

```
List<Integer> annees = new ArrayList<>();   // List<int> : refuse

annees.add(1885);                // auto-boxing de int vers Integer
int premiere = annees.get(0);    // unboxing de Integer vers int
```

Une collection ne contient que des objets. On écrit donc `Integer` et non `int` entre les chevrons. Le module `POO` a présenté ces classes enveloppes, et la conversion automatique entre les deux formes, l'autoboxing (voir `POO §7`). Elles prennent ici leur raison d'être.

En pratique, vous écrivez `Integer` entre les chevrons et vous oubliez la différence. Avec deux exceptions :

- **Comparer deux `Integer` avec `==` compare deux objets**, pas deux nombres : écrivez `equals`, ou convertissez d'abord en `int`.
- **Un `Integer` peut valoir `null`**, ce qu'un `int` ne peut pas : la conversion échoue alors à l'exécution. `COL §4` montre où ce cas se présente.

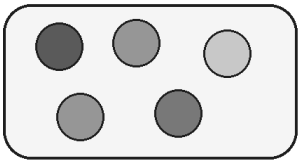
Déclarer par l'interface

```
List<String> titres = new ArrayList<>();
Set<String> auteurs = new HashSet<>();
Map<String, Integer> nombrePages = new HashMap<>();
```

`List`, `Set` et `Map` sont des interfaces ; `ArrayList`, `HashSet` et `HashMap` sont des classes qui les implémentent. On déclare la variable avec l'interface, ce que la variable promet, et on choisit la classe au moment du `new`, comment la promesse est tenue. Changer d'implémentation ne change alors qu'un mot.

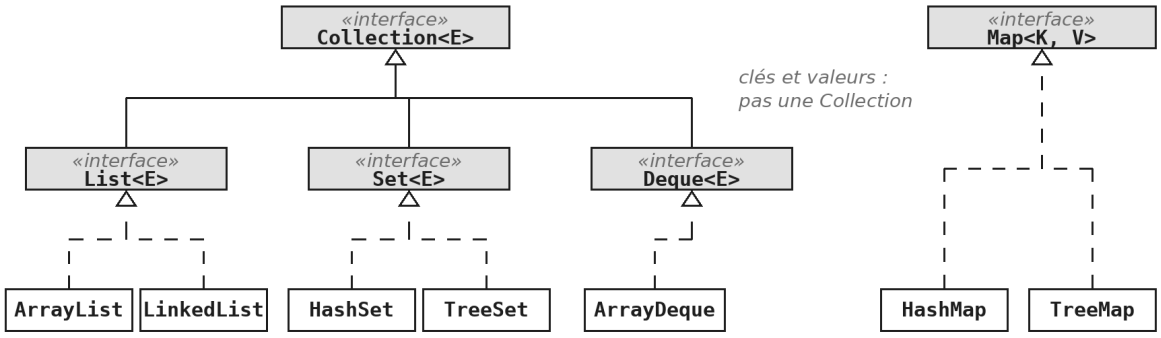
C'est le principe du module `ITF` : le type le plus faible qui suffit (voir `ITF §4`). Le reste du programme ne connaît que la promesse, et l'implémentation peut changer sans qu'il le sache. Toutes ces interfaces et ces classes vivent dans le package `java.util`, qu'il faut importer.

Une *collection* rassemble des objets dans une seule structure, qui grandit et rétrécit au fil des ajouts et des retraits. Chaque sorte de collection ajoute à cette idée sa propre promesse : garder un ordre, refuser les doublons, ranger par clé, ou ne s'ouvrir que par ses bouts. Les sections suivantes les présentent une à une.



Une collection : des objets rassemblés, sans autre promesse

La carte ci-dessous situe les interfaces les unes par rapport aux autres, avec les classes qui les implémentent.



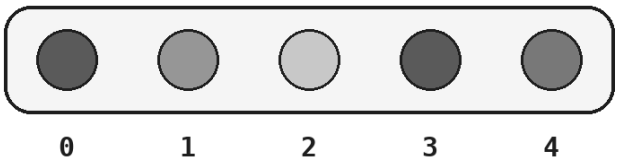
trait plein : hérite de l'interface ; pointillé : la classe l'implémente

Les principales interfaces de collections et leurs implémentations

COL §2 — Une liste, des positions

Une liste (`List`) est une collection *ordonnée* : chaque élément y occupe une position, numérotée à partir de `0`, et l'ordre dans lequel on les a rangés est conservé. Elle accepte les doublons : la même valeur peut y figurer plusieurs fois, à des positions différentes. C'est la collection la plus proche du tableau, sans sa taille fixe : elle grandit à chaque ajout et rétrécit à chaque retrait.

On la choisit quand l'ordre compte, ou quand on a besoin de retrouver un élément par sa position : une liste de lecture, les coups d'une partie, les lignes d'un fichier.



Une liste : des positions numérotées à partir de 0, et des doublons possibles

Les opérations d'une liste

```
List<String> titres = new ArrayList<>();
titres.add("Germinal");
titres.add("Nana");
titres.add(0, "Candide");          // insere en tete

titres.size();                     // 3
titres.get(1);                     // "Germinal"
titres.contains("Nana");           // true
titres.remove("Nana");             // retire l'element
```

Une liste garde l'ordre d'ajout, et accepte les doublons. C'est le remplaçant direct du tableau : des positions numérotées à partir de 0, comme `t[0]`, mais sans taille fixe. La méthode `add` ajoute à la fin, ou à la position demandée si on la donne.

Méthode	Ce qu'elle fait
`get(i)`	retourne l'élément en position `i` ; `IndexOutOfBoundsException` si elle n'existe pas
`set(i, valeur)`	remplace l'élément en position `i` (rappel : les indices commencent à `0`)
`indexOf(valeur)`	retourne la position d'un élément, ou `-1` s'il est absent
`remove(...)`	retire une position, ou un élément

> **Attention** - `remove` sur une liste d'`Integer`
>
> La méthode `remove` existe en deux versions : `remove(int)` retire la position, `remove(Object)` retire l'élément. Sur une `List<Integer>`, `remove(2)` retire donc l'élément en position 2 (donc le troisième), pas le nombre 2.

Parcourir une collection

```
for (String titre : titres) {
    System.out.println(titre);
}

System.out.println(titres);      // [Candide, Germinal]
```

La boucle `for-each`, vue au module `TAB` pour les tableaux, accepte n'importe quelle collection : une liste, un ensemble, et les vues d'une table présentées en `COL §4`. Elle parcourt les éléments dans l'ordre que la collection garantit, s'il y en a un.

Contrairement à un tableau, dont l'affichage produisait un code illisible, une collection s'affiche entre crochets, éléments séparés par des virgules. C'est commode pour vérifier ce que contient une structure en cours de mise au point.

Ce que la boucle for-each utilise : `Iterable` et `Iterator`

```
Iterator<String> it = titres.iterator();
while (it.hasNext()) {
    String titre = it.next();
    System.out.println(titre);
}
```

La boucle `for-each` n'est pas réservée aux collections du JDK : elle accepte tout objet qui implémente l'interface `Iterable`, dont hérite `Collection`. Cette interface ne promet qu'une chose : savoir fournir un `Iterator`, un objet qui parcourt les éléments un par un. Il répond à deux questions : `hasNext()`, reste-t-il un élément ? et `next()`, lequel ?

Le code ci-dessus est, à peu de chose près, ce que le compilateur écrit pour vous quand il rencontre une boucle `for-each` sur une collection. Vous n'aurez presque jamais à écrire un itérateur vous-même. Il faut toutefois le connaître pour un cas précis, que `COL §5` présente : retirer des éléments pendant un parcours.

`ArrayList` ou `LinkedList`

Les deux classes tiennent exactement la même promesse : celle de `List`. Un programme qui déclare ses variables avec l'interface fonctionne avec l'une comme avec l'autre, et le test de `COL.5` le vérifie justement sur une `LinkedList`.

Elles diffèrent par leur fonctionnement interne, donc par leur rapidité selon l'usage. `ArrayList` range ses éléments dans un tableau qu'elle agrandit elle-même, ce qui rend `get(i)` immédiat. `LinkedList` chaîne ses

éléments les uns aux autres, ce qui rend immédiats les ajouts et retraits aux deux bouts, mais lent `get(i)`. Prenez `ArrayList` sauf raison précise.

`Collection` et `Collections`

```
Collections.sort(titres);            // trie la liste elle-meme
Collections.shuffle(titres);         // la melange
String premier = Collections.min(titres);
int fois = Collections.frequency(titres, "Nana");
```

Les deux noms ne diffèrent que d'une lettre, et la confusion est classique.

- **L'interface `Collection`** est l'interface mère de `List` et de `Set` : elle déclare ce que toutes les collections savent faire, `add`, `remove`, `size`, `contains`.
- **La classe `Collections`**, au pluriel, est une classe utilitaire, comme `Math` : uniquement des méthodes statiques, qui travaillent sur des collections.

Les méthodes `sort` et `shuffle` reçoivent une `List`, parce qu'un ensemble n'a pas de positions à réordonner. Elles modifient la liste reçue et ne retournent rien. Les méthodes `min` et `max` retournent le plus petit et le plus grand élément, `frequency` compte les apparitions d'une valeur. La Javadoc de `Collections` en contient une vingtaine d'autres.

Nommer ses bornes

```
private static final int MINIMUM = 0;
private static final int MAXIMUM = 1000;

System.out.printf("Nombres entre %d et %d%n", MINIMUM, MAXIMUM);
int tire = MINIMUM + hasard.nextInt(MAXIMUM - MINIMUM + 1);
```

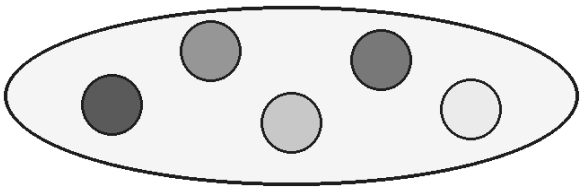
Une première version de ce programme écrivait `hasard.nextInt(1001)`. Le lecteur devait deviner d'où sortait 1001, et pourquoi pas 1000. La réponse tient au module `ENU` : `nextInt(n)` de la classe `Random` retourne un entier de `0` inclus à `n` exclu, et il faut donc compter les valeurs de l'intervalle.

Avec deux constantes, le calcul se lit au lieu de se deviner. Le gain le plus utile est ailleurs : les mêmes constantes servent au message. Le jour où l'intervalle change, le programme ne peut plus annoncer 0 à 1000 en tirant jusqu'à 500. Les squelettes de `COL.1`, `COL.5` et `COL.6` vous donnent ces constantes : servez-vous-en partout. C'est la règle `CC-NOM.7`.

> **Exercices** - `COL.1` Deux façons d'être trié, puis `COL.3` Lire des noms et les trier
>
> `COL.1` oppose deux manières d'obtenir des nombres triés : trier une liste, ou les ranger dans une structure qui reste triée, ce que la démonstration a montré. Lisez bien la consigne de la première méthode : la liste reçue ne doit pas être modifiée, et `Collections.sort` modifie celle qu'elle reçoit. `COL.3` remplit une liste à partir d'un `Scanner`, que la classe fournie `SourceDeTexte` vous donne déjà ouvert : l'exercice porte sur les collections, pas sur les fichiers. Les deux exercices finissent sur une question d'ordre.

COL §3 — Un ensemble, sans doublons

Un ensemble (`Set`) est une collection qui ne contient jamais deux fois la même valeur, comme un ensemble au sens mathématique. Il n'a pas de positions : on ne demande pas « le troisième élément », on demande « cette valeur y est-elle ? ». On le choisit quand l'unicité est la règle : les auteurs d'une bibliothèque, les cours suivis par un étudiant, les numéros déjà tirés.



Un ensemble : chaque valeur une seule fois, sans position

Ce qu'un ensemble refuse

```
Set<String> auteurs = new HashSet<>();

auteurs.add("Zola");              // true
auteurs.add("Zola");              // false : deja present
auteurs.size();                   // 1
auteurs.contains("Zola");         // true
```

Un ensemble ne garde qu'un exemplaire de chaque valeur. Ajouter une valeur présente ne lève aucune erreur : rien ne se passe, et `add` retourne `false` pour le signaler. C'est ce que la démonstration a exploité.

Un ensemble n'a pas de positions : ni `get(i)`, ni `add(i, x)`. On l'interroge avec `contains`, on le parcourt avec la boucle `for-each`, mais on ne demande jamais « le troisième élément ». Ce n'est pas une limite : c'est ce qui lui permet d'autres services, que `COL.6` vous fera découvrir.

Qui décide que deux éléments sont égaux

```
record Exemplaire(String titre, int numero) { }
```

```
Set<Exemplaire> rayon = new HashSet<>();
rayon.add(new Exemplaire("Nana", 1));
rayon.add(new Exemplaire("Nana", 1));

rayon.size();                     // 1
```

Le module `ENU` l'avait annoncé en présentant les records : la question de l'égalité reviendrait. Pour savoir si une valeur est déjà présente, un `HashSet` appelle `hashCode` puis `equals` sur ses éléments.

- **`String` et `Integer`** les définissent comme on s'y attend : deux chaînes de mêmes lettres sont égales.
- **Un `record`** les définit aussi, en comparant ses composants : les deux exemplaires ci-dessus ne comptent donc qu'une fois.
- **Une classe ordinaire** qui ne redéfinit rien compare des références : deux objets construits séparément sont différents, même avec les mêmes valeurs, et l'ensemble en garde deux.

Écrire `equals` et `hashCode` à la main est possible, toujours ensemble[3], et IntelliJ les génère. Ce cours ne vous le demandera pas : un `record` suffit dans tous les exercices. La même règle vaut pour les clés d'une table, en `COL §4`.

`Hash` ou `Tree`

	`HashSet`, `HashMap`	`TreeSet`, `TreeMap`
Ordre de parcours	aucun ordre garanti	trié en permanence
Les éléments doivent	définir `equals` et `hashCode`	savoir se comparer

[3]Les deux méthodes sont liées par un contrat : si `a.equals(b)` retourne `true`, alors `a.hashCode()` et `b.hashCode()` doivent retourner le même nombre. L'inverse n'est pas exigé : deux objets différents peuvent avoir le même code de hachage, ce qui ralentit un peu la recherche sans la fausser. Un `HashSet` cherche d'abord dans la case que désigne `hashCode`, puis compare avec `equals` : si deux objets égaux avaient des codes différents, il les rangerait dans deux cases, et ne verrait jamais le doublon. C'est pourquoi on les redéfinit toujours ensemble.

	`HashSet`, `HashMap`	`TreeSet`, `TreeMap`
Quand le choisir	par défaut	quand l'ordre compte

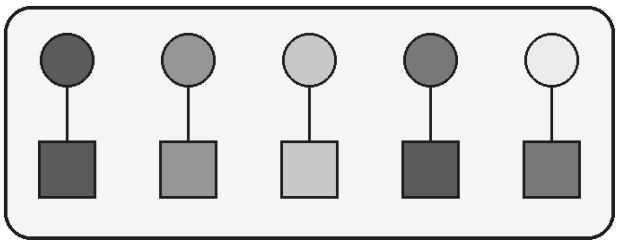
Les ensembles (`Set`) et les tables associatives (`Map`) existent chacun en deux implémentations principales, et la différence est la même des deux côtés. Les classes `Hash` rangent les éléments selon leur code de hachage, ce qui les rend très rapides mais ne garantit aucun ordre de parcours. Les classes `Tree` les rangent dans un arbre trié : le parcours se fait toujours dans l'ordre, au prix d'un peu de vitesse.

Savoir se comparer est le cas des chaînes, des nombres, des dates et des énumérations, dont l'ordre est celui de déclaration (voir `ENU §2`). Pour vos propres classes, il faut leur apprendre à se comparer, ce que `COL §6` montre : sans cela, un `TreeSet` de records lève une exception au premier ajout. Une troisième implémentation, `LinkedHashSet` et `LinkedHashMap`, garde l'ordre d'ajout ; vous ne la croiserez pas dans les exercices.

> **Exercices** - `COL.6` Un ensemble sans doublons
>
> L'exercice fait constater les deux services d'un ensemble : il ne garde qu'un exemplaire de chaque valeur, et il répond très vite à `contains`. La première question demande pourquoi une liste ne le peut pas : cherchez avant de lire la Javadoc. Le squelette du programme vous donne trois constantes, le nombre de tirages et les deux bornes, et le calcul de `COL §2` sert de nouveau. Le résultat de la deuxième question surprend presque toujours.

COL §4 — Une table, des clés

Une table associative (`Map`) range des couples : à chaque *clé* correspond une *valeur*. On ne la parcourt pas pour chercher : on donne la clé, et la table retourne la valeur. Les clés sont uniques, comme les éléments d'un ensemble ; les valeurs peuvent se répéter. On la choisit chaque fois qu'on retrouve une information par un identifiant : un dictionnaire, un annuaire, un stock par référence.



Une table : chaque clé, en haut, mène à sa valeur, en bas

Ranger et retrouver par clé

```
Map<String, Integer> nombrePages = new HashMap<>();
nombrePages.put("Germinal", 591);
nombrePages.put("Candide", 160);

nombrePages.get("Candide");            // 160
nombrePages.containsKey("Nana");       // false
nombrePages.size();                    // 2
```

Une table associative, `Map` en Java, associe une valeur à chaque clé. On y range avec `put`, on retrouve avec `get` en donnant la clé, et on ne parcourt jamais pour chercher. Un dictionnaire, un annuaire, un stock par référence : tout ce qui se consulte par un identifiant est une table. Les chevrons contiennent deux types : celui des clés, celui des valeurs.

Les clés sont uniques, comme les éléments d'un ensemble, et pour la même raison : elles sont comparées par `equals` et `hashCode`, ou rangées dans un arbre pour une `TreeMap`. Un second `put` avec la même clé remplace la valeur. Les valeurs, elles, peuvent se répéter. Remarquez qu'une table n'est pas une `Collection` : l'interface `Map` n'hérite pas de `Collection`, et n'a pas de méthode `add`.

Une clé absente

```
Integer inconnu = nombrePages.get("Nana");           // null
int nombre = nombrePages.getOrDefault("Nana", 0);    // 0

int plantage = nombrePages.get("Nana");    // NullPointerException
```

Une clé inconnue n'est pas une erreur pour une table : `get` retourne simplement `null`, et c'est au programme de le prévoir. La dernière ligne est le cas annoncé en `COL §1` : la valeur `null` ne peut pas être convertie en `int`, et la conversion automatique échoue à l'exécution, par une `NullPointerException` qui ne désigne pas clairement la cause.

La méthode `getOrDefault` évite le problème : elle retourne la valeur si la clé existe, et la valeur par défaut fournie sinon. Elle sert souvent quand une absence a un sens naturel : zéro pour un compte, une liste vide pour un ensemble de résultats.

Trois façons de parcourir une table associative

```
for (String titre : nombrePages.keySet()) { ... }

for (int nombre : nombrePages.values()) { ... }

for (Map.Entry<String, Integer> entree : nombrePages.entrySet()) {
    System.out.println(entree.getKey() + " : " + entree.getValue());
}
```

Une table ne se parcourt pas directement, mais elle offre trois vues, que la boucle `for-each` parcourt : `keySet` retourne l'ensemble des clés, `values` la collection des valeurs, et `entrySet` l'ensemble des couples. Chaque couple est un objet de type `Map.Entry`, avec ses deux accesseurs `getKey` et `getValue`.

Quand on a besoin des deux à la fois, c'est la vue à préférer : parcourir les clés puis appeler `get` pour chacune refait une recherche à chaque tour. Ces vues suivent l'ordre de la table : aucun pour une `HashMap`, l'ordre des clés pour une `TreeMap`.

> **Exercices** - `COL.2` Un annuaire, puis `COL.4` Compter les mots d'un texte
>
> `COL.2` range des fiches par nom, puis pose la question qui fait l'exercice : peut-on obtenir les fiches dans l'ordre des noms ? Répondez d'abord sans changer de table, puis en en changeant : la seconde réponse ne demande qu'une ligne, et le test passe toujours. `COL.4` est l'usage le plus courant d'une table : accumuler un décompte, où `getOrDefault` fait gagner trois lignes. Les deux exercices finissent sur une question de conception ; celle de `COL.2`, sur la méthode `chercher` qui retourne `null`, trouvera sa réponse en `COL.7`.

COL §5 — Passer un comportement

Des méthodes ajoutées aux interfaces

```
// dans l'interface Map<K, V>
default V getOrDefault(Object cle, V parDefaut) { ... }
```

Le module `ITF` l'avait annoncé : vous utiliseriez des méthodes par défaut sans le savoir, dès ce module (voir `ITF §6`). Vous venez de le faire avec `getOrDefault`. Java 8 a ajouté de telles méthodes à `Collection` et à `Map`, parmi elles `getOrDefault`, `removeIf` et `computeIfAbsent`.

Le problème qu'elles ont résolu est celui d'`ITF §6`, à l'échelle du monde. Ajouter une méthode abstraite à `Map` aurait cassé toutes les classes qui l'implémentent, y compris celles écrites hors du JDK. Une méthode par défaut contient son propre code : chaque table l'a reçue d'office. Deux de ces méthodes ont une particularité, qui fait le sujet de cette section : elles reçoivent un comportement en paramètre.

Retirer pendant un parcours

```
for (String titre : titres) {
    if (titre.length() > 20) {
        titres.remove(titre);   // ConcurrentModificationException
    }
}
```

Retirer un élément d'une collection pendant qu'une boucle `for-each` la parcourt lève une `ConcurrentModificationException`. L'itérateur caché dans la boucle détecte que la collection a changé sous ses pieds, et refuse de continuer plutôt que de sauter un élément en silence.

Deux remèdes. Le premier écrit l'itérateur explicitement, et lui demande de retirer l'élément qu'il vient de fournir, ce qu'il sait faire sans se perdre :

```
Iterator<String> it = titres.iterator();
while (it.hasNext()) {
    if (it.next().length() > 20) {
        it.remove();
    }
}
```

Le second, plus court, est la méthode `removeIf`, qui fait exactement ce travail : c'est l'objet de la sous-section suivante.

Une méthode qui reçoit un critère

```
// dans l'interface Collection<E>
default boolean removeIf(Predicate<? super E> filtre) { ... }

// dans l'interface Predicate<T>
boolean test(T valeur);
```

La méthode `removeIf` parcourt la collection et appelle `test` sur chaque élément ; ceux pour lesquels `test` retourne `true` sont retirés. Elle ignore tout du critère : elle sait seulement qu'on lui a donné un objet capable de répondre oui ou non.

`Predicate` est une interface fonctionnelle, au sens d'`ITF §6` : elle possède une seule méthode abstraite. Elle appartient au package `java.util.function`, qui en contient d'autres, dont `Function`, en `COL §7`. Le `? super E` se lit « un critère qui sait tester des `E` » ; il autorise par exemple un critère écrit pour des `Object` sur une liste de `String`, et son écriture sera détaillée au module `GEN`.

Écrire le critère dans une classe

```
public class TitreLong implements Predicate<String> {

    private static final int LONGUEUR_MAX = 20;

    @Override
    public boolean test(String titre) {
        return titre.length() > LONGUEUR_MAX;
    }
}
```

```
titres.removeIf(new TitreLong());
```

Vous connaissez déjà tout ce qui est écrit ici : une classe qui implémente une interface, une méthode redéfinie, et un objet passé en paramètre à une méthode qui ne connaît que l'interface. C'est `ITF §1`, avec une interface que le JDK a écrite à votre place.

La classe porte un nom qui dit ce qu'elle teste. On peut la réutiliser, et la tester séparément. Le critère, lui, est une constante nommée : la règle de `COL §2` vaut partout. Au module `LAM`, ce même critère s'écrira en une ligne, sans classe.

> **Exercices** - `COL.5` Filtrer avec un prédicat
>
> L'exercice applique exactement cette sous-section, sur des nombres pairs. La classe `EstPair` est testée seule, puis employée par deux méthodes, dont une qui compte sans rien supprimer. La première question vous fait essayer de supprimer les pairs autrement, avec une boucle : faites-le vraiment, ce que vous obtiendrez explique pourquoi `removeIf` existe.

COL §6 — Comparer et trier

Une seule question, deux réponses : comment Java sait-il dans quel ordre ranger des objets ? Un `TreeSet`, une `TreeMap` et le tri d'une liste ont tous besoin de savoir, pour deux éléments quelconques, lequel vient avant l'autre. Soit la classe le définit elle-même, c'est son ordre *naturel* ; soit on fournit un ordre de l'extérieur, un parmi d'autres.

L'ordre naturel : `Comparable`

Un `TreeSet` ou une `TreeMap` doit savoir, pour deux éléments quelconques, lequel vient avant l'autre. Les chaînes, les nombres, les dates et les énumérations le savent déjà, parce que leurs classes implémentent l'interface `Comparable`. Une classe que vous écrivez peut faire de même :

```
record Exemplaire(String titre, int numero)
        implements Comparable<Exemplaire> {

    @Override
    public int compareTo(Exemplaire autre) {
        return Integer.compare(numero, autre.numero);
    }
}
```

L'interface ne déclare qu'une méthode, `compareTo`, et son résultat se lit par son signe, pas par sa valeur :

`a.compareTo(b)` retourne	Ce que cela veut dire
un nombre négatif	`a` vient avant `b`
zéro	`a` et `b` sont au même rang
un nombre positif	`a` vient après `b`

Pour comparer deux nombres, `Integer.compare` (ou `Double.compare`) fait le travail ; pour comparer deux chaînes, leur propre `compareTo`. Évitez la soustraction `numero - autre.numero`, qu'on voit souvent : elle déborde pour de très grands écarts, et retourne alors le mauvais signe[4].

C'est l'ordre *naturel* de la classe : celui qu'un `TreeSet` et `Collections.sort` emploient quand on ne leur dit rien. Il doit rester cohérent avec `equals` : un `TreeSet` considère comme doublons deux éléments dont `compareTo` retourne zéro, même s'ils ne sont pas égaux au sens d'`equals`.

Quand on veut un autre ordre que l'ordre naturel, ou quand on ne peut pas modifier la classe, on fournit l'ordre de l'extérieur : c'est la sous-section suivante.

[4]Par exemple, avec `numero` égal à `Integer.MAX_VALUE` (2 147 483 647, le plus grand `int`) et `autre.numero` égal à `-1`, la soustraction devrait donner 2 147 483 648. Ce nombre ne tient pas dans un `int` : le calcul déborde, et le résultat devient -2 147 483 648, le plus petit `int`. Le signe est négatif, et `compareTo` affirme que le plus grand des deux nombres vient avant l'autre. `Integer.compare(numero, autre.numero)` retourne bien un nombre positif.

Un ordre fourni de l'extérieur : `Comparator`

```
public class ParLongueur implements Comparator<String> {

    @Override
    public int compare(String a, String b) {
        return Integer.compare(a.length(), b.length());
    }
}
```

```
titres.sort(new ParLongueur());                    // une liste
Set<String> rayon = new TreeSet<>(new ParLongueur());  // un ensemble trie
```

`Comparator` est une autre interface fonctionnelle : sa méthode `compare` reçoit les deux éléments, et retourne un nombre dont le signe se lit comme celui de `compareTo`. La différence est dans la place : `Comparable` est implémentée par la classe elle-même, et donne *son* ordre ; un `Comparator` est un objet à part, et donne *un* ordre parmi d'autres. On peut en écrire autant qu'on veut pour la même classe : par titre, par longueur, par année.

On le passe à `sort`, pour trier une liste, ou au constructeur d'un `TreeSet` ou d'une `TreeMap`, pour qu'ils rangent selon cet ordre. C'est le mécanisme de `COL §5` : un comportement passé en paramètre, comme le `Predicate` de `removeIf`. Au module `LAM`, ce même comparateur s'écrira en une ligne, sans classe.

`Comparable` ou `Comparator` ?

Choisissez	Quand
`Comparable`	il existe un ordre évident, qu'on voudra presque toujours
`Comparator`	on veut un autre ordre, ponctuellement
`Comparator`	on veut plusieurs ordres pour la même classe
`Comparator`	on ne peut pas modifier la classe, comme `String`

La question qui tranche : l'ordre appartient-il à la classe, ou à l'usage qu'on en fait ? Les deux coexistent sans problème : une classe peut avoir son ordre naturel, et des comparateurs pour les autres besoins.

> **Exercices** - `COL.8` Trois ordres pour une bibliothèque
>
> L'exercice met en pratique toute la section : un record `Livre` qui reçoit son ordre naturel par `compareTo`, deux comparateurs pour les autres ordres, et trois méthodes qui trient une copie de la liste. Pour les années, `Integer.compare`, jamais une soustraction. Faites l'expérience de la première question : retirez le titre du comparateur par auteur, et comptez les livres qui restent dans le `TreeSet`. Un comparateur ne dit pas seulement dans quel ordre ranger : pour un `TreeSet`, il dit aussi ce qui est un doublon.

COL §7 — Des collections emboîtées

Lire un type emboîté

```
Map<String, List<String>> titresParAuteur = new HashMap<>();

List<String> deZola = titresParAuteur.get("Zola");    // null
```

Une collection peut contenir des collections. `Map<String, List<String>>` se lit en partant de l'extérieur : une table, dont les clés sont des chaînes, et dont chaque valeur est une liste de chaînes. Ici, les titres de chaque auteur. La déclaration documente la structure à elle seule : sans lire une ligne de plus, on sait qu'un auteur a plusieurs titres, et qu'ils sont ordonnés.

La table, en revanche, ne crée rien d'elle-même. Tant qu'aucun titre de Zola n'a été rangé, il n'existe aucune liste pour Zola, et `get` retourne `null`. Ajouter un titre demande donc deux étapes : obtenir la liste, en la créant si elle n'existe pas encore, puis y ajouter.

Créer au premier usage

```
public class NouvelleListe implements Function<String, List<String>> {

    @Override
    public List<String> apply(String auteur) {
        return new ArrayList<>();
    }
}
```

```
titresParAuteur.computeIfAbsent("Zola", new NouvelleListe())
               .add("Germinal");
```

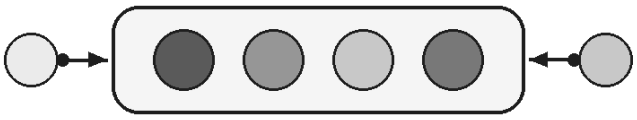
La méthode `computeIfAbsent` fait les deux étapes en une. Si la clé a déjà une valeur, elle la retourne. Sinon, elle appelle la méthode `apply` de l'objet reçu, range le résultat dans la table, et le retourne. On ajoute ensuite le titre à la liste obtenue, qu'elle soit ancienne ou neuve.

`Function<T, R>` est une interface fonctionnelle de `java.util.function`, comme `Predicate` : sa méthode `apply` reçoit un `T` et retourne un `R`. Le mécanisme est celui de `COL §5` : `removeIf` recevait un objet qui sait tester, `computeIfAbsent` reçoit un objet qui sait fabriquer. La méthode `apply` reçoit la clé, ici le nom de l'auteur ; elle n'en a pas besoin pour créer une liste vide, mais l'interface le lui donne.

> **Exercices** - `COL.7` Une table de collections
>
> L'exercice emboîte une table et des ensembles : à chaque étudiant, les cours qu'il suit. Le fascicule explique les deux décisions de conception, un ensemble plutôt qu'une liste, un ensemble vide plutôt que `null` (`CC-ERR.2`) ; comparez la seconde avec la méthode `chercher` de `COL.2`. Vous y écrivez aussi une classe `NouvelEnsemble`, sur le modèle de `NouvelleListe`. La première question vous demande d'écrire la même méthode sans `computeIfAbsent` : faites-le, pour mesurer ce qu'elle épargne. La dernière porte sur ce que `coursDe` livre à son appelant (`CC-OBJ.2`).

COL §8 — Une file, deux bouts

Une file à deux bouts (`Deque`) est une suite d'éléments qu'on ne touche que par ses extrémités : on ajoute et on retire au début ou à la fin, jamais au milieu. C'est le modèle d'une file d'attente, qu'on sert par un bout et qu'on allonge par l'autre, ou d'une pioche de cartes.



Une file à deux bouts : on n'y touche que par les extrémités

Une file à deux bouts

```
Deque<String> attente = new ArrayDeque<>();

attente.addLast("Zola");
attente.addLast("Hugo");
attente.addFirst("Sand");        // passe devant

System.out.println(attente);     // [Sand, Zola, Hugo]
```

Une `Deque`, file à double entrée, se prononce « dèque ». C'est une liste qu'on ne manipule que par ses deux bouts : on ajoute et on retire aux deux extrémités, jamais au milieu. Ce n'est pas une limite gênante : beaucoup de problèmes n'ont besoin de rien d'autre, et la structure le dit alors clairement.

Une file d'attente ajoute à la fin et sert au début. Une pioche de cartes prend au-dessus et, au jeu de bataille, remet les cartes gagnées en dessous : c'est le rôle qu'elle jouera dans le fil rouge. `ArrayDeque` est l'implémentation à préférer ; `LinkedList`, vue en `COL §2`, implémente aussi `Deque`, mais elle est plus lente pour cet usage.

Prendre et remettre

```
String suivant = attente.pollFirst();   // "Sand", retire
String ensuite = attente.peekFirst();   // "Zola", reste

while (!attente.isEmpty()) {
    System.out.println(attente.pollFirst());
}
```

La méthode `pollFirst` retire et retourne le premier élément ; `peekFirst` le retourne sans le retirer. Sur une file vide, les deux retournent `null`. Les méthodes existent par paires symétriques : `addFirst` et `addLast`, `pollFirst` et `pollLast`, `peekFirst` et `peekLast`.

Il existe une seconde famille, `removeFirst` et `getFirst`, qui font la même chose mais lèvent une exception sur une file vide au lieu de retourner `null`. Les deux familles se valent. Ce qui compte est de vérifier que la file n'est pas vide avant de puiser, comme dans la boucle ci-dessus.

Le jeu de bataille tient tout entier dans cette boucle : tant qu'aucune pioche n'est vide, chaque joueur prend sa première carte, et le gagnant remet les deux à la fin de la sienne.

> **Exercices** - `G.21` à `G.27` (fil rouge GameHub)
>
> La quatrième partie de GameHub écrit un jeu de bataille, et elle s'appuie sur ce module de bout en bout. `G.21` reprend le `Deck` d'`ENU.3`, qui rangeait ses cartes dans un tableau, et le passe à une `List` : le mélange se réduit à `Collections.shuffle`. `G.22` donne une valeur à chaque carte, dont l'ordre de déclaration décide de la force. `G.23` écrit une première version du jeu, et distribue le paquet dans deux `Deque`. `G.24` à `G.26` écrivent la méthode `jouer` en trois temps : la mise en place, la boucle (`pollFirst` pour piocher, `addLast` pour remettre), le résultat. `G.27` traite enfin la bataille elle-même, quand deux cartes sont égales.

Aide-mémoire

Cette page regroupe, par besoin, les collections du module et leurs méthodes. Toutes viennent de `java.util`, sauf `Predicate` et `Function`, de `java.util.function`.

Je veux	Je déclare	Je crée	À savoir
garder un ordre et des positions	`List<E>`	`new ArrayList<>()`	doublons acceptés
refuser les doublons	`Set<E>`	`new HashSet<>()`	`TreeSet` pour un ordre trié
retrouver par clé	`Map<K, V>`	`new HashMap<>()`	`TreeMap` pour des clés triées
prendre et remettre aux bouts	`Deque<E>`	`new ArrayDeque<>()`	jamais d'insertion au milieu

Sur	J'appelle	Ce que j'obtiens
toute collection	`add`, `remove`, `contains`, `size`, `isEmpty`	modifier, tester, compter
une liste	`get(i)`, `set(i, x)`, `indexOf(x)`, `sort(comparateur)`	la position, l'ordre
une table	`put`, `get`, `containsKey`, `getOrDefault`	`get` retourne `null` si la clé manque
une table	`keySet`, `values`, `entrySet`	les trois vues à parcourir
une table	`computeIfAbsent(cle, fabrique)`	la valeur, créée au besoin
une file	`addFirst`, `addLast`, `pollFirst`, `peekFirst`	`poll` et `peek` retournent `null` si vide
`Collections`	`sort`, `shuffle`, `min`, `max`, `frequency`	sur une liste

Pour un ensemble ou les clés d'une table, les éléments doivent définir `equals` et `hashCode` (un `record` le fait) ; pour un `TreeSet` ou une `TreeMap`, ils doivent être `Comparable`, ou recevoir un `Comparator`. Les quatre squelettes suivants sont prêts à adapter.

Compter des occurrences

```
Map<String, Integer> compte = new HashMap<>();
for (String mot : mots) {
    compte.put(mot, compte.getOrDefault(mot, 0) + 1);
}
```

Regrouper par clé

```
Map<String, List<String>> titresParAuteur = new HashMap<>();
for (Livre livre : livres) {
    titresParAuteur.computeIfAbsent(livre.auteur(), new NouvelleListe())
                   .add(livre.titre());
}
```

`NouvelleListe` implémente `Function<String, List<String>>`, et sa méthode `apply` retourne une `new ArrayList<>()` (voir `COL §7`).

Parcourir une table

```
for (Map.Entry<String, Integer> entree : compte.entrySet()) {
    System.out.println(entree.getKey() + " : " + entree.getValue());
}
```

Retirer selon un critère, puis trier

```
titres.removeIf(new TitreLong());      // TitreLong implements Predicate<String>
titres.sort(new ParLongueur());        // ParLongueur implements Comparator<String>
Collections.sort(titres);              // l'ordre naturel : Comparable
```

Ne retirez jamais d'élément dans une boucle `for-each` sur la même collection : `ConcurrentModificationException`. Utilisez `removeIf`, ou un `Iterator` et sa méthode `remove` (voir `COL §5`).

Ce qu'il faut retenir

- Une collection grandit toute seule ; on la déclare par son interface, et on choisit la classe au `new`.
- Une liste garde l'ordre et les doublons ; un ensemble refuse les doublons ; une table range par clé ; une file à deux bouts ne se manipule que par ses extrémités.
- `Hash` par défaut, `Tree` quand l'ordre compte.
- Un ensemble et les clés d'une table reposent sur `equals` et `hashCode` : un `record` les écrit pour vous.
- Une clé absente donne `null` : `getOrDefault` et `computeIfAbsent` l'évitent.

- `removeIf` et `computeIfAbsent` reçoivent un comportement, sous la forme d'un objet.
- Un ordre se définit dans la classe (`Comparable`, l'ordre naturel) ou se fournit de l'extérieur (`Comparator`, un ordre parmi d'autres).

La leçon qui dépasse ce module est la façon de choisir : une collection se choisit sur la promesse qu'elle tient, et quand la structure porte une règle, personne n'a plus besoin d'y penser.

Module EXC

EXC — Exceptions

Dire ce qui ne va pas là où on le constate, et le traiter là où l'on peut agir

Jusqu'ici, quand quelque chose tournait mal, le programme s'arrêtait sur un message rouge, ou continuait avec une valeur fausse. Ce module donne un moyen de dire ce qui ne va pas, à l'endroit où on le constate, et de le traiter à l'endroit où l'on peut y faire quelque chose.

Ce sont deux gestes distincts, faits à deux endroits différents : *lever* une exception, et l'*attraper*. Presque tout ce qui suit découle de cette séparation. Le module pèse aussi à l'examen : chaque session commence par une classe qui lève ses propres exceptions.

À la fin de ce module, vous saurez

- expliquer ce qu'est une exception, et comment elle remonte la pile des appels ;
- écrire votre propre classe d'exception, la lever avec `throw` et la déclarer avec `throws` ;
- choisir, pour une exception contrôlée, entre l'attraper et la déclarer ;
- écrire un `try` avec un ou plusieurs `catch`, et un `finally` quand il faut remettre les choses en place ;
- reconnaître une exception non contrôlée, et vous protéger de la plus fréquente, `NullPointerException` ;
- choisir entre une exception du JDK et une exception à vous.

EXC §1 — Un objet qui interrompt

Démonstration - `Demo_EXC` : faire planter, lire la trace, corriger

La démonstration part d'un panier dont on demande la moyenne alors qu'il est vide. Le premier essai ne plante pas du tout : il affiche une valeur absurde, sans rien signaler. Le second plante, et c'est un progrès. On lit alors la trace ensemble, dans le bon sens, puis on discute trois corrections avant d'en coder une. Deux idées à retenir : un plantage bruyant vaut mieux qu'un résultat faux qui continue en silence ; et on lève une exception là où l'on constate le problème, on l'attrape là où l'on peut y faire quelque chose.

Vous levez déjà des exceptions

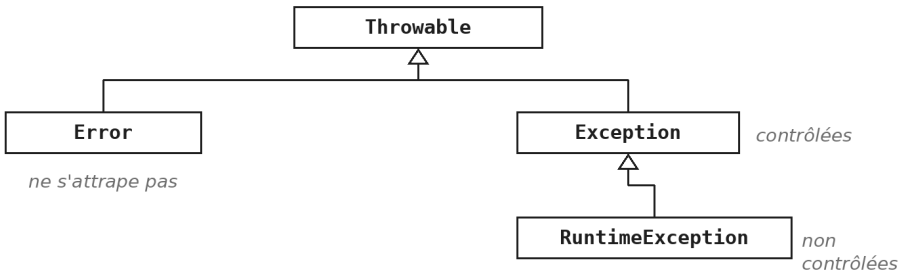
```
public int etageCourant() {
    throw new UnsupportedOperationException(
            "etageCourant n'est pas encore implementee");
}
```

Depuis le module `POO`, chaque méthode à compléter des squelettes contient une ligne de ce genre. Vous avez vu le test passer au rouge avec ce message, sans vous demander d'où il venait : c'était une exception, levée exprès. Quand l'incrément `G.15` du fil rouge vous a demandé de lever « une exception explicite » dans la fabrique, vous avez sans doute recopié ce geste. C'était le bon.

Ce module explique ce qui se passe quand cette ligne s'exécute, comment écrire vos propres exceptions, et comment les attraper.

Une exception est un objet

Une exception n'a rien de magique : c'est un objet, d'une classe comme les autres, que l'on crée avec `new`. Au moment de sa création, il mémorise un message et l'état de la pile des appels. C'est ce qu'affiche la trace.



La famille des exceptions

Toutes les exceptions descendent de la classe `Throwable`.

- **Une `Error`**, comme `OutOfMemoryError` ou `StackOverflowError`, dit que la JVM elle-même ne peut plus continuer. On ne l'attrape pas : il n'y a rien à faire d'utile à ce moment-là.
- **Une `Exception`** signale ce que le programme peut traiter. Ses descendantes sont dites *contrôlées*.
- **Une `RuntimeException`**, sous-branche d'`Exception`, est dite *non contrôlée*, comme toutes ses descendantes.

La différence entre contrôlée et non contrôlée porte sur ce que le compilateur exige ; `EXC §2` la montre, et `EXC §4` revient sur les non contrôlées.

Les exceptions remontent la pile

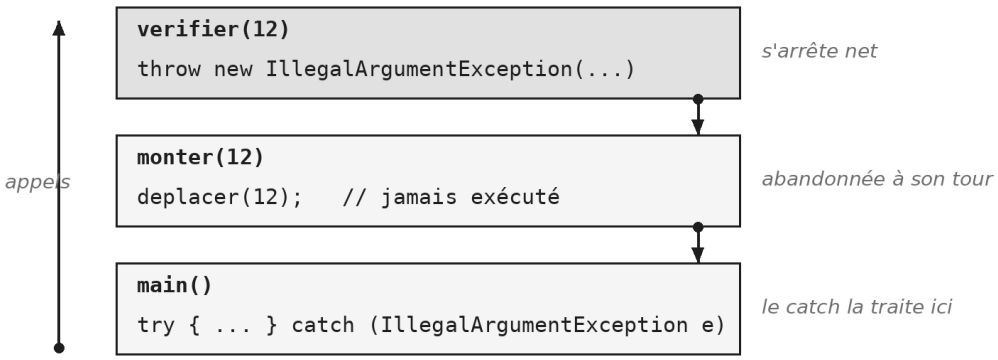
```
void monter(int etage) {
    verifier(etage);       // l'exception traverse monter
    deplacer(etage);       // jamais execute
}

void verifier(int etage) {
    if (etage > dernierEtage) {
        throw new IllegalArgumentException("Etage " + etage
                + " au-dela du dernier, " + dernierEtage);
    }
}
```

Quand on appelle `monter` avec un étage tel que `etage > dernierEtage`, elle appelle `verifier`, qui lève l'exception. La méthode `verifier` ne termine pas son exécution : elle s'arrête net, à l'instant même où le `throw` est exécuté. La méthode `monter` ne reprend pas non plus à la ligne suivante. L'exception traverse chaque méthode de la pile, de la plus récente à la plus ancienne, et chacune est abandonnée à son tour, jusqu'à ce qu'un `catch` l'arrête.

Si aucune ne l'attrape avec un `catch`, l'exception sort du `main`, et la JVM affiche la trace. La trace est exactement la pile des appels décrite dans le mémento technique (`TEC-EXE.1`), figée au moment de la création de l'exception.

Du `throw` au `catch`



Le trajet d'une exception : levée en haut de la pile, elle descend jusqu'au `catch`

Le schéma se lit de deux façons. De bas en haut, ce sont les appels : `main` appelle `monter`, qui appelle `verifier`. Chaque appel empile une méthode, et la plus récente est en haut.

De haut en bas, c'est le trajet de l'exception. `verifier` la lève avec `throw` et s'arrête net. L'exception descend dans `monter`, qui ne la traite pas : `monter` est abandonnée à son tour, et `deplacer` n'est jamais exécutée. Elle arrive enfin dans `main`, où l'appel était entouré d'un `try`. Le `catch` correspond au type de l'exception, il la reçoit, et le programme continue après le `catch`. `EXC §3` montre comment écrire ce `try` et ce `catch`, et surtout où les placer.

EXC §2 — Lever, et déclarer

Écrire sa classe d'exception

```
public class EtageInaccessibleException extends Exception {

    public EtageInaccessibleException(String message) {
        super(message);
    }
}
```

Une exception à soi tient en ces quelques lignes : une classe qui hérite d'`Exception`, et un constructeur qui reçoit le `message` et le transmet à la classe mère par `super`. Tout le travail est déjà fait par `Exception` : mémoriser le `message`, capturer la pile, se laisser attraper. La méthode `getMessage()` retourne le `message` transmis à `super`. Il existe aussi un constructeur sans message, mais indiquez toujours un message qui donne le contexte de l'erreur : c'est lui que lira celui qui découvrira la trace.

Inutile d'y ajouter des champs : un message qui dit ce qui était attendu et ce qui a été reçu suffit presque toujours. C'est la règle `CC-ERR.4`. Le nom se termine par `Exception`, par convention.

Le choix de la classe mère, `Exception` ou `RuntimeException`, a une conséquence que la sous-section « Attraper ou déclarer » montre, et `EXC §4` donne le critère. À l'examen, la première question de chaque session demande d'écrire une classe de ce genre.

Lever avec `throw`, déclarer avec `throws`

```
public void appeler(int etage) throws EtageInaccessibleException {
    if (etage < 0 || etage > dernierEtage) {
        throw new EtageInaccessibleException("Etage " + etage
                + " hors de l'intervalle 0 a " + dernierEtage);
    }
    etageCourant = etage;
}
```

Les deux mots se ressemblent et ne font pas la même chose.

- **`throw` est une instruction**, dans le corps de la méthode : il crée l'objet et l'envoie.
- **`throws` fait partie de la signature** : il prévient l'appelant que cette méthode peut échouer de cette façon.

Une méthode peut déclarer plusieurs exceptions, séparées par des virgules. Un constructeur annonce une exception exactement de la même façon : `throws` après la parenthèse.

Remarquez le message : il nomme l'étage demandé et l'intervalle permis. Quelqu'un qui lira la trace sans le code saura quoi corriger. Remarquez aussi l'ordre : la vérification vient *avant* la modification de `etageCourant`. Une méthode qui refuse une demande doit laisser l'objet dans l'état où elle l'a trouvé.

Attraper ou déclarer

```
void depuisLeBouton(int etage) {
    try {
        ascenseur.appeler(etage);
    } catch (EtageInaccessibleException e) {
        System.out.println("Refus : " + e.getMessage());
    }
}

void depuisLeBadge(int etage) throws EtageInaccessibleException {
    ascenseur.appeler(etage);
}
```

Pour une exception contrôlée, le compilateur exige un choix. Dès qu'une méthode appelle `appeler`, il refuse de compiler tant qu'elle n'a pas choisi :

- **attraper**, c'est-à-dire traiter l'exception ici, avec `try` et `catch` ;
- **déclarer**, c'est-à-dire la laisser passer à l'appelant, qui aura le même choix.

Ce n'est pas une politesse : c'est la seule chose que le compilateur sait vérifier à propos des erreurs, et il le fait pour les exceptions contrôlées seulement. Aucun des deux choix n'est le bon dans l'absolu. On attrape quand on peut faire quelque chose d'utile ; sinon, on déclare, et la question remonte d'un cran. Déclarer jusqu'au `main` revient à laisser le programme s'arrêter sur une trace.

Pour une exception non contrôlée, le compilateur n'exige rien : c'est `EXC §4`. Vous retrouverez ce `throws` dans les tests du module `JUN` : une méthode de test qui appelle une méthode capable d'échouer doit, elle aussi, choisir.

Exercices - `EXC.1` Lever sa propre exception, puis `EXC.2` Un compte, sans exception

`EXC.1` met en pratique cette section : une classe d'exception, une méthode qui la lève, un programme qui l'attrape. Lisez l'encadré sur le vrai piège avant d'écrire la moindre ligne : le test vérifie ce que devient le point quand le déplacement est refusé. Les deux questions de fin portent sur le choix de la classe mère. `EXC.2` fait exprès l'inverse : un compte, sans aucune exception. Gardez la réponse à sa question finale, elle sert à l'exercice suivant. Pour les messages, voir `CC-ERR.4`.

EXC §3 — Attraper là où l'on peut agir

Ce que le `try` saute

```
try {
    ascenseur.appeler(2);
    ascenseur.appeler(9);              // leve l'exception
    System.out.println("Arrive");      // jamais affiche
} catch (EtageInaccessibleException e) {
    System.out.println("Refus : " + e.getMessage());
}
System.out.println("Fin");             // toujours affiche
```

- **Au premier `throw`, le reste du `try` est abandonné.** Le programme n'y retourne jamais : ce qui suivait l'instruction fautive est perdu.
- **Le `catch` s'exécute, puis tout ce qui le suit.**
- **Sans exception, le `catch` est simplement ignoré.**

Le premier appel a réussi : l'ascenseur est bien au deuxième étage. Le second a échoué avant de modifier quoi que ce soit, si `appeler` est bien écrite : c'est tout l'enjeu de l'encadré d'`EXC.1`.

Une variable déclarée dans le `try` n'existe ni dans le `catch`, ni après : déclarez-la avant si vous en avez besoin des deux côtés.

Plusieurs `catch`, ou un seul

```
try {
    ascenseur.appeler(etage);
    porte.ouvrir();
} catch (EtageInaccessibleException e) {
    afficher("Etage refuse : " + e.getMessage());
} catch (PorteBloqueeException | CoupureException e) {
    signalerPanne(e.getMessage());
}
```

Quand une exception sort du `try`, Java essaie les `catch` un par un, de haut en bas, et exécute le premier dont le type convient. Les suivants sont ignorés.

Convenir veut dire la même chose qu'au module `ABS` : une `EtageInaccessibleException` est aussi une `Exception`. Un `catch` écrit pour une classe mère attrape donc toutes ses filles. Il en découle une règle d'ordre : un `catch` pour une classe fille doit venir avant celui de sa mère, sans quoi il ne serait jamais atteint, et le compilateur le refuse.

La barre verticale réunit des types qui n'ont pas de lien d'héritage entre eux, quand le traitement est le même. Faut-il un `catch` par exception, ou un seul pour toute la famille ? Le fascicule pose la question à `EXC.3`, et la règle `CC-ERR.5` y répond.

Un `catch` fait quelque chose

```
try {
    ascenseur.appeler(etage);
} catch (EtageInaccessibleException e) {
    System.out.println("Refus : " + e.getMessage());
    ecran.proposerEtagesValides();
}
```

La démonstration a montré le `catch` vide : le programme ne dit plus rien, et tout semble aller bien. Le plantage était bruyant mais honnête ; ce silence ment. L'erreur ressurgira plus loin, à un endroit qui n'a plus rien à voir avec sa cause. C'est la règle `CC-ERR.6`.

Trois réactions sont honnêtes :

- **traiter :** redemander, proposer autre chose, prendre une valeur de repli que l'on assume ;
- **informer :** dire ce qui a échoué ;
- **ne pas attraper du tout,** et laisser l'exception remonter jusqu'à quelqu'un qui saura quoi en faire.

Une nuance, que vous croiserez dans GameHub. Attraper une exception pour redemander une saisie est un vrai traitement, même quand l'objet exception ne sert plus. On nomme alors la variable `ignoree` : le nom dit que c'est voulu.

Ce que `finally` garantit

```
void deplacer(int etage) throws EtageInaccessibleException {
    porte.verrouiller();
    try {
        ascenseur.appeler(etage);
    } finally {
        porte.deverrouiller();         // dans tous les cas
    }
}
```

Cette méthode n'attrape rien : elle déclare l'exception et la laisse passer. Mais la porte doit être déverrouillée, que l'appel réussisse ou non. Écrire `deverrouiller` après le `try` ne suffirait pas : en cas d'exception, la ligne ne serait jamais atteinte.

Le bloc `finally` s'exécute à la sortie du `try`, que celui-ci se termine normalement ou par une exception, attrapée ou non. Il peut suivre un `catch`, ou se passer de `catch` comme ici. On s'en sert pour libérer ce qu'on a pris : un verrou, une connexion, un fichier ouvert. Le module `FIC` montrera une écriture plus courte pour les fichiers, qui fait ce travail toute seule.

> **Exercices** - `EXC.3` Un numéro de compte vérifié, puis `EXC.6` Ce que `finally` garantit
>
> `EXC.3` reprend le compte de l'exercice précédent et lui ajoute un numéro vérifié. Les messages sont imposés au caractère près : chacun dit ce qui était attendu et ce qui a été reçu. L'encadré du fascicule pose la question qui fait l'exercice : trois exceptions ou une seule. Essayez les quatre numéros fournis avant de décider combien de `catch` écrire, et faites l'expérience de la seconde question : le compilateur a quelque chose à dire. `EXC.6` fait constater ce que `finally` garantit. La première version ouvre la caisse, encaisse, puis la referme, trois gestes à la suite : un seul test échoue, et son nom dit pourquoi. La seconde version déplace la fermeture dans un `finally`, sans ajouter de `catch`. Voir `CC-ERR.4` et `CC-ERR.5`.

EXC §4 — Celles que le compilateur ne voit pas

Une exception que personne n'annonce

```
String saisie = "douze";
int etage = Integer.parseInt(saisie);   // compile sans rien dire

// Exception in thread "main" java.lang.NumberFormatException:
//     For input string: "douze"
```

La méthode `Integer.parseInt` peut échouer, et sa signature n'en dit rien : `NumberFormatException` hérite de `RuntimeException`. Le compilateur ne demande donc ni `try` ni `throws`, et le programme plante à la première saisie qui n'est pas un nombre.

La JVM lève elle-même plusieurs exceptions de cette famille : `ArithmeticException`, rencontrée pendant la démonstration (une division entière par zéro), `ArrayIndexOutOfBoundsException` au module `TAB`, et la plus fréquente de toutes, présentée juste après. Elles peuvent survenir presque partout, et le compilateur ne vous prévient d'aucune.

`NullPointerException` : la plus fréquente

```
String titre = null;

titre.equals("Nana")                    // plantage

titre != null && titre.equals("Nana")  // false
"Nana".equals(titre)                    // false
Objects.equals(titre, "Nana")           // false
```

C'est l'exception que vous rencontrerez le plus souvent. `titre.equals("Nana")` appelle une méthode sur l'objet désigné par `titre`. Si `titre` vaut `null`, il n'y a pas d'objet à qui demander, et la JVM lève une `NullPointerException`.

La protection classique teste d'abord la non-nullité : `titre != null && titre.equals("Nana")`. L'ordre est essentiel. L'opérateur `&&` s'arrête dès que la première condition est fausse, et la seconde n'est alors jamais évaluée ; inverser les deux tests remet le plantage.

Deux écritures plus courtes font la même chose :

- **la constante d'abord**, `"Nana".equals(titre)` : une constante n'est jamais `null`, et l'appel retourne simplement `false` ;
- **`Objects.equals(titre, "Nana")`**, de `java.util`, qui gère le `null` des deux côtés.

Tester partout la non-nullité reste un filet de sécurité. La vraie protection est en amont : ne pas produire de `null`, ni en retour ni en paramètre (`CC-ERR.2` et `CC-ERR.3`), et préférer une valeur vide. La règle `CC-ERR.7` résume le cas de la comparaison.

> **Exercices** - `EXC.4` Provoquer, puis traiter
>
> L'exercice se fait dans un ordre précis, et cet ordre est l'exercice. Écrivez d'abord le programme sans aucun `try`. Lancez-le avec une taille négative, puis avec une position hors du tableau, et lisez les deux messages : ils vous disent quelles exceptions attraper. Constatez surtout que le code a compilé sans rien dire, et que le programme s'arrête net : la boucle ne repart pas. Seulement ensuite, écrivez `placerDouze`, qui traite les deux cas, et relancez avec les mêmes saisies : la boucle survit. Sans la première partie, vous saurez écrire le `try`, mais pas pourquoi il est là.

Réutiliser plutôt que créer

Le JDK contient déjà des exceptions pour les cas les plus courants. Toutes trois sont non contrôlées, et vivent dans `java.lang` : aucun `import` n'est nécessaire.

Exception	Elle signale
`IllegalArgumentException`	un paramètre refusé
`IllegalStateException`	un objet dans le mauvais état
`UnsupportedOperationException`	une opération absente

La première dit que l'appelant a passé une valeur qui n'a pas de sens, comme un dernier étage négatif. La deuxième dit que la demande était peut-être correcte, mais pas à ce moment-là, parce que l'objet n'est pas dans le bon état : appeler un ascenseur en panne. La troisième dit qu'une opération n'existe pas, ou pas encore : c'est celle de vos squelettes.

Quand l'une d'elles dit exactement ce qui se passe, servez-vous-en. On crée sa propre exception quand l'appelant doit pouvoir la distinguer des autres, ou quand il doit être obligé de s'en soucier. Dans ce dernier cas, elle hérite d'`Exception`, et le compilateur imposera le choix d'`EXC §2` ; sinon, elle hérite de `RuntimeException`. À l'examen, l'énoncé précise souvent « il ne faut pas la créer » : c'est l'une de celles-ci.

Lever une exception du JDK

```
public Ascenseur(int dernierEtage) {
    if (dernierEtage < 0) {
        throw new IllegalArgumentException(
                "Le dernier etage doit etre positif ou nul, pas " + dernierEtage);
    }
    this.dernierEtage = dernierEtage;
}
```

Rien ne change par rapport à vos propres exceptions, sinon qu'il n'y a rien à écrire ni à déclarer : le constructeur ci-dessus n'a pas de clause `throws`, puisque l'exception n'est pas contrôlée.

Le message reste votre responsabilité. `new IllegalArgumentException()` sans message ne dit rien à celui qui lira la trace : nommez ce qui était attendu, et la valeur reçue (`CC-ERR.4`). Le module `JUN` vous demandera d'écrire une méthode qui lève une `IllegalArgumentException`, et de le vérifier.

C'est aussi la réponse promise au module `ENU`. Rien n'empêchait de construire un `Student` avec une cote de 47 : un record accepte un constructeur compact, où l'on place exactement cette vérification (voir `ENU §7`). Si la cote sort de 0 à 20, il lève une `IllegalArgumentException`, et l'objet n'est jamais créé.

> **Exercices** - `EXC.5` Lire une trace, puis le fil rouge relu (`G.15` et `G.24`)
>
> `EXC.5` refait seul ce que la démonstration a fait avec vous : lancer un programme qui plante, et lire la trace avant de toucher au code (voir `TEC-EXE.1`). Recopiez exactement ce que dit la trace, ponctuation comprise ; l'une des réponses ne ressemble à aucun nom que vous avez écrit, et c'est elle qui apprend quelque chose. GameHub n'a pas d'incrément consacré aux exceptions, mais il en lève depuis longtemps : relisez `G.15`, dont la fabrique lève une `UnsupportedOperationException`, et `G.24`, qui lève une `IllegalArgumentException` si l'un des joueurs est `null`. Vos messages nomment-ils ce qui a été refusé ? Et si votre GameHub lit des nombres au clavier, votre `catch` redemande-t-il, ou reste-t-il muet (`CC-ERR.6`) ?

Ce qu'il faut retenir

- Une exception est un objet ; levée par `throw`, elle remonte la pile jusqu'au premier `catch` qui lui convient.
- `throw` lève, dans le corps ; `throws` déclare, dans la signature.
- Pour une exception contrôlée, le compilateur exige un choix : attraper ou déclarer.
- Un `catch` fait quelque chose : traiter, informer, ou ne pas attraper du tout.
- `finally` s'exécute toujours, et remet les choses en place.
- Une `RuntimeException` passe inaperçue à la compilation ; `NullPointerException` est la plus fréquente.

- Une exception du JDK quand elle dit exactement ce qui se passe ; la vôtre quand l'appelant doit la distinguer.

La séparation qui structure tout le module tient en une phrase : on lève là où l'on constate le problème, on attrape là où l'on peut y faire quelque chose.

Module JUN

JUN — Tests unitaires avec JUnit

Écrire le code qui vérifie le code

Depuis le début de l'année, un test vous dit si votre code est juste. Ce module vous apprend à écrire ce test vous-mêmes : du code qui dira, tout seul et à chaque lancement, que quelque chose s'est cassé.

Le module ne compte que trois exercices, et ce chiffre est trompeur. Quatre objectifs de l'examen ne reposent que sur lui : écrire la classe de test d'une classe écrite par quelqu'un d'autre, écrire une classe et ses tests, repérer un défaut grâce aux tests, et justifier les cas que l'on a choisi de tester. Chaque session consacre sa troisième question à une classe de test.

À la fin de ce module, vous saurez

- écrire une classe de test `JUnit`, au bon endroit, avec des tests en trois temps ;
- choisir l'assertion la plus précise, et vérifier qu'une exception est levée ;
- traduire une documentation en liste de tests, y compris pour les cas d'échec ;
- choisir les cas à tester par familles et par frontières, et justifier chaque test par le défaut qu'il attrape ;
- lire un test rouge, et décider qui a tort, du code ou du test ;
- rendre testable une classe qui dépend d'une fenêtre, de la date du jour ou du hasard.

JUN §1 — À vous d'écrire les tests

Qui écrit quoi

Jusqu'ici, le test était fourni et vous écriviez le code. Dans ce module, les rôles s'inversent : c'est à vous d'écrire les tests.

Exercice	Fourni	À écrire
jusqu'ici	le test	le code
`JUN.1`	le test	le code, une dernière fois
`JUN.2`	quatre tirelires	la vérification
`JUN.3`	la classe `Moyenne`	sa classe de test
l'examen	une classe d'un tiers	sa classe de test

Lisez ce tableau de haut en bas : la colonne de droite change de nature. À partir de `JUN.2`, le code est déjà écrit, et souvent correct ; ce que vous rendez, c'est ce qui le vérifie. `JUN.1` est la charnière : le test y est encore fourni, mais il ne sert plus seulement à dire vert ou rouge. Il est l'énoncé, et vous apprenez à le lire.

Conséquence pratique : le vert n'est plus la preuve que vous avez fini. Dans `JUN.3`, la classe de test fournie contrôle la *forme* de la vôtre, pas sa valeur. Personne ne peut la contrôler à votre place, et c'est exactement ce que l'examen évalue.

Un programme qui affiche ne vérifie rien

```
public static void main(String[] args)
        throws SalleCompleteException {
    Salle salle = new Salle(100);

    salle.reserver(2);

    System.out.println(salle.placesLibres());   // 98 ?
}
```

C'est ainsi que vous avez vérifié vos programmes jusqu'ici : on lance, on lit ce qui s'affiche, on décide si c'est juste. Cela marche une fois. Personne ne relance quarante programmes après chaque modification, en relisant chaque ligne et en se souvenant de ce qu'elle devait afficher.

Un test fait la même chose, à une différence près : il contient la réponse attendue. Il compare lui-même, et ne dit que vert ou rouge. C'est le « S » de F.I.R.S.T.[5], *auto-vérifiant*, dans `CC-TST.3`.

La salle de spectacle servira d'exemple pour tout le module. Elle a une capacité, on y réserve des places, et elle refuse ce qu'elle ne peut pas servir.

> **Démonstration** - `Demo_JUN` : rouge, vert, puis nettoyer
>
> La démonstration écrit un test avant la classe qu'il vérifie, le regarde échouer, puis le fait passer avec le code le plus bête possible. Un test de plus, et le code doit s'améliorer. Un cas inattendu apparaît, que seul le fait de chercher quoi tester a fait découvrir. Deux idées à retenir : un test qu'on n'a jamais vu échouer ne prouve rien ; et un test sans assertion passe au vert quoi qu'il arrive, ce que la démonstration montre exprès, parce que c'est le défaut le plus fréquent dans les copies d'examen.

[5] F.I.R.S.T. réunit, en cinq mots anglais, les qualités d'un bon test : *Fast*, rapide, parce qu'une suite lente n'est plus lancée ; *Independent*, indépendant, parce qu'aucun test ne suppose qu'un autre a tourné avant lui ; *Repeatable*, répétable, parce qu'il donne le même résultat sur toutes les machines et dans six mois ; *Self-validating*, auto-vérifiant, parce qu'il dit vert ou rouge sans qu'on lise une sortie ; *Timely*, écrit tôt, idéalement avant le code. Le mémento Clean Code les détaille en `CC-TST.3`.

JUN §2 — Ce qu'un test contient

Où vit une classe de test

```
package be.ephec.java.spectacle;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class SalleTest {

    // une methode par comportement verifie
}
```

- **Dans `src/test/java`, sous le même package que la classe testée.** Les deux arborescences se superposent : la classe de test voit sa classe comme une voisine (voir `PKG §1`).
- **Son nom est celui de la classe, suivi de `Test`.**
- **`JUnit` n'est visible que de ce côté.** Le projet Maven du cours ne laisse `JUnit` entrer que dans `src/test` ; tenter de l'importer dans `src/main` ne compile pas, et c'est voulu (`TEC-MVN.2`).

Notez bien : le code « utile » que vous produisez, celui qui partira en production, se trouve dans le dossier `src/main/java` ; le code de test, qui ne doit pas partir en production, est dans `src/test/java`.

L'`import static` est nouveau. Il importe une méthode statique, ici `assertEquals` de la classe `Assertions`, pour l'appeler sans écrire le nom de la classe devant. Toutes les assertions de `JUnit` 5 vivent dans `org.junit.jupiter.api.Assertions`.

Dans IntelliJ, `Ctrl+Shift+T` depuis la classe propose de créer sa classe de test au bon endroit. À l'examen, si les noms de `JUnit` restent en rouge, l'énoncé rappelle l'option qui l'ajoute au projet.

`@Test`, puis une assertion

```
@Test
void uneSalleNeuveEstEntierementLibre() {
    Salle salle = new Salle(100);

    int libres = salle.placesLibres();

    assertEquals(100, libres,
            "Une salle neuve de 100 places doit etre entierement libre");
}
```

L'annotation `@Test` fait d'une méthode un test, que `JUnit` lance seul. Une méthode de test ne reçoit rien et ne retourne rien. Il n'y a pas de `main` : `JUnit` cherche toutes les méthodes annotées `@Test` et les lance une à une.

Les trois temps se lisent dans le code :

1. *préparer* la salle ;
2. *agir* en demandant les places libres ;
3. *vérifier* le résultat.

Une ligne vide les sépare, et c'est la forme à donner à vos propres tests, à l'examen comme ailleurs. Vous ne la retrouverez pas dans tous les tests fournis avec les exercices : beaucoup ne vérifient que la forme d'une classe, en une ligne.

L'ordre des arguments compte : `assertEquals` reçoit l'attendu d'abord, l'obtenu ensuite. En cas d'échec, `JUnit` affiche `expected: <100> but was: <0>`. Si vous les inversez, le message perdra tout son sens.

Enfin, la ligne qui fait le test est la dernière. Sans elle, la méthode appelle du code, ne vérifie rien, et passe au vert : sans assertion, un test ne peut qu'être vert (`CC-TST.4`).

Les assertions dont vous aurez besoin

Assertion	Réussit si
`assertEquals(attendu, obtenu[, message])`	les deux valeurs sont égales
`assertTrue(condition[, message])`	la condition est vraie
`assertFalse(condition[, message])`	la condition est fausse
`assertNull(valeur[, message])`	la valeur est `null`
`assertThrows(Type.class, ... [, message])`	l'appel lève cette exception

Cinq assertions suffisent à tout ce que le cours et l'examen demandent. Chacune a sa contraire (`assertNotEquals`, `assertNotNull`), et toutes acceptent un argument supplémentaire, un message affiché en cas d'échec.

Pour des objets, `assertEquals` appelle `equals` : un objet qui ne le redéfinit pas n'est égal qu'à lui-même (voir `COL §3`). Pour des `double`, elle demande une tolérance, parce qu'un calcul en virgule flottante n'est presque jamais exact au dernier chiffre : le fascicule de `JUN.3` le montre.

Préférez toujours l'assertion la plus précise. Un `assertTrue(libres == 100)` qui échoue dit seulement « expected: <`true> but` was: <`false`> ». Un `assertEquals(100, libres)` dit ce qu'il a obtenu.

Un message qui explique l'échec

```
assertEquals(98, salle.placesLibres(),
        "Deux places reservees sur 100 doivent en laisser 98");

// en cas d'echec, JUnit affiche :
// Deux places reservees sur 100 doivent en laisser 98
//     ==> expected: <98> but was: <100>
```

Le dernier argument, entre crochets dans le tableau, est facultatif pour `JUnit`. Dans ce cours, ajoutez-le systématiquement. Il dit ce que le test vérifiait, avec les mots du métier : celui qui lira l'échec, vous-même dans six mois ou un collègue, comprend ce qui s'est cassé sans ouvrir le test. « expected: <98> but was: <100> » dit qu'un nombre est faux ; le message dit lequel, et pourquoi il aurait dû valoir 98.

Le message décrit l'attente, pas le défaut supposé : « doivent en laisser 98 », et non « la réservation est cassée ». Le test ne sait pas pourquoi il échoue ; il sait seulement ce qu'il attendait. Comme pour les noms de variables, pas d'accent dans la chaîne.

Un test rouge accuse le code, ou le test

```
// une assertion fausse
expected: <98> but was: <100>

// une exception que personne n'attendait
java.lang.UnsupportedOperationException:
    reserver n'est pas encore implementee
```

Un test rouge dit qu'il y a un désaccord, mais ne dit pas qui a tort. Le code peut être faux, mais le test aussi : un attendu mal calculé, des arguments inversés, une préparation incomplète.

L'arbitre n'est ni l'un ni l'autre, c'est la spécification : la documentation de la méthode, ou l'énoncé. On la relit, on décide qui a tort, et on ne corrige que celui-là. Le second cas vous est familier : c'est le message de chacun de vos squelettes. Une exception qui sort d'un test le fait échouer, et sa trace se lit comme au module `EXC`.

C'est le troisième objectif de l'examen : exécuter les tests de quelqu'un d'autre, et s'en servir pour trouver le défaut dans une classe que vous n'avez pas écrite.

Exercices - `JUN.1` Le test comme spécification

C'est le dernier exercice où le test est fourni, et la manière de s'en servir change. Ouvrez `CalculatriceTest` d'abord, et lisez-le comme un énoncé : les signatures, les valeurs, et ce qui doit arriver quand on divise par zéro y sont, et nulle part ailleurs. Vous y croiserez `assertThrows`, que la section suivante explique. La question sur les `assertEquals` multiples mérite d'être confrontée à `CC-TST.2`. Voir aussi `CC-TST.1`.

JUN §3 — Vérifier tout le contrat

Une documentation est une liste de tests

```
/**
 * Reserve des places.
 *
 * @throws IllegalArgumentException si places <= 0
 * @throws SalleCompleteException   s'il reste moins de
 *         places que demande ; rien n'est alors reserve
 */
public void reserver(int places)
        throws SalleCompleteException
```

Quand vous testez la classe de quelqu'un d'autre, vous ne partez pas de son code, vous partez de ce qu'elle promet. Chaque phrase de la documentation est une promesse, et chaque promesse mérite son test, y compris celles faites en cas d'échec. Ici, quatre promesses :

1. une réservation diminue les places libres d'autant ;
2. zéro place est refusé ;
3. une demande trop grande est refusée ;
4. ce refus ne réserve rien.

La dernière est la plus souvent oubliée. Elle est écrite en fin de phrase, presque en passant, et c'est pourtant elle qui distingue une salle correcte d'une salle qui réserve quand même la moitié de la demande. C'est ce que l'examen appelle tester « sur base d'une classe implémentée par un tiers et/ou de spécifications techniques » : la spécification, c'est ceci.

Vérifier qu'une exception est levée

```
@Test
void reserverZeroPlaceEstRefuse() {
    Salle salle = new Salle(100);

    assertThrows(IllegalArgumentException.class,
            () -> salle.reserver(0),
            "Reserver zero place doit etre refuse");
}
```

Appeler `salle.reserver(0)` directement ne marcherait pas : l'exception interromprait le test, qui passerait au rouge précisément quand la salle fait ce qu'on attend d'elle. `assertThrows` fait l'appel pour vous, attrape l'exception, et vérifie son type. Le test échoue si rien n'est levé, ou autre chose.

L'écriture `() -> salle.reserver(0)` emballe l'appel pour qu'`assertThrows` l'exécute elle-même, au bon moment. Pour l'instant, retenez la forme : des parenthèses vides, une flèche, l'appel. Le module `LAM` expliquera cette forme plus en détail. La notation `IllegalArgumentException.class` désigne la classe elle-même.

Nommez le type le plus précis. Une sous-classe convient aussi, si bien qu'`assertThrows(Exception.class, ...)` accepterait n'importe quelle exception, y compris celle d'un bug. `assertThrows` retourne l'exception attrapée : on peut en vérifier le message.

Le `throws` d'une méthode de test

```
@Test
void deuxPlacesReserveesNeSontPlusLibres()
        throws SalleCompleteException {
    Salle salle = new Salle(100);

    salle.reserver(2);

    assertEquals(98, salle.placesLibres(),
            "Deux places reservees sur 100 doivent en laisser 98");
}
```

Le module `EXC` l'avait annoncé : une méthode de test qui appelle `reserver` doit choisir, attraper ou déclarer (voir `EXC §2`). Dans un test, on déclare. Si l'exception survient alors qu'on ne l'attendait pas, `JUnit` marque le test en rouge avec sa trace : c'est exactement ce qu'on veut.

L'attraper serait pire que tout. Un `catch` vide, ou qui affiche un message, fait passer le test au vert alors que la réservation a échoué. C'est un test sans assertion déguisé, et un `catch` qui ne traite rien, contraire à `CC-ERR.6`. Dans `assertThrows`, en revanche, rien à déclarer : c'est `JUnit` qui attrape.

Après l'échec, l'état

```
@Test
void unePlaceDeTropNeReserveRien()
        throws SalleCompleteException {
    Salle salle = new Salle(10);
    salle.reserver(8);

    assertThrows(SalleCompleteException.class,
            () -> salle.reserver(3),
            "Une demande trop grande doit etre refusee");

    assertEquals(2, salle.placesLibres(),
            "Le refus ne doit rien reserver");
}
```

Lever la bonne exception ne suffit pas : le refus ne doit rien avoir changé. Une salle qui réserverait les deux places restantes avant de lever l'exception passerait la première assertion ; seule la seconde la démasque.

Les deux assertions ne contredisent pas `CC-TST.2` : elles décrivent un seul comportement, le refus d'une demande trop grande, vu sous ses deux faces. C'est la même idée qu'en `EXC.1`, où le test vérifiait que le point ne bougeait pas quand le déplacement était refusé. Cette fois, c'est vous qui l'écrivez.

> **Exercices** - `JUN.2` Détecter un défaut
>
> L'exercice met la section en pratique sans `JUnit`. Vous n'y écrivez pas de `@Test` : chaque clause du contrat devient une vérification dans une méthode ordinaire, qui retourne `true` ou `false`, avec ce que vous savez depuis le module `EXC`. L'intérêt est que la qualité de votre vérification devient mesurable : chaque tirelire défectueuse n'est démasquée que par une clause précise, et en oublier une, c'est en laisser passer une. Résistez à l'envie d'ouvrir les classes pour trouver le défaut : le repérer en lisant ne prouve rien, et ne servira plus le jour où le code changera.

JUN §4 — Choisir ses cas, et les justifier

Où se logent les défauts

On ne peut pas tout tester. On découpe donc les entrées en *familles* qui devraient se comporter de la même façon, on prend un représentant de chacune, et l'on vise les *frontières* entre familles, parce que c'est là que les défauts se logent : un `>` au lieu d'un `>=`, un indice qui commence à 1.

Famille de cas	Pour une salle de 10 places
le cas ordinaire	réserver 3 places
la borne atteinte	réserver exactement les places libres
la borne dépassée	réserver une place de trop
la valeur interdite	réserver 0 place, ou -1
la répétition	deux réservations successives

Le cas ordinaire est nécessaire et ne suffit jamais : un test qui ne couvre que lui ne protège que de ce qui n'arrive pas. La répétition est la famille la plus oubliée : beaucoup de défauts n'apparaissent qu'au deuxième appel, quand l'objet n'est plus neuf.

Le fascicule de `JUN.3` donne la liste des cas de `Moyenne`. Ce tableau est la méthode qui produit une telle liste quand personne ne vous la donne.

Un cas, un test, un nom

```
@Test
@DisplayName("Reserver toutes les places libres remplit la salle")
void reserverToutesLesPlacesLibres()
        throws SalleCompleteException {
    Salle salle = new Salle(10);

    salle.reserver(10);

    assertEquals(0, salle.placesLibres(),
            "Reserver les 10 places doit remplir la salle");
}
```

Une méthode par comportement (`CC-TST.2`), et un nom qui dit ce qui est vérifié, jamais un numéro (`CC-NOM.1`). Quand ce test passera au rouge, son nom sera la première chose que vous lirez, et souvent la seule dont vous aurez besoin. `test1` ne dit rien. L'ancienne habitude de commencer chaque nom par `test` n'apporte rien non plus : l'annotation le dit déjà.

L'annotation `@DisplayName` énonce le test en clair, avec des espaces ; IntelliJ l'affiche à la place du nom de la méthode. Pas d'accent dans la chaîne, comme partout dans le code du cours. `JUN.3` exige un `@DisplayName` sur chaque test, et refuse les noms numérotés.

Une exception : si l'énoncé impose un nom, on le respecte à la lettre. L'examen de 2022 imposait `testConcatene`, et la correspondance exacte à l'énoncé fait partie des critères.

Une valeur, un test

```
@Test
void reserverUnePlaceEnLaisseNeuf()
        throws SalleCompleteException {
    Salle salle = new Salle(10);

    salle.reserver(1);

    assertEquals(9, salle.placesLibres(),
            "Une place reservee sur 10 doit en laisser 9");
}

// un autre test pour 5 places, un autre pour 10
```

La question revient chaque année : faut-il vérifier plusieurs valeurs dans une même méthode de test, ou écrire une méthode par valeur ? La règle est une méthode par valeur. Une assertion qui échoue arrête la méthode : si la première de cinq valeurs est fausse, les quatre autres ne sont jamais vérifiées, et vous corrigez à l'aveugle. Et le nom de la méthode ne dit pas quelle valeur a cassé, alors qu'un test par valeur le dit d'un coup d'œil.

La règle connaît des exceptions raisonnables. Après une seule action, on peut vérifier plusieurs choses sur son résultat : « Après l'échec, l'état » vérifie l'exception, puis que rien n'a changé. Et un test qui contrôle la forme d'une classe peut vérifier tous ses champs d'un coup. Ce qu'on évite, c'est d'enchaîner plusieurs actions indépendantes dans une seule méthode.

> **Pour aller plus loin - `@ParameterizedTest`**
>
> Quand une même vérification doit porter sur beaucoup de valeurs, `JUnit` propose un compromis : une seule méthode, lancée une fois par cas, et rapportée comme autant de tests séparés. Avec `@ParameterizedTest` et `@CsvSource({"1, 9", "5, 5", "10, 0"})`, la méthode reçoit en paramètres l'entrée et l'attendu de chaque ligne. Si le cas « 5, 5 » échoue, les deux autres sont quand même vérifiés. Les annotations viennent de `org.junit.jupiter.params`, déjà présent dans le projet. Rien n'oblige à s'en servir : une méthode par valeur reste toujours correcte.

Chaque test repart de zéro

```
class SalleTest {

    private Salle salle;

    @BeforeEach
    void preparer() {
        salle = new Salle(10);
    }

    @Test
    void uneSalleNeuveEstEntierementLibre() {
        assertEquals(10, salle.placesLibres(),
                "Une salle neuve doit etre libre");
    }
}
```

`JUnit` crée un nouvel objet de test pour chaque méthode, et la méthode annotée `@BeforeEach` s'exécute juste avant elle. Un test qui remplit la salle ne gêne donc pas le suivant. `JUnit` ne promet aucun ordre d'exécution : un test qui a besoin qu'un autre ait tourné avant lui finira par échouer sans raison apparente. Aucun test ne compte sur un autre (`CC-TST.3`).

N'en abusez pas. Si la valeur qui compte pour un test est cachée dans `preparer`, le lecteur doit remonter la chercher. Ce qui fait le cas d'un test reste visible dans le test.

Un test se justifie par le défaut qu'il attrape

```
// le defaut :  >=  au lieu de  >
if (places >= placesLibres()) {
    throw new SalleCompleteException(
            "Plus que " + placesLibres() + " places");
}

// un seul test passe au rouge :
// reserverToutesLesPlacesLibres
```

Justifier un test, c'est nommer le défaut qu'il attraperait. Ici, la salle refuse une demande qu'elle pouvait servir. Le cas ordinaire ne le voit pas, la place de trop non plus : seul le test de la borne atteinte passe au rouge. C'est sa justification, en une phrase.

La question se pose dans les deux sens. Un test pour lequel on ne trouve aucun défaut qu'il attraperait est décoratif. Deux tests qui attrapent exactement les mêmes défauts en font un de trop. C'est le quatrième objectif de l'examen, « justifier ses choix quant aux tests implémentés » : la réponse la plus solide ne dit pas « pour tester le cas limite », elle dit quelle erreur ce cas limite démasque.

À l'examen : écrire, lancer, corriger

La troisième question de l'examen a toujours la même forme : une classe fournie, une classe de test à écrire, puis lancer les tests. La classe fournie peut contenir une erreur, à corriger si vos tests la trouvent.

- **Le plus souvent, l'énoncé dicte les cas :** vous traduisez chaque phrase en méthode, comme en `JUN §3`. Quand il ne dit que « envisagez les différents cas possibles », c'est le tableau des familles de cas qui répond.
- **Vos tests peuvent révéler un défaut : c'est prévu.** Revenez alors à « Un test rouge accuse le code, ou le test ».
- **Si l'énoncé vous donne raison, la classe est fausse :** corrigez-la, et relancez tout. Modifier l'attendu pour retrouver le vert, c'est rendre un test qui ment.

> **Exercices** - `JUN.3` Écrire une classe de test
>
> L'exercice est la forme exacte de la question d'examen, sans la correction : la classe `Moyenne` est juste. Vous écrivez `MoyenneTest`, au moins cinq tests, chacun avec son `@DisplayName`. `RedactionTest` vérifie que votre classe existe et qu'elle a la bonne forme ; elle ne peut pas juger si vos tests sont pertinents. Ce jugement-là, c'est vous qui le portez. Faites l'expérience proposée par le fascicule : glisser un défaut dans `Moyenne`, et regarder si vos tests le voient. C'est la seule mesure honnête de ce que vous avez écrit. Voir `CC-TST.1`, `CC-TST.2` et `CC-TST.3`.

JUN §5 — Du code qui se laisse tester

Pourquoi certaines classes résistent aux tests

Un test prépare une situation, déclenche une action, vérifie le résultat : seul, et en un instant. Certaines classes l'en empêchent.

- **Une classe qui ouvre une fenêtre** ne peut pas tourner dans un test : il faudrait que quelqu'un regarde l'écran.
- **Une classe qui lit la date du jour** donne un résultat qui change d'une année à l'autre : un test juste aujourd'hui sera faux demain.
- **Une classe qui tire au hasard** ne donne jamais deux fois la même chose : que vérifier ?

Dans les trois cas, ce n'est pas le test qu'il faut changer, c'est la classe. Le remède est dans la classe elle-même : une interface à la place de la fenêtre, un paramètre à la place de la date ou du hasard. Retenez l'idée générale : une classe difficile à tester est souvent une classe mal découpée.

Une doublure à la place de la fenêtre

Au module `ITF`, le contrôleur ne dépend plus que d'une interface, `Afficheur`, qu'il reçoit par `setAfficheur` (voir `ITF §4`). Qui affiche, et comment, le contrôleur l'ignore. C'est exactement cette ignorance qui rend le test possible.

```
class AfficheurQuiRetient implements Afficheur {

    String dernier;

    @Override
    public void affiche(String message) {
        dernier = message;
    }
}
```

Pour le test, on écrit un faux afficheur : il implémente `Afficheur`, mais au lieu d'afficher, il range le message dans un champ, que le test pourra lire. On appelle ce genre de remplaçant une *doublure*, comme au cinéma : elle tient le rôle à la place de la vraie, le temps d'une scène. Elle ne sert qu'au test, et vit donc dans `src/test`.

```
@Test
void leControleurSignaleLEvenement() {
    Controleur controleur = new Controleur();
    AfficheurQuiRetient afficheur = new AfficheurQuiRetient();
    controleur.setAfficheur(afficheur);

    controleur.annoncer("bouton presse");

    assertEquals("Evenement : bouton presse", afficheur.dernier,
            "Le controleur doit transmettre l'evenement a l'afficheur");
}
```

Le test suit les trois temps habituels : préparer un contrôleur et lui brancher la doublure, annoncer un événement, vérifier que la doublure a reçu le bon message. Aucune fenêtre ne s'ouvre, et le test tourne en quelques millisecondes. Le contrôleur n'a pas changé d'une ligne pour être testé : c'est le même que celui qui pilotera la vraie fenêtre. Sans l'interface du module `ITF`, ce test serait impossible.

Ni le jour, ni le hasard

Au module `POO`, l'âge d'un livre n'était pas stocké : `getAge` le calculait à partir de la date du jour. Juste, mais difficile à tester : pour *Dune*, paru en 1965, faut-il attendre 61 ou 62 ? Cela dépend du jour où le test tourne. Un test écrit aujourd'hui deviendra rouge le premier janvier, alors que personne n'aura touché au code.

```
public int getAge(int anneeCourante) {           // testable
    return anneeCourante - anneeDeParution;
}
public int getAge() {                             // pratique
    return getAge(LocalDate.now().getYear());
}
```

```
assertEquals(61, dune.getAge(2026),     // vrai aujourd'hui, et dans dix ans
        "Dune, paru en 1965, a 61 ans en 2026");
```

On sépare le calcul de la commodité. Le calcul reçoit l'année en paramètre : le test la fixe, et le résultat ne bouge plus. La version sans paramètre, celle qu'on appelle d'habitude, se contente d'aller chercher l'année et de déléguer. Vous avez déjà fait ce découpage au module `POO`, avec `computeAge(int annee)`.

Le hasard se traite de la même façon : la classe reçoit son `Random` au lieu de le créer, et le test lui en donne un avec une graine, qui tire toujours les mêmes nombres (voir la note sur le générateur pseudo-aléatoire en `ENU §6`). Le mémento Clean Code montre l'exemple complet en `CC-TST.3`.

Ce qu'il faut retenir

- Un test contient la réponse attendue, et vérifie seul ; sans assertion, il ne vérifie rien.
- Trois temps : 1. préparer, 2. agir, 3. vérifier ; `assertEquals(attendu, obtenu, message)`, dans cet ordre.
- Une documentation est une liste de tests, y compris pour les échecs : `assertThrows`, puis l'état.
- Dans un test, on déclare les exceptions ; on ne les attrape jamais.
- Des familles de cas, leurs frontières, et un test par valeur, au nom qui dit ce qu'il vérifie.
- Un test se justifie par le défaut qu'il attrape.
- Une classe difficile à tester est souvent mal découpée : une interface, ou un paramètre, la rend testable.

La leçon qui dépasse ce module : un test rouge ne dit pas qui a tort. L'arbitre est la spécification, et c'est elle qu'on relit avant de corriger quoi que ce soit.

Module GEN

GEN — Types génériques

Écrire une classe une fois, et laisser son utilisateur choisir le type

Depuis le module `COL`, vous écrivez `List<String>` et `Map<String, Integer>` sans vous demander qui a écrit les chevrons. Ce module vous fait passer de l'autre côté : écrire une classe une fois, et laisser celui qui l'utilise décider du type qu'elle contient.

Le module compte six exercices, qui montent sans rupture : un type, deux types, trois, une famille de types, une méthode, un héritage. Il pèse à l'examen : chaque session contient une question entière sur une classe générique, et presque toujours une classe qui enveloppe une collection.

À la fin de ce module, vous saurez

- écrire une classe générique à un ou plusieurs paramètres de type, et l'utiliser ;
- écrire une classe générique qui enveloppe une collection, la forme de la question d'examen ;
- expliquer ce que l'effacement de type permet, et ce qu'il interdit ;
- choisir entre `? extends` et `? super` dans la signature d'une méthode ;
- écrire une méthode générique, y compris une fabrique statique ;
- hériter d'une classe générique, en gardant le paramètre ou en le fixant.

GEN §1 — Un type en paramètre

Quelqu'un a écrit ce `E`

```
// ce que vous avez ecrit dans le module COL
List<String> titres = new ArrayList<>();

// ce qu'a ecrit l'auteur de List (extrait simplifie)
public interface List<E> {
    boolean add(E element);
    E get(int index);
}
```

Le `E` de la déclaration n'est pas un type : c'est une place laissée vide, que chaque utilisateur remplit. Quand vous écrivez `List<String>`, tous les `E` de l'interface valent `String`, et le compilateur vérifie que chaque `add` reçoit bien une `String`.

Imaginez que l'auteur de `List` n'ait pas eu ce `E` à sa disposition. Il aurait fallu une liste capable de tout recevoir, des `Object`. Que se passerait-il alors à la lecture ? Et surtout, à quel moment verrait-on une erreur ?

> **Démonstration** - `Demo_GEN` : l'erreur qui change de camp
>
> La démonstration part d'une classe écrite pour accepter n'importe quel objet, puis la rend générique en n'y changeant presque rien. Regardez à quel moment l'erreur apparaît dans chacune des deux versions, et où se trouve le transtypage. Retenez une phrase plutôt que la syntaxe : la généricité ne rend rien possible qui ne l'était pas déjà, elle avance le moment où l'erreur se voit, de l'exécution à la compilation.

Déclarer un paramètre, le fixer à l'usage

```
public class Casier<T> {
    private T contenu;

    public void deposer(T objet) {
        contenu = objet;
    }

    public T retirer() {
        T objet = contenu;
        contenu = null;
        return objet;
    }
}
```

```
Casier<Valise> casier = new Casier<>();
```

- **`<T>` après le nom de la classe** déclare un paramètre de type.
- **`T` s'écrit ensuite partout où un type peut s'écrire :** champ, paramètre, retour, variable locale.
- **À l'usage, un argument de type prend sa place :** ici, `Valise`.

Deux mots à distinguer, comme pour les méthodes. Le `T` de la déclaration est le *paramètre de type* ; le `Valise` de l'usage est l'*argument de type*. Une méthode reçoit des valeurs, une classe générique reçoit des types.

À l'intérieur de la classe, `T` se comporte comme n'importe quel type, avec une seule limite, et elle compte : la classe ne sait rien de ce que sera `T`. Elle ne peut donc appeler sur un contenu que ce que tout objet sait faire, comme `toString` ou `equals`. Le diamant de droite, vu au module `COL`, laisse le compilateur déduire l'argument de la déclaration.

Un `T` se transmet

```
public class Consigne<T> {
    private final List<T> depots = new ArrayList<>();

    public void deposer(T objet) {
        depots.add(objet);
    }

    public int nombreDeDepots() {
        return depots.size();
    }
}
```

`T` peut servir d'argument à un autre type générique. Le champ est une `List<T>` : le `E` de la liste reçoit le `T` de la consigne. Une `Consigne<Valise>` contient donc une `List<Valise>`, et une `Consigne<String>` une `List<String>`, sans une ligne de plus.

La classe n'invente rien : elle *délègue* à la liste. Ce qu'elle ajoute, c'est un nom et un choix d'opérations : elle n'expose que ce qu'une consigne doit savoir faire.

> **À retenir** - la forme de la question d'examen
>
> C'est la forme de la question 2 de l'examen : une classe générique, une collection privée, quelques méthodes qui délèguent, et un `Programme` qui montre la même classe au service de deux types. Aucun exercice du module ne la pratique telle quelle : regardez-la bien.

> **Exercices** - `GEN.1` Une paire, un seul type
>
> L'exercice applique cette section sur une paire immuable : deux champs, deux accesseurs, un `toString` au format imposé. Faites l'expérience proposée avant d'écrire le `Programme`, et lisez ce que dit le compilateur : c'est la démonstration, refaite par vous-même. La seconde question de fin, `Paire<int>`, mérite d'être essayée vraiment : `GEN §3` dira pourquoi le compilateur refuse.

GEN §2 — Autant de types qu'il en faut

Plusieurs paramètres

```
public class Ticket<K, V> {
    private final K numero;
    private final V objet;

    public Ticket(K numero, V objet) {
        this.numero = numero;
        this.objet = objet;
    }
}
```

```
Ticket<Integer, Valise> t = new Ticket<>(42, valise);
```

Une classe générique peut avoir plusieurs paramètres, déclarés dans les mêmes chevrons et séparés par des virgules. Chacun s'utilise indépendamment des autres, et l'usage donne un argument à chacun, dans l'ordre.

Un ticket de consigne relie un numéro à un objet. Rien n'oblige le numéro à être un nombre, ni l'objet à être une valise : la classe laisse les deux choix à son utilisateur. C'est la forme de la `Map<K, V>` du module `COL`. L'ordre compte : `Ticket<Valise, Integer>` est un autre type, et le compilateur refuserait alors `new Ticket<>(42, valise)`.

Un argument de type peut être générique

```
Map<String, List<Valise>> parVoyageur = new HashMap<>();

Casier<List<Valise>> casierFamille = new Casier<>();

Ticket<Integer, Casier<Valise>> ticket = new Ticket<>(7, new Casier<>());
```

Un argument de type peut contenir lui-même des chevrons ; on le lit de l'extérieur vers l'intérieur, et le compilateur vérifie chaque niveau. La première ligne se lit : une table dont les clés sont des chaînes et les valeurs des listes de valises, la forme rencontrée en `COL §7`.

La deuxième montre que rien ne distingue une classe du JDK d'une classe que vous écrivez : `Casier<List<Valise>>` est un casier qui contient une liste, et sa méthode `retirer` retourne une `List<Valise>`, sans transtypage. Dans la troisième, le diamant fonctionne même imbriqué.

> **Exercices** - `GEN.2` Un couple, deux types, puis `GEN.3` Deux sortes de triplets
>
> `GEN.2` applique la première sous-section ; lisez la remarque de l'énoncé sur le premier des deux couples demandés. `GEN.3` refait le geste deux fois, `TripletMemeType` puis `TripletTroisTypes` : écrivez-les l'une après l'autre, posez-les côte à côte, comptez les lignes qui diffèrent et notez où elles se trouvent. Les deux questions de fin sont l'exercice : discutez-les avant de regarder le corrigé, le compilateur ne peut pas y répondre à votre place.

GEN §3 — Ce que le compilateur efface

Une seule classe à l'exécution

```
Casier<Valise> pourValises = new Casier<>();
Casier<String> pourMessages = new Casier<>();

System.out.println(pourValises.getClass() == pourMessages.getClass());   // true
```

Pour le compilateur, `Casier<Valise>` et `Casier<String>` diffèrent. À l'exécution, il n'existe qu'une classe `Casier`. Les arguments de type n'existent que pour le compilateur : une fois qu'il a tout vérifié, il les efface, et un objet `Casier` ne sait pas de quoi il est le casier. On appelle ce mécanisme l'*effacement de type*, en anglais *type erasure*.

La raison est historique. La généricité est arrivée avec Java 5, en 2004. Tout le code écrit avant devait continuer à tourner sur la même machine virtuelle, sans être recompilé. Le *type brut* que la démonstration a montré, `Casier` sans chevrons, vient de là : c'est ce qui reste d'une classe générique une fois ses arguments effacés.

Vérifier, puis effacer

```
// vous ecrivez
Casier<Valise> casier = new Casier<>();
casier.deposer(new Valise("bleue"));
Valise v = casier.retirer();
```

```
// le compilateur en fait
Casier casier = new Casier();
casier.deposer(new Valise("bleue"));
Valise v = (Valise) casier.retirer();
```

Le compilateur vérifie chaque usage de `T`, puis il remplace `T` par `Object` dans la classe, et il écrit lui-même le transtypage chez l'appelant. Le transtypage n'a donc pas disparu : c'est le compilateur qui l'écrit, à chaque appel.

La différence avec la première version de la démonstration est ailleurs. Ce transtypage-ci ne peut pas échouer, parce que le compilateur a vérifié tous les `deposer` avant de l'écrire. C'est la vérification qui refuse une erreur de type, pas le transtypage : elle a lieu avant l'effacement.

Ce que l'effacement interdit

Refusé	Parce qu'à l'exécution
`Casier<int>`	`T` devient `Object`, et un `int` n'est pas un objet
`new T()`	on ne sait plus quelle classe construire
`new T[10]`	un tableau doit connaître le type de ses cases
`static T champ;`	tous les casiers partagent une seule classe

Chaque refus découle de la même cause : au moment de l'exécution, `T` n'existe plus.

- **La première ligne répond à la question de `GEN.1`.** Une classe générique ne connaît que des objets : on écrit `Casier<Integer>`, et l'autoboxing fait le reste (voir `COL §1`).
- **La troisième explique ce que la démonstration a laissé dans sa classe :** un tableau d'`Object`, et non un tableau de `T`. Un tableau vérifie à l'exécution ce qu'on y range, il doit donc savoir quel type il contient.
- **La dernière tient à ce qu'un membre `static` appartient à la classe** (voir `POO §4`), et que `Casier<Valise>` et `Casier<String>` n'en font qu'une.

Le transtypage qu'on ne peut pas éviter

```
// dans une classe generique, sur un tableau d'Object
@SuppressWarnings("unchecked")
T objet = (T) cases[i];

// au module FIC, en relisant un fichier
List<Valise> valises = (List<Valise>) entree.readObject();
```

Parfois, seul le programmeur sait ce que contient un `Object`. Après effacement, un transtypage vers `T` devient un transtypage vers `Object` : il ne vérifie rien. Un transtypage vers `List<Valise>` ne vérifie que `List`. Si le contenu n'est pas le bon, l'erreur surgira plus tard, à l'endroit où on lira l'élément, c'est-à-dire exactement ce que la généricité devait empêcher. Le compilateur le signale par un avertissement « unchecked ».

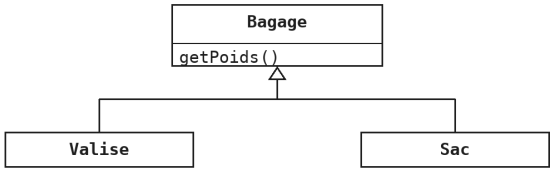
On l'accepte dans deux cas : dans une classe qui contrôle seule ce qu'elle range, comme la classe de la démonstration ; et quand une donnée revient d'un monde qui ignore la généricité, comme un fichier au module `FIC`. L'annotation `@SuppressWarnings("unchecked")` signe alors : « je l'ai vérifié ». Elle se pose sur la plus petite portée possible, une variable ou une méthode, et jamais pour faire taire un avertissement qu'on ne comprend pas.

GEN §4 — Une famille de types

Une `Valise` est un `Bagage`

```
// Valise extends Bagage, Sac extends Bagage

List<Bagage> soute = new ArrayList<>();
soute.add(new Valise("bleue"));       // accepte
soute.add(new Sac());                 // accepte
```



une Valise est un Bagage ; une List<Valise> n'est pas une List<Bagage>

La famille des bagages, qui sert de fil à `GEN §4`

Rien de neuf ici : c'est la substitution du module `ABS`. Partout où l'on attend un `Bagage`, on peut donner une `Valise`. Une liste de bagages peut donc contenir des valises et des sacs mélangés.

Mais une `List<Valise>` peut-elle passer là où l'on attend une `List<Bagage>` ? La réponse est non, et le compilateur le refuse. S'il l'acceptait, la méthode qui reçoit une `List<Bagage>` pourrait y ajouter un `Sac`, et la liste de valises de l'appelant contiendrait un intrus. Une `Valise` est un `Bagage`, mais une `List<Valise>` n'est pas une `List<Bagage>`. Les jokers répondent à ce problème.

> **Exercices** - `GEN.4` Le joker
>
> L'exercice se fait dans un ordre précis, et cet ordre est l'exercice. Écrivez d'abord la méthode avec un `Triplet<Personne>` en paramètre, comme l'énoncé le propose. Passez-lui un `Triplet<Etudiant>`, et lisez le refus du compilateur avant de lire l'encadré. Seulement ensuite, écrivez la version avec le joker.

`? extends` : on lit

```
public static double poidsTotal(List<? extends Bagage> bagages) {
    double total = 0;
    for (Bagage b : bagages) {
        total += b.getPoids();
    }
    // bagages.add(new Sac());   ne compile pas
    return total;
}
```

Le point d'interrogation se lit « quelque chose » : `? extends Bagage` est une liste de quelque chose qui est un `Bagage`. Une `List<Valise>` ou une `List<Sac>` y passent. La méthode ne sait pas quelle liste elle reçoit, mais elle sait que chaque élément est au moins un `Bagage` : elle peut donc lire et peser.

Ajouter, en revanche, lui est interdit. Si la liste reçue est une `List<Valise>`, un `Sac` y serait un intrus, et la méthode n'a aucun moyen de le savoir. Le compilateur refuse donc tout ajout, sauf `null`. Retirer ou vider la liste restent possibles : on ne peut rien y faire entrer, ce qui n'est pas tout à fait « lecture seule ».

`? super` : on ajoute

```
public static void charger(List<? super Valise> soute, int n) {
    for (int i = 0; i < n; i++) {
        soute.add(new Valise("grise"));
    }
}
```

```
charger(new ArrayList<Bagage>(), 3);    // accepte
```

C'est le joker symétrique : `? super Valise` est une liste de quelque chose dont `Valise` hérite. Une `List<Valise>`, une `List<Bagage>` ou une `List<Object>` y passent, et quelle que soit la liste reçue, une `Valise` y a sa place. En revanche, la méthode ne sait pas ce qu'elle lirait : seulement des `Object`.

Le module `COL` a montré `removeIf(Predicate<? super E> filtre)`. Vous pouvez maintenant le lire : un critère écrit pour des `Bagage` sait tester des `Valise`, et peut donc servir sur une `List<Valise>`. Les reconnaître dans les signatures du JDK suffit : en écrire est rare.

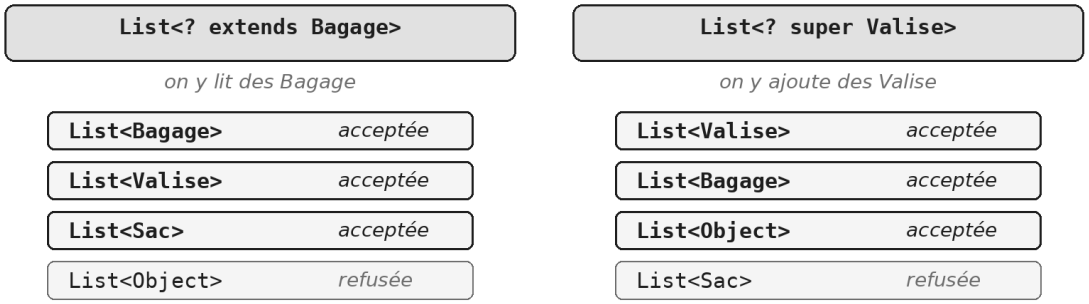
`? extends` ou `? super`

Tout dépend de ce que la méthode fait de la liste : la lire, ou la remplir.

	`? extends Bagage`	`? super Valise`
Accepte	`Bagage`, `Valise`, `Sac`	`Valise`, `Bagage`, `Object`
Lire	oui, des `Bagage`	des `Object` seulement
Ajouter	non	oui, des `Valise`
La liste	fournit	reçoit
Exemple	`poidsTotal(...)`	`charger(...)`

La question à se poser porte sur la méthode, pas sur la liste. Si la méthode lit, si la liste lui *fournit* des éléments, le joker est `extends`. Si elle ajoute, si la liste *reçoit* des éléments, le joker est `super`. Les anglophones retiennent la règle sous le nom PECS, *Producer Extends, Consumer Super*. Une méthode qui fait les deux, lire et ajouter, n'utilise pas de joker : elle prend une `List<Bagage>`.

Des exemples, et des contre-exemples



Ce que chaque joker accepte, et ce qu'il refuse

```
List<Valise> valises = new ArrayList<>();
List<Sac> sacs = new ArrayList<>();
List<Bagage> bagages = new ArrayList<>();
List<Object> objets = new ArrayList<>();

poidsTotal(valises);     // accepte : chaque Valise est un Bagage
poidsTotal(sacs);        // accepte : chaque Sac aussi
poidsTotal(objets);      // refuse : un Object n'est pas forcement un Bagage

charger(bagages, 2);     // accepte : une Valise a sa place parmi des Bagage
charger(objets, 2);      // accepte : et parmi des Object
charger(sacs, 2);        // refuse : une Valise n'a pas sa place parmi des Sac
```

Le raisonnement est toujours le même : se demander ce que la méthode va faire de la liste. `poidsTotal` lit des éléments et les traite comme des `Bagage` : toute liste dont les éléments sont des `Bagage` convient, quelle que soit leur sorte exacte. `charger` ajoute des `Valise` : toute liste où une `Valise` a sa place convient, c'est-à-dire une liste de `Valise` ou d'un type plus général. Le compilateur refuse exactement les cas où la méthode pourrait se tromper.

GEN §5 — Une méthode générique

Le compilateur déduit le `T`

```
public static <T> T auHasard(List<T> elements) {
    int i = (int) (Math.random() * elements.size());
    return elements.get(i);
}
```

```
Valise v = Tirage.auHasard(valises);    // T vaut Valise
String s = Tirage.auHasard(messages);   // T vaut String
Valise w = Tirage.<Valise>auHasard(valises);
```

Une méthode peut déclarer son propre paramètre de type, entre chevrons, avant le type de retour. Le `T` d'une classe est fixé une fois, à la création de l'objet ; celui d'une méthode est fixé à chaque appel, et peut valoir autre chose à l'appel suivant.

Personne ne l'écrit : le compilateur regarde l'argument, voit une `List<Valise>`, et en déduit `T`. Il sait alors que l'appel retourne une `Valise`. La dernière ligne montre la forme explicite ; vous la croiserez dans des documentations, rarement dans du code.

Une méthode statique déclare son `T`

```
public class Casier<T> {
    // ...

    public static <T> Casier<T> contenant(T objet) {
        Casier<T> casier = new Casier<>();
        casier.deposer(objet);
        return casier;
    }
}
```

```
Casier<Valise> casier = Casier.contenant(valise);
```

Le `T` d'une classe appartient à chacun de ses objets. Une méthode statique est appelée sur la classe, sans objet : elle ne le voit pas, pour la même raison que le `static T champ` de `GEN §3`. Elle déclare donc son propre paramètre. Il porte ici la même lettre, comme dans le JDK (`List.of` fait pareil), mais c'est un autre `T` : celui de la méthode, déduit à chaque appel.

C'est une *fabrique*, au sens du module `POO` (voir `POO §4`) : une méthode statique qui construit et retourne un objet prêt à servir.

> **Exercices** - `GEN.5` Une méthode générique
>
> L'exercice déclare un `T` sur une méthode d'une classe qui n'est pas générique ; l'indice de l'énoncé vous évite d'écrire la boucle d'inversion. Lisez la remarque honnête de l'énoncé, puis faites l'expérience de la première question : la méthode réécrite avec un `Object`. Comparez avec `auHasard`, et cherchez ce qui fait la différence entre les deux.

GEN §6 — Hériter d'une classe générique

Deux façons d'hériter

```
public class CasierFerme<T> extends Casier<T> {
    private final String code;

    public CasierFerme(String code) {
        this.code = code;
    }
}
```

```
public class CasierVelo extends Casier<Velo> {
    // deposer(Velo) et retirer() : Velo, herites tels quels
}
```

Une fille peut rester générique, ou fixer le type une fois pour toutes. `CasierFerme` reste générique : on peut créer un `CasierFerme<Valise>` ou un `CasierFerme<Sac>`, et chacun hérite des méthodes de `Casier` pour ce type. `CasierVelo` ne l'est plus : il est un `Casier<Velo>`, et rien d'autre. On écrit `new CasierVelo()` sans chevrons, et sa méthode `retirer` retourne un `Velo`.

Pour le reste, l'héritage fonctionne comme au module `ABS` : redéfinition, `super`, polymorphisme.

Une interface générique

```
public class Vestiaire implements Depot<Manteau> {
    private Manteau accroche;

    @Override
    public void deposer(Manteau m) { accroche = m; }

    @Override
    public Manteau retirer() { return accroche; }
}
```

Une interface aussi peut être générique. L'interface `Depot<T>` déclare ici `deposer(T)` et `retirer()`, qui retourne un `T`. La classe qui l'implémente transmet le type, ou le fixe : `Vestiaire` fixe le type, et dans ses méthodes, `T` est remplacé par `Manteau`, ce que l'annotation `@Override` confirme. `Casier<T>` pourrait de son côté implémenter `Depot<T>`, et transmettre son paramètre.

Vous connaissez déjà plusieurs interfaces génériques : `List<E>`, et les `Predicate<T>`, `Function<T, R>` et `Comparator<T>` du module `COL`. Le module `LAM` vous les fera implémenter sans écrire de classe.

> **Exercices** - `GEN.6` Hériter d'une classe générique, puis `G.31` Un objet par joueur (fil rouge GameHub)
>
> `GEN.6` fournit `Couple<T>` et vous fait écrire `CoupleNomme<T>` ; lisez l'encadré sur `super` et `this` avant d'écrire le constructeur. La première question de fin rejoint le module `ABS` : une méthode de la mère qui appelle du code écrit après elle (`CC-CLS.3`). Côté fil rouge, le jeu de Nim garde deux stratégies dans deux champs jumeaux, et choisit la bonne par un opérateur ternaire. `G.31` vous fait écrire une seule classe, `ParJoueur<T>`, qui garde une chose pour chaque joueur, et l'introduire dans le Nim : la même classe y sert pour des stratégies et pour des joueurs, sans transtypage. Elle stocke la même chose qu'une `Paire<T>` : que dit-elle de plus ?

Ce qu'il faut retenir

- Une classe générique reçoit un type en paramètre ; l'utilisateur le fixe, et le compilateur vérifie chaque usage.
- La généricité ne rend rien possible : elle avance l'erreur de l'exécution à la compilation.
- La forme de l'examen : une classe générique qui enveloppe une collection privée et lui délègue le travail.
- À l'exécution, les arguments de type sont effacés : ni `new T()`, ni `new T[]`, ni `static T`, ni `Casier<int>`.
- Une `List<Valise>` n'est pas une `List<Bagage>` : `? extends` pour lire, `? super` pour ajouter.
- Une méthode peut déclarer son propre `T`, que le compilateur déduit à chaque appel.

La phrase qui résume le module : la généricité ne change pas ce que le programme fait, elle change le moment où l'on découvre qu'il se trompe.

Module LAM

LAM — Lambdas et références de méthodes

Passer un comportement aussi simplement qu'une valeur

Une méthode reçoit d'habitude des valeurs : un nombre, une chaîne, un objet. Ce module montre comment lui passer un comportement, c'est-à-dire ce qu'il faut faire, aussi simplement qu'on lui passe une valeur.

Vous savez déjà le faire, avec une classe qui implémente une interface. Ce module n'ajoute aucune capacité : il ajoute une écriture qui rend la chose assez légère pour qu'on la fasse vraiment, et partout. À l'examen, les lambdas reviennent surtout dans la question `JavaFX` : chaque bouton reçoit le comportement à déclencher quand on clique.

À la fin de ce module, vous saurez

- écrire une classe anonyme, et dire ce qu'elle a de plus et de moins qu'une classe nommée ;
- réécrire une classe anonyme en lambda, quand l'interface est fonctionnelle ;
- expliquer ce qu'est le type cible, et pourquoi il décide de tout ;
- écrire une expression lambda sous ses différentes formes ;
- choisir parmi les interfaces de `java.util.function` avant d'écrire la vôtre ;
- remplacer une expression lambda par une référence de méthode, et donner un nom à un comportement.

LAM §1 — Une classe qui perd son nom

Vous passez déjà des comportements

```
public class TitreLong implements Predicate<String> {

    private static final int LONGUEUR_MAX = 20;

    @Override
    public boolean test(String titre) {
        return titre.length() > LONGUEUR_MAX;    // la seule ligne utile
    }
}
```

```
titres.removeIf(new TitreLong());
```

Vous avez écrit ce code au module `COL` (voir `COL §5`). Regardez-le à nouveau, en vous demandant ce que `removeIf` reçoit vraiment. Un objet, oui. Mais elle ne lui demande rien d'autre que d'appeler `test`. Ce qu'on lui transmet, c'est un comportement : la façon de décider si un titre est trop long.

Pour transmettre cette seule ligne, il a fallu un fichier, une classe, un nom, une annotation, une signature. Quand le critère sert souvent, ce prix se justifie : la classe porte un nom, et on peut la tester seule. Quand il ne sert qu'une fois, le prix devient dissuasif. On finit par écrire une boucle à la place, et le critère se perd dans la boucle.

La classe anonyme

```
titres.removeIf(new Predicate<String>() {
    @Override
    public boolean test(String titre) {
        return titre.length() > LONGUEUR_MAX;
    }
});
```

La première chose à retirer, c'est le nom. Ce code déclare une classe qui implémente `Predicate<String>`, et crée aussitôt son unique objet. La classe n'a pas de nom, d'où son appellation : *classe anonyme*. Elle est écrite à l'endroit même où l'on crée son objet, et ne servira qu'ici.

Lisez-la de l'extérieur vers l'intérieur. `new Predicate<String>()` ne crée pas un objet de l'interface, ce qui est impossible depuis le module `ITF` : les accolades qui suivent sont le corps d'une classe qui l'implémente. Le mot `new` peut aussi être suivi d'une classe abstraite, dont la classe anonyme hérite alors.

La constante `LONGUEUR_MAX` vit désormais dans la classe qui contient cet appel : une classe anonyme voit les champs de la classe où elle est écrite. Le fichier a disparu, mais regardez ce qui reste autour de la ligne utile : cinq lignes « de cérémonie ».

> **Démonstration** - `Demo_LAM` : ce qu'une lambda peut remplacer, et ce qu'elle ne peut pas
>
> La démonstration part de deux classes anonymes qui se ressemblent comme deux gouttes d'eau. L'une implémente une interface, l'autre hérite d'une classe abstraite. IntelliJ propose de transformer la première, pas la seconde. Ne retenez pas le raccourci, retenez la question : qu'est-ce que l'outil sait, et que nous n'avons pas encore dit ? La réponse tient en une idée, le type cible. Retenez aussi le geste de la fin : une lambda et une classe anonyme sont deux écritures de la même chose, et IntelliJ passe de l'une à l'autre dans les deux sens. Quand une lambda vous semble obscure, dépliez-la.

LAM §2 — Ce que le compilateur sait déjà

La lambda : ce qui reste

```
titres.removeIf(new Predicate<String>() {    // impose par removeIf
    @Override
    public boolean test(String titre) {       // la seule methode a ecrire
        return titre.length() > LONGUEUR_MAX;
    }
});
```

```
titres.removeIf( titre -> titre.length() > LONGUEUR_MAX );
```

Reprenez la classe anonyme ligne par ligne, et demandez-vous ce que le compilateur savait déjà :

- que l'interface est `Predicate<String>` : c'est le type du paramètre de `removeIf` ;
- que la méthode est `test` : c'est la seule méthode abstraite de l'interface ;
- que `titre` est une `String`, et que le résultat est un `boolean` : c'est la signature de `test`.

Tout cela s'efface. Il reste ce que le compilateur ne peut pas deviner : le paramètre qu'on passe à la méthode, et ce que la méthode en fait. La flèche les sépare. C'est une *expression lambda*.

Le programme fait exactement la même chose qu'avec `TitreLong`. Seul le coût d'écriture a changé, et avec lui l'envie de le faire.

Rappel : l'interface fonctionnelle

```
@FunctionalInterface
public interface Predicate<T> {      // java.util.function

    boolean test(T t);               // la seule methode abstraite

    // ... plus des methodes default, qui ne comptent pas
}
```

Le module `ITF` a défini le terme (voir `ITF §6`). Une interface est dite *fonctionnelle* quand elle déclare une seule méthode abstraite. Elle peut avoir en plus des méthodes `default` ou `static` : elles ont déjà un corps, et ne comptent donc pas.

C'est ce qui rend la lambda possible. Une lambda n'apporte qu'un corps de méthode, sans nom. Pour que le compilateur sache quelle méthode elle écrit, il n'en faut qu'une à écrire. L'annotation `@FunctionalInterface` n'est pas obligatoire, mais elle protège : si quelqu'un ajoute une seconde méthode abstraite à l'interface, le compilateur refuse, au lieu de casser toutes les lambdas qui l'utilisent.

Le type de la lambda vient du contexte

```
// la variable fixe le type de la lambda
Predicate<String> estLong = titre -> titre.length() > LONGUEUR_MAX;

// le parametre de la methode assertThrows fixe le type de la lambda :
// Executable, une seule methode, execute()
assertThrows(IllegalArgumentException.class, () -> salle.reserver(0));
```

Une lambda seule ne veut rien dire. Le compilateur la comprend en regardant où elle va : la variable qui la reçoit, ou le paramètre auquel on la passe. Ce type de destination s'appelle le *type cible*, et c'est lui qui dit quelle méthode la lambda implémente.

Voilà la règle que la démonstration a fait trouver : le type cible doit être une interface fonctionnelle. S'il avait deux méthodes abstraites, le compilateur ne saurait pas laquelle la lambda écrit. Si c'était une classe abstraite, il faudrait un objet complet, avec son état et son constructeur, alors qu'une lambda n'apporte qu'une méthode.

Le module `JUN` vous avait demandé de recopier `() -> salle.reserver(0)` sans explication (voir `JUN §3`). Le paramètre d'`assertThrows` a pour type `Executable`, une interface de `JUnit` dont la seule méthode, `execute`, ne reçoit rien et ne retourne rien : d'où les parenthèses vides. La lambda emballe l'appel, et `assertThrows` l'exécute quand elle est prête à attraper l'exception.

Une lambda est donc un objet comme un autre : on la range dans une variable, on la passe en paramètre, on la met dans un tableau.

Les formes d'une lambda

```
Runnable saluer = () -> System.out.println("Bonjour");

Comparator<String> parLongueur = (a, b) -> Integer.compare(a.length(), b.length());

Predicate<String> estLong = titre -> {
    String sansEspaces = titre.strip();
    return sansEspaces.length() > LONGUEUR_MAX;
};
```

La forme la plus courante a un seul paramètre et une seule expression. Les autres suivent la signature de la méthode à écrire.

- **Aucun paramètre :** la méthode `run` de `Runnable` ne reçoit rien, et la lambda commence par des parenthèses vides.
- **Deux paramètres :** la méthode `compare` de `Comparator` en reçoit deux, qu'on nomme entre parenthèses, séparés par une virgule. C'est le `ParLongueur` de `COL §6`, en une ligne.
- **Plusieurs instructions :** dès que le corps compte plus d'une instruction, il prend des accolades, et le `return` redevient obligatoire, comme dans une méthode.

Un corps en bloc est permis, mais s'il s'allonge, c'est le signe qu'il mérite un nom. `LAM §3` dit comment le lui donner.

> **Exercices** - `LAM.1` Un tableau de comportements, puis `LAM.2` Écrire son interface fonctionnelle
>
> `LAM.1` range quatre lambdas dans un tableau, et les applique sans savoir ce qu'elles font. Comparez avec `ITF.1` : la même idée y demandait un fichier par calcul. Faites l'expérience de la deuxième question, en ajoutant une méthode à l'interface : regardez sur quelle ligne le compilateur proteste. `LAM.2` passe de l'autre côté : vous écrivez la méthode qui reçoit le critère, et l'interface qui le décrit, avec `@FunctionalInterface`. Le test lui passe des lambdas qu'elle n'a jamais vues, et elle s'en sort sans les connaître. Essayez vraiment la question sur la variable locale : le compilateur a un avis. Pour déplier une lambda, voir `IDE.15` ; pour nommer ce qui n'a pas de nom, `CC-MET.1`.

LAM §3 — Réutiliser ce qui existe

Les interfaces de `java.util.function`

Interface	Méthode	Elle sait
`Predicate<T>`	`boolean test(T)`	répondre oui ou non
`Function<T, R>`	`R apply(T)`	transformer un `T` en `R`
`Consumer<T>`	`void accept(T)`	faire quelque chose d'un `T`
`Supplier<T>`	`T get()`	fournir un `T`

Avant d'écrire une interface fonctionnelle, regardez si le JDK ne l'a pas déjà écrite. Le package `java.util.function` en contient une quarantaine, et ces quatre-là couvrent la plupart des besoins. Vous connaissez les deux premières depuis le module `COL`.

Toutes sont génériques, comme au module `GEN`. Pour les types primitifs, le package contient aussi des variantes qui évitent de passer par `Integer`, comme `IntPredicate` : une question de `LAM.2` vous invite à les chercher. Écrire sa propre interface reste utile quand son nom porte un sens métier ; c'est alors pour le nom, pas pour la signature.

Les méthodes de `COL`, sans classe

```
titres.removeIf(titre -> titre.length() > LONGUEUR_MAX);

titresParAuteur.computeIfAbsent("Zola", auteur -> new ArrayList<>()).add("Germinal");

titres.forEach(titre -> System.out.println(titre));
```

Au module `COL`, `removeIf` et `computeIfAbsent` recevaient des objets de classes nommées, `TitreLong` et `NouvelleListe`. Chacune tient désormais sur une ligne, à l'endroit même où elle sert, et le lecteur voit le critère sans ouvrir un autre fichier.

La méthode `forEach`, que possède toute collection, reçoit un `Consumer` et l'applique à chaque élément, dans l'ordre : c'est la boucle `for-each`, écrite comme un appel de méthode.

Remarquez le paramètre `auteur`, inutile ici puisqu'une liste vide ne dépend de personne. La signature d'`apply` l'impose, comme au module `COL` : la lambda doit le nommer, même sans s'en servir.

La référence de méthode

```
public class Titres {
    private static final int LONGUEUR_MAX = 20;

    public static boolean estLong(String titre) {
        return titre.length() > LONGUEUR_MAX;
    }
}
```

```
titres.removeIf(titre -> Titres.estLong(titre));
titres.removeIf(Titres::estLong);
```

Dernière manière de raccourcir le code. La lambda de la première ligne ne fait rien d'autre que transmettre son paramètre à une méthode qui existe : le nom `titre` entre et ressort aussitôt. On peut alors désigner la méthode elle-même : la classe, deux deux-points, et le nom, sans parenthèses.

`Titres::estLong` n'appelle pas la méthode : il la désigne, pour que `removeIf` l'appelle plus tard, sur chaque titre. Les deux lignes produisent le même objet, un `Predicate<String>`. La condition est la même que pour une lambda : la méthode doit correspondre à la méthode abstraite du type cible. Ici, `estLong` reçoit une `String` et retourne un `boolean`, comme `test`.

Le chemin est complet : une classe nommée, puis une classe sans nom, puis une lambda, puis le seul nom d'une méthode. À chaque étape, la même idée, avec moins à écrire. Entre la lambda et la référence, la différence n'est plus le coût, c'est le nom : une méthode nommée se lit, se réutilise et se teste seule.

> **Pour aller plus loin** - un stream, en passant
>
> Un *stream* fait passer les éléments d'une collection par une suite d'étapes, chacune décrite par une lambda. `titres.stream().filter(titre -> titre.length() > LONGUEUR_MAX).map(titre -> titre.toUpperCase()).collect(Collectors.joining(", "))` ouvre le flux, ne garde que les titres longs (`filter` reçoit un `Predicate`), les met en majuscules (`map` reçoit une `Function`), puis les rassemble en une seule chaîne. Avec quatre titres, « Nana », « La Débâcle », « Germinal » et « Le Docteur Pascal », et une limite de 8 caractères, il en sort « LA DÉBÂCLE, LE DOCTEUR PASCAL ». Contrairement à `removeIf`, la liste d'origine n'est pas modifiée : le stream produit un résultat neuf. Pour un tableau d'entiers, `Arrays.stream(tableau)` ouvre le même enchaînement, et `mapToObj(String::valueOf)` transforme les `int` en chaînes avant de les joindre.

Exercices - `LAM.3` Celle du JDK, et les références de méthodes

L'exercice répond à la question gardée depuis `LAM.2` : votre interface `Filtrage` avait déjà son équivalent dans le JDK. Remplacez-la, et constatez que rien d'autre ne change. La seconde partie fait le dernier dépouillement : trois critères écrits comme des méthodes statiques, chacune sur une ligne, puis passés par référence. Le test vérifie chaque critère seul avant le rapport : c'est ce que le nom permet, et que la lambda ne permet pas. Les questions de fin ouvrent sur les autres formes de référence de méthode. Voir `CC-NOM.1` et `CC-MET.3`.

Vous les retrouverez au module `FX`

```
bouton.setOnAction(evenement -> afficherTotal());
```

Un bouton reçoit le comportement à déclencher au clic. Le type cible est `EventHandler`, une interface fonctionnelle que le module `ITF` citait déjà : sa méthode `handle` reçoit l'événement, et la lambda dit quoi faire quand il survient.

Sans lambda, chaque bouton demanderait une classe, anonyme ou non. Avec elle, l'interface graphique se lit ligne par ligne : tel bouton, tel effet. C'est ce qui rend le code `JavaFX` lisible, et c'est à l'examen que vous en aurez besoin : la question `JavaFX` demande d'écrire ces gestionnaires.

Exercices - `G.32` Une fabrique sans `switch` (fil rouge GameHub)

L'incrément reprend la fabrique de `G.15`. Son `switch` associe chaque type de jeu à la façon d'en créer un ; `G.32` range ces associations dans une `Map`, dont chaque valeur est un `Supplier<Jeu>` : une lambda qui, à chaque appel, fabrique un jeu neuf. Ajouter un jeu revient alors à ajouter une ligne au catalogue. Répondez d'abord à la question qui compte : pourquoi ranger la façon de fabriquer un jeu, et pas le jeu lui-même ? Relisez aussi `G.17` : `StrategieNim` est fonctionnelle, et `StrategieTimide` tiendrait en une lambda, `(pile, regles) -> 1`.

Ce qu'il faut retenir

- Passer un comportement, c'est passer un objet dont on n'attend qu'une méthode.
- Une classe anonyme se déclare et s'instancie au même endroit, sans nom.
- Une lambda écrit la seule méthode d'une interface fonctionnelle : le paramètre, une flèche, le corps.
- Son type vient du contexte : le type cible, qui doit être une interface fonctionnelle.
- `Predicate`, `Function`, `Consumer`, `Supplier` : regardez dans `java.util.function` avant d'écrire votre interface.
- Une référence de méthode, `Classe::methode`, désigne une méthode qui existe ; elle donne un nom au comportement.

Le module n'ajoute aucune capacité au langage. Il en change le coût, et un comportement qui ne coûte plus qu'une ligne se passe enfin partout où il le faut.

Module FX

FX — Interfaces graphiques avec JavaFX

Une fenêtre qui réagit, des valeurs qui se suivent, et un écran séparé du calcul

Jusqu'ici, vos programmes posaient des questions et attendaient la réponse dans la console. Ce module change de rapport : c'est l'utilisateur qui décide de ce qui arrive, et quand. Votre programme attend, et réagit.

Le module est le plus gros du cours, et ses slides sont réparties en trois decks. Le premier apprend à réagir à un geste : un clic, une touche. Le deuxième change de point de vue : au lieu d'écouter ce que fait l'utilisateur, on observe des valeurs, puis on sépare ce qu'on voit de ce qui calcule. Le troisième assemble le tout sur une calculatrice, pas à pas. La question `JavaFX` de l'examen repose surtout sur la première partie : chaque année, elle demande de relier un composant au code qu'il déclenche.

> **À la fin de ce module, vous saurez**
>
> - lancer une application `JavaFX`, et diagnostiquer un démarrage qui échoue ;
> - construire un écran : une fenêtre, une scène, des conteneurs et des contrôles ;
> - brancher un composant sur le code qu'il déclenche, et retirer ce branchement ;
> - lire un événement, et réagir au clavier et à la souris ;
> - expliquer le voyage d'un événement, et l'intercepter avec un filtre ;
> - observer une propriété, et lier une valeur à une autre ;
> - séparer le modèle, la vue et le contrôleur, et décrire une vue en FXML.

FX §1 — Lancer une application JavaFX

JavaFX n'est plus dans le JDK

```
<dependency>
    <groupId>org.openjfx</groupId>
    <artifactId>javafx-controls</artifactId>
    <version>21.0.2</version>
</dependency>
```

`JavaFX` a été livrée avec le JDK pendant quelques années, puis en a été retirée, à partir de Java 11. C'est devenu un projet indépendant, OpenJFX. Installer un JDK ne vous donne donc pas `JavaFX` : il faut l'ajouter au projet, comme n'importe quelle bibliothèque.

Le projet du cours le fait pour vous depuis le premier jour. Ces lignes sont déjà dans son `pom.xml`, à côté de celles de `JUnit`, et Maven a téléchargé la bibliothèque au premier chargement : rien à installer, rien à configurer. Vous trouverez en ligne des procédures qui font télécharger un SDK `JavaFX` et le déclarer dans IntelliJ : ne les suivez pas, elles concernent les projets sans Maven (voir `IDE.28`). Pourquoi Java a connu trois bibliothèques graphiques successives, AWT, Swing et `JavaFX`, le mémento technique l'explique en `TEC-FX.1`.

La classe à lancer s'appelle `Lanceur`

```
public class LanceurLampe {

    public static void main(String[] args) {
        Application.launch(Lampe.class, args);
    }
}
```

Une application `JavaFX` hérite de la classe `Application`. Pourtant, ce n'est pas elle qu'on lance : chaque exercice et chaque démonstration de ce module contient une classe dont le nom commence par `Lanceur`, et c'est elle qu'il faut exécuter.

La raison est un contrôle du lanceur de Java. Quand la classe principale hérite d'`Application`, il exige que `JavaFX` soit chargé d'une façon particulière, comme module. Or Maven place les bibliothèques sur le classpath ordinaire. Le lanceur s'arrête alors avant d'avoir exécuté une seule ligne de votre code. Une classe qui n'hérite de rien ne déclenche pas ce contrôle : son `main` s'exécute, et c'est `Application.launch` qui démarre `JavaFX`. Ce n'est pas un bricolage, c'est la forme recommandée pour une application qui ne passe pas par les modules (`TEC-FX.4`). Vous n'avez rien à écrire dans ces classes : elles sont fournies.

Si rien ne démarre

Le message le plus fréquent des premières minutes est aussi le plus trompeur : « `JavaFX` runtime components are missing ». Rien ne manque. Il signifie seulement que la bibliothèque n'a pas été annoncée à la machine virtuelle au démarrage.

- **Premier réflexe :** vérifier la classe lancée. Vous avez sans doute lancé la classe d'application au lieu du `Lanceur`.
- **En secours :** passer par Maven. Dans la fenêtre Maven, sous Plugins puis javafx, un double-clic sur `javafx:run` construit lui-même la bonne ligne de commande (`IDE.28`). Le message lui-même est expliqué en `ERR.7`.

FX §2 — Une fenêtre et ses composants

Qui appelle `start` ?

```
public class Lampe extends Application {

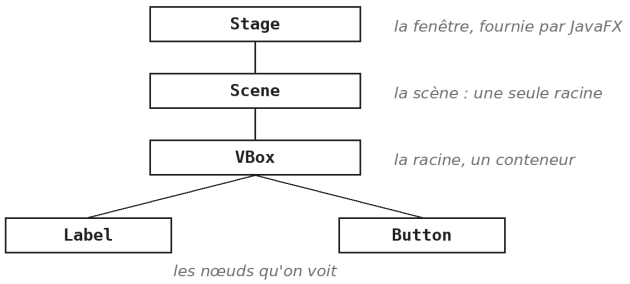
    @Override
    public void start(Stage fenetre) {
        Label etat = new Label("Eteinte");
        fenetre.setScene(new Scene(new StackPane(etat), 300, 150));
        fenetre.setTitle("Lampe");
        fenetre.show();
    }
}
```

Vous n'écrivez jamais `new Lampe()`, et vous ne créez pas la fenêtre. C'est `JavaFX` qui le fait : `launch` construit un objet de votre classe, avec son constructeur sans paramètre, puis appelle sa méthode `start` en lui donnant une fenêtre déjà prête. Votre travail se limite à remplir cette fenêtre et à l'afficher.

La classe `Application` est abstraite, et `start` est sa méthode abstraite : c'est le module `ABS` appliqué à une bibliothèque (voir `ABS §6`). `JavaFX` sait *quand* appeler `start` ; vous seul savez *quoi* y mettre.

Remarquez la dernière ligne. La méthode `start` se termine en quelques millisecondes, mais la fenêtre reste ouverte et le programme continue de tourner. Ce qu'il fait pendant ce temps est le sujet de `FX §3`.

Le graphe de scène



Le graphe de scène : une fenêtre, une scène, une racine, et les nœuds qu'on voit

Tout ce qu'une fenêtre affiche est un *nœud*, un objet d'une sous-classe de `Node` : un libellé, un bouton, un cercle, et aussi les conteneurs qui les rangent. Les nœuds s'emboîtent : une racine contient des enfants, qui contiennent leurs propres enfants. On appelle cet arbre le *graphe de scène*.

Le schéma se lit de haut en bas, et le vocabulaire vient du théâtre.

La fenêtre, le `Stage`, est fournie par `JavaFX` : c'est le plateau sur lequel évoluent les acteurs. Cette fenêtre affiche une scène, la `Scene` : c'est le décor dans lequel les acteurs évoluent.

La scène n'accepte qu'un nœud, sa racine. Et cette racine contient tous les autres nœuds : ce sont eux qu'on voit. Pour afficher plusieurs composants, on les range donc dans un conteneur, qui devient la racine.

Construire le graphe de scène

```
Label etat = new Label("Eteinte");
Button interrupteur = new Button("Interrupteur");

VBox racine = new VBox(etat, interrupteur);
Scene scene = new Scene(racine, 300, 150);

// un enfant peut s'ajouter apres coup
racine.getChildren().add(new Label("Bureau"));
```



La fenêtre obtenue : un libellé et un bouton, rangés dans une VBox

Le même arbre, construit en code, dans l'ordre où on l'écrit : d'abord les nœuds qu'on verra, puis le conteneur qui les range, qui devient la racine, enfin la scène qui reçoit cette racine. Il ne reste qu'à donner la scène à la fenêtre, ce que fait `start`.

Les enfants d'un conteneur se lisent et se modifient par `getChildren`, qui retourne une liste : vous savez vous en servir depuis le module `COL`. Si on ne donne pas de taille à la scène, elle s'adapte à son contenu. Gardez cet arbre en tête : il est la clé de `FX §5`.

Les conteneurs

Conteneur	Range ses enfants	Usage typique
`HBox`	côte à côte	une rangée de boutons
`VBox`	les uns sous les autres	un formulaire
`BorderPane`	en cinq zones	une application
`StackPane`	empilés et centrés	un texte sur une forme

En `JavaFX`, on ne place pas un composant à des coordonnées fixes. On le confie à un conteneur, qui décide de sa position et la recalcule quand la fenêtre change de taille : c'est ce qui permet à une interface de rester lisible quand on l'agrandit.

Un `BorderPane` ne range pas ses enfants dans une liste : il a cinq zones (le haut, le bas, la gauche, la droite et le centre), qu'on remplit une à une par `setTop`, `setCenter` ou `setBottom`.

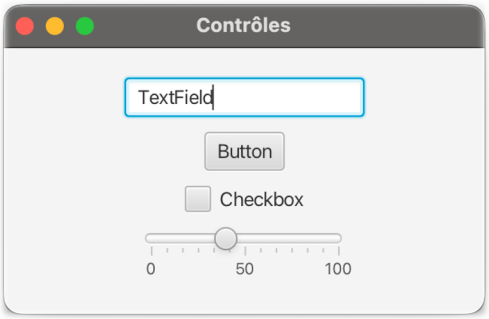
Un `StackPane` empile ses enfants dans l'ordre où on les ajoute : le dernier est au-dessus.

La Javadoc en présente d'autres : `FlowPane`, qui passe à la ligne comme un texte, et `GridPane`, qui range en lignes et en colonnes.

Les contrôles

```
Label consigne = new Label("Piece :");
TextField saisie = new TextField("Bureau");
Button valider = new Button("Valider");

saisie.setPromptText("ex. Cuisine");
String piece = saisie.getText();
saisie.setEditable(false);
```



Quelques contrôles : un `TextField`, un Button, une `CheckBox` et un Slider

Un *contrôle* est un nœud avec lequel l'utilisateur interagit.

- **Un `Label`** affiche un texte que l'utilisateur ne modifie pas.
- **Un `TextField`** reçoit une saisie, que `getText` relit. Un texte d'invite, grisé, s'affiche tant que le champ est vide ; `setEditable(false)` le met en lecture seule.
- **Un `Button`** déclenche une action quand on l'actionne.

Les trois partagent `setText` et `getText`. La Javadoc de chaque contrôle est longue : ne cherchez pas à la lire en entier, cherchez-y ce dont vous avez besoin, au moment où vous en avez besoin. `Ctrl+Q` sur un nom de classe l'ouvre sans quitter l'éditeur (`IDE.14`). La case à cocher et le curseur viendront en `FX §6`.

FX §3 — Brancher un bouton

Le programme attend

Dans un programme console, c'est vous qui décidez de ce qui se passe, comme dans une recette de cuisine : le programme affiche une question, lit la réponse, passe à la suivante. L'ordre est écrit dans le code.

Une interface graphique renverse ce rapport. L'utilisateur peut cliquer sur n'importe quel bouton, dans n'importe quel ordre, ou taper au clavier. Le programme ne sait pas ce qui va arriver : il attend. Chaque geste devient un objet, un *événement*, que `JavaFX` transmet au code qui a demandé à en être prévenu, et à lui seul. On appelle cette façon de programmer la *programmation événementielle*.

Vous avez déjà rencontré ce mécanisme sous un autre nom. Au module `ITF`, le magasin prévenait ses abonnés sans les connaître : c'est le patron Observateur (voir `ITF §5`). Un bouton fonctionne exactement ainsi. Il ne connaît pas votre programme ; il connaît une interface, et votre code vient s'y abonner. Un événement sans abonné se perd.

> **Démonstration** - `Demo_FX` : le bouton qui ne fait rien
>
> La démonstration part d'un bouton qui s'affiche, s'enfonce quand on clique, et ne déclenche rien. Tout y est correct ; il manque une seule chose. Elle le branche ensuite de trois façons qui produisent le même objet. Retenez moins la syntaxe que la question : quel est cet objet que le bouton reçoit, et qu'en fait-il ?

Un gestionnaire est un objet

```
// javafx.event.EventHandler, simplifiee
public interface EventHandler<T extends Event> {
    void handle(T evenement);
}
```

```
interrupteur.setOnAction(evenement -> basculer());
```

Les trois écritures de la démonstration produisent la même chose : un objet qui implémente `EventHandler`, appelé un *gestionnaire*. Le bouton le range, et quand on clique, il appelle sa méthode `handle` en lui passant l'événement.

L'interface est générique : son paramètre dit quel type d'événement le gestionnaire reçoit, un `ActionEvent` pour un bouton (voir `GEN §6`). Elle est aussi fonctionnelle, puisqu'elle ne déclare qu'une méthode : c'est pourquoi une lambda suffit, comme le module `LAM` l'annonçait (voir `LAM §3`). Lisez la lambda en deux temps : la flèche dit *quand*, au moment où l'événement survient ; la méthode appelée dit *quoi*, basculer la lampe. Le paramètre `evenement` est reçu, même si on ne s'en sert pas ici.

`setOnAction` n'est qu'un raccourci

```
private final EventHandler<ActionEvent> basculement = evenement -> basculer();
```

```
private void verrouiller() {
    interrupteur.removeEventHandler(
        ActionEvent.ACTION, basculement);
}
```

```
private void deverrouiller() {
    interrupteur.addEventHandler(
        ActionEvent.ACTION, basculement);
}
```

La méthode `setOnAction` n'accepte qu'un gestionnaire à la fois : un second appel remplace le premier. La forme longue, `addEventHandler`, nomme explicitement le type d'événement écouté, ici `ActionEvent.ACTION`, et elle cumule : deux gestionnaires ajoutés s'exécutent tous les deux.

Pour retirer un gestionnaire, `removeEventHandler` demande l'objet exact qui a été ajouté. Écrire deux fois la même lambda ne suffit pas : chaque écriture crée un objet distinct, et le retrait ne trouverait rien à retirer. C'est la promesse du module `LAM` : une lambda est une valeur, qu'on range dans un champ pour la retrouver plus tard.

C'est aussi une question d'examen classique : activer un bouton et en désactiver un autre, en ajoutant et en retirant leurs gestionnaires.

> **Exercices** - `FX.1` Un compteur de participants, `FX.2` Un bouton Reset, puis `FX.3` Deux compteurs, et une mise en page
>
> Les trois exercices construisent la même application, de plus en plus riche : `FX.1` branche un bouton, `FX.2` en ajoute un second qui agit sur le même compteur, `FX.3` double les compteurs et compose les conteneurs. Ce module n'a aucune classe de test : chaque énoncé se termine par la liste de ce que vous devez observer en lançant l'application, et c'est elle qui remplace la barre verte de `JUnit`. Lisez les deux réflexes de l'introduction du fascicule avant `FX.1` : ils vous éviteront l'erreur la plus fréquente, un composant déclaré dans `start` et introuvable ensuite.

FX §4 — Ce que dit un événement

> **Démonstration** - `Demo_FX` : une touche frappée
>
> La démonstration ajoute un gestionnaire de clavier à l'application de tout à l'heure. Observez deux choses : ce que le gestionnaire reçoit, et l'endroit où on le pose.

Ce que transporte un événement

```
private void noterBouton(ActionEvent evenement) {
    Button presse = (Button) evenement.getSource();
    journal.setText("Dernier bouton : " + presse.getText());
}

// dans start
allumer.setOnAction(this::noterBouton);
eteindre.setOnAction(this::noterBouton);
```

Jusqu'ici, le paramètre du gestionnaire était ignoré. Il contient pourtant tout ce qu'on sait de l'événement : son type (`getEventType`), le composant concerné (`getSource`), et, selon la famille, la touche frappée ou la position de la souris. Un même gestionnaire peut alors servir à plusieurs composants.

La méthode `getSource` retourne un `Object` : il faut le transtyper pour s'en servir comme bouton. Le transtypage est sûr ici, puisque seuls des boutons ont reçu ce gestionnaire. Vous retrouverez ce geste dans la calculatrice, pour un gestionnaire partagé par les dix chiffres. La référence de méthode `this::noterBouton` vaut la lambda `evenement -> noterBouton(evenement)` (voir `LAM §3`).

Les familles d'événements

Classe	Produit par	Types utiles ici
`ActionEvent`	un bouton actionné	`ACTION`
`KeyEvent`	le clavier	`KEY_PRESSED`, `KEY_TYPED`
`MouseEvent`	la souris	`MOUSE_CLICKED`
`WindowEvent`	la fenêtre	`WINDOW_CLOSE_REQUEST`

Toutes ces classes héritent d'`Event`. Chacune déclare des champs statiques qui désignent ses types précis : ce sont eux qu'on passe à `addEventHandler`, comme `ActionEvent.ACTION` en `FX §3`. Il y en a d'autres, comme `KEY_RELEASED`, `MOUSE_MOVED` ou `MOUSE_ENTERED`, et chaque famille a un type `ANY`, qui les regroupe tous. Un `ActionEvent` vient aussi d'un champ de saisie quand on y presse Entrée.

> **Attention** - les imports de `java.awt`
>
> Il existe un `KeyEvent` et un `MouseEvent` dans `java.awt.event`, hérités d'une bibliothèque plus ancienne, et ils n'ont rien à voir avec ceux de `JavaFX`. Quand la complétion vous propose un nom, lisez le package avant de valider : il doit commencer par `javafx` (voir `TEC-FX.1` et `IDE.11`).

> **Exercices** - `FX.4` Compter au clavier
>
> L'exercice reprend l'application de `FX.3` et lui ajoute le clavier : les touches 1, 2 et 0 font ce que font les boutons, qui continuent de fonctionner. L'encadré de l'énoncé dit où poser le gestionnaire. La question de fin demande pourquoi cela fonctionne même quand un bouton a le focus : `FX §5` y répond.

Une touche, deux événements

```
racine.addEventHandler(KeyEvent.KEY_PRESSED, this::toucheEnfoncee);

private void toucheEnfoncee(KeyEvent evenement) {
    switch (evenement.getCode()) {
        case UP -> augmenterIntensite();
        case DOWN -> diminuerIntensite();
        default -> { }
    }
}
```

Une touche enfoncée produit toujours un `KEY_PRESSED`, puis un `KEY_RELEASED` quand on la lâche. Si elle produit un caractère, un `KEY_TYPED` s'y ajoute. Les deux familles ne répondent pas à la même question.

- **`KEY_TYPED` dit quel texte l'utilisateur a tapé :** `getCharacter` le donne.
- **`KEY_PRESSED` dit quelle touche physique il a enfoncée :** `getCode` la nomme par une valeur de l'énumération `KeyCode`.

Les flèches, Échap ou F1 n'ont pas de caractère : seul `KEY_PRESSED` les voit. Dans l'autre cas, `getCharacter` et `getCode` retournent une valeur vide, ce qui explique qu'un gestionnaire de `KEY_TYPED` ne verrait rien de la flèche haut. Le `switch` sur une énumération vient du module `ENU` : les étiquettes s'écrivent sans le nom du type, `UP` et non `KeyCode.UP` (voir `ENU §4`).

La souris

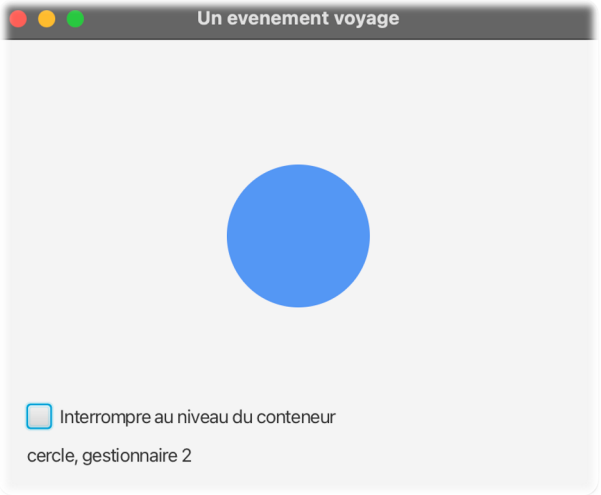
```
ampoule.addEventHandler(MouseEvent.MOUSE_CLICKED, evenement -> {
    if (evenement.getClickCount() == 2) {
        basculer();
    }
});
```

N'importe quel nœud peut recevoir des événements de souris, pas seulement un bouton : ici, l'ampoule est un `Circle`. Un `MOUSE_CLICKED` survient quand on presse puis relâche sur le même nœud. Les méthodes `getX` et `getY` donnent la position dans le nœud, `getClickCount` distingue le simple clic du double clic, et `getButton` dit quel bouton de la souris a servi. Un double clic produit deux `MOUSE_CLICKED`, le premier avec un compte de 1, le second avec un compte de 2 : le test ne réagit donc qu'au second.

Un bouton produit à la fois un `ActionEvent` et des `MouseEvent`, et ce ne sont pas les mêmes déclencheurs. L'`ActionEvent` survient aussi quand on actionne le bouton au clavier, avec la barre d'espace ; le `MOUSE_CLICKED` ne survient qu'à la souris. Quand un énoncé demande d'utiliser le clic de souris comme déclencheur, c'est le second qu'il faut écouter.

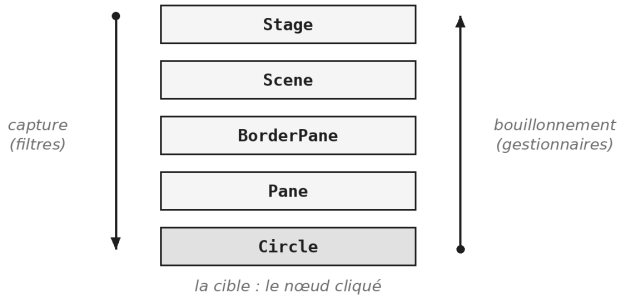
FX §5 — Un événement voyage

> **Démonstration** - `Demo_FX` : un événement voyage
>
> La démonstration pose trois gestionnaires : deux sur un cercle, un sur le conteneur qui l'entoure. Un seul clic les déclenche tous les trois. Notez l'ordre dans lequel ils s'affichent, avant qu'on ne l'explique. Puis une case à cocher change le comportement du conteneur : regardez ce que deviennent les gestionnaires du cercle.



La fenêtre de la démonstration : un cercle dans un conteneur, et la case qui interrompt l'événement au niveau du conteneur

L'aller et le retour



Le voyage d'un événement : il descend jusqu'à la cible, puis remonte

Un clic sur le cercle ne concerne pas que le cercle. Le cercle est dans un conteneur, qui est dans la scène, qui est dans la fenêtre : le clic a eu lieu dans chacun d'eux. `JavaFX` en tient compte en faisant voyager l'événement le long de la branche du graphe de scène qui mène au nœud cliqué, la *cible*.

`JavaFX` trace d'abord le chemin, de la fenêtre jusqu'à la cible, puis le fait parcourir deux fois.

- **La capture :** l'événement descend, du `Stage` vers la cible, en passant par la scène et chaque conteneur.
- **Le bouillonnement :** il remonte le même chemin, de la cible vers le `Stage`.

Chaque nœud traversé peut intervenir, à l'aller comme au retour. C'est ce qui explique la question de `FX.4` : un gestionnaire de clavier posé sur la racine reçoit la touche même quand un bouton a le focus, parce que l'événement remonte jusqu'à elle.

Filtre ou gestionnaire

	Filtre	Gestionnaire
Phase	la descente	la remontée
Ordre	du haut vers la cible	de la cible vers le haut
Ajout	`addEventFilter`	`addEventHandler`
Sert à	intercepter avant les enfants	réagir à ce qui s'est passé

Filtre et gestionnaire sont le même type d'objet, un `EventHandler`. Ce qui les distingue, c'est le moment où on les appelle : un filtre pendant la descente, un gestionnaire pendant la remontée. Les raccourcis `setOnAction` et `setOnMouseClicked` posent des gestionnaires.

Un filtre posé sur un conteneur voit donc passer l'événement avant ses enfants, et avant la cible. Dans la démonstration, le filtre du conteneur s'affichait avant les gestionnaires du cercle : ce n'était pas une question d'ordre d'écriture, mais de phase.

`consume()`

```
panneau.addEventFilter(MouseEvent.MOUSE_CLICKED, evenement -> {
    if (verrouille) {
        evenement.consume();
    }
});
```

La méthode `consume` marque l'événement comme traité : les nœuds suivants du chemin ne le reçoivent plus. Consommé par un filtre, il n'atteint jamais la cible. Les autres gestionnaires du même nœud, eux, sont encore appelés.

C'est ce qu'a fait la case à cocher de la démonstration. On s'en sert pour bloquer une saisie, ou pour qu'un panneau verrouillé ignore les clics destinés à ses enfants, sans toucher au code de ces enfants. Vous connaissez maintenant deux façons de rendre un composant sourd : retirer son gestionnaire, comme en `FX §3`, ou consommer l'événement avant qu'il ne l'atteigne.

> **Exercices** - `FX.5` Un événement voyage, puis `G.33` Le Nim dans une fenêtre (fil rouge GameHub)
>
> `FX.5` fait construire ce que la démonstration a montré, avec un empilement de quatre niveaux : vous verrez le filtre, puis les gestionnaires, et vous constaterez ce qu'un clic à côté du cercle déclenche. La seconde partie tient en une ligne : ne la cherchez pas dans le cercle. `G.33` donne une fenêtre au jeu de Nim, vous contre la machine : un libellé affiche la pile, trois boutons prennent une, deux ou trois allumettes, et les touches 1, 2 et 3 font la même chose. Les classes du Nim servent telles quelles, sauf sa boucle.

FX §6 — Une valeur qui prévient

Réagir à un geste, ou à une valeur

Supposez qu'un libellé doive toujours afficher l'intensité de la lampe. Avec ce que vous savez, il faudrait le mettre à jour dans chaque gestionnaire qui modifie l'intensité : le bouton plus, le bouton moins, la touche flèche, et le jour où on ajoute un curseur, le curseur. Oublier un seul endroit, c'est afficher une valeur fausse.

Le problème est mal posé. Ce qui compte n'est pas le geste, c'est le changement de valeur, qui peut aussi venir d'un collage, d'un calcul ou du programme lui-même. `JavaFX` rend donc les valeurs observables : ce sont les *propriétés*.

Une propriété est un objet

```
private final IntegerProperty intensite = new SimpleIntegerProperty(50);

public int getIntensite() { return intensite.get(); }

public void setIntensite(int valeur) { intensite.set(valeur); }

public IntegerProperty intensiteProperty() { return intensite; }
```

Java n'a pas de notion de propriété dans le langage. Il a une convention, plus ancienne que `JavaFX` : un champ privé, un accesseur `get` et un mutateur `set` qui portent son nom, comme vous en écrivez depuis le module `POO`. `JavaFX` garde cette convention et y ajoute un troisième accès : une méthode dont le nom finit par `Property`, et qui retourne non pas la valeur, mais l'objet qui la contient. C'est sur cet objet qu'on peut s'abonner.

Il existe une classe par type : `IntegerProperty`, `DoubleProperty`, `StringProperty`, `BooleanProperty`.

S'abonner à un changement

```
intensite.addListener((propriete, ancienne, nouvelle) ->
        System.out.println(ancienne + " -> " + nouvelle));

intensite.set(80);   // affiche 50 -> 80
intensite.set(80);   // n'affiche rien : la valeur n'a pas change
```

La méthode `addListener` abonne un `ChangeListener` à la propriété. C'est une interface fonctionnelle, comme `EventHandler` : sa méthode unique, `changed`, reçoit trois paramètres, d'où les trois noms de la lambda. Le premier est la propriété elle-même, qu'on ignore presque toujours ; les deux autres sont l'ancienne valeur et la nouvelle.

Le listener n'est appelé qu'aux changements, jamais à l'abonnement : une fenêtre qui s'ouvre n'affiche donc rien tant que la valeur n'a pas bougé. Et une affectation qui ne change pas la valeur ne déclenche rien.

Les contrôles en ont aussi

```
CheckBox veilleuse = new CheckBox("Veilleuse");

veilleuse.selectedProperty().addListener(
        (propriete, avant, apres) -> passerEnVeilleuse(apres));
```

Presque tout ce qu'affiche un contrôle est une propriété : son texte, sa largeur, sa visibilité, son état coché ou désactivé. Une `CheckBox` a une `selectedProperty`, un `Label` une `textProperty`, un `Slider` une `valueProperty`. On s'y abonne exactement comme à la sienne. La case à cocher prévient quand elle change d'état, qu'on l'ait cochée à la souris, au clavier, ou que le programme l'ait cochée par `setSelected`.

> **Exercices** - `FX.6` Écouter une propriété, puis `FX.7` Un Slider et un Label
>
> `FX.6` vous fait chercher, dans la Javadoc de `TextField`, la propriété à laquelle s'abonner : ne cherchez pas ailleurs que dans l'éditeur, la démarche compte autant que la réponse. `FX.7` refait le geste sur un curseur, et vous fait rencontrer trois pièges, que l'énoncé annonce ; vous connaissez déjà deux des trois : le `Number`, et le listener muet à l'ouverture.

FX §7 — Lier plutôt qu'écouter

Une dépendance déclarée

```
Label affichage = new Label();

affichage.textProperty().bind(intensite.asString());

intensite.set(80);   // le libelle affiche 80, sans autre ligne
```

Un listener dit : *quand ceci change, exécute ce code*. Un *binding* dit : *ceci est égal à cela*. La différence paraît mince, elle ne l'est pas. Avec un listener, c'est votre code qui recopie la valeur, et il peut oublier ou se tromper. Avec un binding, il n'y a plus de code de recopie : la relation est déclarée une fois, et `JavaFX` la maintient, pour toujours.

La méthode `bind` lie une propriété à une valeur observable. Rien à afficher au départ : le lien s'applique dès qu'il est posé. Une propriété liée ne s'écrit plus que par son lien : un `setText` sur elle lèverait une exception.

Des valeurs qui se calculent

Sur	La méthode	Produit
un nombre	`add`, `multiply`	un nombre
un nombre	`greaterThan`, `lessThan`	un booléen
un nombre	`asString`	un texte
un texte	`length`, `isEmpty`	un nombre, un booléen
un booléen	`not`, `and`, `or`	un booléen

Une propriété ne se lie pas seulement à une autre propriété : elle se lie à une expression. Chaque méthode du tableau fabrique une nouvelle valeur observable, qui se recalcule quand ce dont elle dépend change. Les expressions s'enchaînent : `intensite.multiply(2).greaterThan(100)` est un booléen qui suit l'intensité. C'est ainsi qu'on désactive un bouton sans écrire de condition dans aucun gestionnaire : `plus.disableProperty().bind(intensite.greaterThanOrEqualTo(100))`.

Dans les deux sens

```
Slider curseur = new Slider(0, 100, 50);

curseur.valueProperty().bindBidirectional(reglage);

reglage.set(75);   // le curseur se place sur 75
// et si l'utilisateur deplace le curseur, reglage le suit
```

Un binding ordinaire a une source et une cible, et la cible ne s'écrit plus. Le binding bidirectionnel garde deux propriétés égales, dans les deux sens : chacune peut être modifiée, et l'autre suit. C'est ce qu'on veut pour un réglage qu'on peut changer de deux endroits, ici un curseur et une valeur du programme. Aucun exercice ne l'emploie : retenez seulement qu'il existe.

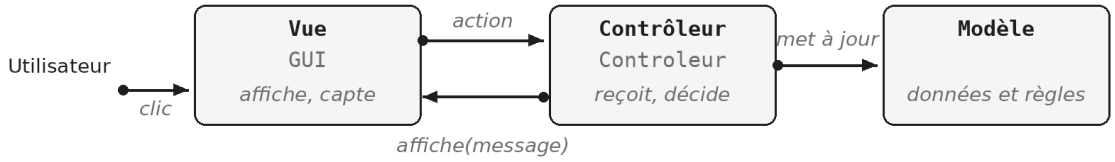
> **Exercices** - `FX.8` Lier plutôt qu'écouter, puis `G.34` Des boutons qui savent quand se taire (fil rouge GameHub)
>
> `FX.8` impose une contrainte : aucun listener, aucun gestionnaire de clavier. Trois composants dépendent d'un champ de saisie, et chaque dépendance s'écrit en une instruction. Comparez votre code à celui de `FX.7` une fois fini : la question de fin demande ce que le binding fait disparaître, et ce qu'il vous fait perdre. `G.34` répond par un binding à une question laissée ouverte par `G.33` : la fenêtre du Nim tient une `IntegerProperty` qui suit la taille de la pile, et chaque bouton se désactive par une seule instruction, celui qui prend trois allumettes quand il en reste moins de trois.

FX §8 — Séparer l'écran du calcul

Trois responsabilités

Jusqu'ici, tout tenait dans une classe : les composants, les gestionnaires, et la règle elle-même. Ça suffit pour un compteur. Dès qu'une application contient une vraie logique, une calculatrice ou un jeu, ce mélange coûte cher. On sépare alors trois rôles, sous le nom de MVC, déjà rencontré au module `ITF` (voir `ITF §4`) :

- **le modèle** tient les données et les règles, et ignore tout de l'écran ;
- **la vue** montre, et ne décide de rien ;
- **le contrôleur** écoute la vue, interroge le modèle, met la vue à jour.



Le découpage MVC, déjà rencontré au module `ITF`

Le gain le plus visible : le modèle se teste par `JUnit`, une fenêtre ne se teste pas (voir `JUN §5`).

Qui connaît qui

```
// le modele, sans JavaFX
public class Lampe {
    private boolean allumee;

    public void basculer() {
        allumee = !allumee;
    }

    public boolean estAllumee() {
        return allumee;
    }
}
```

```
// dans le controleur
private void basculer() {
    lampe.basculer();
    etat.setText(lampe.estAllumee()
            ? "Allumee" : "Eteinte");
}
```

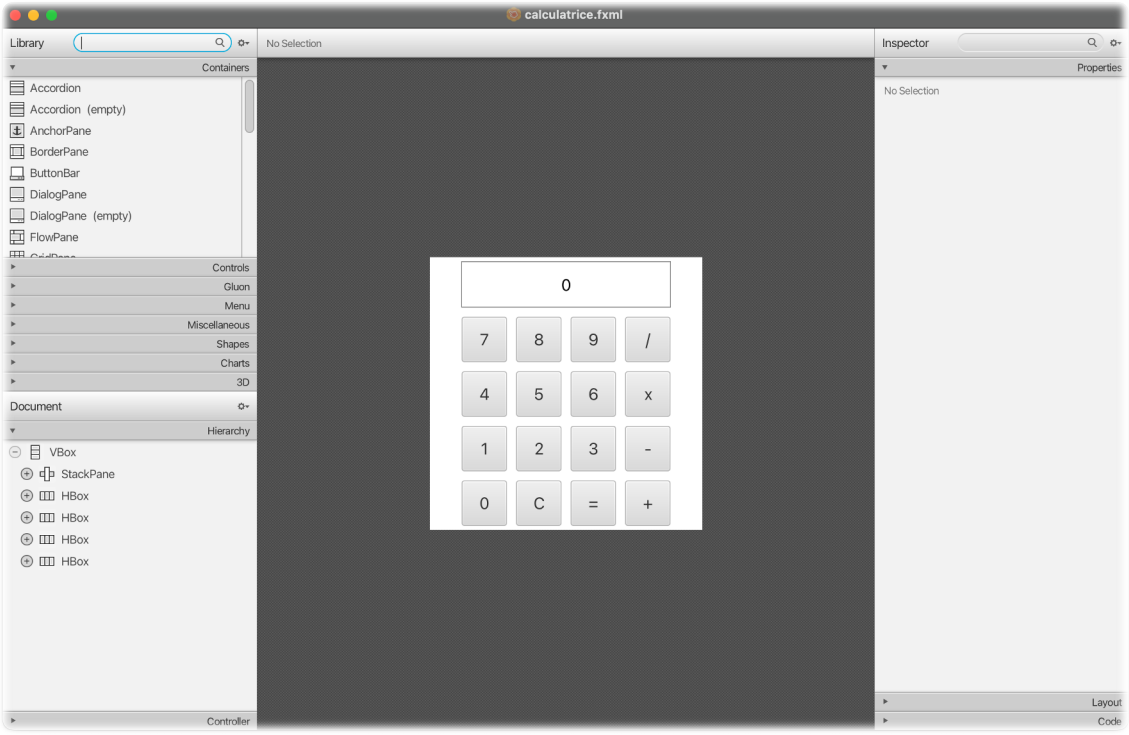
La règle tient dans les dépendances. Le contrôleur connaît le modèle et la vue ; le modèle ne connaît personne ; la vue ne connaît pas le modèle. Le modèle ne connaît ni les boutons ni les libellés : il dit seulement si la lampe est allumée, et sait la basculer. Le contrôleur reçoit le clic, demande au modèle d'agir, puis lit son état pour mettre la vue à jour.

La méthode `basculer` du contrôleur porte le même nom que celle de la lampe, mais elle ne décide plus de rien : elle transmet, puis affiche.

FX §9 — Décrire l'écran en FXML

Écrire l'écran, ou le décrire

Jusqu'ici, la vue était du code Java, dans `start`. Une vue construite en Java mélange deux choses : la structure de l'écran (quels conteneurs, quels contrôles), et la façon de la construire (les `new` et les `getChildren`). Le FXML ne garde que la structure, dans un fichier à part, que `JavaFX` transforme en objets au chargement. Le fichier décrit ce qu'on voit, le contrôleur ce qui se passe. L'outil Scene Builder dessine ce fichier à la souris (`IDE.29`).



La vue de la calculatrice dans Scene Builder : la bibliothèque, l'arbre des nœuds, et l'aperçu

> **À retenir**
>
> Le FXML n'est pas à l'examen. Il sert à la calculatrice de `FX.9`, et à toute application de taille réelle.

Un élément, un objet

```
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.layout.VBox?>

<VBox spacing="10" alignment="CENTER">
    <Label text="Eteinte"/>
    <Button text="Interrupteur"/>
</VBox>
```

Le FXML est du XML : des balises qui encadrent des données. Un élément s'ouvre et se ferme, `<VBox>` puis `</VBox>`, ou se ferme aussitôt, `<Label/>`, quand il n'a pas de contenu. Il contient des attributs, des paires nom et valeur entre guillemets. Le FXML donne un sens précis à chaque morceau :

- **un élément crée un objet** de la classe du même nom ;
- **un attribut appelle le mutateur** du même nom : `spacing` appelle `setSpacing` ;
- **les éléments imbriqués deviennent les enfants ;**
- **les classes s'importent,** comme en Java.

Cette règle vous rend autonome : pour savoir ce qu'on peut écrire sur un élément, on cherche les setters de sa classe dans la Javadoc.

Charger une vue

```
@Override
public void start(Stage fenetre) throws IOException {
    FXMLLoader chargeur = new FXMLLoader(
            ApplicationLampe.class.getResource("lampe.fxml"));

    fenetre.setScene(new Scene(chargeur.load()));
    fenetre.show();
}
```

Un fichier FXML n'est pas du code : il se range dans `src/main/resources`, dans le même package que la classe, et Maven le copie à côté des classes compilées. La méthode `getResource` le retrouve à partir de la classe. Le `FXMLLoader` lit le fichier, crée tous les objets qu'il décrit, et retourne la racine. Il peut échouer si le fichier est absent ou mal écrit : `load` déclare une `IOException`, contrôlée, que `start` laisse passer (voir `EXC §2`).

La vue et son contrôleur

```
public class ControleurLampe {
    private final Lampe lampe = new Lampe();

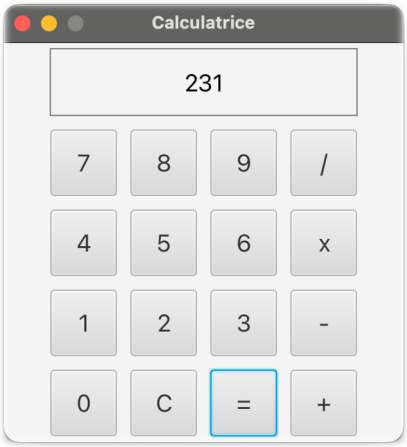
    @FXML private Label etat;

    @FXML
    private void basculer(ActionEvent evenement) {
        lampe.basculer();
        etat.setText(lampe.estAllumee() ? "Allumee" : "Eteinte");
    }
}
```

Côté fichier, la vue porte trois ajouts : `fx:controller` sur la racine, qui nomme le contrôleur ; `fx:id="etat"` sur le libellé ; `onAction="#basculer"` sur le bouton. Au chargement, le `FXMLLoader` crée un objet de la classe désignée par `fx:controller`, sans que vous écriviez de `new`. Puis, pour chaque élément qui porte un `fx:id`, il range l'objet créé dans le champ du contrôleur qui porte ce nom : c'est l'*injection*. Le contrôleur ne crée pas ses composants, il les reçoit. Enfin, `onAction="#basculer"` désigne la méthode à appeler à chaque clic : c'est `setOnAction`, écrit dans le FXML. Le dièse est indispensable, et son oubli est l'erreur la plus fréquente.

L'annotation `@FXML` autorise le chargeur à remplir un champ privé ou à appeler une méthode privée : sans elle, il faudrait les rendre publics.

FX §10 — La calculatrice, pas à pas



La calculatrice de `FX.9`, terminée

Le troisième deck n'apprend rien de neuf : il assemble ce que les sections précédentes ont posé, sur une seule application. C'est l'exercice `FX.9`. Cette section en donne le fil ; le deck et l'énoncé le détaillent étape par étape.

Ce qu'on va construire

Une calculatrice simplifiée : les quatre opérations, sur des entiers seulement, et sans priorité des opérations. Chaque opérateur calcule ce qui précède : `2 x 3 + 4 x 5` donne donc 50, et non 26. Trois fichiers, trois rôles :

```
src/main/java/be/ephec/java/fx/e09/
    ApplicationCalculatrice.java   // fournie
    Lanceur.java                   // fournie
    Controleur.java                // a ecrire
    Modele.java                    // a ecrire
src/main/resources/be/ephec/java/fx/e09/
    calculatrice.fxml              // a ecrire
```

C'est aussi l'ordre de travail : la vue d'abord, parce qu'on la voit tout de suite ; puis le contrôleur, branché sur ses boutons ; enfin le modèle, que le contrôleur appelle.

La vue

Avant d'écrire une ligne, on dessine l'arbre. La racine est une `VBox`, qui empile ses enfants. Son premier enfant est l'écran, un `StackPane` qui superpose un `Rectangle` (le cadre) et un `Text` (le nombre affiché). Les quatre suivants sont les lignes de touches, quatre `HBox` de quatre `Button`. Les dimensions tombent juste : quatre boutons de 50 et trois espaces de 10 font 230, exactement la largeur du cadre.

```
<StackPane alignment="CENTER">
    <Rectangle width="230" height="50" fill="TRANSPARENT" stroke="GRAY"/>
    <Text fx:id="affichage" text="0"/>
</StackPane>

<HBox spacing="10" alignment="CENTER">
    <Button text="7" prefWidth="50" prefHeight="50" onAction="#chiffrePresse"/>
    <!-- ... -->
</HBox>
```

Dans un `StackPane`, le dernier enfant est au-dessus : le texte s'affiche par-dessus le cadre. Le `fx:id` du texte correspond au champ `@FXML private Text affichage` du contrôleur. Les classes utilisées s'importent en tête du fichier, sans quoi le chargement échoue, et Scene Builder refuse de les afficher.

Le contrôleur

Seize touches, mais trois comportements : les dix chiffres ajoutent leur chiffre au nombre affiché, les opérateurs et le « = » déclenchent un calcul, et « C » remet tout à zéro. D'où trois méthodes dans le contrôleur, `chiffrePresse`, `operateurPresse` et `effacer`, et non seize.

```
String chiffre = ((Button) evenement.getSource()).getText();
```

Dix boutons, une seule méthode : c'est l'événement qui dit quelle touche a été pressée (voir `FX §4`). Si l'écran montre « 0 », le chiffre le remplace ; sinon, il s'accole à la fin.

Le modèle

```
public long calculer(long premier, long second, String operateur) {
    return switch (operateur) {
        case "+" -> premier + second;
        // ... les trois autres, et la division par zero
    };
}
```

Le modèle ne connaît rien de `JavaFX` : deux nombres, un opérateur, un résultat. La division par zéro lève une `ArithmeticException`, plutôt que de retourner 0, ce qui ferait passer une erreur pour un résultat (`CC-ERR.1`) : c'est le contrôleur qui décidera d'afficher « Erreur ». Le modèle se vérifie par une classe de test, avant d'être branché.

Une opération se tape en plusieurs clics : entre deux clics, le contrôleur doit se souvenir du premier nombre et de l'opérateur choisi. Ce sont donc des champs, puisqu'une variable locale disparaît à la fin de la méthode :

```
private final Modele modele = new Modele();

private long premierNombre = 0;
private String operateurEnAttente = "";
private boolean nouvelleSaisie = true;
```

Le booléen `nouvelleSaisie` dit si le prochain chiffre commence un nouveau nombre : sans lui, `5 + 6` afficherait 56.

Vérifier, puis aller plus loin

La fenêtre n'a pas de test `JUnit` : la vérification passe par une liste d'observations, dans l'énoncé de `FX.9`. Le modèle, lui, se vérifie en une seconde. C'est toute la différence que fait l'architecture.

Les améliorations possibles sont de bons exercices libres : taper les chiffres au clavier, respecter la priorité des opérations, ajouter des opérations. La priorité change le modèle bien plus que le contrôleur, et c'est le signe d'un bon découpage : la vue n'a pas à bouger.

> **Exercices** - `FX.9` Une calculatrice, en MVC
>
> L'exercice suit exactement le troisième deck : la vue d'abord, puis le contrôleur branché, puis le modèle et sa classe de test, enfin les opérateurs. L'énoncé donne la logique des opérateurs et la liste de ce qu'il faut observer. Pour aller plus loin, un démineur complet est proposé à part, facultatif : vous écrivez la grille et ses règles, la fenêtre est fournie, et onze tests `JUnit` vérifient votre grille sans ouvrir de fenêtre.

Aide-mémoire

Cette page regroupe, par tâche, les classes et les méthodes du module. Elles viennent des packages `javafx.*` : vérifiez l'`import`, et non `java.awt`.

Je veux	J'écris	À savoir
lancer l'application	la classe `Lanceur`, fournie	jamais la classe `Application`
remplir la fenêtre	`start(Stage fenetre)` : `setScene`, `setTitle`, `show`	une `Scene` n'a qu'une racine
ranger des nœuds	`HBox`, `VBox`, `BorderPane`, `StackPane`	`getChildren().add(...)`
lire une saisie	`TextField` : `getText`, `setText`	`Label` pour afficher
réagir à un bouton	`bouton.setOnAction(e -> ...)`	remplace le gestionnaire précédent
ajouter ou retirer	`addEventHandler`, `removeEventHandler`	retirer demande le même objet
réagir au clavier	`addEventHandler(KeyEvent.KEY_PRESSED, ...)`	`getCode()` : `KeyCode.UP`…
réagir à la souris	`addEventHandler(MouseEvent.MOUSE_CLICKED, ...)`	`getClickCount()`, `getX()`
savoir qui a déclenché	`evenement.getSource()`	à transtyper
intercepter avant les enfants	`addEventFilter(...)`, puis `consume()`	pendant la descente
suivre une valeur	`propriete.addListener((p, avant, apres) -> ...)`	rien à l'abonnement
lier une valeur	`cible.bind(source)`, `bindBidirectional`	une cible liée ne s'écrit plus
décrire la vue en FXML	`FXMLLoader`, `fx:id`, `onAction="#methode"`, `@FXML`	le fichier dans `src/main/resources`

Les quatre squelettes suivants sont prêts à adapter.

Une fenêtre, un bouton, un libellé

```
public class Compteur extends Application {
    private int valeur = 0;

    @Override
    public void start(Stage fenetre) {
        Label affichage = new Label("0");
        Button plus = new Button("+1");
        plus.setOnAction(evenement -> {
            valeur++;
            affichage.setText(String.valueOf(valeur));
        });
        fenetre.setScene(new Scene(new VBox(10, affichage, plus), 200, 120));
        fenetre.setTitle("Compteur");
        fenetre.show();
    }
}
```

Activer et désactiver un gestionnaire

```
private final EventHandler<ActionEvent> action = evenement -> agir();

private void activer() {
    bouton.addEventHandler(ActionEvent.ACTION, action);
}

private void desactiver() {
    bouton.removeEventHandler(ActionEvent.ACTION, action);
}
```

Le gestionnaire est rangé dans un champ : c'est le même objet qu'on ajoute et qu'on retire (voir `FX §3`).

Réagir au clavier, sur la racine

```
racine.addEventHandler(KeyEvent.KEY_PRESSED, evenement -> {
    switch (evenement.getCode()) {
        case DIGIT1 -> plusUn();
        case DIGIT0 -> remettreAZero();
        default -> { }
    }
});
```

Lier plutôt qu'écouter

```
IntegerProperty valeur = new SimpleIntegerProperty(0);

affichage.textProperty().bind(valeur.asString());
moins.disableProperty().bind(valeur.lessThanOrEqualTo(0));
```

Une fois liés, le libellé suit la valeur et le bouton se désactive seul à zéro : aucun gestionnaire ne s'en occupe (voir `FX §7`).

Ce qu'il faut retenir

- On lance le `Lanceur`, jamais la classe qui hérite d'`Application`.
- Une fenêtre affiche une scène, qui a une racine ; les conteneurs rangent les nœuds.
- Une interface graphique attend : chaque geste est un événement, transmis à un gestionnaire, un objet qui implémente `EventHandler`.
- `setOnAction` remplace, `addEventHandler` cumule ; pour retirer un gestionnaire, il faut le même objet.

- Un événement descend jusqu'à la cible (les filtres), puis remonte (les gestionnaires) ; `consume` l'arrête.
- Une propriété prévient de ses changements ; un binding déclare qu'une valeur en suit une autre.
- MVC sépare ce qu'on voit de ce qui calcule ; le modèle se teste, la fenêtre se regarde.

Module FIC

FIC — Fichiers

Faire survivre une information à l'arrêt du programme

Tout ce que vos programmes construisent depuis le début de l'année disparaît quand ils s'arrêtent. Ce module montre comment faire persister une information : l'écrire dans un fichier, et savoir la relire.

Il y a deux façons de s'y prendre, le texte et les objets, et elles n'ont pas les mêmes propriétés. Le module ne vous dira pas laquelle choisir : la démonstration et les exercices vous en feront juger vous-même.

À la fin de ce module, vous saurez

- expliquer ce qu'est un flux, et lire le nom d'une classe de flux ;
- emboîter des flux, et fermer un flux quoi qu'il arrive, avec un `try` avec ressources ;
- traiter ou déclarer une `IOException` ;
- écrire un fichier texte, et le relire ligne par ligne en reconstruisant ses champs ;
- écrire un objet entier dans un fichier, et le relire ;
- choisir entre le texte et les objets, en sachant ce que chacun coûte.

FIC §1 — Ce qui entre, ce qui sort

Tout s'efface à l'arrêt

Relancez n'importe quel programme écrit cette année : il repart de zéro. Les objets qu'il avait créés, les saisies de l'utilisateur, tout est perdu, parce que tout vivait dans la mémoire du programme, le temps qu'il tourne. Un fichier, lui, vit sur le disque, et survit à l'arrêt.

Écrire un fichier, c'est envoyer une suite d'octets vers le disque. Un fichier ne contient que des octets : il ne sait pas s'il contient du texte, des nombres ou des objets. C'est le programme qui l'a écrit qui le sait, et celui qui relit le fichier doit le savoir aussi. Reste donc à décider ce que représentent ces octets : c'est la question de tout le module, et il y répond de deux façons très différentes.

Vous écrivez déjà dans un flux

```
// System.out est un flux de sortie, ouvert pour vous
PrintStream console = System.out;
console.println("Uccle : 12 degres");

// System.in est un flux d'entree, lu par le Scanner
Scanner clavier = new Scanner(System.in);
String saisie = clavier.nextLine();
```

Un *flux*, *stream* en anglais, est un tuyau : on y écrit d'un côté, les données sortent de l'autre, dans l'ordre et dans un seul sens. Un flux de sortie reçoit ce que le programme envoie ; un flux d'entrée fournit ce que le programme lit.

Vous utilisez deux flux depuis le premier module sans les avoir nommés.

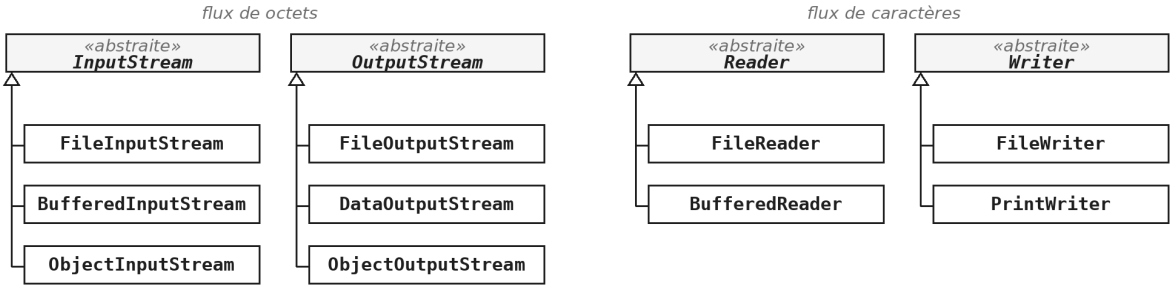
- `System.out` est un objet de la classe `PrintStream`, que la machine virtuelle ouvre vers la console.
- `System.in` est un flux d'entrée branché sur le clavier, que le `Scanner` découpe en lignes.

Écrire dans un fichier ne demande aucune notion nouvelle : on ouvre un autre flux, dont l'extrémité est un fichier au lieu de la console.

Deux familles de flux, quatre classes racines

Famille	Lire	Écrire
Octets, la racine	`InputStream`	`OutputStream`
Octets, vers un fichier	`FileInputStream`	`FileOutputStream`
Caractères, la racine	`Reader`	`Writer`
Caractères, vers un fichier	`FileReader`	`FileWriter`

Toutes les classes de flux du package `java.io` descendent de quatre classes abstraites. Ce sont les *racines* de leurs hiérarchies d'héritage : chaque classe de flux en descend, directement ou non.



chaque classe de flux descend, directement ou non, de l'une des quatre classes abstraites

Les quatre classes racines et quelques-unes de leurs descendantes

Le tableau se lit en deux temps : la famille, puis le sens.

- **Les flux d'octets** transportent des octets tels quels, sans les interpréter.

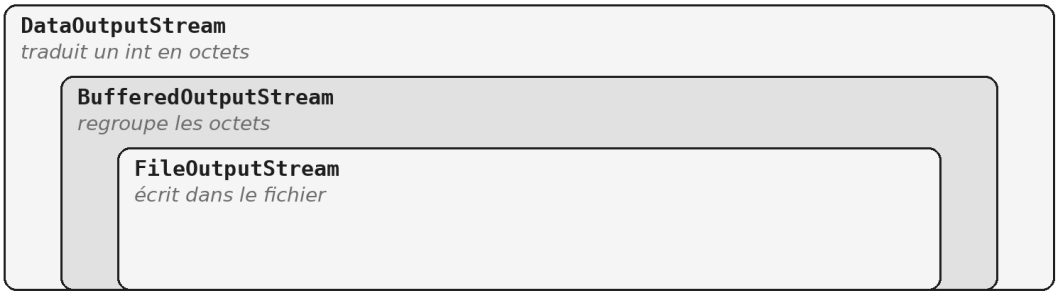
- **Les flux de caractères** transportent du texte : ils convertissent chaque caractère en octets à l'écriture, et les octets en caractères à la lecture, selon un encodage.

Les noms suivent une règle simple. Le suffixe dit la famille et le sens : `InputStream`, `OutputStream`, `Reader`, `Writer`. Le préfixe dit ce que la classe ajoute : `File` pour un fichier, `Buffered` pour un tampon, `Object` pour des objets. Une classe inconnue se range ainsi d'un coup d'œil.

Les flux s'emboîtent

```
OutputStream fichier = new FileOutputStream("temperatures.bin");
OutputStream tampon = new BufferedOutputStream(fichier);
DataOutputStream sortie = new DataOutputStream(tampon);

sortie.writeInt(12);        // quatre octets
sortie.writeDouble(12.5);   // huit octets
sortie.close();             // ferme les trois
```



L'emboîtement des flux : chaque couche ajoute une capacité à celle qu'elle enveloppe

Aucune classe ne fait tout. La première sait où écrire, mais seulement des octets, un par un. La deuxième garde les octets dans un tampon, en mémoire, et ne touche le disque que lorsque le tampon est plein : beaucoup moins d'accès, beaucoup plus rapide. La troisième sait traduire un `int` ou un `double` en octets.

On les emboîte comme des poupées russes, et on ne parle qu'à la dernière. Le constructeur de chaque couche reçoit la couche du dessous : c'est pour cela qu'on lit souvent ces créations sur une seule ligne, de droite à gauche, `new DataOutputStream(new BufferedOutputStream(new FileOutputStream(...)))`. Fermer la couche extérieure ferme toutes les autres.

> **Exercices** - `FIC.1` Trois classes qui se connaissent
>
> L'exercice ne touche à aucun fichier. Il construit trois petites classes, un étudiant inscrit à des cours donnés par des enseignants, que `FIC.2` et `FIC.3` enregistreront de deux façons différentes. Lisez l'encadré sur ce que retourne `getCours` avant d'écrire la méthode : un test vérifie qu'on ne peut pas vider un étudiant à travers sa liste (`CC-OBJ.2`).

FIC §2 — Fermer quoi qu'il arrive

Un flux ouvert se ferme

```
Writer sortie = new FileWriter("journal.txt");
try {
    sortie.write("Uccle;12");
} finally {
    sortie.close();     // meme si write echoue
}
```

Un flux ouvert n'est pas qu'un objet Java. Le système d'exploitation a ouvert le fichier pour lui, et ne le libère qu'à la fermeture. Sur certains systèmes, un fichier resté ouvert ne peut plus être déplacé ni supprimé ; partout, le nombre de fichiers ouverts à la fois est limité.

Il y a plus sournois. La plupart des flux d'écriture gardent ce qu'on leur envoie dans un tampon. Sans fermeture, le tampon n'est jamais vidé et le contenu peut ne jamais atteindre le disque : le fichier existe, mais il est vide.

Le module `EXC` vous a donné l'outil : le bloc `finally` s'exécute toujours, exception ou non (voir `EXC §3`). Il garantit la fermeture même si une erreur survient dans le bloc, à condition de ne jamais l'oublier.

Le `try` avec ressources

```
try (Writer sortie = new FileWriter("journal.txt")) {
    sortie.write("Uccle;12");
}   // sortie.close() est appele ici, exception ou non
```

C'est la même garantie que le `finally`, mais c'est le compilateur qui écrit la fermeture à votre place : on ne peut plus l'oublier, ni mal l'écrire. On déclare la ressource entre les parenthèses du `try`, et elle est fermée à la sortie du bloc, quoi qu'il arrive.

- **La variable déclarée entre parenthèses n'existe que dans le bloc.**
- **On peut en déclarer plusieurs,** séparées par un point-virgule : elles sont fermées dans l'ordre inverse de leur ouverture.
- **Un `catch` et un `finally` peuvent suivre,** comme pour un `try` ordinaire ; la ressource est déjà fermée quand ils s'exécutent.

Toute classe qui implémente l'interface `AutoCloseable` s'y prête. Elle ne déclare qu'une méthode, `close` ; toutes les classes de flux l'implémentent, et `Scanner` aussi. C'est l'écriture que promettait `EXC §3`, et désormais la seule à utiliser pour un fichier.

`IOException`, une exception contrôlée

```
public static String premiereLigne(File fichier) throws IOException {
    try (Scanner entree = new Scanner(fichier)) {
        return entree.nextLine();
    }
}
```

Une entrée-sortie dépend de ce que le programme ne contrôle pas : un fichier peut être absent ou verrouillé, le disque peut être plein. `IOException` est donc une exception contrôlée : comme au module `EXC`, le compilateur vous oblige à décider ce qu'il faut faire quand elle se produit : l'attraper là où l'on peut agir, ou la déclarer et laisser l'appelant décider (voir `EXC §2`). Ici, la méthode déclare.

Ses filles précisent l'échec. Le constructeur de `Scanner` lève une `FileNotFoundException` si le fichier n'existe pas ; comme c'est une fille d'`IOException`, la déclaration de la mère la couvre. `EOFException` signale qu'on a voulu lire au-delà de la fin d'un fichier binaire : `FIC §4` dira quand on la rencontre.

FIC §3 — Écrire et relire du texte

Écrire comme sur la console

```
File fichier = new File("uccle.txt");
try (PrintWriter sortie = new PrintWriter(new FileWriter(fichier))) {
    sortie.println("Uccle;12");
    sortie.printf("%s;%d%n", "Ostende", 9);
}
```

C'est l'emboîtement de `FIC §1`, dans la famille des caractères. Le `FileWriter` sait écrire des caractères dans un fichier ; le `PrintWriter` ajoute `print`, `println` et `printf`, que vous connaissez depuis le premier module (voir `TMP §4`). Tout ce que vous saviez afficher, vous savez maintenant l'écrire.

Le fichier produit contient deux lignes, lisibles dans n'importe quel éditeur. Chaque nombre y a été converti en chiffres, exactement comme `println` le fait à la console.

À l'ouverture, `new FileWriter(fichier)` vide le fichier s'il existait déjà. Pour ajouter des lignes à la fin, on écrit `new FileWriter(fichier, true)`.

> **Attention** - un `PrintWriter` ne lève pas d'exception
>
> Les méthodes de `PrintWriter` ne lèvent pas d'`IOException` : en cas d'échec d'écriture, elles le notent en silence, et seule la méthode `checkError` permet de le savoir. C'est commode pour écrire, mais un disque plein passe inaperçu.

Relire ligne par ligne

```
File fichier = new File("uccle.txt");
try (Scanner entree = new Scanner(fichier)) {
    while (entree.hasNextLine()) {
        System.out.println(entree.nextLine());
    }
}
```

La classe `Scanner` lit un fichier comme elle lit le clavier. Seul le constructeur change : `new Scanner(fichier)` au lieu de `new Scanner(System.in)`. La méthode `hasNextLine` dit s'il reste une ligne, `nextLine` la retourne, et la boucle s'arrête d'elle-même à la fin du fichier.

La lecture n'est pas le miroir de l'écriture. On a écrit des nombres ; on relit des lignes de caractères, et rien d'autre. Retrouver les nombres est l'affaire de la sous-section suivante.

Vous rencontrerez aussi une écriture plus ancienne : un `BufferedReader` posé sur un `FileReader`, dont la méthode `readLine` retourne `null` quand le fichier est épuisé. Les deux font le même travail ; les corrigés du module utilisent `Scanner`.

Une ligne, plusieurs champs

```
String ligne = "Uccle;12";
String[] champs = ligne.split(";");

String station = champs[0];
int temperature = Integer.parseInt(champs[1]);
```

Un fichier texte ne sait pas qu'il contient une station et une température : il contient huit caractères. C'est le *format*, choisi par le programmeur, qui dit où s'arrête un champ et où commence le suivant : ici, un point-virgule entre les champs, et une ligne par relevé. La méthode `split` découpe la ligne, et `Integer.parseInt` reconvertit le texte en nombre (voir `POO §7`).

Ce format est un contrat entre deux morceaux de code : celui qui écrit et celui qui relit. Changez l'un sans l'autre, et la relecture se trompe. Et si un champ peut lui-même contenir le séparateur, un nom avec un point-virgule par exemple, le découpage casse : c'est l'avertissement de `FIC.3`.

Vous croiserez peut-être `StringTokenizer` dans du code plus ancien : cette classe existe toujours, mais sa documentation elle-même conseille d'utiliser `split` à sa place.

Un nom n'est pas un fichier

```
File dossier = new File("releves");
File fichier = new File(dossier, "uccle.txt");

dossier.mkdirs();                       // cree le dossier s'il manque
System.out.println(fichier.exists());
System.out.println(fichier.length());   // en octets
```

Le nom de la classe trompe : un `File` n'est pas un fichier, c'est son adresse. Le créer ne crée aucun fichier ; on construit d'ailleurs un `File` vers un fichier qui n'existe pas chaque fois qu'on s'apprête à en écrire un. Il permet d'interroger le système : `exists`, `length`, `mkdirs`. Les constructeurs de flux acceptent un `File` comme une chaîne, et les signatures des exercices du module en reçoivent un.

Le constructeur à deux arguments, un dossier et un nom, évite d'écrire le séparateur de dossiers à la main. Ce séparateur n'est pas le même sous Windows et sous macOS ; Java le pose pour vous. La méthode `mkdirs` crée le dossier et tous ses parents manquants : sans elle, ouvrir un flux vers un fichier d'un dossier absent lève une `FileNotFoundException`.

> **Pour aller plus loin** - `Path` et `Files`
>
> Depuis Java 7, l'interface `Path` désigne un chemin, comme `File`, et la classe `Files` lui ajoute des raccourcis statiques : `Files.writeString(chemin, "Uccle;12\n")` écrit une chaîne en une ligne, et `Files.readAllLines(chemin)` relit toutes les lignes dans une `List<String>`. Ces méthodes ouvrent et ferment le fichier elles-mêmes, et lèvent les mêmes `IOException`. Un chemin se construit par `Path.of("releves", "uccle.txt")` ; vous trouverez aussi `Paths.get(...)` dans du code plus ancien. `File` n'est pas déprécié pour autant, et les exercices l'emploient : les deux se convertissent l'un dans l'autre par `fichier.toPath()` et `chemin.toFile()`.

FIC §4 — Écrire l'objet tel quel

Un format à inventer pour chaque classe

Un relevé tient sur une ligne : une station, une température. Mais un objet qui en contient d'autres, une liste qui contient des objets qui en contiennent d'autres encore, demande un format plus élaboré, et autant de code pour le relire. Le texte demande, pour chaque classe, un format, une écriture et une relecture, et plus l'objet est riche, plus le format se complique.

Java pourrait-il écrire un objet entier, d'une seule instruction, et le reconstruire à la lecture ? Et si oui, qu'y gagne-t-on, et qu'y perd-on ?

> **Démonstration** - `Demo_FIC` : les mêmes données, deux fois
>
> La démonstration écrit les mêmes données de deux façons : la première avec ce que vous venez de voir, le texte ; la seconde avec une instruction nouvelle. Regardez ce que chaque version demande d'écrire, ce qui échoue, et ce que contient chaque fichier.

Les flux d'objets

```
// ecrire : les methodes de DataOutputStream, plus une
sortie.writeInt(2026);
sortie.writeObject(releve);

// relire : dans le meme ordre
int annee = entree.readInt();
Releve lu = (Releve) entree.readObject();
```

`sortie` est un `ObjectOutputStream` posé sur un `FileOutputStream`, et `entree` un `ObjectInputStream` posé sur un `FileInputStream`, chacun ouvert dans un `try` avec ressources. C'est l'emboîtement de `FIC §1`, avec une couche qui sait aussi écrire des objets : elle ajoute `writeObject` aux méthodes des types primitifs. On appelle cette écriture la *sérialisation*.

Le fichier ne garde pas d'étiquettes : la lecture doit suivre l'ordre de l'écriture, un `int` puis un objet. Lire dans un autre ordre donne des valeurs fausses ou une exception. La méthode `readObject` retourne un `Object`, d'où le transtypage. Si la variable est générique, une `List<Releve>` par exemple, le transtypage ne peut pas être vérifié : c'est l'un des deux cas où l'on accepte l'avertissement « unchecked » (voir `GEN §3`).

Une étiquette, pas un contrat

```
public class Releve implements Serializable {
    private final String station;
    private final int temperature;

    // constructeur et accesseurs : rien d'autre a ajouter
}
```

Au module `ITF`, une interface était un contrat : des méthodes que la classe s'engage à fournir. `Serializable` n'en déclare aucune : c'est un marqueur, une étiquette (voir `ITF §6`). La classe donne son accord pour que ses objets soient transformés en octets.

Le compilateur ne vérifie rien : une classe sans l'étiquette compile, et c'est l'écriture qui échoue, une fois le programme lancé, par une `NotSerializableException`. La démonstration a montré ce qui arrive. Les classes du JDK qui contiennent des données sont presque toutes déjà étiquetées, comme `String`, `Integer`, `ArrayList` ou `LocalDate` : c'est pourquoi une liste de chaînes ou de dates s'écrit sans qu'on y pense, alors qu'une classe écrite par vous doit le demander. Et tous les objets qu'un objet contient doivent l'être aussi.

Où s'arrête un fichier

Méthode de lecture	Comment s'annonce la fin
`Scanner.nextLine`	`hasNextLine()` retourne `false` avant
`BufferedReader.readLine`	la méthode retourne `null`
`DataInputStream.readInt`	elle lève une `EOFException`
`ObjectInputStream.readObject`	elle lève une `EOFException`

Une lecture finit toujours par atteindre la fin du fichier, et chaque classe le dit à sa manière. Le texte prévient : on demande avant de lire, ou on reçoit `null`. Le binaire ne prévient pas : on lit, et une exception répond qu'il n'y avait plus rien. *EOF* signifie *end of file*, fin de fichier.

L'exception signale ici une situation normale, et non une erreur. On peut boucler jusqu'à elle, mais il est plus simple de savoir d'avance combien d'éléments relire : par exemple en écrivant d'abord leur nombre, ou en écrivant la liste entière en un seul `writeObject`.

> **Exercices** - `FIC.2` Enregistrer des objets, `FIC.3` Enregistrer du texte, puis `G.35` Un journal qui survit (fil rouge GameHub)
>
> `FIC.2` et `FIC.3` enregistrent exactement les mêmes étudiants, l'un en objets, l'autre en texte. Faites-les dans cet ordre, et ouvrez les deux fichiers produits, comme dans la démonstration. `FIC.2` vous fournit les classes de `FIC.1`, déjà étiquetées. `FIC.3` vous impose un format : lisez l'avertissement sur `split` avant d'écrire la relecture. Les questions de fin des deux exercices n'ont pas de réponse ici : cherchez-les vraiment, elles disent ce qu'on choisit en choisissant un format. Côté fil rouge, le journal de GameHub ne garde rien depuis `G.13` ; `G.35` lui donne une mémoire : chaque partie terminée s'écrit dans un fichier, relu au lancement suivant pour afficher l'historique. Avant d'écrire une ligne, décidez du format, avec la question des exercices : qui lira ce journal, et quand ?

Aide-mémoire

Cette page regroupe, par tâche, les classes et les méthodes du module. Toutes viennent de `java.io`, sauf `Scanner`, de `java.util`.

Je veux	Je construis	J'appelle	À savoir
écrire du texte	`new PrintWriter(new FileWriter(fichier))`	`println`, `printf`	ne lève rien : `checkError`
écrire à la suite	`new FileWriter(fichier, true)`	idem	sans `true`, le fichier est vidé

Je veux	Je construis	J'appelle	À savoir
relire du texte	`new Scanner(fichier)`	`hasNextLine`, `nextLine`	`FileNotFoundException`
découper une ligne	rien : la ligne lue est une `String`	`split(";")`, `Integer.parseInt`	`NumberFormatException`
écrire des objets	`new ObjectOutputStream(new FileOutputStream(fichier))`	`writeObject`, `writeInt`	classe `Serializable`
relire des objets	`new ObjectInputStream(new FileInputStream(fichier))`	`readObject`, puis transtyper	`EOFException` en fin
désigner un fichier	`new File(dossier, nom)`	`exists`, `mkdirs`, `length`	ne crée rien

Chaque flux s'ouvre dans un `try` avec ressources, et la méthode qui l'ouvre déclare `throws IOException`, sauf à l'attraper. Les quatre squelettes suivants sont prêts à adapter.

Écrire des lignes

```
public static void ecrire(List<Releve> releves, File fichier) throws IOException {
    try (PrintWriter sortie = new PrintWriter(new FileWriter(fichier))) {
        for (Releve r : releves) {
            sortie.println(r.station() + ";" + r.temperature());
        }
    }
}
```

Relire des lignes, et les découper

```
public static List<Releve> relire(File fichier) throws IOException {
    List<Releve> releves = new ArrayList<>();
    try (Scanner entree = new Scanner(fichier)) {
        while (entree.hasNextLine()) {
            String[] champs = entree.nextLine().split(";");
            releves.add(new Releve(champs[0], Integer.parseInt(champs[1])));
        }
    }
    return releves;
}
```

Écrire une liste d'objets

```
public static void ecrire(List<Releve> releves, File fichier) throws IOException {
    try (ObjectOutputStream sortie =
            new ObjectOutputStream(new FileOutputStream(fichier))) {
        sortie.writeObject(releves);    // la liste entiere, en une fois
    }
}
```

Relire une liste d'objets

```
@SuppressWarnings("unchecked")
public static List<Releve> relire(File fichier)
        throws IOException, ClassNotFoundException {
    try (ObjectInputStream entree =
            new ObjectInputStream(new FileInputStream(fichier))) {
        return (List<Releve>) entree.readObject();
    }
}
```

La méthode `readObject` déclare aussi une `ClassNotFoundException`, contrôlée : la classe des objets relus doit exister dans le programme qui les relit. Écrire la liste en un seul `writeObject` évite de se demander où s'arrête le fichier : un seul `readObject` la reconstruit entière. Dans ces squelettes, `Releve` est un `record`

`Releve(String station, int temperature) implements Serializable` : ses accesseurs s'appellent `station()` et `temperature()` (voir ENU §7).

Ce qu'il faut retenir

- Un fichier ne contient que des octets ; c'est le format qui leur donne un sens.
- Un flux va dans un seul sens ; octets ou caractères, on les emboîte et on ne parle qu'à la couche extérieure.
- Un flux ouvert se ferme : le `try` avec ressources le garantit.
- `IOException` est contrôlée : on la traite, ou on la déclare.
- Le texte : `PrintWriter` pour écrire, `Scanner` pour relire, `split` et `parseInt` pour retrouver les champs.
- Les objets : `writeObject` et `readObject`, dans le même ordre, sur des classes étiquetées `Serializable`.

Le texte se lit par un humain et par n'importe quel programme, au prix d'un format à écrire et à relire ; les objets s'écrivent en une instruction, au prix d'un fichier que seul Java sait relire. Le choix dépend de la question des exercices : qui lira ce fichier, et quand ?

Annexe A

Annexe A — Guide IntelliJ IDEA

Installer, ouvrir, écrire, exécuter, dépanner

À quoi sert ce guide

IntelliJ IDEA est l'outil dans lequel vous passerez toutes vos heures de Java cette année. Ce guide n'en fait pas le tour — personne n'en fait le tour — mais il couvre ce dont vous aurez besoin, dans l'ordre où vous en aurez besoin.

Les deux premières sections servent le premier soir : installer, et ouvrir le projet du cours. Les suivantes servent toute l'année et se consultent au coup par coup. L'annexe rassemble les messages d'erreur les plus fréquents, avec leur cause et leur remède.

Chaque section porte un code — `IDE.17` pour le lancement des tests, par exemple — et les énoncés d'exercices y renvoient au moment utile. Les codes sont cliquables dans ce document.

Deux conventions

Les noms de menus sont donnés en anglais, tels qu'ils apparaissent à l'écran dans l'installation par défaut : `File → Project Structure`. Les explications, elles, restent en français.

Les raccourcis sont donnés dans les deux systèmes, séparés par une barre oblique : `Ctrl+Alt+L` sous Windows et Linux, `⌥⌘L` sous macOS. Les symboles Mac se lisent ⌘ commande, ⌥ option, ⇧ majuscule, ^ contrôle.

Les captures d'écran

Elles ont été prises sous macOS avec un thème sombre. Sous Windows, les fenêtres ont une autre apparence mais le même contenu, et les menus portent les mêmes noms. Ne vous laissez pas troubler par les couleurs.

Sommaire

Avant de commencer	
`IDE.1`	Quelle édition d'IntelliJ
`IDE.2`	Installer IntelliJ
`IDE.3`	Le JDK ne vient pas avec IntelliJ

Ouvrir le projet du cours	
`IDE.4`	Ouvrir par le pom.xml, jamais par le dossier
`IDE.5`	Installer un JDK depuis IntelliJ
`IDE.6`	Vérifier le SDK du projet

L'écran	
IDE.7	Les quatre zones
IDE.8	La fenêtre Project, créer une classe
IDE.9	La fenêtre Maven
IDE.10	Ouvrir et fermer les fenêtres d'outils

Écrire du code	
IDE.11	Complétion et import automatique
IDE.12	Générer accesseurs et constructeurs
IDE.13	Les modèles : psvm, sout, fori
IDE.14	Lire la Javadoc sans quitter l'éditeur
IDE.15	Les avertissements et leurs corrections

Exécuter	
IDE.16	Lancer une classe
IDE.17	Lancer les tests et lire le rapport
IDE.18	Passer des arguments au programme
IDE.19	Le débogueur

Transformer du code sans le casser	
IDE.20	Renommer
IDE.21	Déplacer une classe
IDE.22	Extraire une méthode

Naviguer	
IDE.23	Aller à la déclaration, et revenir
IDE.24	Chercher n'importe quoi
IDE.25	La structure d'un fichier

Raccourcis et réglages	
IDE.26	Les raccourcis à connaître
IDE.27	Plugins et apparence
IDE.28	JavaFX : rien à configurer, ou presque
IDE.29	Installer Scene Builder

Annexe — Erreurs et solutions	
ERR.1	JDK isn't specified for module
ERR.2	Cannot resolve symbol
ERR.3	Tout est rouge sans raison
ERR.4	Pas de flèche verte sur une classe de test
ERR.5	Deux classes du même nom

Annexe — Erreurs et solutions	
ERR.6	Lancer malgré une erreur de compilation
ERR.7	JavaFX runtime components are missing
ERR.8	source level should be in … : 21
ERR.9	Un exercice inachevé bloque tous les autres
ERR.10	Un fichier disparaît de la fenêtre Project
ERR.11	Les modules Maven ont disparu : plus rien ne se lance

A — Avant de commencer

IDE.1 — Quelle édition d'IntelliJ

IntelliJ existe en deux éditions. L'édition **Community** est gratuite et suffit intégralement pour ce cours : elle gère Java, Maven, `JUnit` et le débogueur. L'édition **Ultimate** est payante et ajoute des outils dont nous n'aurons pas l'usage — serveurs d'applications, frameworks web, bases de données.

Prenez Community. Rien dans ce guide ni dans les exercices ne demande Ultimate.

IDE.2 — Installer IntelliJ

Téléchargez depuis `jetbrains.com/idea/download`, en veillant à prendre la version correspondant à votre machine. Sur un Mac récent, choisissez la variante `Apple Silicon` et non `Intel` : la seconde fonctionne, mais plus lentement.

L'installation ne demande aucun réglage particulier. Au premier démarrage, IntelliJ propose d'importer des paramètres existants : répondez que vous n'en avez pas.

IDE.3 — Le JDK ne vient pas avec IntelliJ

C'est la source de confusion la plus fréquente en début d'année, alors autant l'énoncer tout de suite.

IntelliJ est un *éditeur*. Il ne compile pas votre code lui-même : il confie ce travail à un JDK, qui est un logiciel distinct. Installer IntelliJ n'installe donc pas Java, et sans JDK rien ne compilera.

IntelliJ affiche bien un numéro de version de Java dans sa fenêtre `About` — c'est celui du Java qui fait tourner IntelliJ lui-même, pas celui qui compilera votre code. Les deux n'ont aucun rapport.

Deux façons d'installer un JDK. Soit vous le téléchargez chez son éditeur et vous l'installez comme n'importe quel logiciel. Soit vous le laissez faire à IntelliJ, ce qui est plus simple et se fait à l'étape suivante, une fois le projet ouvert. Nous prendrons cette seconde voie : voir IDE.5.

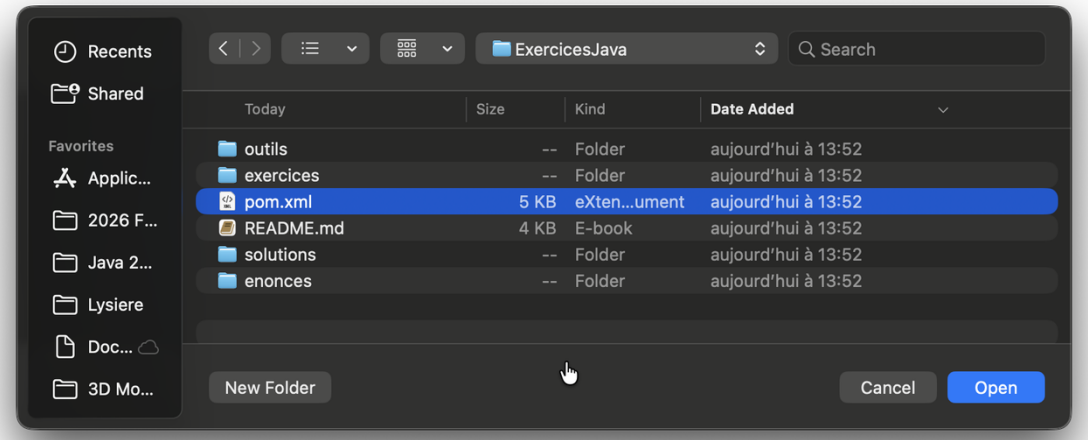
> **Quelle version, quelle distribution**
>
> Ce cours utilise **Java 21**, qui est une version à support long terme. N'installez pas une version plus récente : votre code pourrait compiler chez vous et pas à l'examen.
>
> La distribution — Amazon Corretto, Eclipse Temurin, Oracle — importe peu : ce sont des empaquetages du même Java. Corretto et Temurin sont gratuits sans condition.

B — Ouvrir le projet du cours

IDE.4 — Ouvrir par le pom.xml, jamais par le dossier

Le projet du cours est un projet **Maven** composé de trois modules Maven, `exercices`, `solutions` et `demos`. Sa description tient dans un fichier `pom.xml` placé à la racine.

`File → Open`, puis sélectionnez le fichier `pom.xml` lui-même — pas le dossier qui le contient. IntelliJ propose alors « Open as Project » : acceptez.



Sélectionner le `pom.xml` de la racine, et non le dossier.

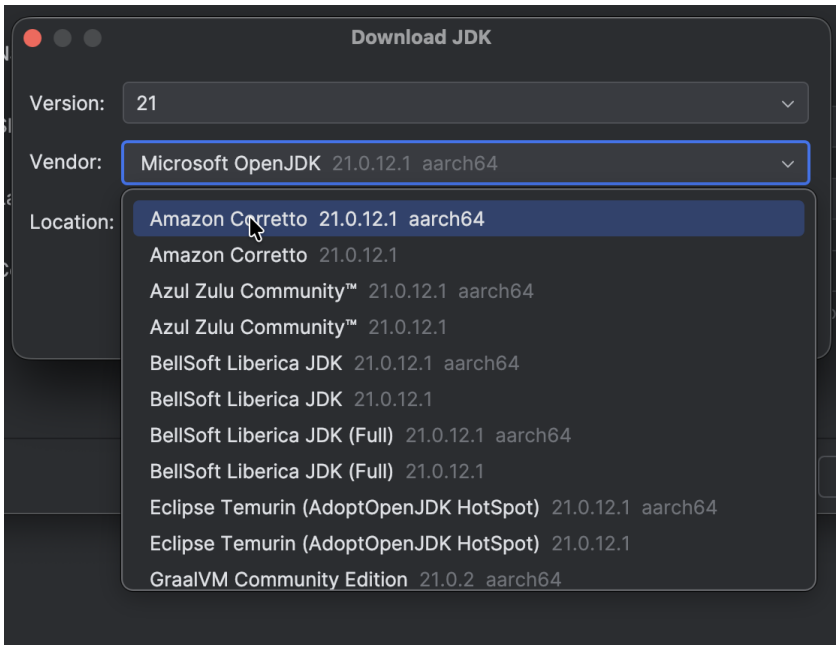
Le `pom.xml` est le fichier qui décrit le projet à Maven : ses modules, ses bibliothèques, sa version de Java. Le mémento technique le détaille en `TEC-MVN.2`, et explique à quoi sert Maven en `TEC-MVN.1`. Notre projet en compte plusieurs modules — voir `TEC-MVN.3`.

Pourquoi cette insistance. Si vous ouvrez le dossier, IntelliJ crée parfois des modules ordinaires au lieu de modules Maven. Ces modules n'ont pas de JDK, ne connaissent pas les bibliothèques déclarées dans le `pom.xml`, et rien ne compile. C'est la cause de l'erreur décrite en `ERR.1`.

Au premier chargement, IntelliJ télécharge les bibliothèques dont le projet a besoin — `JUnit`, `JavaFX`. Comptez une à deux minutes, avec une barre de progression en bas de la fenêtre. C'est un téléchargement unique : les fois suivantes, l'ouverture est immédiate.

IDE.5 — Installer un JDK depuis IntelliJ

`File → Project Structure → Project`. Dépliez la liste du champ `SDK` : elle contient les JDK qu'IntelliJ a trouvés sur votre machine, et une entrée `Download JDK…` en bas.



La liste du champ SDK, avec l'entrée Download JDK.

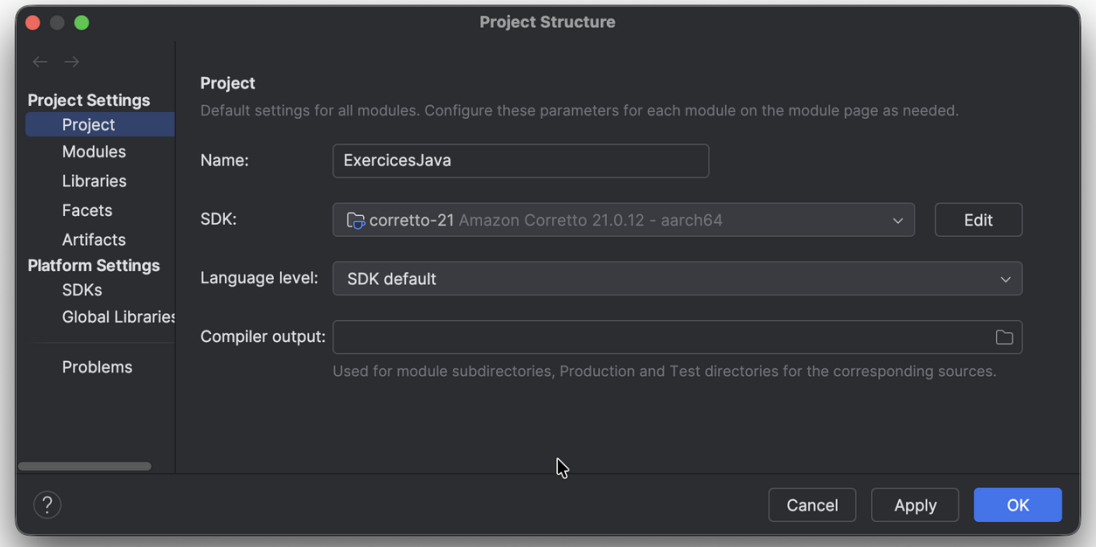
Choisissez la version 21 et une distribution Amazon Corretto ou Eclipse Temurin. IntelliJ télécharge et installe le JDK tout seul, puis le sélectionne.

Attention à l'architecture. Sur un Mac Apple Silicon, prenez la variante `aarch64` et non `x86_64`. Si la liste ne le précise pas, elle propose déjà la bonne.

IDE.6 — Vérifier le SDK du projet

Toujours dans `File → Project Structure → Project`, deux champs comptent.

- **SDK** : le JDK qui compilera votre code. Il doit nommer un JDK 21.
- **Language level** : la version du langage que vous êtes autorisé à écrire. Mettez-la sur 21 également.

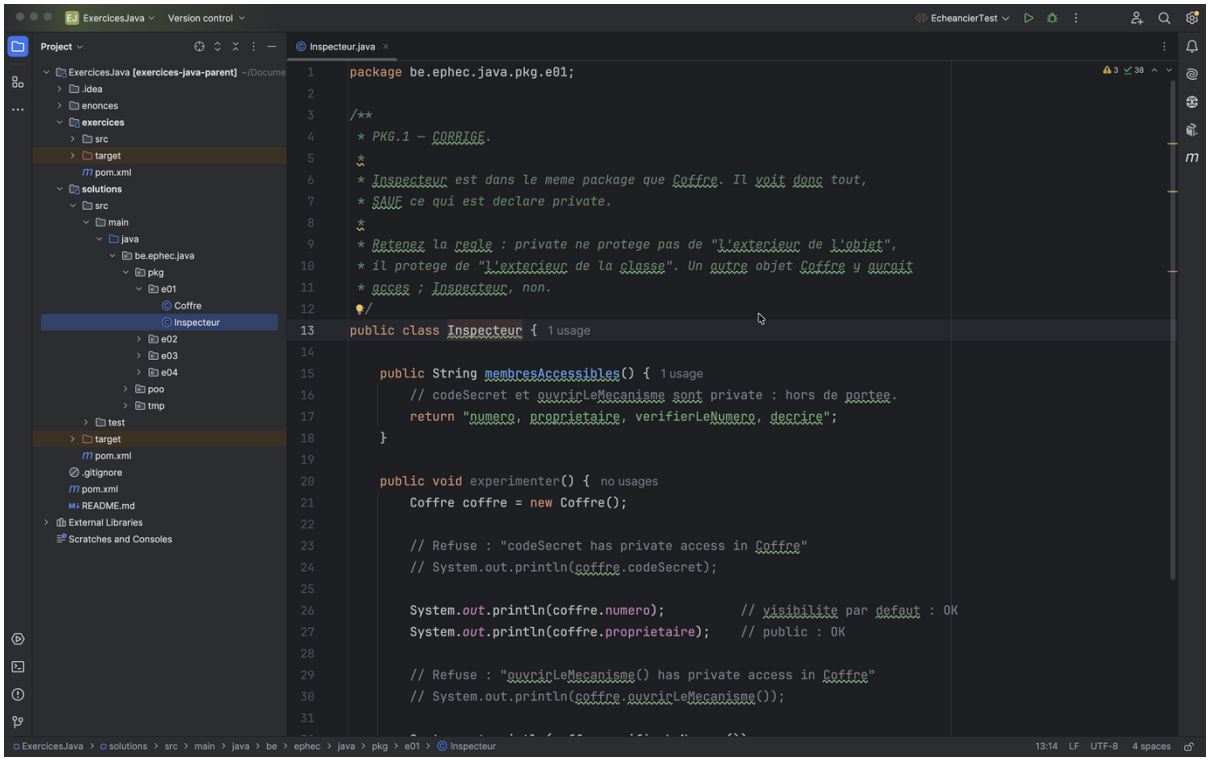


Les deux champs à vérifier : SDK et Language level.

Les modules Maven n'ont pas de JDK propre : ils héritent de celui du projet. Vous n'avez donc rien à régler module par module — et si un module affiche un JDK explicite au lieu de « Project SDK », c'est une anomalie à corriger. Voir `ERR.1`.

C — L'écran

IDE.7 — Les quatre zones



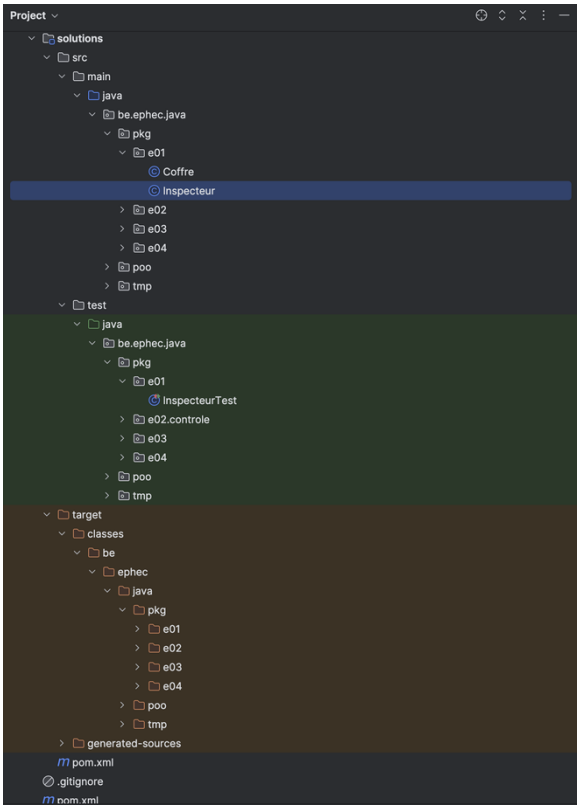
IntelliJ, projet ouvert.

L'écran se lit en quatre zones.

- **À gauche**, les fenêtres d'outils : Project le plus souvent, mais aussi Maven, Structure, Debug. Elles s'ouvrent et se ferment à volonté.
- **Au centre**, l'éditeur, avec un onglet par fichier ouvert. C'est là que vous passerez l'essentiel de votre temps.
- **En bas**, les fenêtres de résultat : Run pour l'exécution, Debug pour le débogage, Terminal. Elles apparaissent quand vous lancez quelque chose.
- **En haut à droite**, la barre de lancement : la configuration courante, la flèche verte, l'insecte du débogueur.

Tout le reste — les bandeaux de boutons sur les bords — n'est qu'un raccourci vers ces mêmes fenêtres. Ne cherchez pas à tout retenir : trois ou quatre suffisent, et les autres se découvriront d'elles-mêmes.

IDE.8 — La fenêtre Project, créer une classe



L'arborescence du projet, dépliée.

L'arborescence a l'air compliquée, mais elle obéit à une convention Maven simple et invariable.

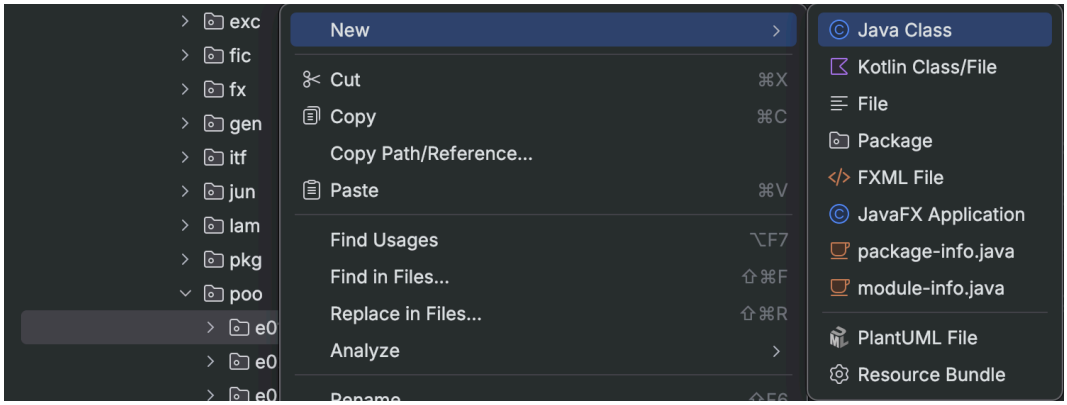
Dossier	Contenu
`src/main/java`	Votre code. Les classes que vous écrivez.
`src/test/java`	Les classes de test. Fournies, à ne pas modifier.
`target`	Le résultat de la compilation. Généré, jamais édité à la main.
`pom.xml`	La description du projet : modules, bibliothèques, version de Java.

Le dossier `target` se recrée intégralement à chaque compilation. C'est pourquoi il est ignoré par Git et absent des archives : il ne contient rien qu'on ne puisse refabriquer. Le mémento technique décrit ce qu'il contient en `TEC-CMP.4`, et ce qui distingue votre source, le bytecode et le programme en cours d'exécution en `TEC-CMP.1`.

Notez aussi que `src/main/java` et `src/test/java` contiennent les *mêmes* noms de packages. Ce n'est pas une erreur : une classe de test vit dans le même package que la classe qu'elle teste, ce qui lui donne accès à ses membres non publics.

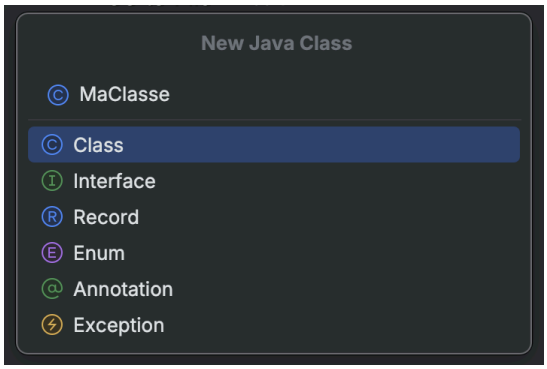
Créer une classe

Dans les exercices, les classes existent déjà. À l'examen, certaines questions vous demandent d'en créer une. Faites un clic droit sur le package qui doit l'accueillir, par exemple `be.ephec.java.poo.e01`, puis choisissez New, puis Java Class.



Clic droit sur le package, New, puis Java Class.

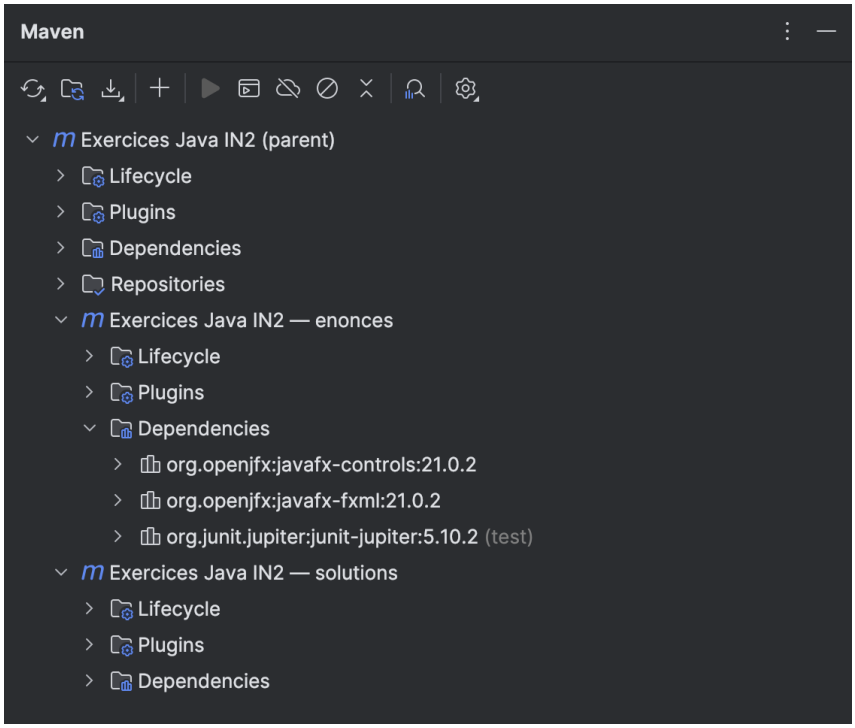
IntelliJ demande alors le nom de la classe et son genre : une classe ordinaire, une interface, un `record`, une énumération (`enum`)… Le nom suit les conventions : une majuscule initiale, sans espace ni accent, et sans l'extension `.java`, qu'IntelliJ ajoute lui-même.



Le nom de la classe, puis son genre.

Le fichier créé contient déjà la ligne `package` et la déclaration de la classe, vide : il ne reste qu'à l'écrire. Choisir le bon package compte : une classe créée ailleurs ne sera pas trouvée par la classe de test qui l'attend.

IDE.9 — La fenêtre Maven



Le projet parent et ses modules.

Cette fenêtre montre le projet tel que Maven le voit : un projet parent, et sous lui un dossier par module — `exercices`, `solutions` et `demos`. Si l'un d'eux n'y figure pas, c'est qu'IntelliJ ne l'a pas reconnu comme module Maven, et il faut relire `IDE.4`.

Le bouton de rechargement, en haut à gauche de cette fenêtre, force IntelliJ à relire les `pom.xml`. C'est le premier geste à faire quand une bibliothèque semble introuvable alors qu'elle est déclarée. Voir `ERR.2`.

Sous chaque module, quatre dossiers. Deux vous serviront, les deux autres presque jamais.

Dossier	Ce qu'il contient
`Lifecycle`	les étapes de construction : clean, compile, test, package…
`Dependencies`	les bibliothèques du module, telles que le `pom.xml` les déclare
`Plugins`	les outils que Maven ajoute, comme javafx ou surefire
`Repositories`	les serveurs où Maven va chercher ce qui lui manque

Dépliez Lifecycle : vous y trouverez les étapes du cycle de construction. Un double-clic sur l'une d'elles la lance, et Maven exécute au passage toutes celles qui la précèdent — lancer `test` compile donc d'abord. Le mémento technique détaille cet enchaînement en `TEC-MVN.4`.

Dépliez Dependencies : vous y verrez les bibliothèques réellement utilisées, avec leur version. C'est l'endroit où vérifier qu'une dépendance déclarée dans le `pom.xml` est bien arrivée.

Elles ne sont téléchargées qu'une fois : Maven les conserve ensuite sur votre machine, ce qui explique pourquoi le premier chargement d'un projet est long et les suivants immédiats. Voir `TEC-MVN.5`.

IDE.10 — Ouvrir et fermer les fenêtres d'outils

Les fenêtres d'outils portent un numéro, et le raccourci qui les ouvre reprend ce numéro. Le même raccourci les referme, ce qui rend l'écran modulable en une frappe.

Fenêtre	Windows / Linux	macOS
Project	`Alt+1`	`⌘1`
Run	`Alt+4`	`⌘4`
Debug	`Alt+5`	`⌘5`
Terminal	`Alt+F12`	`⌥F12`
Tout masquer sauf l'éditeur	`Ctrl+Maj+F12`	`⇧⌘F12`

La dernière ligne est la plus utile : elle bascule entre un écran chargé et l'éditeur seul, plein écran. Pratique quand on écrit, et pour se retrouver quand tout est ouvert.

D — Écrire du code

IDE.11 — Complétion et import automatique

IntelliJ propose des complétions au fil de la frappe : tapez les premières lettres et la liste apparaît. Entrée valide la proposition surlignée.

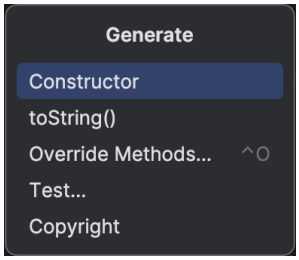
Le point important est ailleurs. Quand vous acceptez une complétion sur une classe qui n'est pas encore importée — `Scanner`, `LocalDate`, `ArrayList` — IntelliJ ajoute l'`import` en tête de fichier tout seul. Vous n'avez jamais à écrire un `import` à la main.

Si un nom de classe apparaît en rouge parce que son `import` manque, placez le curseur dessus et faites `Alt+Entrée` / ⌥⏎ : la première proposition sera de l'importer.

Une nuance qui compte. La complétion propose parfois plusieurs classes de même nom venant de bibliothèques différentes. `Date` existe en trois versions, et `KeyEvent` en deux — une pour `JavaFX`, une pour AWT. Lisez le nom complet affiché à droite de la proposition avant de valider : importer la mauvaise donne un code qui ne compile pas, avec un message qui n'aide pas.

IDE.12 — Générer accesseurs et constructeurs

Placez le curseur dans le corps d'une classe et faites `Alt+Inser` / `⌘N`. Le menu Generate propose de fabriquer ce qui se déduit mécaniquement des champs.



Le menu Generate.

- **Getter and Setter** : les accesseurs des champs que vous choisissez.
- **Constructor** : un constructeur initialisant les champs sélectionnés.
- **`Override` Methods** : les méthodes héritées que vous voulez redéfinir.

> **Attention aux doublons**
>
> Les squelettes d'exercices déclarent souvent déjà les accesseurs, avec un corps qui lève une exception. Dans ce cas, complétez ce corps — ne régénérez pas, vous obtiendriez deux méthodes du même nom et la classe ne compilerait plus.
>
> Générer ne dispense pas de réfléchir : un getter et un setter pour chaque champ annule l'encapsulation qu'on vient de mettre en place. Voir la règle `CC-OBJ.2` du mémento Clean Code.

IDE.13 — Les modèles : psvm, sout, fori

Certaines suites de caractères se déplient en code quand on les tape puis qu'on appuie sur Tab. Trois valent d'être connues dès la première séance.

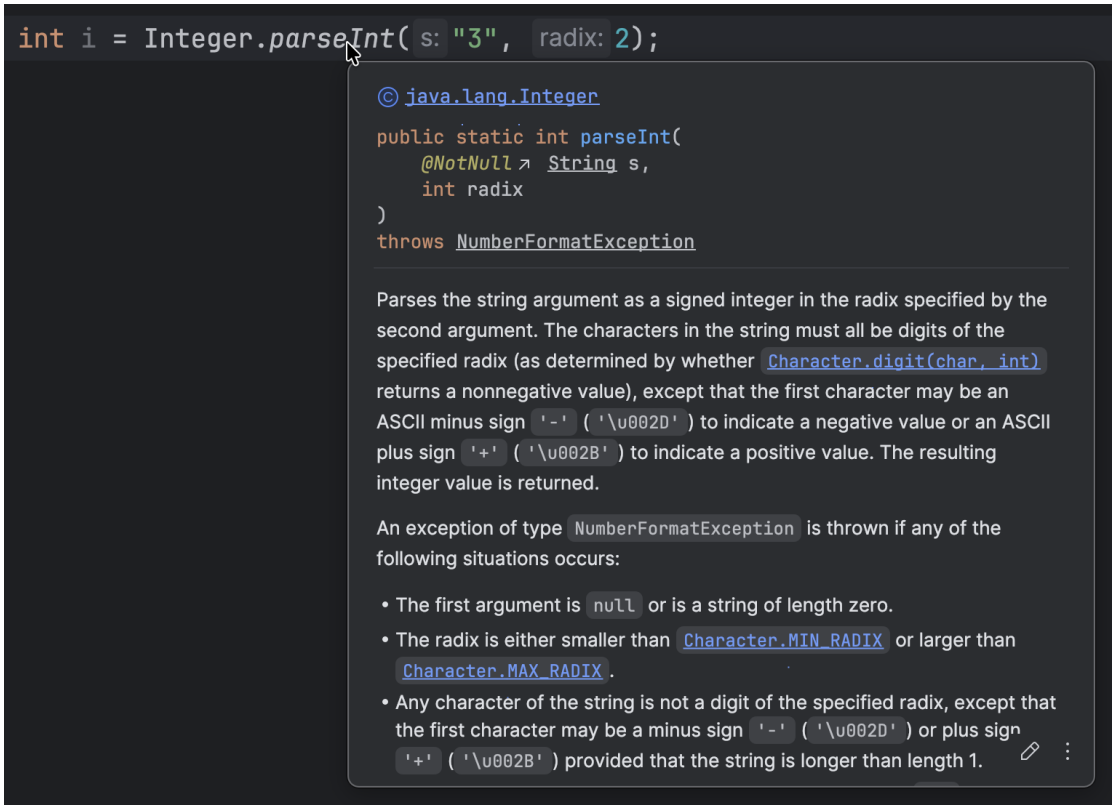
Vous tapez	Vous obtenez
`psvm`	`public static void main(String[] args) { }`
`sout`	`System.out.println();`
`fori`	`une boucle for indexée complète`
`iter`	`une boucle for-each sur la collection la plus proche`

Il en existe des dizaines d'autres. `Ctrl+J` / `⌘J` affiche la liste de ceux disponibles à l'endroit où vous êtes.

IDE.14 — Lire la Javadoc sans quitter l'éditeur

Plusieurs exercices vous demandent d'aller lire la documentation d'une méthode du JDK. Vous n'avez pas besoin d'ouvrir un navigateur : la documentation est dans IntelliJ.

Survolez le nom de la méthode avec la souris, ou placez-y le curseur et faites `Ctrl+Q` / `F1`.



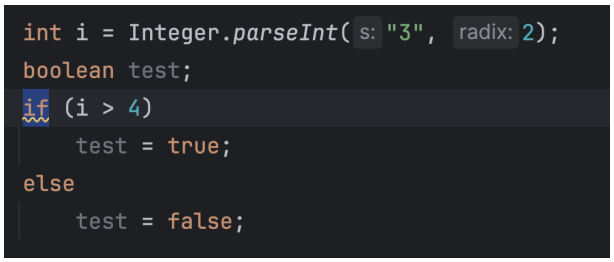
La documentation de `Integer.parseInt(String, int)`.

Cette bulle contient tout ce qu'un énoncé peut vous demander de chercher : la signature exacte, ce que la méthode retourne, et surtout la liste des cas où elle lève une exception. Sur la capture, on lit que le radix doit se situer entre `MIN_RADIX` et `MAX_RADIX` — exactement ce que demande l'exercice `POO.5`.

Les liens de la bulle sont cliquables : de proche en proche, on remonte à toute la bibliothèque standard.

IDE.15 — Les avertissements et leurs corrections

Un souligné rouge signale une erreur : le code ne compile pas. Un souligné jaune ou gris signale autre chose, et c'est celui qui nous intéresse ici — le code compile parfaitement, mais IntelliJ estime qu'on peut faire mieux.



Le code compile. Le souligné jaune signale autre chose.

Survolez la partie soulignée : le message explique ce qui est en cause.



Le diagnostic au survol.

Puis `Alt+Entrée` / ⌥⏎ ouvre la liste des corrections possibles. La première est en général la bonne, et un aperçu montre le résultat avant que vous ne validiez.



Les corrections proposées, avec l'aperçu du résultat.

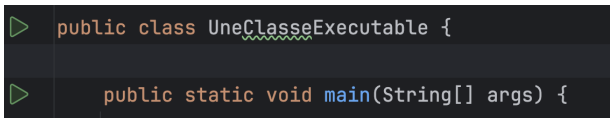
Dans cet exemple, un `if/else` qui affecte `true` dans une branche et `false` dans l'autre ne fait que recopier sa propre condition. IntelliJ propose de le remplacer par `test = i > 4;`, qui dit la même chose en une ligne.

> **Ces avertissements recoupent le mémento Clean Code**
>
> La variable jamais utilisée, la méthode qui pourrait être `static`, la condition inutilement compliquée : IntelliJ effectue gratuitement une partie du travail de relecture que le mémento vous demande de faire à la main.
>
> Prenez l'habitude de lire ces messages au lieu de les ignorer. Ils ne sont pas tous pertinents — vous apprendrez à faire le tri — mais ils sont bien plus souvent justes que le contraire.

E — Exécuter

IDE.16 — Lancer une classe

Une classe est exécutable si elle contient une méthode `main`. IntelliJ le repère et affiche une flèche verte dans la marge, à hauteur de la déclaration.



La flèche verte, dans la marge, à côté du main.

Cliquez dessus et choisissez `Run`. La fenêtre Run s'ouvre en bas et affiche la sortie du programme. Le raccourci `Maj+F10` / `^R` relance ce que vous avez lancé la dernière fois, sans repasser par la marge.

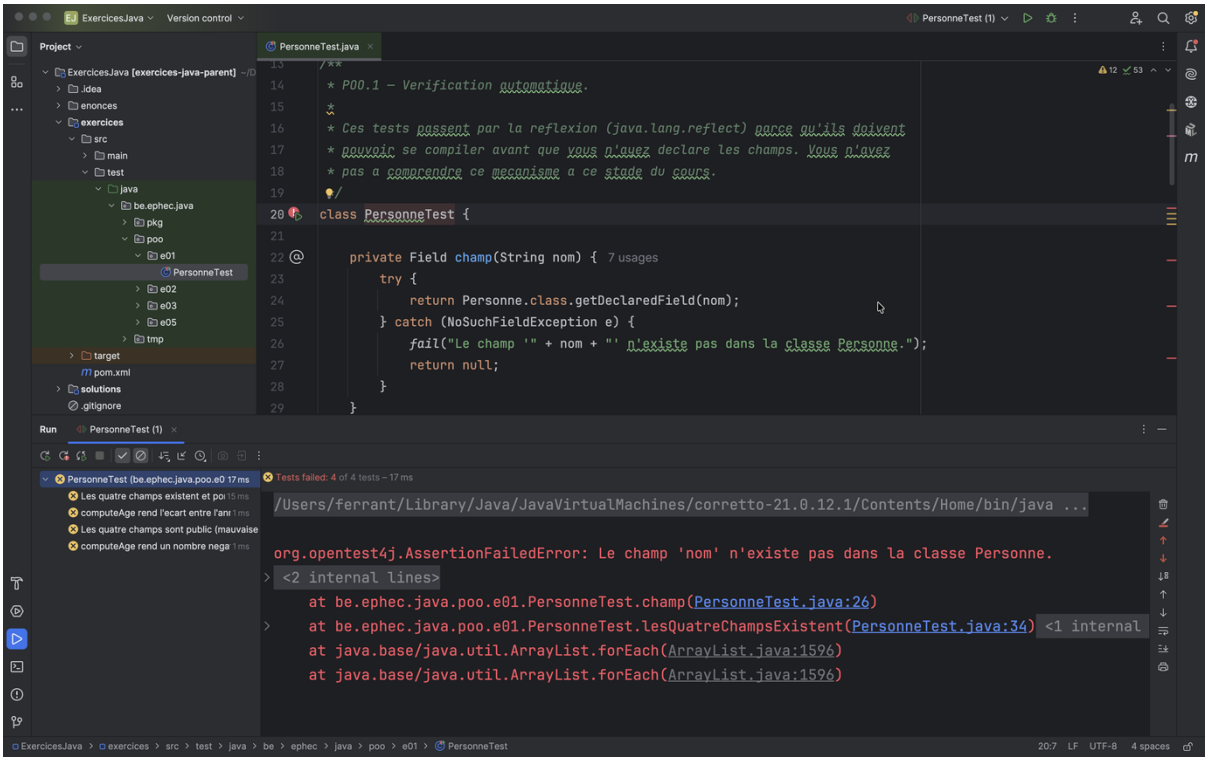
Si votre programme lit au clavier avec un `Scanner`, c'est dans cette fenêtre Run que vous tapez : cliquez dedans avant de saisir.

IDE.17 — Lancer les tests et lire le rapport

C'est le geste que vous répéterez le plus souvent de toute l'année, et celui qui vous dira si votre travail est correct.

Ouvrez la classe de test indiquée en fin d'énoncé. Une double flèche verte apparaît dans la marge à hauteur du nom de la classe : elle lance tous les tests du fichier. Une flèche simple, en face de chaque méthode, n'en lance qu'un. Le raccourci est `Ctrl+Maj+F10` / `^⇧R`.

Au départ, tout échoue. C'est normal : vous n'avez encore rien écrit.

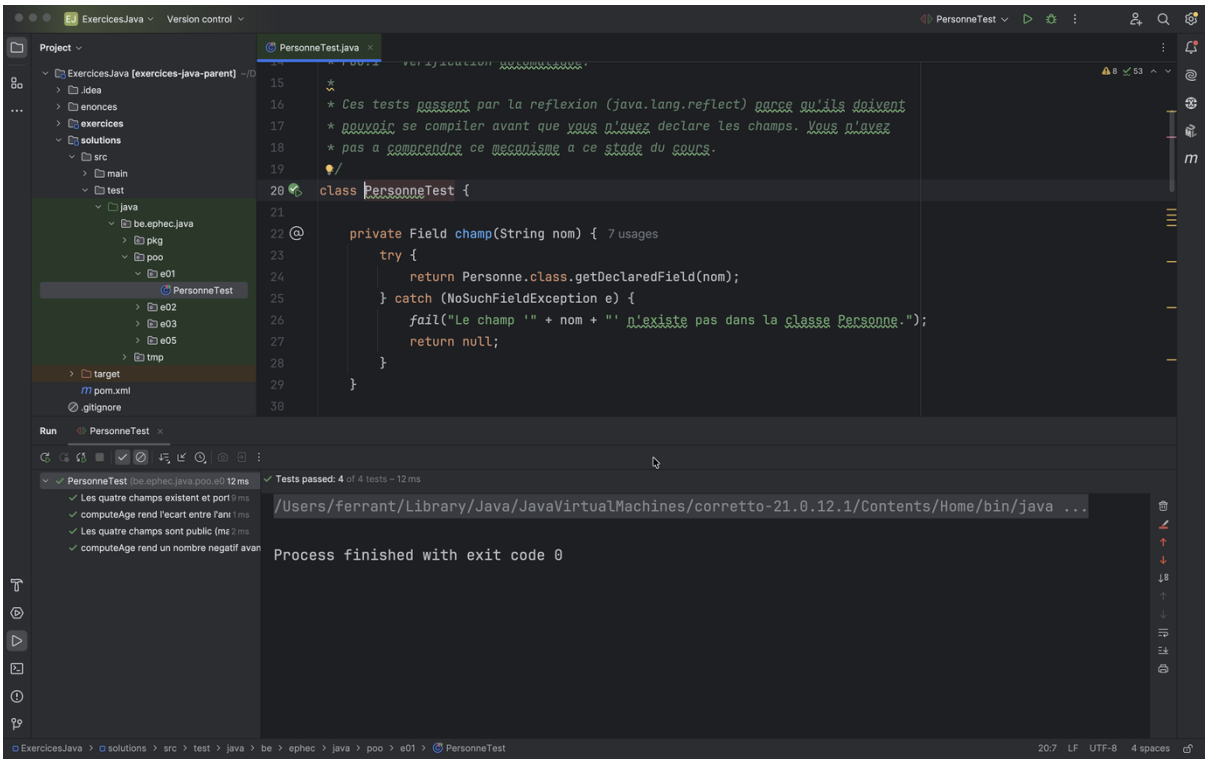


Quatre tests sur quatre échouent : l'état initial.

Le panneau se lit en trois temps. À gauche, la liste des tests, chacun avec sa croix ou sa coche. En haut, le compte. À droite, pour le test sélectionné, la raison de l'échec.

Lisez cette raison. Les messages sont écrits pour vous : ici, « Le champ 'nom' n'existe pas dans la classe Personne » vous dit exactement quoi faire. Se contenter de voir la croix rouge et retourner au code à l'aveugle vous fera perdre bien plus de temps que de lire trois lignes.

Quand tout est correct, l'ensemble passe au vert.



Le même fichier de test, une fois le travail fait.

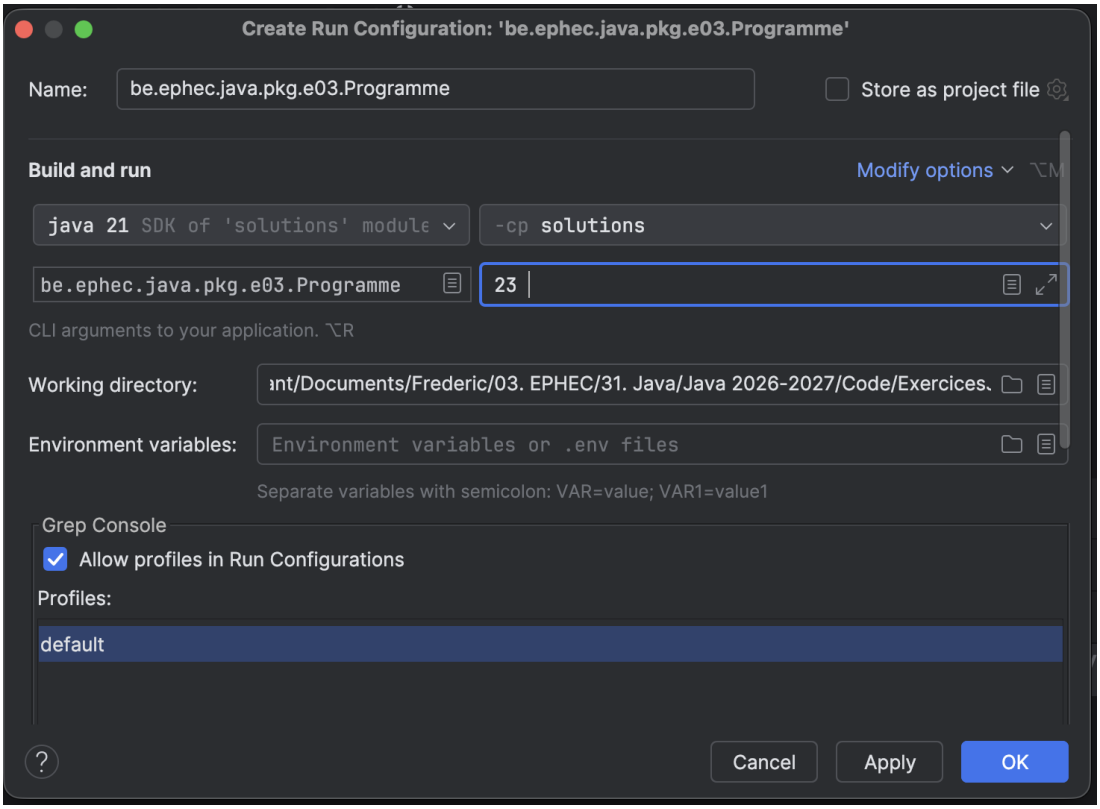
Pourquoi les deux captures ne montrent pas le même module

La première est prise dans le module exercices, la seconde dans solutions. Les deux modules contiennent les mêmes classes de test, à l'identique : le corrigé subit exactement les tests que vous subissez, il n'en a pas de plus indulgents.

Vous obtiendrez le second résultat sur votre propre code, dans le module exercices.

IDE.18 — Passer des arguments au programme

La méthode `main` reçoit un tableau de chaînes, `String[] args`. Pour le remplir, il faut passer par la configuration de lancement : `Run → Edit Configurations`, champ `Program arguments`.



Le champ Program arguments d'une configuration de lancement.

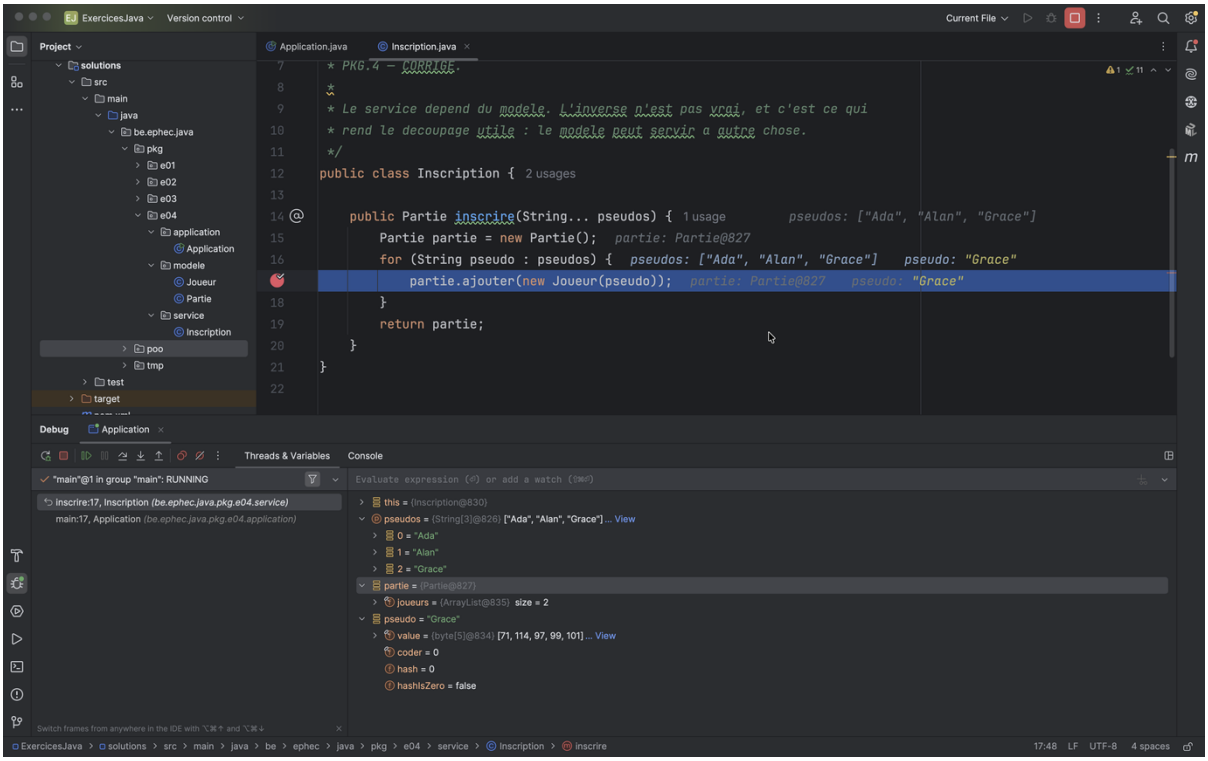
Les arguments sont séparés par des espaces. Pour en passer un qui contient une espace, entourez-le de guillemets.

IDE.19 — Le débogueur

Le débogueur suspend le programme à l'endroit de votre choix et vous laisse regarder ce qui s'y passe. C'est de loin le meilleur moyen de comprendre ce que fait vraiment votre code — et bien plus efficace que de semer des affichages un peu partout.

Le poser

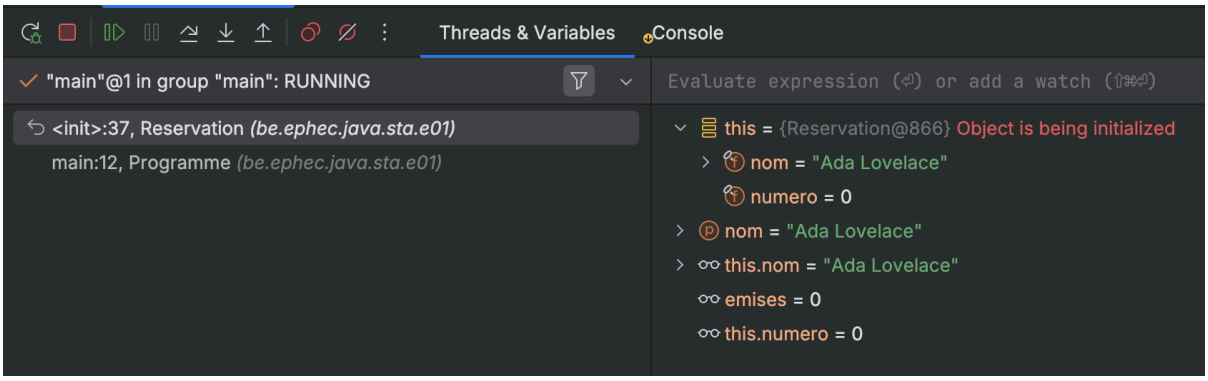
Cliquez dans la marge, à gauche d'une ligne : un point rouge apparaît. C'est un point d'arrêt. Lancez ensuite le programme avec l'insecte plutôt qu'avec la flèche verte. L'exécution s'arrête sur cette ligne, avant de l'exécuter.



Arrêt au troisième tour de boucle.

Le lire

- **Frames**, à gauche : la pile des appels. Sur la capture, `inscrire` a été appelée depuis `main`. C'est la pile dont parle le mémento technique, rendue visible.
- **Variables**, à droite : tout ce qui existe à cet instant. Les objets se déplient et montrent leur contenu — ici, `partie` contient une liste de deux joueurs, ce qui confirme qu'on est au troisième tour.
- Les annotations grises dans l'éditeur affichent la valeur des variables directement à côté du code.
- **Evaluate expression**, en haut du panneau Variables : tapez une expression — `Reservation.emises`, `liste.size()` — pour voir sa valeur à l'instant de l'arrêt. C'est utile pour ce que le panneau n'affiche pas de lui-même.



Arrêt dans le constructeur de Reservation : nom et numero sont dans `this`, le compteur `static` emises est affiché à part.

Un champ `static` n'apparaît pas quand on déplie `this` : il n'appartient pas à l'objet. IntelliJ l'affiche à part quand la ligne d'arrêt l'utilise ; sinon, demandez-le dans Evaluate expression.

Avancer

Action	Windows / Linux	macOS
Ligne suivante, sans entrer dans les appels	`F8`	`F8`
Entrer dans la méthode appelée	`F7`	`F7`
Sortir de la méthode courante	`Maj+F8`	`⇧F8`
Reprendre jusqu'au prochain arrêt	`F9`	`⌥⌘R`

Action	Windows / Linux	macOS

Trois choses qui surprennent au début

Au début d'un constructeur, les champs valent encore 0 ou `null`, et IntelliJ annonce `Object is being initialized`. Avancez avec F8 pour les voir se remplir.

Les objets portent un numéro — Partie@827, `ArrayList`@835. Ce numéro n'a aucune signification en soi et change à chaque exécution : il sert seulement à distinguer deux objets l'un de l'autre. Deux variables qui affichent le même numéro désignent le même objet, et c'est une observation précieuse.

Déplier une chaîne de caractères montre ses entrailles — un tableau d'octets, un champ hash. Ignorez-les : c'est la mécanique interne de `String`, pas quelque chose que vous ayez à comprendre.

F — Transformer du code sans le casser

Renommer une classe, déplacer une méthode, extraire un bout de code : ces opérations sont mécaniques et IntelliJ les fait à votre place, en corrigeant partout où c'est nécessaire. Les faire à la main, c'est se garantir un oubli.

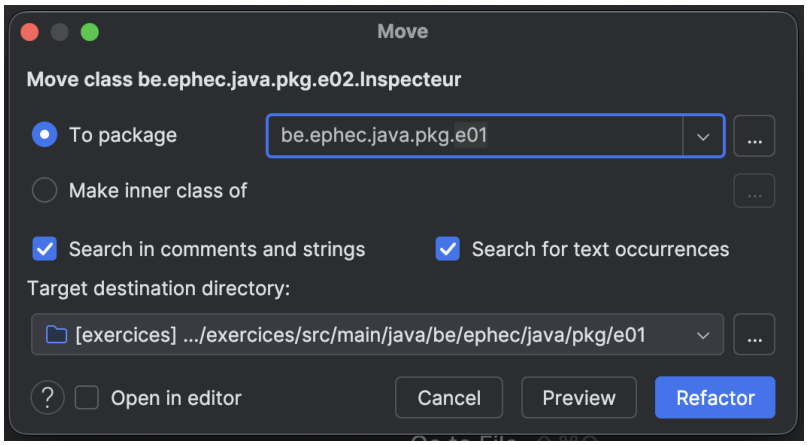
IDE.20 — Renommer

Curseur sur le nom, puis `Maj+F6`. Tapez le nouveau nom et validez : IntelliJ met à jour toutes les utilisations, dans tout le projet, y compris le nom du fichier si c'est une classe.

Cela vaut pour tout : classes, méthodes, variables, paramètres, packages. Il n'y a donc aucune raison de conserver un mauvais nom par crainte du travail de correction — voir `CC-NOM.2` du mémento Clean Code, qui repose entièrement sur cette possibilité.

IDE.21 — Déplacer une classe

Sélectionnez la classe dans la fenêtre Project, puis `F6`. Indiquez le package de destination.



La boîte Move, avec le package de destination.

IntelliJ déplace le fichier, corrige sa déclaration de `package`, et ajoute les `import` nécessaires dans toutes les classes qui l'utilisaient.

Déplacer, pas copier-coller

La tentation est de créer le nouveau package, d'y coller le fichier et de supprimer l'ancien. Cela marche parfois, et donne parfois deux classes du même nom dans deux packages, ce qui produit des erreurs incompréhensibles. Voir `ERR.5`.

L'exercice `PKG.4` est bâti sur ce geste, et sa classe de test vérifie explicitement que rien ne subsiste à l'ancien emplacement.

IDE.22 — Extraire une méthode

Sélectionnez les lignes à extraire, puis `Ctrl+Alt+M` / `⌥⌘M`. IntelliJ crée une méthode contenant ces lignes, détermine seul les paramètres nécessaires et la valeur de retour, et remplace les lignes d'origine par un appel.

C'est l'outil du mémento Clean Code : `CC-MET.2` demande des méthodes courtes, `CC-MET.3` qu'une méthode ne fasse qu'une chose, et `CC-COM.1` de remplacer un commentaire par une méthode bien nommée. Les trois se font avec ce raccourci.

G — Naviguer

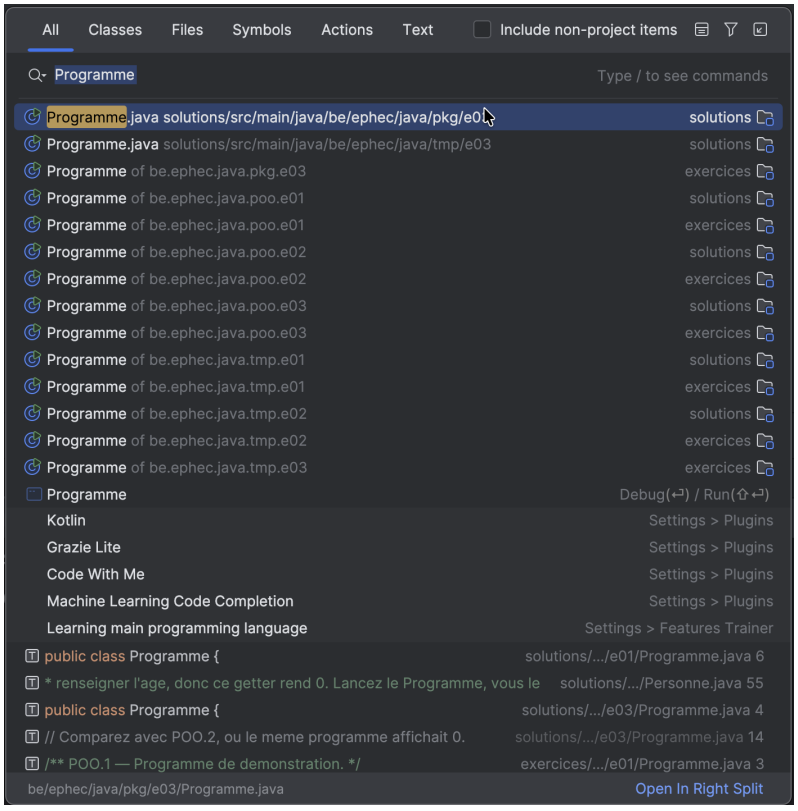
IDE.23 — Aller à la déclaration, et revenir

`Ctrl+clic` / `⌘clic` sur n'importe quel nom saute à sa déclaration. Cela fonctionne sur vos classes comme sur celles du JDK : vous pouvez lire le code source de `ArrayList` en un clic.

Pour revenir d'où vous venez : `Ctrl+Alt+←` / `⌘[`. C'est le bouton « page précédente » de l'éditeur, et il mémorise tout un historique.

IDE.24 — Chercher n'importe quoi

Appuyez deux fois sur `Maj`. Une barre de recherche s'ouvre et cherche partout à la fois : classes, fichiers, symboles, et jusqu'aux commandes d'IntelliJ.



Search Everywhere.

C'est le raccourci à retenir si vous n'en retenez qu'un. Vous cherchez une classe dont vous ne savez plus dans quel package elle vit, ou un réglage dont vous ne savez plus dans quel menu il se cache : les deux se trouvent ici.

IDE.25 — La structure d'un fichier

`Ctrl+F12` / `⌘F12` affiche la liste des champs et méthodes du fichier courant. Tapez quelques lettres pour filtrer, validez pour y sauter.

Utile dès qu'une classe dépasse l'écran — ce qui, si vous suivez le mémento Clean Code, ne devrait pas arriver souvent.

H — Raccourcis et réglages

IDE.26 — Les raccourcis à connaître

IntelliJ en compte plusieurs centaines. Voici ceux qui servent réellement, regroupés par usage. Apprenez-en deux ou trois par semaine plutôt que la liste entière.

Écrire

Action	Windows / Linux	macOS
Compléter	`Ctrl+Espace`	`^Espace`
Corriger, importer, proposer	`Alt+Entrée`	`⌥⏎`
Générer (accesseurs, constructeur)	`Alt+Inser`	`⌘N`
Reformater le code	`Ctrl+Alt+L`	`⌥⌘L`
Dupliquer la ligne	`Ctrl+D`	`⌘D`
Déplacer la ligne	`Alt+Maj+↑ ↓`	`⌥⇧↑ ↓`
Commenter la sélection	`Ctrl+/`	`⌘/`
Coller un contenu précédent	`Ctrl+Maj+V`	`⇧⌘V`

Naviguer

Action	Windows / Linux	macOS
Chercher partout	`Maj Maj`	`⇧⇧`
Aller à la déclaration	`Ctrl+clic`	`⌘clic`
Revenir en arrière	`Ctrl+Alt+←`	`⌘[`
Structure du fichier	`Ctrl+F12`	`⌘F12`
Fichier récent	`Ctrl+E`	`⌘E`

Exécuter

Action	Windows / Linux	macOS
Relancer la dernière exécution	`Maj+F10`	`^R`
Lancer ce qui est sous le curseur	`Ctrl+Maj+F10`	`^⇧R`
Déboguer la dernière configuration	`Maj+F9`	`^D`
Ligne suivante en débogage	`F8`	`F8`

Transformer

Action	Windows / Linux	macOS
Renommer	`Maj+F6`	`⇧F6`
Déplacer	`F6`	`F6`
Extraire une méthode	`Ctrl+Alt+M`	`⌥⌘M`
Extraire une variable	`Ctrl+Alt+V`	`⌥⌘V`

Si vous ne devez en retenir que deux : `Maj Maj` pour trouver n'importe quoi, et `Alt+Entrée` / `⌥⏎` pour qu'IntelliJ propose la correction de ce qu'il vient de signaler.

IDE.27 — Plugins et apparence

`Settings → Plugins` sous Windows, `IntelliJ IDEA → Settings → Plugins` sous macOS. Deux méritent le détour.

- **Key Promoter X** affiche le raccourci correspondant chaque fois que vous faites quelque chose à la souris. C'est le moyen le plus efficace d'apprendre les raccourcis : ils arrivent au moment où ils serviraient.
- **PlantUML integration** affiche les diagrammes de classes utilisés dans certains énoncés.

L'apparence se règle dans `Settings → Appearance`. Thème clair ou sombre, taille de la police de l'éditeur : réglez-les à votre confort, et augmentez la police si vous devez projeter votre écran.

IDE.28 — JavaFX : rien à configurer, ou presque

Vous trouverez en ligne des procédures expliquant qu'il faut télécharger un SDK `JavaFX`, l'installer, puis indiquer à IntelliJ où il se trouve. Ne les suivez pas pour ce cours : elles concernent les projets sans Maven.

`JavaFX` ne fait plus partie du JDK depuis Java 11 — voir `TEC-FX.2` du mémento technique. C'est donc une bibliothèque comme une autre, et le projet du cours la déclare dans son `pom.xml` au même titre que `JUnit` :

```
<dependency>
    <groupId>org.openjfx</groupId>
    <artifactId>javafx-controls</artifactId>
    <version>21.0.2</version>
</dependency>
```

Maven la télécharge au premier chargement et la place sur le classpath. Vous n'avez rien à installer, rien à désigner, rien à régler.

Une seule chose peut être ajoutée, et elle est facultative : Scene Builder, l'éditeur visuel de fichiers `.fxml`. Voir `IDE.29`.

Lancer une application JavaFX

C'est le seul point où `JavaFX` se distingue vraiment. Une application `JavaFX` a besoin que la bibliothèque soit annoncée à la JVM au démarrage, ce qu'un simple clic sur la flèche verte ne fait pas toujours. Deux façons de procéder, selon ce que l'énoncé indique.

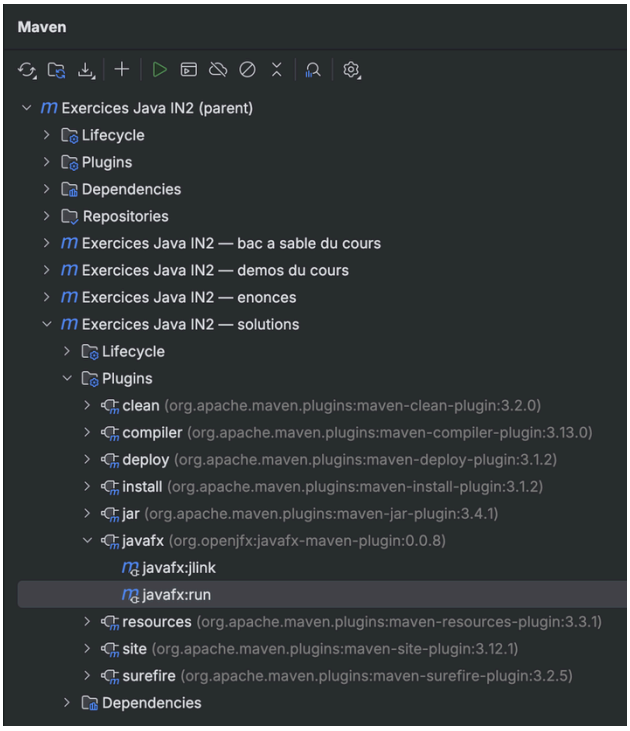
- **Depuis IntelliJ**, en lançant une classe de démarrage distincte de la classe d'application : le `Lanceur`. C'est le montage que le projet du cours utilise, et il est expliqué le moment venu dans le fascicule `FX`.
- **Depuis Maven**, par le greffon `JavaFX`, qui construit lui-même la ligne de commande correcte. C'est la solution de secours quand le lancement normal échoue : la procédure suit.

Lancer par Maven : javafx:run

1. Ouvrez la fenêtre Maven. Cliquez sur l'icône `m` de la barre verticale de droite, ou passez par `View → Tool Windows → Maven`.

2. Dépliez le bon module. La fenêtre montre le projet parent, puis ses modules, chacun nommé « Exercices Java IN2 — … » : *enonces* (vos exercices), *solutions*, *demos du cours* et *bac a sable du cours*. Dépliez celui qui contient la classe à lancer, en général *enonces*. Pas le nœud du haut, marqué « (parent) » : depuis lui, Maven lancerait la commande dans chaque module, l'un après l'autre.

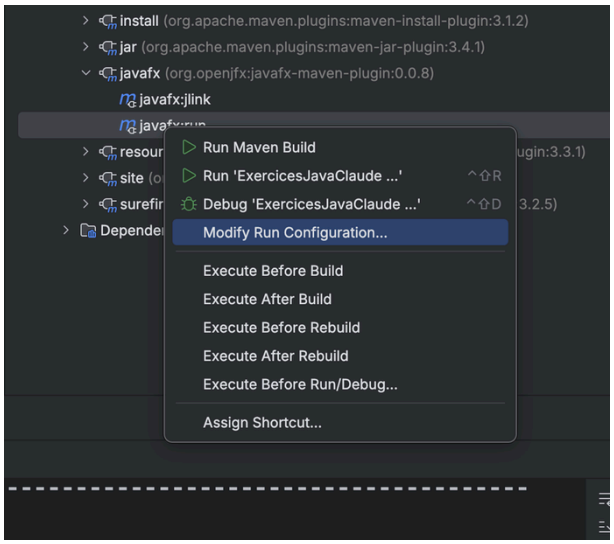
3. Lancez. Sous le module, dépliez `Plugins → javafx`, puis double-cliquez sur `javafx:run`. La console Maven s'ouvre en bas, puis la fenêtre de l'application apparaît ; la fermer termine l'exécution. La première fois, Maven télécharge le greffon : il faut une connexion internet, et quelques secondes de patience.



La fenêtre Maven : le greffon javafx, sous Plugins, dans le module à lancer.

Quelle classe est lancée ? Celle que désigne la propriété `javafx.mainClass` dans le `pom.xml` du module : pour les énoncés et les solutions, le premier exercice `JavaFX`, `be.ephec.java.fx.e01.CompteurParticipants`. Le greffon accepte directement la classe d'application : inutile de passer par le Lanceur.

Lancer un autre exercice, sans toucher au `pom.xml` : faites un clic droit sur `javafx:run` et choisissez `Modify Run Configuration…`.



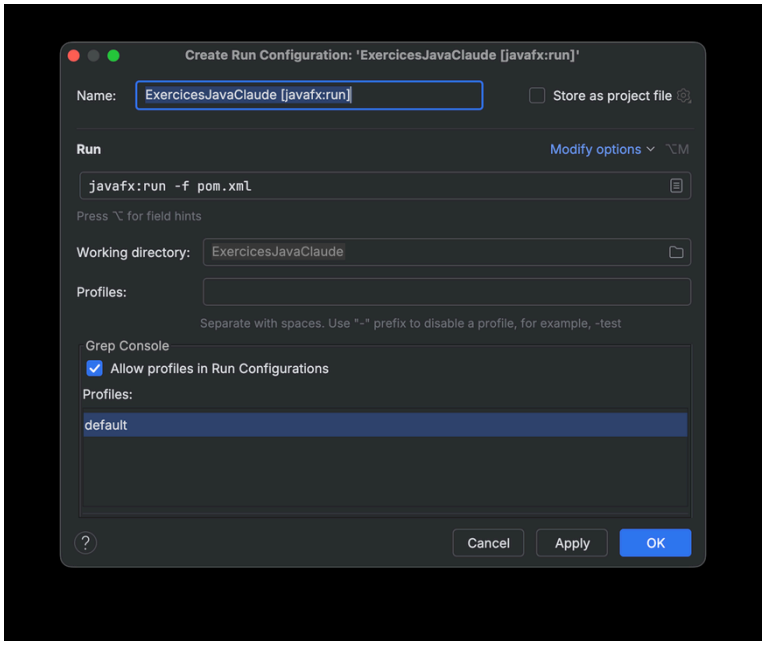
Le menu contextuel de javafx:run.

Dans la fenêtre qui s'ouvre, trois réglages :

- **Name** : un nom parlant, « `FX.2` » par exemple.
- **Run** : la commande. Gardez ce qu'IntelliJ a écrit, et ajoutez la classe à lancer après `javafx:run` :

```
javafx:run -Djavafx.mainClass=be.ephec.java.fx.e02.CompteurParticipants -f pom.xml
```

- **Working directory** : le dossier dont Maven lit le `pom.xml`. Vérifiez qu'il désigne bien le module, par exemple le dossier `exercices`. S'il indique la racine du projet, choisissez le dossier du module avec l'icône de dossier : sinon, Maven partirait du `pom.xml` parent et lancerait la commande dans chaque module.



La configuration d'exécution : son nom, la commande Maven, le dossier de travail.

Validez par OK. La configuration apparaît ensuite dans la liste déroulante en haut à droite d'IntelliJ, et la flèche verte la relance.

Si vous obtenez le message `JavaFX runtime components are missing`, c'est exactement ce problème : voir ERR.7.

IDE.29 — Installer Scene Builder

Scene Builder est un éditeur visuel de fichiers `.fxml` : on y compose la fenêtre en déposant les composants à la souris, et il écrit le XML correspondant. Il est gratuit.

Ce n'est pas un greffon d'IntelliJ, mais un logiciel séparé. IntelliJ ne fait que lui passer la main quand vous ouvrez un fichier FXML. Il faut donc l'installer, puis dire à IntelliJ où il se trouve — deux étapes distinctes, et c'est la seconde qu'on oublie.

1. Télécharger

Sur `gluonhq.com/products/scene-builder`, en prenant la version correspondant à votre système. Sur un Mac Apple Silicon, vérifiez qu'il s'agit bien de la variante `aarch64` et non `x64`.

Quelle version. Les numéros de Scene Builder suivent ceux de `JavaFX`. Le cours utilise `JavaFX` 21 : prenez Scene Builder 21, ou la version disponible la plus proche. Un Scene Builder plus récent que votre `JavaFX` peut écrire dans le FXML des attributs que votre version ne comprend pas — le fichier s'ouvre alors correctement dans l'éditeur et échoue au lancement.

2. Installer

Installation ordinaire : un .dmg à glisser dans Applications sous macOS, un installateur sous Windows. Aucun réglage à faire pendant l'installation.

Notez le chemin où il atterrit, vous en aurez besoin à l'étape suivante. Par défaut :

```
macOS     /Applications/SceneBuilder.app
Windows   C:\Users\<vous>\AppData\Local\SceneBuilder\SceneBuilder.exe
```

3. Le déclarer dans IntelliJ

`Settings → Languages & Frameworks → JavaFX`, champ `Path to SceneBuilder`. Désignez le fichier repéré ci-dessus.

C'est la seule ligne à renseigner. Tant qu'elle est vide, IntelliJ ouvre les fichiers FXML comme du XML ordinaire, sans onglet d'édition visuelle — et rien n'indique qu'il manque quelque chose.

4. Vérifier

Ouvrez n'importe quel fichier `.fxml` du projet. Deux onglets doivent apparaître en bas de l'éditeur : `Text` et `Scene Builder`. Le second affiche la fenêtre telle qu'elle sera, avec la palette de composants sur le côté.

Si l'onglet n'apparaît pas, le chemin est erroné ou pointe vers le dossier plutôt que vers l'exécutable.

S'en servir, ou pas

Scene Builder fait gagner du temps sur les fenêtres complexes, où placer les composants à la main devient fastidieux. Il rend aussi visible ce qu'un gestionnaire de placement fait réellement, ce qui aide à comprendre pourquoi une fenêtre se réorganise quand on la redimensionne.

Mais vous pouvez très bien vous en passer, et je vous conseille de commencer sans. Écrire quelques fichiers FXML à la main apprend leur structure, et cette structure est ce que l'examen évalue — pas votre habileté à la souris.

> **Deux attributs à ne jamais perdre de vue**
>
> Un fichier FXML contient un attribut fx:controller sur son élément racine, qui désigne la classe contrôleur, et des attributs fx:id sur les composants que le contrôleur manipule. Ce sont eux qui relient la vue au code.
>
> Scene Builder les gère, mais ils se règlent dans un panneau discret. Si votre application se lance avec une fenêtre vide, ou lève une exception au démarrage en évoquant un champ `null`, vérifiez d'abord ces deux attributs dans l'onglet Text.

Annexe — Erreurs et solutions

Les messages les plus fréquents, leur cause, et ce qu'il faut faire. Cherchez le message que vous avez à l'écran plutôt que de lire l'annexe en entier.

ERR.1 — JDK isn't specified for module

```
java: JDK isn't specified for module 'solutions'
```

Un module Maven ne définit pas son propre JDK : il hérite de celui du projet. Ce message signifie donc presque toujours qu'aucun JDK n'est déclaré au niveau du projet, et il nomme simplement le premier module qu'IntelliJ a essayé de compiler.

`File → Project Structure → Project`, champ SDK : choisissez un JDK 21. S'il n'y en a aucun dans la liste, voir `IDE.5`.

Si le SDK du projet est correct et que le message persiste, vérifiez le module lui-même : `Project Structure → Modules`, onglet `Dependencies`, champ `Module SDK`. Il doit indiquer « Project SDK » et non un JDK nommé.

En dernier recours, fermez le projet, supprimez le dossier `.idea` et les fichiers `.iml`, puis rouvrez par le `pom.xml`. Vous ne perdez rien : ce dossier ne contient que des préférences d'affichage.

ERR.2 — Cannot resolve symbol

```
java: cannot find symbol
  symbol:   class Test
  location: package org.junit.jupiter.api
```

Une classe venant d'une bibliothèque est introuvable. Le plus souvent, IntelliJ n'a pas encore chargé ce que déclare le `pom.xml` — après un premier chargement interrompu, ou après avoir déplié une nouvelle archive dans le projet.

Ouvrez la fenêtre Maven et cliquez sur le bouton de rechargement. Voir `IDE.9`. Si le téléchargement échoue, vérifiez votre connexion : Maven va chercher les bibliothèques sur Internet à la première utilisation.

ERR.3 — Tout est rouge sans raison

Des dizaines de classes soulignées en rouge alors que le code n'a pas changé, souvent après une mise à jour d'IntelliJ ou l'ajout de fichiers dans le projet. Les index de résolution sont désynchronisés.

`File → Invalidate Caches… → Invalidate and Restart`. IntelliJ reconstruit ses index au redémarrage, ce qui prend une minute. Rien n'est perdu : ni votre code, ni vos réglages.

ERR.4 — Pas de flèche verte sur une classe de test

Trois causes possibles, dans l'ordre de fréquence.

- Le fichier n'est pas sous `src/test/java` mais sous `src/main/java`. Déplacez-le avec `F6`.
- La déclaration de `package` ne correspond pas à l'emplacement du fichier dans l'arborescence. Les deux doivent coïncider exactement.
- `JUnit` n'est pas chargé : voir `ERR.2`.
- Aucune classe du projet n'a de flèche verte, pas seulement les tests : voir `ERR.11`.

ERR.5 — Deux classes du même nom

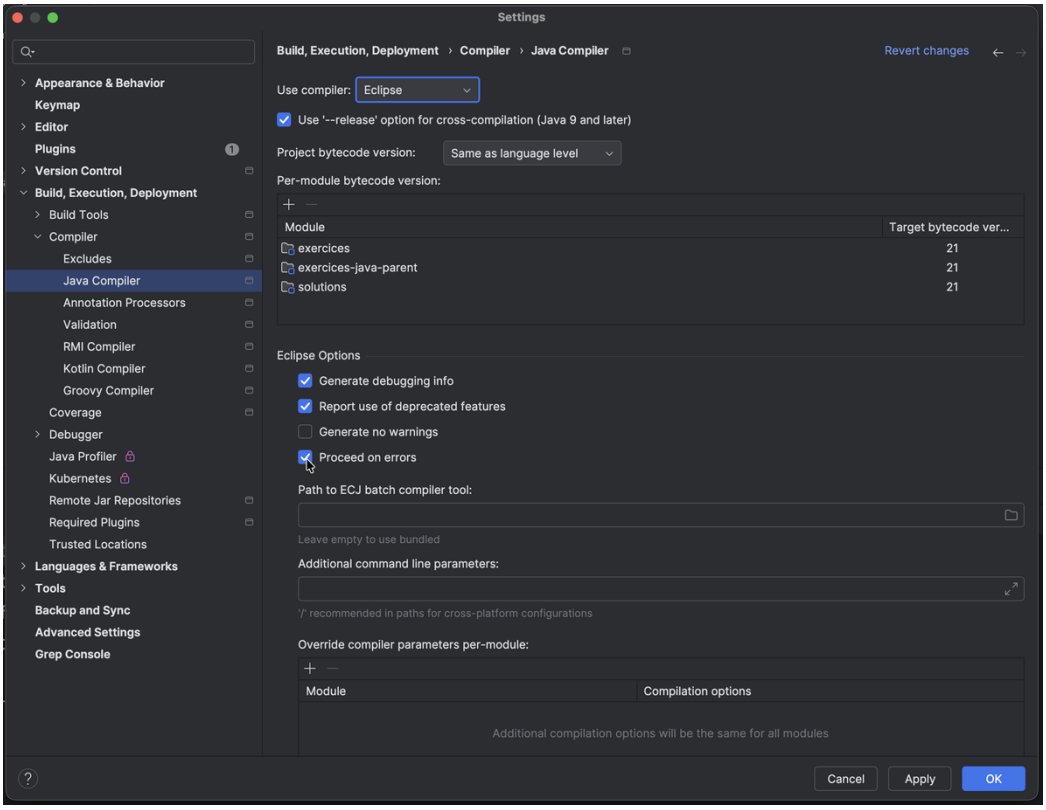
Des erreurs incompréhensibles après une réorganisation : une méthode existe pourtant, un `import` semble correct mais ne l'est pas. Presque toujours, une classe a été copiée-collée dans un nouveau package sans être supprimée de l'ancien. Vous en modifiez une, le programme utilise l'autre.

Cherchez le nom de la classe avec `Maj Maj` : si deux résultats apparaissent dans deux packages différents, vous tenez le coupable. Supprimez celle qui est à la mauvaise place, et à l'avenir déplacez avec `F6` au lieu de copier — voir `IDE.21`.

ERR.6 — Lancer malgré une erreur de compilation

Par défaut, IntelliJ compile tout le module avant de lancer quoi que ce soit. Une erreur dans une classe empêche donc de lancer une autre classe, même sans rapport avec elle.

Il est possible de contourner ce comportement : `Settings → Build, Execution, Deployment → Compiler → Java Compiler`, basculez `Use compiler` sur `Eclipse`, puis cochez `Proceed on errors`. L'option n'apparaît qu'après avoir changé de compilateur.



L'option Proceed on errors, disponible avec le compilateur Eclipse.

> **Ne le faites pas dans ce projet**
>
> Le compilateur Eclipse embarqué dans IntelliJ 2024.3 ne gère Java que jusqu'à la version 18. Ce cours utilise Java 21 : dès que vous basculez dessus, plus rien ne compile, et vous obtenez le message décrit en `ERR.8`.
>
> L'option est donc inutilisable ici, et c'est aussi bien : changer de compilateur pour contourner une erreur de compilation est un pis-aller. Le bon réflexe reste de lire le message et de corriger.
>
> Dans le projet du cours, le problème que l'option prétend résoudre ne se pose de toute façon pas : les squelettes fournis compilent toujours, puisqu'ils lèvent une exception au lieu d'être vides.

ERR.7 — JavaFX runtime components are missing

```
Error: JavaFX runtime components are missing,
and are required to run this application
```

La bibliothèque `JavaFX` est bien présente dans le projet, mais elle n'a pas été annoncée à la JVM au démarrage. Ce n'est pas une dépendance manquante : c'est une histoire de mode de lancement.

Lancez par `javafx:run` depuis la fenêtre Maven, ou par la classe de démarrage prévue par l'énoncé. Voir `IDE.28`.

L'explication de fond — pourquoi `JavaFX` est sortie du JDK et ce que cela change — se trouve en `TEC-FX.2` du mémento technique.

ERR.8 — source level should be in ... : 21

```
java: java.lang.IllegalArgumentException: source level should be in
'1.1'...'1.8','9'...'18' (or '5.0'..'18.0'): 21
```

Ce message ne vient pas de `javac` mais du compilateur Eclipse. Il dit qu'on lui demande de compiler en Java 21 alors qu'il ne connaît le langage que jusqu'à la version 18.

La cause est presque toujours la même : le compilateur du projet a été basculé sur Eclipse, souvent pour accéder à l'option décrite en `ERR.6`. Une fois basculé, il le reste.

Le remède. `Settings → Build, Execution, Deployment → Compiler → Java Compiler`, champ `Use compiler` : remettez-le sur `Javac`. Relancez ensuite la compilation.

Si le message persiste, vérifiez dans le même écran que le champ « Project bytecode version » et les « Target bytecode version » par module indiquent 21 ou « Same as language level ».

> **Un message qui désigne le coupable**
>
> La liste des versions acceptées est signée : elle s'arrête à 18, ce qui ne correspond à aucun JDK que vous auriez installé. C'est le genre de détail qui permet d'identifier l'outil fautif sans chercher plus loin.
>
> À l'inverse, javac dirait quelque chose comme « release version 21 not supported », en nommant la version qu'il connaît.

ERR.9 — Un exercice inachevé bloque tous les autres

Vous laissez un exercice à moitié écrit, avec une accolade manquante ou un type qui ne correspond pas. Et soudain plus rien ne se lance — pas même un exercice fait la semaine dernière, qui n'a aucun rapport.

Ce n'est pas une panne, c'est ainsi que Java compile. Tout le dossier `src/main/java` du module se compile d'un seul bloc, et `src/test/java` en dépend. Un seul fichier fautif suffit donc à empêcher la production de tous les autres.

Ce qui n'est PAS bloqué

Le projet compte trois modules Maven indépendants — `exercices`, `solutions` et `demos` — et aucun ne dépend des deux autres. Une erreur dans votre code n'empêche donc ni d'ouvrir un corrigé, ni de lancer une démonstration du cours.

Trois façons de s'en sortir

1. Commenter le code fautif. Sélectionnez les lignes en chantier et faites `Ctrl+/` / ⌘/. Tout recompile, vous reprenez votre travail plus tard. C'est la solution en dix secondes, et celle à prendre comme réflexe.

2. Remettre le squelette d'origine. Extrayez l'archive de la séance dans un dossier temporaire — et non par-dessus votre projet, voir `TEC-ZIP.1` — puis recopiez le seul fichier concerné. Vous perdez ce que vous aviez écrit dans ce fichier-là, rien d'autre : `TEC-ZIP.2` du mémento technique explique pourquoi.

3. Demander à IntelliJ de lancer malgré tout. Dans `Run → Edit Configurations`, section `Before launch`, remplacez la tâche `Build` par `Build, no error check`. IntelliJ compile alors ce qu'il peut et lance votre classe si elle-même est correcte.

> **La troisième solution a ses limites**
>
> Elle ne fonctionne que si la classe que vous lancez a réellement compilé. Si c'est elle qui contient l'erreur, rien ne la sauvera.
>
> Et elle se règle configuration par configuration, ce qui devient vite fastidieux. Considérez-la comme un dépannage ponctuel, pas comme un réglage à laisser en place.
>
> Ne cherchez surtout pas à obtenir le même effet en changeant de compilateur : voir `ERR.6`, qui explique pourquoi cela ne marche pas dans ce projet.

Ce qu'il faut en retenir

Laissez votre code dans un état qui compile avant de passer à autre chose. Un exercice incomplet mais compilable ne gêne personne ; un exercice à moitié tapé bloque toute la soirée.

C'est d'ailleurs pourquoi les squelettes qu'on vous fournit lèvent une `UnsupportedOperationException` au lieu d'être vides : un squelette compile dès le premier instant, avant même que vous n'ayez écrit quoi que ce soit.

ERR.10 — Un fichier disparaît de la fenêtre Project

Vous travaillez sur un fichier, et il s'efface de l'arborescence. Parfois le dossier entier. Il revient de lui-même au bout d'un moment, ou au prochain clic dans la fenêtre.

Votre fichier n'est pas perdu. Il est sur le disque, exactement là où vous l'avez laissé. C'est la *vue* qu'IntelliJ en a qui s'est désynchronisée.

Pourquoi cela arrive sur un disque réseau

IntelliJ ne relit pas le disque à chaque affichage : ce serait trop lent. Il garde une image de l'arborescence en mémoire, et demande au système d'exploitation de le prévenir quand un fichier change.

Ce mécanisme d'alerte ne fonctionne pas sur un partage réseau. IntelliJ le sait, et il l'annonce parfois lui-même dans une bulle au démarrage :

```
External file changes sync may be slow:
Project files cannot be watched (are they under network mount?)
```

Faute d'être prévenu, IntelliJ se rabat sur une relecture périodique. Entre deux relectures, sa vue peut être en retard. Et si le partage répond lentement au moment exact où il relit un dossier, la lecture échoue : IntelliJ conclut que le dossier est vide, et l'affiche vide. La relecture suivante, réussie, remet tout en place — d'où le retour spontané.

Ce qu'il faut faire

Forcez la relecture : `File → Reload All from Disk` — `Ctrl+Alt+Y` / `⌥⌘Y`. L'arborescence est reconstruite à partir du disque, et le fichier réapparaît.

Si vous voulez vous rassurer avant, ouvrez le dossier dans l'explorateur de fichiers du système : il y verra le fichier, lui.

> **Ce qu'il ne faut surtout pas faire**
>
> Recréer le fichier. Vous en obtiendriez un second, vide, qui écraserait le vôtre à l'enregistrement — et là, cette fois, le travail serait bel et bien perdu.
>
> Le réflexe est compréhensible, et c'est précisément pour cela qu'il faut l'avoir vu écrit avant de se trouver devant l'écran.

Le même symptôme, une autre cause

Si le fichier ne revient pas après un `Reload All from Disk`, ce n'est plus un problème d'affichage. Vérifiez alors qu'il n'a pas été déplacé par un `F6` involontaire, ou filtré par la barre de recherche de la fenêtre Project — quelques lettres tapées par erreur dans l'arborescence y font office de filtre, et masquent tout le reste.

En séance d'examen, signalez-le plutôt que d'improviser : cinq secondes de vérification valent mieux qu'un fichier écrasé.

ERR.11 — Les modules Maven ont disparu : plus rien ne se lance

Les dossiers `exercices`, `solutions` ou `demos` apparaissent comme des dossiers ordinaires : `src/main/java` n'est plus en bleu, aucune flèche verte ne s'affiche à côté des `main`, et les tests ne se lancent plus. Cela arrive souvent après avoir récupéré une nouvelle version du projet ou déplié une archive dedans : IntelliJ n'a pas relu les `pom.xml`.

IntelliJ propose normalement de le faire lui-même, par une petite icône Maven qui apparaît en haut à droite de l'éditeur. Elle est discrète, et on la manque facilement.

Ouvrez la fenêtre Maven et cliquez sur le bouton de rechargement. Voir `IDE.9`. Si la fenêtre Maven est vide, faites un clic droit sur le `pom.xml` à la racine du projet, puis `Add as Maven Project`. Si le problème persiste, voir `ERR.3`.

Avant de recharger, vérifiez que chaque module déclaré par le `pom.xml` racine existe bien, avec son propre `pom.xml` : s'il en manque un, Maven refuse de charger le projet entier, et tous les modules disparaissent d'un coup.

Annexe B — Mémento technique

Ce qu'IntelliJ fait à votre place

À quoi sert ce mémento

Vous allez écrire du Java toute l'année sans avoir besoin de savoir ce qui se passe entre le moment où vous cliquez sur la flèche verte et celui où le texte s'affiche à l'écran. Jusqu'au jour où quelque chose se passe mal — et là, sans ces notions, on cherche à l'aveugle.

Ce document répond aux questions que le cours ne pose pas directement. Que fait exactement le JDK. Pourquoi un programme Java tourne sur Windows comme sur Mac. Où vivent vos objets. Ce que Maven télécharge pendant la minute d'attente du premier lancement.

Il se consulte, il ne se lit pas d'une traite. Chaque section porte un code — `TEC-EXE.2` pour le tas, par exemple — et les énoncés d'exercices y renvoient au moment utile. Les codes sont cliquables.

Les deux autres documents

Ce mémento fait partie d'un ensemble de trois. Le mémento Clean Code rassemble les règles d'écriture. Le guide IntelliJ explique comment faire les choses dans l'outil.

La frontière est simple : ce qui est vrai de Java est ici, ce qui est propre à IntelliJ est dans le guide. Comprendre ce qu'est un JDK relève de ce document ; savoir où cliquer pour en changer relève du guide.

Sommaire

La plateforme Java	
`TEC-PLA.1`	JVM, JRE, JDK : qui fait quoi
`TEC-PLA.2`	Les trois éditions
`TEC-PLA.3`	Les versions et le support long terme
`TEC-PLA.4`	Que contient ma machine

Les interfaces graphiques	
`TEC-FX.1`	Trois générations, et pourquoi
`TEC-FX.2`	JavaFX ne fait plus partie du JDK
`TEC-FX.3`	Les modules Java de JavaFX
`TEC-FX.4`	Pourquoi une application JavaFX refuse de démarrer

De votre code au programme qui tourne	
TEC-CMP.1	Trois états de votre programme
TEC-CMP.2	Pourquoi Java tourne partout
TEC-CMP.3	Le classpath
TEC-CMP.4	Le dossier target

Ce qui se passe pendant l'exécution	
TEC-EXE.1	La pile
TEC-EXE.2	Le tas
TEC-EXE.3	Ce que contient vraiment une variable
TEC-EXE.4	Le ramasse-miettes

Maven	
TEC-MVN.1	À quoi sert Maven
TEC-MVN.2	Le pom.xml
TEC-MVN.3	Un projet, plusieurs modules Maven
TEC-MVN.4	Le cycle de vie
TEC-MVN.5	Le dépôt local

Recevoir les fichiers du cours	
TEC-ZIP.1	Extraire une archive au bon endroit
TEC-ZIP.2	Ne pas écraser son propre travail

Git, en deux mots	
TEC-GIT.1	Ce qu'est un gestionnaire de versions
TEC-GIT.2	Pourquoi certains dossiers sont ignorés
TEC-GIT.3	Le dépôt du fil rouge, et comment le parcourir

La plateforme Java

TEC-PLA.1 — JVM, JRE, JDK : qui fait quoi

Trois sigles, trois choses emboîtées l'une dans l'autre. Chacune contient la précédente.

Sigle	Nom	Contenu
`JVM`	Java Virtual Machine	Le programme qui exécute du bytecode Java
`JRE`	Java Runtime Environment	La JVM, plus la bibliothèque standard (`String`, `List`, `LocalDate`…)
`JDK`	Java Development Kit	Le JRE, plus les outils de développement : javac, javadoc, jar, jdb

```
JDK  =  JRE  +  javac, javadoc, jar, jdb, ...
        |
        +--  JVM  +  bibliotheque standard
```

La conséquence pratique tient en une phrase : pour exécuter un programme Java il suffit d'un JRE, pour en écrire un il faut un JDK. Comme vous écrivez du code, c'est un JDK qu'il vous faut.

> **Une correction par rapport à ce que vous lirez ailleurs**
>
> Beaucoup de schémas trouvés sur Internet présentent le JRE comme un produit qu'on télécharge séparément. C'était vrai jusqu'à Java 8. Depuis Java 11, le JRE n'est plus distribué seul : on installe un JDK, point. Le mot reste utile pour désigner la partie « exécution » de ce JDK, mais vous ne trouverez pas de JRE à télécharger.

TEC-PLA.2 — Les trois éditions

Java se décline en trois éditions, qui ne visent pas les mêmes machines.

Édition	Pour quoi	Vous concerne ?
Java SE	Standard. Applications de bureau, consoles, bibliothèques.	Oui — c'est ce cours
Jakarta EE	Entreprise. Applications serveur, web, transactions.	Plus tard, en spécialisation
Java ME	Micro. Systèmes embarqués, cartes à puce.	Non, et de plus en plus marginal

Quand une documentation parle de « Java » sans préciser, il s'agit presque toujours de Java SE.

Note de vocabulaire. L'édition entreprise s'appelait Java EE ou J2EE. Oracle en a transféré le développement à la fondation Eclipse à la fin des années 2010, et pour des raisons de marque elle a été renommée **Jakarta EE**. Vous rencontrerez les deux noms selon l'âge du document que vous lisez.

TEC-PLA.3 — Les versions et le support long terme

Depuis 2018, une nouvelle version de Java sort tous les six mois, en mars et en septembre. La plupart ne sont maintenues que six mois, jusqu'à la suivante.

Tous les deux ans environ, une version est désignée **LTS** — `Long` Term Support. Celle-là est maintenue pendant plusieurs années : elle reçoit des correctifs de sécurité longtemps après sa sortie. Les versions LTS récentes sont 8, 11, 17, 21 et 25.

Ce cours utilise Java 21. N'installez pas une version plus récente en pensant bien faire. Une version supérieure accepte des constructions que 21 ignore : votre code compilerait chez vous et refuserait de compiler ailleurs — sur la machine d'un condisciple, ou à l'examen.

Le projet du cours fixe d'ailleurs cette contrainte lui-même, dans son `pom.xml` : voir TEC-MVN.2.

TEC-PLA.4 — Que contient ma machine

Deux commandes, dans un terminal sous macOS ou Linux, dans une invite de commandes sous Windows.

```
java -version      # la version qui EXECUTE
javac -version     # la version qui COMPILE
```

La première doit répondre quelque chose comme :

```
openjdk version "21.0.12" 2024-07-16 LTS
OpenJDK Runtime Environment Corretto-21.0.12.7.1
OpenJDK 64-Bit Server VM Corretto-21.0.12.7.1 (build 21.0.12+7-LTS, mixed mode)
```

Si javac répond que la commande est introuvable alors que `java` fonctionne, vous avez un environnement d'exécution mais pas de kit de développement. Vous pouvez lancer des programmes Java, pas en compiler.

Si les deux répondent des versions différentes, plusieurs JDK cohabitent sur votre machine et ce n'est pas le même qui est trouvé en premier. Ça n'a rien d'anormal, mais il faut savoir lequel IntelliJ utilise — qui n'est d'ailleurs pas nécessairement celui de ces commandes.

Lister tous les JDK installés

Sous macOS :

```
/usr/libexec/java_home -V
```

Sous Windows, dans PowerShell :

```
Get-ChildItem "C:\Program Files\Java", "C:\Program Files\Eclipse Adoptium" -EA 0
```

C'est la commande à connaître le jour où IntelliJ vous annonce une version que vous ne vous souvenez pas d'avoir installée. Le réglage effectivement utilisé par IntelliJ, lui, se vérifie dans `Project Structure` : voir la section `IDE.6` du guide IntelliJ.

Les interfaces graphiques

TEC-FX.1 — Trois générations, et pourquoi

Java a connu trois bibliothèques successives pour construire des fenêtres. Elles ne se sont pas remplacées par mode : chacune corrige un défaut de la précédente.

Bibliothèque	Apparue	Principe
`AWT`	1995, avec Java 1.0	Chaque bouton est un vrai bouton du système d'exploitation
`Swing`	1998, avec Java 2	Java dessine lui-même tous les composants
`JavaFX`	2008, intégrée au JDK de 2012 à 2018	Description déclarative de l'interface, styles, écrans tactiles

AWT et sa limite

AWT confie l'affichage au système : un bouton AWT sous Windows est un bouton Windows, et sous macOS un bouton macOS. L'idée paraît bonne — l'application ressemble aux autres applications de la machine — mais elle impose de se limiter au dénominateur commun des trois systèmes, et les comportements diffèrent d'un poste à l'autre. Le même programme ne se comporte pas tout à fait pareil partout, ce qui est exactement ce que Java cherchait à éviter.

Swing et son principe

Swing prend le contre-pied : Java dessine lui-même chaque composant, pixel par pixel. L'apparence et le comportement deviennent identiques partout, au prix d'une interface qui ne ressemble à aucune autre sur la machine.

Swing introduit une organisation qu'on retrouvera à l'identique dans `JavaFX`, et qu'il vaut donc la peine de connaître :

- **les composants** — boutons, champs de texte, images ;
- **les conteneurs** — des composants qui en contiennent d'autres ;
- **les gestionnaires de placement** — à qui l'on confie la disposition, plutôt que de fixer des coordonnées, ce qui permet à la fenêtre de s'adapter quand on la redimensionne ;
- **les gestionnaires d'événements** — un clic devient un objet, transmis à du code qui décide quoi en faire.

JavaFX

`JavaFX` conserve ces quatre familles et y ajoute trois choses : la possibilité de décrire l'interface dans un fichier séparé plutôt qu'en code, une mise en forme par feuilles de style comparable au CSS du web, et la prise en charge des écrans tactiles.

C'est aujourd'hui la bibliothèque officielle. Le développement d'AWT et de Swing est arrêté — ils restent livrés avec le JDK, pour que les applications existantes continuent de fonctionner, mais ils n'évoluent plus.

> **AWT n'a pourtant pas disparu de votre écran**
>
> Swing est bâti sur AWT, et certaines classes AWT restent utilisées partout — notamment celles qui représentent les événements. Vous trouverez donc un `KeyEvent` dans `java.awt.event` et un autre dans `javafx.scene.input`.
>
> Les deux portent le même nom et n'ont rien à voir. Quand la complétion vous en propose un, lisez le nom complet avant de valider : importer celui d'AWT dans une application `JavaFX` donne un code qui ne compile pas, avec un message qui n'aide pas. Voir `IDE.11` du guide IntelliJ.

TEC-FX.2 — JavaFX ne fait plus partie du JDK

`JavaFX` a été livrée avec le JDK depuis Java 8, puis en a été retirée à partir de Java 11. Depuis, c'est un projet distinct, OpenJFX, développé en logiciel libre.

La conséquence est directe. Installer un JDK 21 ne vous donne pas `JavaFX`. Il faut l'ajouter au projet comme n'importe quelle bibliothèque externe, en la déclarant dans le `pom.xml` — voir `TEC-MVN.2`. C'est ce que fait le projet du cours, et c'est pourquoi le premier chargement télécharge davantage de choses qu'un projet ordinaire.

Cela explique aussi une bizarrerie que vous rencontrerez : une application `JavaFX` ne se lance pas toujours d'un simple clic sur la flèche verte, parce que la bibliothèque doit être signalée à la JVM au démarrage. Le projet du cours contient un greffon Maven qui s'en charge, et l'application se lance alors par la commande Maven correspondante.

Si vous voyez le message `JavaFX runtime components are missing`, c'est exactement ce problème : la bibliothèque est présente mais n'a pas été annoncée au lancement.

TEC-FX.3 — Les modules Java de JavaFX

`JavaFX` est découpée en sept modules. Un projet ne déclare que ceux dont il a besoin, ce qui évite d'embarquer un lecteur vidéo dans une application qui n'affiche que des boutons.

Module	Contenu
`javafx.base`	L'API de base : propriétés, liaisons, collections observables
`javafx.graphics`	Conteneurs, formes, images, animations, effets
`javafx.controls`	Les composants : boutons, champs, tableaux, listes
`javafx.fxml`	Le format FXML, qui décrit une interface en XML
`javafx.media`	Lecture audio et vidéo
`javafx.web`	Affichage de contenu web
`javafx.swing`	Passerelle pour intégrer du `JavaFX` dans une application Swing

Ce cours utilise les quatre premiers. `javafx.fxml` mérite une mention particulière : il permet de décrire la fenêtre dans un fichier `.fxml` séparé du code Java. La description de l'interface d'un côté, le comportement de l'autre — c'est ce qui rend praticable la séparation en modèle, vue et contrôleur.

TEC-FX.4 — Pourquoi une application JavaFX refuse de démarrer

Vous rencontrerez ce message, et il déroute parce qu'il semble faux : les bibliothèques sont bien là, Maven les a téléchargées, IntelliJ les affiche dans la ligne de commande.

```
Erreur : des composants d'execution JavaFX obligatoires pour
executer cette application sont manquants.
```

Ce qui se passe réellement

Une bibliothèque peut être fournie de deux façons. Sur le **classpath**, la manière historique, décrite en TEC-CMP.3. Ou sur le **chemin de modules**, apparu avec Java 9, où chaque bibliothèque déclare ce qu'elle offre et ce qu'elle exige.

Le lanceur du JDK examine la classe qu'on lui demande d'exécuter. **Si elle hérite d'Application**, il en déduit que c'est une application `JavaFX`, et il exige alors que `JavaFX` soit chargé comme module. Il vérifie, ne le trouve pas, et s'arrête — avant d'avoir exécuté la moindre ligne de votre code.

Or Maven, lui, place les bibliothèques sur le classpath ordinaire. Les deux ne s'entendent pas, et c'est tout le problème : *rien ne manque*, les jars sont présents et complets. C'est leur mode de chargement qui ne convient pas au contrôle.

La parade : une classe de lancement

Le contrôle ne se déclenche que si la classe principale hérite d'Application. Il suffit donc de lui en présenter une autre.

```
public class LanceurDemineur {

    public static void main(String[] args) {
        Application.launch(Demineur.class, args);
    }
}
```

Cette classe n'hérite de rien. Le lanceur ne reconnaît pas une application `JavaFX`, ne cherche pas de module, et exécute le `main`. C'est ensuite `Application.launch` qui démarre `JavaFX` — depuis le classpath, sans protester, parce que le contrôle est derrière nous.

> **Ce n'est pas un bricolage**
>
> C'est la forme recommandée dès qu'on distribue une application `JavaFX` sans passer par les modules, et vous la retrouverez telle quelle dans beaucoup de projets.
>
> Conséquence pratique : la classe à lancer n'est pas celle qui hérite d'Application. Dans ce cours, c'est toujours celle dont le nom commence par Lanceur.

Les deux autres solutions, et leur prix

Solution	Ce qu'elle coûte
`Ajouter --module-path et --add-modules aux options de la machine virtuelle`	le chemin dépend de la machine : la configuration d'exécution n'est plus transportable d'un poste à l'autre
`Déclarer un module-info.java`	la solution propre, mais elle impose que toutes les dépendances soient modulaires, tests compris
`Passer par mvn javafx:run`	le plugin monte le chemin de modules lui-même ; il faut penser à passer par la fenêtre Maven plutôt que par le bouton Run

La classe de lancement a l'avantage de ne rien demander : ni configuration, ni ligne de commande, ni fichier supplémentaire à tenir à jour.

De votre code au programme qui tourne

TEC-CMP.1 — Trois états de votre programme

Ce que vous écrivez et ce que la machine exécute ne sont pas la même chose. Entre les deux, il y a une traduction.

```
  Personne.java          votre code, du texte que vous lisez
        |
```

```
       |   javac        le compilateur
       v
Personne.class         du bytecode, illisible pour vous
       |
       |   java         la machine virtuelle
       v
le programme s'execute
```

`javac` lit vos fichiers `.java` et produit des fichiers `.class`. C'est là que sont détectées les erreurs de compilation : une parenthèse manquante, un type qui ne correspond pas, une méthode qui n'existe pas.

`java` prend un fichier `.class` et l'exécute. C'est là qu'apparaissent les erreurs d'exécution : une division par zéro, un `null` inattendu, un fichier introuvable.

> **Pourquoi cette distinction compte**
>
> Elle vous dit dans quelle direction chercher. Une erreur signalée avant que le programme ne démarre vient de javac : le code est mal écrit. Une erreur qui interrompt un programme en cours de route vient de la JVM : le code est correct mais ce qu'il rencontre ne l'est pas.
>
> IntelliJ vous le dit d'ailleurs : les messages de compilation commencent par « java: », ceux d'exécution apparaissent dans la fenêtre Run avec une trace d'appels.

TEC-CMP.2 — Pourquoi Java tourne partout

Un programme écrit en C doit être recompilé pour chaque système. En Java, non : le même fichier .class s'exécute sur Windows, macOS et Linux.

```
        Personne.class
       /      |       \
    JVM      JVM      JVM
Windows     Mac     Linux
```

Le bytecode n'est pas destiné à un processeur : il est destiné à la JVM. C'est elle qui diffère d'un système à l'autre, et qui traduit le bytecode en instructions que la machine comprend. Vous compilez une fois, vous exécutez partout — à condition qu'une JVM soit installée.

C'est ce qui explique aussi qu'un programme Java soit un peu plus lent à démarrer qu'un programme compilé directement : il y a une traduction supplémentaire au moment de l'exécution.

TEC-CMP.3 — Le classpath

Quand votre programme utilise `ArrayList`, la JVM doit trouver le fichier `ArrayList.class` quelque part. La liste des endroits où elle cherche s'appelle le **classpath**.

Il contient trois sortes de choses : la bibliothèque standard, fournie avec le JDK ; les bibliothèques externes, comme `JUnit` ; et vos propres classes compilées.

Vous n'aurez normalement jamais à le construire à la main : Maven le fabrique à partir des dépendances déclarées dans le `pom.xml`, et IntelliJ le transmet à la JVM au lancement.

Le connaître sert surtout à comprendre un message. `ClassNotFoundException` et `NoClassDefFoundError` veulent tous deux dire la même chose : une classe manque au classpath. Presque toujours, une dépendance n'a pas été téléchargée.

TEC-CMP.4 — Le dossier target

Maven range tout ce qu'il produit dans un dossier `target`, à la racine de chaque module. Vous y trouverez vos `.class`, les rapports de test, et l'archive `.jar` si vous en fabriquez une.

Rien de ce que contient ce dossier n'a de valeur. Tout s'y recrée intégralement à partir de `src` et du `pom.xml`. C'est pourquoi il est ignoré par Git, absent des archives que vous recevez, et pourquoi vous pouvez le supprimer sans crainte.

Le supprimer est d'ailleurs un réflexe utile quand quelque chose se comporte bizarrement : cela force une recompilation complète et élimine l'hypothèse d'un fichier compilé périmé.

Ce qui se passe pendant l'exécution

Pendant qu'il tourne, votre programme occupe deux zones de mémoire aux comportements très différents. Comprendre laquelle contient quoi éclaire une bonne partie de ce qui surprend en début d'apprentissage.

TEC-EXE.1 — La pile

Chaque fois qu'une méthode est appelée, la JVM empile un bloc contenant ses paramètres et ses variables locales. Quand la méthode se termine, le bloc est retiré et tout ce qu'il contenait disparaît.

D'où le nom : les blocs s'empilent et se dépilent, et on ne retire jamais que celui du dessus — c'est-à-dire la méthode en cours.

```
main()                      <-- appelle inscrire()
  inscrire(pseudos)         <-- appelle ajouter()
    ajouter(joueur)         <-- methode en cours d'execution

  ^ la pile : trois blocs, le dernier appele est en haut
```

Vous avez déjà vu cette pile sans le savoir : c'est exactement ce qu'affiche une trace d'exception, lue de bas en haut. Et le débogueur la montre en direct dans son panneau `Frames` — voir `IDE.19` du guide IntelliJ.

La pile a une taille limitée. Une méthode qui s'appelle elle-même sans condition d'arrêt empile des blocs jusqu'à la saturer, et la JVM lève alors une `StackOverflowError`. Ce message ne veut rien dire d'autre que : trop d'appels imbriqués, cherchez une récursion sans fin.

TEC-EXE.2 — Le tas

Les objets, eux, ne vivent pas dans la pile. Chaque `new` réserve de la place dans une seconde zone, le **tas**, et y construit l'objet.

La différence essentielle est de durée de vie. Le contenu de la pile disparaît dès que la méthode se termine ; un objet du tas survit tant que quelque chose le désigne. C'est ce qui permet à une méthode de créer un objet et de le retourner à son appelant — l'objet reste, seul le bloc de pile a disparu.

```
public Partie inscrire(String... pseudos) {
    Partie partie = new Partie();   // l'objet va dans le TAS
    ...                             // la variable est dans la PILE
    return partie;                  // le bloc de pile disparait,
}                                   // l'objet du tas survit
```

Le tas aussi a une limite. La saturer donne une `OutOfMemoryError` : trop d'objets créés, ou des objets qu'on n'arrive plus à libérer. Retenez la paire — `StackOverflowError` concerne les appels, `OutOfMemoryError` concerne les objets.

TEC-EXE.3 — Ce que contient vraiment une variable

Voici le point qui explique le plus de surprises de première année.

Une variable de type primitif — `int`, double, `boolean` — contient sa valeur. Une variable d'objet, non : elle contient une référence, c'est-à-dire de quoi retrouver l'objet dans le tas. L'objet est ailleurs.

```
  PILE                          TAS
  ----------------              ---------------------
  nombre  =  5
  personne  =  ---------------->  [ Personne: nom, prenom ]
```

Première conséquence : le passage de paramètres

Java passe toujours les arguments *par valeur* : la méthode reçoit une copie. Pour un `int`, la valeur copiée est le nombre, et modifier la copie ne change rien à l'original. Pour un objet, la valeur copiée est la *référence* — la copie désigne donc le même objet, et les modifications faites à travers elle sont visibles de partout.

Ce n'est pas un « passage par référence » : réaffecter le paramètre à l'intérieur de la méthode ne changerait rien chez l'appelant. C'est le comportement que l'exercice `POO.4` vous fait constater.

Seconde conséquence : deux noms, un objet

```
Personne a = new Personne();
Personne b = a;              // b designe le MEME objet

b.setNom("Lovelace");
System.out.println(a.getNom());   // affiche Lovelace
```

Le débogueur rend cela visible : il affiche un numéro à côté de chaque objet, `Personne@827` par exemple. Deux variables qui portent le même numéro désignent le même objet. Ce numéro n'a aucune signification en soi et change à chaque exécution — il ne sert qu'à distinguer les objets entre eux.

TEC-EXE.4 — Le ramasse-miettes

En Java, on ne libère jamais la mémoire à la main. Il n'existe pas de `delete`. Un programme séparé, le **ramasse-miettes** — garbage collector —, tourne en arrière-plan et récupère la place des objets que plus personne ne désigne.

```
Personne p = new Personne();   // l'objet est cree dans le tas
p = null;                      // plus rien ne le designe
                               // le ramasse-miettes le recuperera
                               // un jour, sans qu'on sache quand
```

Le critère est l'accessibilité, pas la portée : un objet reste vivant tant qu'une variable, un champ ou une collection le désigne, même indirectement. Un objet placé dans une liste survit à la disparition de la variable qui l'a créé.

Vous ne contrôlez pas le moment. Il existe bien une méthode `System.gc()`, mais elle ne fait que suggérer, et l'usage recommandé est de ne pas l'appeler. Écrivez votre code sans vous préoccuper de la libération : c'est précisément ce que Java vous offre par rapport à des langages où l'oubli d'un `free` est une fuite de mémoire.

À faire une fois, dans le débogueur

> **Voir la pile et le tas de ses propres yeux**
>
> Ouvrez la classe Inscription de l'exercice `PKG.4`. Posez un point d'arrêt sur la ligne qui ajoute un joueur à la partie, puis lancez Application en mode débogage.
>
> Au deuxième ou troisième arrêt, regardez le panneau Frames : vous y lisez la pile, inscrire appelée depuis main. Puis dépliez la variable partie dans le panneau Variables.
>
> Répondez alors à trois questions. Combien d'objets Joueur existent à cet instant ? Qui les désigne ? Et que deviendraient-ils si la méthode retournait autre chose que partie ?
>
> La marche à suivre dans l'outil est décrite en `IDE.19` du guide IntelliJ.

Maven

TEC-MVN.1 — À quoi sert Maven

Maven résout trois problèmes que vous n'avez pas encore rencontrés parce qu'il s'en occupe déjà.

- **Il impose une structure.** Le code dans `src/main/java`, les tests dans `src/test/java`, le résultat dans `target`. Cette convention est la même dans tous les projets Maven du monde, ce qui fait qu'on s'y retrouve immédiatement dans un projet inconnu.

- **Il gère les bibliothèques.** Vous déclarez que le projet a besoin de `JUnit` ; Maven va le chercher, ainsi que tout ce dont `JUnit` a lui-même besoin.
- **Il rend la compilation reproductible.** Le même projet compilé sur une autre machine donne le même résultat, avec les mêmes versions de tout.

Sans Maven, il faudrait télécharger les bibliothèques à la main, les ranger quelque part, et construire soi-même le classpath. C'est faisable, et c'est fastidieux.

TEC-MVN.2 — Le pom.xml

Tout tient dans un fichier, le `pom.xml` — Project `Object` Model. Il décrit le projet et ses besoins.

```
<groupId>be.ephec.java</groupId>       <!-- qui publie -->
<artifactId>exercices</artifactId>     <!-- quoi -->
<version>1.0-SNAPSHOT</version>        <!-- quelle version -->

<properties>
    <maven.compiler.release>21</maven.compiler.release>
</properties>

<dependencies>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>5.10.2</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

Les trois premières lignes forment les coordonnées du projet : elles l'identifient de façon unique, exactement comme elles identifient `JUnit` un peu plus bas. Toute bibliothèque du monde Java se désigne par ce triplet.

`maven.compiler.release` fixe la version de Java. C'est cette ligne qui garantit que votre code compilera de la même façon partout, quelle que soit la version du JDK installé — voir `TEC-PLA.3`.

`scope test` signifie que `JUnit` n'est disponible que pour le code de test. C'est volontaire : si vous tentez d'importer une classe `JUnit` dans `src/main/java`, cela ne compilera pas — et c'est tant mieux, du code de production n'a rien à faire avec une bibliothèque de test.

TEC-MVN.3 — Un projet, plusieurs modules Maven

Le projet du cours est composé de trois modules Maven, `exercices`, `solutions` et `demos`, chapeautés par un projet parent.

```
ExercicesJava/
    pom.xml            <-- le parent : declare les trois modules
    exercices/
        pom.xml        <-- herite du parent
        src/
    solutions/
        pom.xml
        src/
    demos/
        pom.xml
        src/
```

Le parent contient ce qui est commun : la version de Java, les versions des bibliothèques. Les modules n'ont plus qu'à déclarer ce qui leur est propre. Changer la version de Java se fait donc à un seul endroit.

Les trois modules sont indépendants les uns des autres : le module des solutions ne dépend pas de celui des exercices, ni celui des démonstrations d'aucun des deux, ce qui garantit qu'aucune classe de l'un ne peut être importée par un autre par accident. Une erreur de compilation dans vos exercices n'empêche donc pas de lancer une démonstration.

TEC-MVN.4 — Le cycle de vie

Maven définit une suite d'étapes ordonnées. Demander une étape exécute automatiquement toutes celles qui la précèdent.

Étape	Ce qu'elle fait
`compile`	Compile src/main/java vers target/classes
`test`	Compile les tests et les exécute (compile d'abord)
`package`	Fabrique le .jar (compile et test d'abord)
`clean`	Supprime target — s'utilise seule, hors du cycle

Vous n'aurez pas souvent à les lancer vous-même : IntelliJ le fait pour vous quand vous cliquez sur la flèche verte. Elles apparaissent dans la fenêtre Maven, sous `Lifecycle`, et se lancent d'un double clic.

Le seul cas où vous y viendrez : `clean` puis un nouveau lancement, quand vous soupçonnez un fichier compilé périmé. Voir `TEC-CMP.4`.

TEC-MVN.5 — Le dépôt local

Les bibliothèques téléchargées ne sont pas rangées dans le projet, mais dans un dossier commun à tous vos projets : `.m2/repository`, dans votre dossier personnel.

C'est ce qui explique la minute d'attente au premier chargement, et son absence les fois suivantes : Maven regarde d'abord dans ce dépôt local, et ne va sur Internet que si la bibliothèque y manque. Un second projet utilisant `JUnit` 5.10.2 la trouvera déjà là.

Ce dossier peut être supprimé sans risque : tout se retéléchargera. C'est le remède de dernier recours quand une bibliothèque semble corrompue — mais comptez alors à nouveau le temps de téléchargement.

Recevoir les fichiers du cours

TEC-ZIP.1 — Extraire une archive au bon endroit

Chaque séance donne lieu à une archive. Elle contient elle-même un dossier `ExercicesJava`, au même nom que le vôtre :

```
SeanceNN_..._exercices.zip
    ExercicesJava/
        exercices/src/main/java/be/ephec/java/tab/...
        exercices/src/test/java/be/ephec/java/tab/...
        enonces/TAB_enonces.docx
```

Elle doit donc être extraite dans le dossier PARENT du vôtre, pas à sa racine. Les deux arborescences fusionnent alors, et les nouveaux packages viennent s'ajouter aux anciens. Extraite à la racine, elle produirait un absurde `ExercicesJava/ExercicesJava/...` que Maven ignorerait.

Pour retrouver ce dossier parent sans se tromper : dans IntelliJ, clic droit sur la racine du projet, `Open In` puis `Finder` ou `Explorer`, et remontez d'un niveau.

macOS : passez par le Terminal

Le double-clic ne convient pas. L'Utilitaire d'archive extrait toujours à côté du fichier zip, et s'il rencontre un dossier du même nom il en crée un second — « ExercicesJava 2 » — au lieu de fusionner. Vous vous retrouvez avec deux projets, dont un incomplet.

```
unzip -o ~/Downloads/SeanceNN_..._exercices.zip -d ~/chemin/du/parent
```

`unzip` fusionne dans les dossiers existants, ce que le Finder ne fait pas de façon fiable. L'option `-o` évite qu'il pose une question pour chaque fichier.

Astuce : tapez `unzip -o ` puis faites glisser le fichier zip depuis le Finder dans la fenêtre du Terminal — son chemin s'écrit tout seul. Même chose pour le dossier de destination après `-d`.

> **Pourquoi pas le glisser-déposer**
>
> On serait tenté d'extraire n'importe où puis de faire glisser le dossier ExercicesJava sur celui du projet. Le Finder propose alors « Remplacer », ce qui supprimerait tout votre travail, et ne propose « Fusionner » que dans certains cas.
>
> Le risque n'en vaut pas la peine : une ligne de Terminal fait le travail sans ambiguïté.

Windows : Extraire tout, en corrigeant le chemin

Clic droit sur le zip, `Extraire tout`. Une boîte de dialogue demande la destination, et Windows fusionne correctement les dossiers.

Attention au chemin proposé. Windows ajoute par défaut un sous-dossier portant le nom du zip. Effacez cette partie pour ne garder que le dossier parent de votre projet, sinon vous obtiendrez `...\SeanceNN_..._exercices\ExercicesJava\...` — un niveau de trop.

En PowerShell, si vous préférez :

```
Expand-Archive -Path "$HOME\Downloads\SeanceNN_..._exercices.zip" `
               -DestinationPath "C:\chemin\parent" -Force
```

TEC-ZIP.2 — Ne pas écraser son propre travail

L'archive d'une séance n'apporte que les packages de cette séance-là. Elle ne touche donc pas à ce que vous avez écrit lors des séances précédentes, et il n'y a aucun risque à l'extraire.

En revanche, extraire deux fois l'archive d'une même séance écraserait votre travail sur ces exercices-là. Les squelettes reviendraient à leur état d'origine, avec leurs `UnsupportedOperationException`, et votre code aurait disparu.

Si vous devez reprendre une archive déjà extraite — parce que vous avez cassé quelque chose, ou pour comparer —, extrayez-la dans un dossier temporaire et recopiez seulement le fichier dont vous avez besoin.

C'est exactement le genre d'accident qu'un gestionnaire de versions rend impossible, puisqu'il conserve l'état de chaque fichier à chaque instant. Voir `TEC-GIT.1`.

Git, en deux mots

Git n'est au programme d'aucun exercice, et il ne sera pas évalué. Vous n'aurez jamais à enregistrer vos modifications, ni à travailler à plusieurs, ni à envoyer quoi que ce soit sur un serveur.

Une exception, et en lecture seule. Le corrigé du fil rouge GameHub peut vous être remis sous la forme d'un dépôt Git, qui conserve les trente incréments et permet de se placer à n'importe lequel. Quatre gestes suffisent à le parcourir, et ils sont décrits en `TEC-GIT.3`.

Les deux premières sections existent pour que le mot ne soit pas un mystère, et pour expliquer certains fichiers que vous verrez dans le projet.

TEC-GIT.1 — Ce qu'est un gestionnaire de versions

Un gestionnaire de versions enregistre l'historique d'un projet. À intervalles choisis, on lui demande de mémoriser l'état complet des fichiers — cela s'appelle un commit — avec un message décrivant ce qui a changé.

Il devient alors possible de revenir à n'importe quel état antérieur, de voir ce qui a changé entre deux moments, et de savoir qui a modifié quoi. À plusieurs, chacun travaille de son côté et le système fusionne les contributions.

Git est le plus répandu d'entre eux, et de très loin. GitHub, GitLab et Bitbucket sont des services qui hébergent des dépôts Git — ce ne sont pas des outils différents, mais des endroits où poser un dépôt.

La conséquence pratique pour vous. Puisque l'historique est conservé, on ne garde jamais de code en commentaire « au cas où ». C'est le fondement de la règle `CC-COM.2` du mémento Clean Code.

TEC-GIT.2 — Pourquoi certains dossiers sont ignorés

Le projet contient un fichier `.gitignore` listant ce que Git doit laisser de côté. Vous y trouverez au moins ceci :

```
target/      # tout se recree a la compilation
.idea/       # preferences d'affichage d'IntelliJ, propres a chaque poste
*.iml
```

La règle qui les réunit : on ne conserve dans l'historique que ce qu'on ne peut pas refabriquer. Un fichier compilé se refabrique à partir de la source. Une préférence d'affichage n'a d'intérêt que pour la personne qui l'a réglée, et l'inclure ferait entrer en conflit les réglages de chacun.

C'est aussi pourquoi vous pouvez supprimer `.idea` sans rien perdre d'important quand IntelliJ se comporte étrangement — remède mentionné en `ERR.1` du guide IntelliJ.

TEC-GIT.3 — Le dépôt du fil rouge, et comment le parcourir

Le corrigé de GameHub vous est remis sous la forme d'un **dépôt Git**. Ce n'est pas un fichier particulier : c'est un dossier ordinaire, qui contient en plus un sous-dossier caché `.git` où vit l'historique des trente incréments.

Vous l'ouvrez dans IntelliJ comme n'importe quel projet, et vous pouvez alors vous placer à n'importe quelle étape du fil rouge.

Trois notions, et rien d'autre

Terme	Ce que c'est
`commit`	l'état complet du projet à un instant, sous un nom lisible
`historique`	la suite des commits, chacun ne retenant que ce qui a changé
`étiquette (tag)`	un nom collé sur un commit — ici G.1 à G.30

Ce que le dépôt apporte et qu'un document ne peut pas donner : la comparaison. Choisissez un incrément, et IntelliJ affiche les deux versions d'un fichier côte à côte, en marquant en vert ce qui a été ajouté et en bleu ce qui a été modifié.

> **Le mode d'emploi est ailleurs, et c'est voulu**
>
> Git est un outil vaste, et il n'est pas au programme. Vous l'utiliserez en lecture seule, et quatre gestes suffisent.
>
> Ils sont décrits pas à pas, captures d'écran à l'appui, dans `Git_pour_GameHub.docx` — livré avec le dépôt, dans son dossier _documents. Ce mémento ne les recopie pas : deux documents qui disent la même chose finissent toujours par se contredire.

Deux choses à savoir avant d'ouvrir le dépôt

- **Git doit être installé sur la machine.** IntelliJ ne l'embarque pas. Sans lui, l'onglet Git n'apparaît pas et rien de ce qui précède ne fonctionne.
- **Déplacez toujours le dossier entier.** Le `.git` est caché : copier seulement les sources fait disparaître les trente versions sans qu'aucun message ne le signale.

Et un conseil qui évite la plupart des ennuis : **n'écrivez pas dans le dépôt**. C'est un corrigé qui se lit. Pour essayer des choses, copiez le code dans un projet à vous.

Annexe C — Mémento Clean Code

Des règles simples pour écrire du code lisible

À quoi sert ce mémento

Un programme est lu bien plus souvent qu'il n'est écrit — par d'autres, et par vous-même dans six mois. Les règles rassemblées ici ne rendent pas votre code plus rapide ni plus correct : elles le rendent lisible. C'est ce qui décide, en pratique, si un programme peut encore être corrigé un an plus tard.

Elles viennent pour l'essentiel de *Clean Code: A Handbook of Agile Software Craftsmanship*, de Robert C. Martin. Elles ne sont pas des lois : ce sont des réflexes par défaut, dont on s'écarte parfois en sachant pourquoi.

Comment l'utiliser

Ne le lisez pas d'une traite. Chaque règle porte un code, et les énoncés d'exercices y renvoient au moment où elle devient utile : les règles de nommage dès la première séance, celles sur les erreurs avec le module `EXC`, celles sur les tests avec JUN.

La table qui suit sert de sommaire : cliquez sur une règle pour aller à son explication. Les renvois d'une règle à l'autre dans le texte sont eux aussi cliquables.

Le code se lit ainsi : `CC-ERR.2` désigne la deuxième règle de la section « Traiter les erreurs ». Le préfixe encode le sujet, pas le rang : l'ajout d'une règle ne renumérote jamais les autres.

Ces règles comptent à l'examen. Le critère de style de la grille d'évaluation s'appuie sur elles, et un commentaire de correction peut y renvoyer directement.

Les règles en un coup d'œil

Nommer	
`CC-NOM.1`	Le nom communique l'intention
`CC-NOM.2`	Le nom ne dit pas autre chose que ce qu'il désigne
`CC-NOM.3`	Le nom se prononce
`CC-NOM.4`	Le nom n'encode pas le type
`CC-NOM.5`	Noms pour les choses, verbes pour les actions, prédicats pour les booléens
`CC-NOM.6`	La longueur du nom suit la portée
`CC-NOM.7`	Pas de nombre magique : nommez-le
`CC-NOM.8`	Déclarez une variable près de l'endroit où elle sert

Écrire une méthode	
CC-MET.1	Le nom de la méthode dit ce qu'elle fait
CC-MET.2	Une méthode reste courte
CC-MET.3	Une méthode ne fait qu'une seule chose
CC-MET.4	Le vocabulaire est cohérent d'une méthode à l'autre
CC-MET.5	Pas de drapeau booléen en paramètre
CC-MET.6	Deux paramètres au plus
CC-MET.7	Pas d'effet de bord
CC-MET.8	Une méthode agit ou répond, jamais les deux
CC-MET.9	Un booléen se retourne tel quel

Commenter	
CC-COM.1	Un commentaire nécessaire signale un code à réécrire
CC-COM.2	Pas de code laissé en commentaire
CC-COM.3	Une Javadoc ne répète pas la signature

Objets et encapsulation	
CC-OBJ.1	Demandez à l'objet d'agir, ne lui demandez pas ses entrailles
CC-OBJ.2	Un accesseur n'est pas une obligation

Traiter les erreurs	
CC-ERR.1	Une erreur se signale par une exception, pas par une valeur de retour
CC-ERR.2	Ne retournez pas null
CC-ERR.3	Ne passez pas null en paramètre
CC-ERR.4	Le message d'exception dit ce qui a échoué et avec quelle valeur
CC-ERR.5	Définissez vos exceptions selon ce que l'appelant doit distinguer
CC-ERR.6	N'avalez pas une exception en silence
CC-ERR.7	Une comparaison se protège de null

Écrire des tests	
CC-TST.1	Le code de test se soigne comme le code de production
CC-TST.2	Un test vérifie une seule chose
CC-TST.3	Un test est rapide, indépendant, répétable et auto-vérifiant
CC-TST.4	Un test se termine par une assertion

Structurer une classe	
CC-CLS.1	Une classe n'a qu'une raison de changer
CC-CLS.2	Une classe reste petite
CC-CLS.3	Ne vous répétez pas
CC-CLS.4	Préférez le polymorphisme à une chaîne de tests sur un type

Nommer

Les noms que vous choisissez sont un moyen de communication : avec les autres développeurs, avec le correcteur, et avec vous-même dans trois ans. C'est la seule partie du code que personne ne peut deviner à votre place.

CC-NOM.1 — Le nom communique l'intention

Si vous avez besoin d'un commentaire pour expliquer ce que contient une variable, c'est que son nom ne le dit pas. Mettez l'explication dans le nom et supprimez le commentaire.

À ÉVITER
```
int d;   // temps ecoule en jours
```

PRÉFÉREZ
```
int tempsEcouleEnJours;
```

L'effet se voit surtout sur un calcul un peu long. Voici la méthode que Robert C. Martin donne en exemple, avant et après :

À ÉVITER
```
public static Date addMonths(int months, Date base) {
    int yy = (12 * base.getYear() + base.getMonth() + months - 1) / 12;
    int mm = (12 * base.getYear() + base.getMonth() + months - 1) % 12 + 1;
    int dd = Math.min(base.getDayOfMonth(), Date.lastDayOfMonth(mm, yy));
    return Date.createInstance(dd, mm, yy);
}
```

PRÉFÉREZ
```
public static Date addMonths(int monthsToAdd, Date base) {
    int baseYear         = base.getYear();
    int baseMonth        = base.getMonth() - JANUARY;
    int monthsInBaseYear = 12 * baseYear;
    int baseMonths       = baseMonth + monthsInBaseYear;
    int resultMonths     = baseMonths + monthsToAdd;
    int resultYear       = resultMonths / 12;
    int resultMonth      = resultMonths % 12 + JANUARY;
    int lastDay          = Date.lastDayOfMonth(resultMonth, resultYear);
    int resultDay        = Math.min(base.getDayOfMonth(), lastDay);
    return Date.createInstance(resultDay, resultMonth, resultYear);
}
```

La seconde version est plus longue et fait exactement la même chose. Mais on peut la lire : les noms racontent le calcul, là où la première demandait de le reconstituer.

CC-NOM.2 — Le nom ne dit pas autre chose que ce qu'il désigne

Un nom qui ment est pire qu'un nom vague : il installe une certitude fausse. Veillez à ce que le mot signifie ce qu'il indique, et qu'il indique ce qu'il signifie.

À ÉVITER
```
public class Pair {
    private Object first, second, third;

    public Pair(Object first, Object second, Object third) { ... }
}
```

PRÉFÉREZ
```
public class Triplet {
    private Object first, second, third;

    public Triplet(Object first, Object second, Object third) { ... }
}
```

Ce cas arrive rarement d'emblée : il naît d'une évolution. La classe était bien une paire, on lui a ajouté un troisième élément, et personne n'a renommé. Quand le code change, les noms doivent suivre — `Maj+F6` dans IntelliJ renomme partout sans rien casser.

Même piège avec un nom trop précis : appeler `listeEtudiants` une variable de type `Set` induit en erreur. Préférez etudiants.

CC-NOM.3 — Le nom se prononce

Le test est simple : imaginez que vous devez en parler à voix haute à un collègue, au téléphone. Un nom qu'on doit épeler est un nom à changer.

À ÉVITER
```
private int pq_gwta;

private int getYYYY() {
    return this.year;
}
```

PRÉFÉREZ
```
private int pointsGagnesAuTotal;

private int getAnnee() {
    return this.annee;
}
```

Ce n'est pas un détail de confort : un nom qu'on ne peut pas prononcer ne peut pas être discuté, et un nom qui ne peut pas être discuté ne sera jamais corrigé.

CC-NOM.4 — Le nom n'encode pas le type

La notation hongroise, inventée en 1991, préfixait chaque variable par une abréviation de son type. Elle avait un sens à l'époque des éditeurs de texte sans coloration ni saut à la déclaration. Aujourd'hui votre IDE vous donne le type en survolant l'identifiant, et le préfixe ne fait qu'ajouter du bruit — sans compter qu'il devient faux dès que le type change.

À ÉVITER
```
private boolean bDanger;
private int     iValeur;
private String  strNom;
```

PRÉFÉREZ
```
private boolean enDanger;
private int     valeur;
private String  nom;
```

Une exception tolérée : le préfixe `I` devant un nom d'interface (`IForme`). C'est une habitude venue du monde .NET, encore répandue, et elle a le mérite de distinguer l'interface de son implémentation. Vous la rencontrerez ; en Java, la convention majoritaire est de ne pas l'employer.

CC-NOM.5 — Noms pour les choses, verbes pour les actions, prédicats pour les booléens

Un code bien nommé se lit comme une phrase. Cela suppose de respecter la nature grammaticale de ce qu'on nomme.

Ce qu'on nomme	
`classe`	un nom : Compte, `CompteEpargne`, `MessageParser`, `DocumentArchiver`
`variable`	un nom : solde, `joueurCourant`, `listeDeCartes`
`méthode`	un verbe : regrouper, imprimer, envoyer, identifier, creer
`booléen`	un prédicat, qui se lit après un `if` : `estActif`, `isClosed`, `estVide`
`enum`	souvent un adjectif ou un état : TODO, BUSY, DONE, SMALL, LARGE

Le résultat se lit à voix haute sans effort :

PRÉFÉREZ
```
if (panier.estVide())
    afficherMessageDeBienvenue();

if (paiement.estPossible())
    enregistrerPaiement(paiement);

if (etudiant.getPoints(cours).estCodeEchec())
```

```
    etudiant.recommence(cours);
```

CC-NOM.6 — La longueur du nom suit la portée

Pour une variable, plus la portée est large, plus le nom doit être explicite. Un compteur de boucle utilisé sur trois lignes peut s'appeler i ; une variable déclarée en début de méthode et relue trente lignes plus bas ne le peut pas.

À ÉVITER
```
Tracker tr = new Tracker();

// ... trente lignes de code ...

for (int i = 0; i < 10; i++) {
    Message m = new Message();
    m.setSujet(tr.getNom());
    m.envoyer();
}
```

PRÉFÉREZ
```
Tracker tracker = new Tracker();

// ... trente lignes de code ...

for (int i = 0; i < 10; i++) {
    Message message = new Message();
    message.setSujet(tracker.getNom());
    message.envoyer();
}
```

Pour les méthodes et les classes, la règle s'inverse. Ce qui est public et largement utilisé porte un nom court ; ce qui est privé et sert à un seul endroit porte un nom long et descriptif.

PRÉFÉREZ
```
public void agir() {
    tournerDUnAngleAleatoire();
    chercherUnVerDeTerre();
}
```

CC-NOM.7 — Pas de nombre magique : nommez-le

Un nombre écrit en dur au milieu du code ne dit ni d'où il vient ni ce qu'il représente. Le lecteur doit le deviner, et celui qui devra le changer devra d'abord tous les retrouver.

À ÉVITER
```
if (joueur.getPoints() > 10) {
    joueur.promouvoir();
}

double total = montant * 1.21;
```

PRÉFÉREZ
```
private static final int    POINTS_POUR_PROMOTION = 10;
private static final double TAUX_TVA_STANDARD     = 1.21;

if (joueur.getPoints() > POINTS_POUR_PROMOTION) {
    joueur.promouvoir();
}

double total = montant * TAUX_TVA_STANDARD;
```

La constante fait deux choses à la fois : elle explique le nombre, et elle le rassemble en un seul endroit. Le jour où le taux de TVA change, il y a une ligne à modifier, pas une chasse à mener dans tout le projet.

Tous les nombres ne sont pas magiques. Le critère n'est pas la valeur, c'est ce qu'elle encode. Un nombre qui traduit une décision — un seuil, un taux, une limite — pourrait changer un jour, et il mérite un nom. Un nombre qui appartient à la mécanique du code ne changera jamais et se passe très bien du sien.

À ÉVITER
```
for (int i = 0; i < joueurs.size(); i++) {
    ...
}

if (joueurs.size() > 4) {
    ...
}
```
PRÉFÉREZ
```
// la boucle ne change pas : le 0 et le i++ sont la mecanique du parcours
for (int i = 0; i < joueurs.size(); i++) {
    ...
}

// seul le 4 disparait : lui seul encodait une decision
if (joueurs.size() > SEUIL_GRAND_GROUPE) {
    ...
}
```

Comparez les deux blocs : la boucle est identique. Ce n'est pas un oubli — c'est la démonstration que ses nombres n'avaient rien à se reprocher. Un seul des trois méritait un nom.

Le test à s'appliquer : si quelqu'un vous demandait pourquoi ce nombre-là plutôt qu'un autre, auriez-vous quelque chose à répondre ? Si oui, la réponse est le nom de la constante.

Le problème n'est d'ailleurs pas propre aux nombres. Une chaîne de caractères recopiée à plusieurs endroits pose exactement la même difficulté, et une faute de frappe y passe inaperçue — le compilateur ne vérifie pas le contenu d'un littéral.

À ÉVITER
```
if (utilisateur.getRole().equals("ADMIN")) { ... }

if (autre.getRole().equals("ADMLN")) { ... }      // qui verra la faute ?
```
PRÉFÉREZ
```
private static final String ROLE_ADMIN = "ADMIN";

if (utilisateur.getRole().equals(ROLE_ADMIN)) { ... }

if (autre.getRole().equals(ROLE_ADMLN)) { ... }   // ne compile pas
```

`CC-NOM.8` — Déclarez une variable près de l'endroit où elle sert

Grouper toutes les déclarations en tête de méthode est une habitude héritée de langages qui l'imposaient. En Java, elle oblige le lecteur à faire des allers-retours et élargit inutilement la portée de chaque variable.

À ÉVITER
```
public void distribuer(List<Joueur> joueurs) {
    Carte carte;
    int i;
    Joueur joueur;

    Collections.shuffle(paquet);

    for (i = 0; i < joueurs.size(); i++) {
        joueur = joueurs.get(i);
        carte = paquet.remove(0);
        joueur.recevoir(carte);
    }
}
```
PRÉFÉREZ
```
public void distribuer(List<Joueur> joueurs) {
    Collections.shuffle(paquet);

    for (Joueur joueur : joueurs) {
        Carte carte = paquet.remove(0);
        joueur.recevoir(carte);
```

```
    }
}
```

Chaque variable naît là où elle sert et disparaît avec le bloc. C'est CC-NOM.6 vu sous un autre angle : plutôt que d'allonger le nom pour compenser une portée large, réduisez la portée.

Écrire une méthode

Une méthode est l'unité de lecture d'un programme. Si elle tient dans un écran et porte un nom exact, on peut la comprendre sans lire ce qu'elle appelle — et c'est ce qui rend un programme entier abordable.

CC-MET.1 — Le nom de la méthode dit ce qu'elle fait

Le nom doit suffire à l'appelant. S'il faut ouvrir le corps de la méthode pour savoir ce qu'elle fait exactement, le nom est incomplet.

À ÉVITER
```
public LocalDate addToDate(LocalDate date, int month) {
    return date.plus(month, ChronoUnit.MONTHS);
}

LocalDate date = LocalDate.now();
date = addToDate(date, 3);      // trois quoi ? jours ? mois ?
```

PRÉFÉREZ
```
public LocalDate addMonthsToDate(LocalDate date, int months) {
    return date.plus(months, ChronoUnit.MONTHS);
}

LocalDate date = LocalDate.now();
date = addMonthsToDate(date, 3);
```

Notez au passage que le paramètre est passé au pluriel : `months` et non `month`, puisqu'il en désigne plusieurs. C'est CC-NOM.1 appliqué à un paramètre.

CC-MET.2 — Une méthode reste courte

Deux à cinq lignes est l'objectif. C'est court, et volontairement : le but n'est pas d'écrire moins de code, mais de le répartir dans des méthodes qui portent chacune un nom.

Le moyen d'y arriver n'est pas de compter les lignes, c'est de limiter l'indentation. Le corps d'une méthode devrait tenir en une conditionnelle, une boucle ou un `try/catch`, contenant un seul appel.

À ÉVITER
```
public void afficherElements(List<String> liste) {
    if (liste.isEmpty()) {
        System.out.println("La liste est vide.");
    } else {
        for (String element : liste) {
            if (!element.isEmpty()) {
                System.out.println("- " + element);
            }
        }
    }
}
```

PRÉFÉREZ
```
public void afficherElements(List<String> liste) {
    if (liste.isEmpty()) {
        System.out.println("La liste est vide.");
    } else {
        afficherElementsNonVides(liste);
    }
}

private void afficherElementsNonVides(List<String> liste) {
    for (String element : liste) {
        if (!element.isEmpty()) {
```

```
            System.out.println("- " + element);
        }
    }
}
```

Il y a maintenant deux méthodes au lieu d'une, et plus de lignes au total. Ce n'est pas un échec : chacune se lit d'un coup d'œil, et `afficherElementsNonVides` porte un nom qui explique ce que faisait la boucle imbriquée.

CC-MET.3 — Une méthode ne fait qu'une seule chose

Le test pratique : si vous décrivez ce que fait la méthode et que votre phrase contient un « et », ou descend dans le détail de l'implémentation, elle en fait plusieurs.

À ÉVITER
```
public static void emailClients(List<Client> clients) {
    for (Client client : clients) {
        ClientRecord record = database.lookup(client);
        if (record.isActive()) {
            sendEmail(client);
        }
    }
}
```

PRÉFÉREZ
```
public static void emailClients(List<Client> clients) {
    for (Client client : clients) {
        if (isActiveClient(client)) {
            sendEmail(client);
        }
    }
}

private static boolean isActiveClient(Client client) {
    ClientRecord record = database.lookup(client);
    return record.isActive();
}
```

La première version mélange deux niveaux : l'intention (envoyer un courriel aux clients actifs) et le moyen de savoir si un client est actif (une consultation en base). La seconde les sépare. Effet secondaire agréable, `isActiveClient` devient réutilisable ailleurs.

CC-MET.4 — Le vocabulaire est cohérent d'une méthode à l'autre

Des actions semblables portent le même verbe. Sinon le lecteur cherche une différence de sens là où il n'y en a pas.

À ÉVITER
```
public void loadSingleData();
public void fetchDataFiltered();
public void getAllData();
```

PRÉFÉREZ
```
public void getSingleData();
public void getFilteredData();
public void getAllData();
```

Quelle est la différence entre load, fetch et get ? Aucune, dans cet exemple — et c'est précisément le problème : le lecteur suppose qu'il y en a une. Choisissez un verbe par famille d'actions et tenez-vous-y, dans une classe comme dans une équipe.

CC-MET.5 — Pas de drapeau booléen en paramètre

Un booléen qui pilote le comportement d'une méthode annonce qu'elle fait deux choses. Il rend aussi les appels illisibles : rien ne dit ce que signifie `true`.

À ÉVITER
```
double d = distance(pointA, pointB, true);
```

PRÉFÉREZ

```
double d = distanceEnKilometres(pointA, pointB);
double d = distanceEnMiles(pointA, pointB);
```

Deux méthodes au lieu d'une, chacune avec un nom qui dit ce qu'elle retourne. C'est CC-MET.3 vu depuis la signature : un drapeau booléen est presque toujours le symptôme d'une méthode qui n'a pas été coupée en deux.

CC-MET.6 — Deux paramètres au plus

Chaque paramètre supplémentaire est un ordre à retenir et une occasion de se tromper. Au-delà de deux, c'est en général que plusieurs d'entre eux forment un ensemble qui mérite sa propre classe.

```
À ÉVITER
public void setCustomerAddress(String firstName, String lastName,
                               String streetAddress, String city,
                               String zipCode, String state,
                               String country, String phoneNumber)

PRÉFÉREZ
public void setCustomerAddress(Address address)
```

Le regroupement n'est pas qu'un artifice de comptage : une adresse est bel et bien une chose, et lui donner une classe permet de la valider, de la comparer et de la transporter d'un bloc.

> **Deux précisions**
>
> Un paramètre variadique en dernière position (`String...` valeurs) compte pour un seul argument.
>
> Un record est souvent le moyen le plus léger de regrouper des paramètres liés — vous le verrez au module `ENU`.

CC-MET.7 — Pas d'effet de bord

Un effet de bord, c'est ce que la méthode fait en plus de ce que son nom annonce. Personne ne s'y attend, donc personne ne s'en méfie.

Premier cas, l'effet caché sur les données. Imaginez ces deux méthodes :

```
À ÉVITER
public Product getProductById(List<Product> products, int id) { ... }

public int getProductIndexInList(List<Product> products, Product product) { ... }
```

Si l'auteur de `getProductById` décide de trier la liste pour accélérer sa recherche, tous les indices retournés auparavant par `getProductIndexInList` deviennent faux. Le nom de la méthode ne laissait rien deviner de ce tri. Une méthode dont le nom commence par `get` ne doit rien modifier.

Second cas, le mélange du traitement et de l'affichage. C'est de loin le plus fréquent dans vos exercices :

```
À ÉVITER
public class CompteBancaire {
    private double solde;

    public void retirerArgent(double montant) {
        if (solde >= montant) {
            solde -= montant;
            System.out.println("Retrait de " + montant + " effectue.");
        } else {
            System.out.println("Fonds insuffisants.");
        }
    }
}
```

La méthode retire de l'argent, ce qui est son travail, mais elle décide aussi de ce qui s'affiche à l'écran — une décision qui ne lui appartient pas. Elle devient inutilisable dans une application graphique, et intestable : il n'y a rien à vérifier qu'un texte parti sur la console.

```
PRÉFÉREZ
public class CompteBancaire {
    private double solde;
```

```
    public boolean retirerArgent(double montant) {
        if (solde < montant) {
            return false;
        }
        solde -= montant;
        return true;
    }
}

// la couche de presentation decide de ce qui s'affiche
if (compte.retirerArgent(100)) {
    System.out.println("Retrait de 100 effectue.");
} else {
    System.out.println("Fonds insuffisants.");
}
```

Une variante à ne pas retenir. On voit parfois la méthode rester `void` et la classe exposer un `dernierRetraitOK()` que l'appelant interroge après coup. Cela corrige bien l'affichage, mais introduit un défaut plus grave : la classe mémorise le résultat de la dernière opération, donc deux appels successifs se marchent dessus et l'ordre des instructions devient porteur de sens. Préférez la valeur de retour — ou l'exception, voir CC-ERR.1.

CC-MET.8 — Une méthode agit ou répond, jamais les deux

Une méthode modifie l'état de l'objet — c'est une commande, et elle ne retourne rien — ou bien elle répond à une question sans rien modifier — c'est une requête, et elle retourne une valeur. Mélanger les deux produit des appels dont on ne sait pas lire l'intention.

```
À ÉVITER
public boolean setAttribut(String cle, String valeur) { ... }

if (setAttribut("nom", "Lovelace")) {
    ...
}
```

Que teste ce `if` ? Que l'attribut existait déjà ? Que l'écriture a réussi ? Impossible de le savoir sans ouvrir la méthode. Séparez :

```
PRÉFÉREZ
public void    setAttribut(String cle, String valeur) { ... }   // commande
public boolean contientAttribut(String cle)           { ... }   // requete

if (contientAttribut("nom")) {
    setAttribut("nom", "Lovelace");
}
```

Cette règle éclaire la nuance de CC-ERR.1 : `estDecouvert()` est une requête, elle retourne légitimement un booléen ; `retirer()` est une commande, donc son échec se signale par une exception et non par une valeur de retour.

Un repère de nommage utile : une commande porte un verbe d'action (`ajouter`, `retirer`, `demarrer`), une requête porte un nom ou un prédicat (`solde`, `estVide`). Voir CC-NOM.5.

CC-MET.9 — Un booléen se retourne tel quel

Une comparaison comme `a > b` vaut déjà `true` ou `false` : c'est un booléen. Le reconstruire avec un `if` n'ajoute que des lignes.

```
À ÉVITER
public boolean estMajeur() {
    if (age >= 18) {
        return true;
    } else {
        return false;
    }
}
```

Le ternaire ne fait pas mieux : `return age >= 18 ? true : false;` est la même redondance, en une ligne. La comparaison se retourne directement :

PRÉFÉREZ
```
public boolean estMajeur() {
    return age >= 18;
}
```

La même règle vaut pour les comparaisons avec `true` : `if (estMajeur() == true)` s'écrit `if (estMajeur())`, et `if (estMajeur() == false)` s'écrit `if (!estMajeur())`. Un prédicat bien nommé (CC-NOM.5) se lit alors comme une phrase.

Commenter

Un commentaire n'est ni compilé, ni testé, ni mis à jour par personne. Il vieillit donc plus vite que le code qu'il décrit, et un commentaire faux fait plus de dégâts qu'un commentaire absent.

CC-COM.1 — Un commentaire nécessaire signale un code à réécrire

Avant d'écrire un commentaire, demandez-vous s'il n'existe pas un moyen de le dire dans le code. La plupart du temps, il suffit d'extraire une méthode et de lui donner le nom que vous alliez écrire en commentaire.

À ÉVITER
```
// verifie si l'employe a droit a tous les avantages
if (employe.getAnciennete() > 5 && employe.getStatut() == Statut.TEMPS_PLEIN) {
    ...
}
```

PRÉFÉREZ
```
if (employe.aDroitATousLesAvantages()) {
    ...
}
```

Le commentaire a disparu et l'information est toujours là — mieux, elle est désormais réutilisable et testable. C'est le même mouvement qu'en CC-NOM.1, appliqué cette fois à une expression.

Restent les commentaires qui ont leur place : expliquer un pourquoi qu'aucun nom ne peut porter — une contrainte légale, un contournement de bug, une décision de conception surprenante. Ceux-là, écrivez-les.

PRÉFÉREZ
```
// L'arrondi se fait a 2 decimales : impose par la reglementation
// belge sur la facturation (AR du 21/12/2012).
montant = montant.setScale(2, RoundingMode.HALF_UP);
```

CC-COM.2 — Pas de code laissé en commentaire

Du code commenté pose une question sans réponse à tous ceux qui le liront : est-ce important ? Est-ce que quelqu'un compte le remettre ? Personne n'osera le supprimer, et il restera là des années.

À ÉVITER
```
public double calculerTarif() {
    // ancienne formule, a garder au cas ou
    // return prixBase * 1.21;
    return prixBase * tauxTva;
}
```

PRÉFÉREZ
```
public double calculerTarif() {
    return prixBase * tauxTva;
}
```

Supprimez. Si vous travaillez avec Git, la version précédente est conservée et retrouvable ; si vous n'utilisez pas encore Git, c'est une raison de plus de vous y mettre.

CC-COM.3 — Une Javadoc ne répète pas la signature

Une Javadoc qui paraphrase les noms des paramètres n'apporte rien et donne l'illusion que la méthode est documentée.

À ÉVITER
```
/**
 * Retire un montant.
 *
 * @param montant le montant
 * @return un boolean
 */
public boolean retirerArgent(double montant) { ... }
```

PRÉFÉREZ
```
/**
 * Retire un montant du compte, si le solde le permet.
 *
 * @param montant le montant a retirer, strictement positif
 * @return true si le retrait a eu lieu, false si le solde etait insuffisant
 */
public boolean retirerArgent(double montant) { ... }
```

Une Javadoc utile répond à ce que la signature ne dit pas : quelles valeurs sont acceptables, ce qui se passe dans les cas limites, ce qui est levé et quand. Le reste est déjà écrit juste en dessous.

Objets et encapsulation

Encapsuler ne consiste pas à rendre les champs privés puis à ouvrir un accesseur pour chacun : cela revient au même en plus verbeux. Encapsuler, c'est décider de ce que l'objet accepte de faire, pas de ce qu'il accepte de montrer.

`CC-OBJ.1` — Demandez à l'objet d'agir, ne lui demandez pas ses entrailles

Une chaîne d'appels qui traverse plusieurs objets révèle leur structure interne à qui ne devrait pas la connaître. Le jour où l'un d'eux change, tout le code qui traversait la chaîne casse.

À ÉVITER
```
partie.getVainqueur().getCompte().crediter(10);
```

PRÉFÉREZ
```
partie.crediterVainqueur(10);
```

L'appelant sait maintenant une seule chose : une partie peut créditer son vainqueur. Il ignore qu'un `Joueur` possède un `Compte`, et le jour où ce n'est plus le cas, seule la classe `Partie` est à modifier.

Cette règle porte le nom de loi de Déméter, résumée par la formule « ne parlez qu'à vos amis proches ». En pratique, méfiez-vous de tout enchaînement de plus de deux points sur des objets de votre domaine.

Attention à ne pas sur-appliquer. La règle vise les objets de votre domaine, pas les chaînes de construction ou les bibliothèques conçues ainsi. `LocalDate.now().getYear()` est parfaitement correct — vous l'avez d'ailleurs écrit en `POO.3`.

`CC-OBJ.2` — Un accesseur n'est pas une obligation

Générer mécaniquement un getter et un setter pour chaque champ annule l'encapsulation qu'on venait de mettre en place. Chaque accesseur est une décision : ce champ doit-il vraiment être lisible, et surtout modifiable, depuis l'extérieur ?

À ÉVITER
```
public class Partie {
    private List<Joueur> joueurs;

    public List<Joueur> getJoueurs()              { return joueurs; }
    public void setJoueurs(List<Joueur> joueurs)  { this.joueurs = joueurs; }
}

// n'importe qui peut vider la partie de ses joueurs :
partie.getJoueurs().clear();
```

PRÉFÉREZ

```
public class Partie {
    private final List<Joueur> joueurs = new ArrayList<>();

    public void ajouterJoueur(Joueur joueur) {
        joueurs.add(joueur);
    }

    public int nombreDeJoueurs() {
        return joueurs.size();
    }
}
```

La seconde version n'expose aucune liste. Elle expose deux opérations, qui sont ce dont l'appelant a réellement besoin. Et le jour où une partie doit refuser un cinquième joueur, il y a un endroit où écrire cette règle.

Vous avez rencontré cette question dès `POO.2`, à propos du champ `age` : un getter sans setter, un setter, ou rien. La réponse était « rien », parce que la donnée elle-même n'avait pas lieu d'être.

Traiter les erreurs

Ces six règles s'appliquent au module `EXC`, mais elles décident aussi de la lisibilité du code qui les appelle. Mal traitée, une erreur oblige chaque appelant à vérifier, à commenter, et finalement à oublier de vérifier.

`CC-ERR.1` — Une erreur se signale par une exception, pas par une valeur de retour

Une valeur de retour qui code un échec oblige l'appelant à s'en souvenir. Rien dans le langage ne l'y contraint, et le jour où il oublie, le programme continue avec des données fausses — sans rien signaler.

À ÉVITER

```
public boolean retirer(double montant) {
    if (solde < montant) {
        return false;
    }
    solde -= montant;
    return true;
}

// chez l'appelant, rien n'oblige a ecrire ceci :
compte.retirer(500);   // et si le retrait a echoue ?
```

PRÉFÉREZ

```
public void retirer(double montant) {
    if (solde < montant) {
        throw new SoldeInsuffisantException(
                "Retrait de " + montant + " refuse : solde de " + solde);
    }
    solde -= montant;
}

// chez l'appelant, l'echec ne peut plus passer inapercu :
compte.retirer(500);
```

Nuance. Un booléen de retour n'est pas toujours mauvais : il est bien meilleur qu'un `System.out.println` dans la classe (voir `CC-MET.7`). La question à se poser est : s'agit-il d'une *réponse* ou d'une *erreur* ? `estDecouvert()` retourne légitimement un booléen, parce que les deux réponses sont normales. `retirer()` ne le devrait pas, parce que l'échec est anormal.

`CC-ERR.2` — Ne retournez pas null

Retourner `null` revient à demander à chaque appelant d'écrire une vérification, pour toujours. Un seul oubli et le programme s'arrête sur une `NullPointerException`, à un endroit qui n'a rien à voir avec la cause.

À ÉVITER

```
public Joueur chercherJoueur(String pseudo) {
```

```
    for (Joueur joueur : joueurs) {
        if (joueur.getPseudo().equals(pseudo)) {
            return joueur;
        }
    }
    return null;
}
```

PRÉFÉREZ

```
public Joueur chercherJoueur(String pseudo) {
    for (Joueur joueur : joueurs) {
        if (joueur.getPseudo().equals(pseudo)) {
            return joueur;
        }
    }
    throw new JoueurInconnuException("Aucun joueur nomme " + pseudo);
}
```

La règle vaut aussi pour les collections, et là c'est encore plus simple : retournez une collection vide, jamais `null`. L'appelant peut alors faire son `for` sans rien vérifier, et la boucle ne tourne simplement pas.

À ÉVITER

```
if (partie.getJoueurs() != null) {
    for (Joueur joueur : partie.getJoueurs()) { ... }
}
```

PRÉFÉREZ

```
for (Joueur joueur : partie.getJoueurs()) { ... }
```

CC-ERR.3 — Ne passez pas null en paramètre

Le symétrique de la règle précédente. Une méthode qui accepte `null` pour l'un de ses paramètres doit le tester, et son comportement devient difficile à décrire. Quand un paramètre est facultatif, écrivez plutôt deux méthodes.

À ÉVITER

```
public void demarrer(Joueur premier, Joueur second, Strategie strategie) {
    if (strategie == null) {
        strategie = new StrategieTimide();
    }
    ...
}

partie.demarrer(alice, bob, null);
```

PRÉFÉREZ

```
public void demarrer(Joueur premier, Joueur second) {
    demarrer(premier, second, new StrategieTimide());
}

public void demarrer(Joueur premier, Joueur second, Strategie strategie) {
    ...
}

partie.demarrer(alice, bob);
```

La surcharge dit explicitement ce que veut l'appelant, et la méthode principale n'a plus de cas particulier à traiter.

CC-ERR.4 — Le message d'exception dit ce qui a échoué et avec quelle valeur

Le message d'une exception est souvent la seule information dont vous disposerez pour comprendre un incident. Un message qui ne nomme ni l'opération ni la valeur fautive ne sert à rien.

À ÉVITER

```
throw new IllegalArgumentException("Erreur");
throw new IllegalArgumentException("Valeur invalide");
throw new IllegalArgumentException();
```

PRÉFÉREZ

```
throw new IllegalArgumentException(
        "Nombre de cartes invalide : " + nombre + " (attendu entre 1 et 52)");
```

Trois ingrédients suffisent : ce qui était attendu, ce qui a été reçu, et le nom de ce qu'on manipulait. Écrivez le message en pensant à quelqu'un qui lira la trace sans avoir le code sous les yeux.

CC-ERR.5 — Définissez vos exceptions selon ce que l'appelant doit distinguer

Quand vous créez une classe d'exception, la question n'est pas « quelle erreur technique s'est produite ». Elle est : celui qui l'attrape va-t-il réagir différemment ? Si la réponse est non, une seule classe suffit.

À ÉVITER
```
try {
    compte.retirer(montant);
} catch (SoldeInsuffisantException e) {
    afficherErreur(e.getMessage());
} catch (CompteBloqueException e) {
    afficherErreur(e.getMessage());
} catch (MontantNegatifException e) {
    afficherErreur(e.getMessage());
}
```

Trois classes d'exception, trois blocs identiques : la distinction n'a servi à rien, sinon à alourdir chaque appelant. Une seule classe, avec un message précis, aurait suffi.

PRÉFÉREZ
```
try {
    compte.retirer(montant);
} catch (RetraitImpossibleException e) {
    afficherErreur(e.getMessage());
}
```

L'inverse est vrai aussi : si l'appelant doit vraiment réagir autrement — proposer un découvert dans un cas, appeler le service client dans l'autre — alors deux classes distinctes se justifient pleinement. C'est le besoin de l'appelant qui décide, jamais la cause technique.

> **Ce que le message ne doit pas faire**
>
> Ne construisez pas le message avec des informations sensibles — mot de passe, numéro de compte complet. La trace d'exception finit souvent dans un fichier de log que beaucoup de gens peuvent lire.
>
> N'écrivez pas non plus le message en pensant à l'utilisateur final : ce n'est pas lui qui le lira. Le message d'exception est destiné au développeur ; le message affiché à l'écran est une autre affaire, décidée par l'appelant.

CC-ERR.6 — N'avalez pas une exception en silence

Un `catch` vide fait taire l'erreur sans la résoudre. Le programme continue comme si tout allait bien, dans un état faux, et l'incident ressurgit plus loin, à un endroit qui n'a plus rien à voir avec sa cause. Le plantage était bruyant mais honnête ; ce silence ment.

À ÉVITER
```
try {
    journal.enregistrer(partie);
} catch (IOException e) {
    // ne devrait pas arriver
}
```

Trois réactions sont honnêtes. Traiter : redemander, proposer autre chose, prendre une valeur de repli que l'on assume. Informer : dire ce qui a échoué. Ou ne pas attraper du tout, et laisser l'exception remonter jusqu'à quelqu'un qui saura quoi en faire.

PRÉFÉREZ
```
public void terminer(Partie partie) throws IOException {
    journal.enregistrer(partie);    // c'est l'appelant qui decidera
}
```

Si vous ne savez pas quoi faire d'une exception à cet endroit, ce n'est pas l'endroit pour l'attraper : déclarez-la.

> **Quand l'objet exception ne sert plus**
>
> Attraper une exception pour redemander une saisie est un vrai traitement, même quand l'objet exception n'est plus utilisé. Nommez alors la variable ignoree : le nom dit que c'est voulu, et non oublié.

`CC-ERR.7` — Une comparaison se protège de null

Appeler une méthode sur une variable qui vaut `null` lève une `NullPointerException` : il n'y a pas d'objet à qui demander. La comparaison est l'endroit où l'on se fait le plus souvent prendre.

ÀÉVITER
```
if (titre.equals("Nana")) {   // plante si titre vaut null
    ...
}
```

PRÉFÉREZ
```
if (titre != null && titre.equals("Nana")) { ... }

if ("Nana".equals(titre)) { ... }          // la constante d'abord

if (Objects.equals(titre, "Nana")) { ... } // java.util.Objects
```

Dans la première écriture, l'ordre est essentiel : l'opérateur `&&` s'arrête dès que la première condition est fausse, et la seconde n'est jamais évaluée. Inverser les deux tests remet le plantage. La deuxième met la constante devant : elle n'est jamais `null`, et l'appel retourne simplement `false`. La troisième gère le `null` des deux côtés.

Nuance. Ces tests sont un filet de sécurité. La vraie protection est en amont : ne pas produire de `null`, ni en retour (`CC-ERR.2`), ni en paramètre (`CC-ERR.3`). Un code qui teste la nullité partout signale souvent un `null` qui n'aurait pas dû exister.

Écrire des tests

Une classe de test est du code que vous relirez et modifierez pendant toute la vie du projet. Elle mérite donc le même soin que le reste — et elle obéit à quelques règles qui lui sont propres.

`CC-TST.1` — Le code de test se soigne comme le code de production

Un test illisible ne sera pas maintenu ; un test non maintenu finit désactivé ; une suite désactivée ne protège plus rien. Les règles de nommage et de longueur s'appliquent donc intégralement aux tests.

ÀÉVITER
```
@Test
void test1() {
    Compte c = new Compte();
    c.deposer(100);
    c.retirerArgent(30);
    assertEquals(70, c.getSolde());
}
```

PRÉFÉREZ
```
@Test
@DisplayName("Un retrait diminue le solde du montant retire")
void unRetraitDiminueLeSolde() {
    Compte compte = new Compte();
    compte.deposer(100);

    compte.retirerArgent(30);

    assertEquals(70, compte.getSolde());
}
```

Le nom de la méthode de test énonce le comportement attendu. Quand le test passe au rouge, ce nom est la première chose que vous lirez — et souvent la seule dont vous aurez besoin pour comprendre ce qui est cassé.

Notez aussi la disposition en trois temps, séparés par une ligne vide : préparer, agir, vérifier. Elle se retrouve dans presque tous les tests et se repère d'un coup d'œil.

`CC-TST.2` — Un test vérifie une seule chose

Quand un test vérifie trois comportements, son échec ne vous dit pas lequel est cassé, et les vérifications situées après la première qui échoue ne sont même pas exécutées.

À ÉVITER

```
@Test
void testCompte() {
    Compte compte = new Compte();
    compte.deposer(100);
    assertEquals(100, compte.getSolde());

    assertTrue(compte.retirerArgent(30));
    assertEquals(70, compte.getSolde());

    assertFalse(compte.retirerArgent(1000));
    assertEquals(70, compte.getSolde());
}
```

PRÉFÉREZ

```
@Test
void unDepotAugmenteLeSolde() { ... }

@Test
void unRetraitPossibleDiminueLeSolde() { ... }

@Test
void unRetraitTropGrandEstRefuseEtLaisseLeSoldeIntact() { ... }
```

Plusieurs assertions dans un même test restent acceptables tant qu'elles décrivent un seul comportement — vérifier le résultat retourné et l'état de l'objet après la même action, par exemple. La règle porte sur le nombre de comportements, pas sur le nombre de lignes `assertXxx`.

> **Un test sans assertion ne teste rien**
>
> Une méthode annotée `@Test` qui se contente d'appeler du code passe au vert quoi qu'il arrive : le seul cas où elle échoue est celui d'une exception inattendue. Elle donne donc une impression de couverture sans vérifier aucun comportement.
>
> C'est l'erreur la plus fréquente dans les classes de test rendues à l'examen. Avant de considérer un test comme écrit, cherchez son assertion.

`CC-TST.3` — Un test est rapide, indépendant, répétable et auto-vérifiant

Ces qualités sont résumées par l'acronyme F.I.R.S.T., cinq mots anglais.

Qualité	
`Fast`	Rapide. Une suite lente n'est plus lancée, donc ne sert plus à rien.
`Independent`	Indépendant. Aucun test ne suppose qu'un autre a tourné avant lui, ni dans quel ordre.
`Repeatable`	Répétable. Même résultat sur votre machine, celle du voisin, et dans six mois.
`Self-validating`	Auto-vérifiant. Le test dit vert ou rouge, sans que personne n'ait à lire une sortie console.
`Timely`	Écrit tôt. Idéalement avant le code, comme dans l'exercice `JUN.1`.

La répétabilité est celle qui vous piégera le plus tôt. Un test qui s'appuie sur la date du jour, sur un tirage aléatoire ou sur un fichier présent seulement chez vous passera au rouge un matin, sans que rien n'ait changé dans le code.

À ÉVITER

```
@Test
void ageDeLaPersonne() {
    Personne personne = new Personne();
```

```
    personne.setAnneeDeNaissance(1990);

    assertEquals(36, personne.getAge());     // faux des le 1er janvier
}
```

PRÉFÉREZ

```
@Test
void ageDeLaPersonne() {
    Personne personne = new Personne();
    personne.setAnneeDeNaissance(1990);

    assertEquals(35, personne.computeAge(2025));
}
```

C'est exactement la raison pour laquelle `computeAge(int annee)` existe à côté de `getAge()` en `POO.3` : le noyau déterministe est testable, la commodité qui dépend du jour ne l'est qu'indirectement.

`CC-TST.4` — Un test se termine par une assertion

Sans assertion, un test appelle du code et ne vérifie rien : il passe au vert quoi qu'il arrive, sauf si une exception l'interrompt. Il donne l'illusion d'une vérification, ce qui est pire que pas de test du tout. C'est aussi le défaut le plus fréquent dans les copies d'examen.

À ÉVITER

```
@Test
void reserverDeuxPlaces() throws SalleCompleteException {
    Salle salle = new Salle(100);

    salle.reserver(2);

    System.out.println(salle.placesLibres());   // affiche, ne verifie rien
}
```

PRÉFÉREZ

```
@Test
void reserverDeuxPlaces() throws SalleCompleteException {
    Salle salle = new Salle(100);

    salle.reserver(2);

    assertEquals(98, salle.placesLibres(),
            "Deux places reservees sur 100 doivent en laisser 98");
}
```

Un test qui vérifie qu'une exception est levée a lui aussi son assertion : `assertThrows`. Pour vous assurer qu'un test peut échouer, faites-le échouer une fois : changez l'attendu, et regardez-le passer au rouge. Un test qu'on n'a jamais vu rouge ne prouve rien.

Structurer une classe

Les deux dernières règles sont les mêmes que celles sur les méthodes, appliquées un cran au-dessus. Ce n'est pas un hasard : une bonne conception répète le même principe à chaque échelle.

`CC-CLS.1` — Une classe n'a qu'une raison de changer

Décrivez la classe en une phrase. Si vous devez employer « et », elle porte plusieurs responsabilités — et chacune la fera évoluer pour des motifs différents, à des moments différents, souvent par des personnes différentes.

À ÉVITER

```
public class Facture {
    public double calculerTotal()   { ... }   // regles metier
    public void enregistrerEnBase() { ... }   // persistance
    public String enHtml()          { ... }   // presentation
}
```

PRÉFÉREZ

```
public class Facture {
    public double calculerTotal() { ... }
}

public class DepotFactures {
    public void enregistrer(Facture facture) { ... }
}

public class FactureEnHtml {
    public String convertir(Facture facture) { ... }
}
```

Trois raisons de changer, trois classes : un taux de TVA qui bouge, une base de données qu'on remplace, une maquette qu'on refait. Chacune touche un seul fichier.

Vous reconnaîtrez ici la même faute qu'en CC-MET.7 avec le `CompteBancaire` qui affichait ses messages : mélanger le traitement et la présentation. C'est le cas de figure le plus courant, et le plus coûteux.

CC-CLS.2 — Une classe reste petite

La taille d'une classe ne se mesure pas en lignes mais en responsabilités. Une classe de quarante méthodes est presque toujours plusieurs classes qui s'ignorent.

Un indice fiable : regardez quels champs chaque méthode utilise. Si un groupe de méthodes ne touche qu'à un sous-ensemble des champs, et un autre groupe au reste, vous tenez deux classes.

À ÉVITER

```
public class Joueur {
    private String pseudo;
    private String email;
    private String rue;
    private String codePostal;
    private String ville;
    private String pays;

    public String adresseFormatee()   { ... }   // n'utilise que les 4 derniers
    public boolean adresseValide()    { ... }   // idem
    public boolean emailValide()      { ... }
}
```

PRÉFÉREZ

```
public class Joueur {
    private String pseudo;
    private String email;
    private Adresse adresse;

    public boolean emailValide() { ... }
}

public record Adresse(String rue, String codePostal, String ville, String pays) {
    public String formatee() { ... }
    public boolean valide()  { ... }
}
```

C'est le même regroupement qu'en CC-MET.6 : des données qui voyagent ensemble et que les mêmes méthodes manipulent forment une chose, et cette chose mérite un nom.

CC-CLS.3 — Ne vous répétez pas

Toute duplication est une occasion de divergence. Le jour où la règle change, on modifie une copie et on oublie les autres — et le programme se met à donner deux réponses différentes à la même question.

La duplication la plus fréquente, et la plus coûteuse, se cache entre les sous-classes d'une même hiérarchie :

À ÉVITER

```
public class JeuDeCartes extends Jeu {
    public double calculerTarif() {
        double tarif = prixBase * duree;
```

```
        if (nombreDeJoueurs > 4) {
            tarif *= 1.2;
        }
        return tarif;
    }
}

public class JeuDePlateau extends Jeu {
    public double calculerTarif() {
        double tarif = prixBase * duree;
        if (nombreDeJoueurs > 4) {
            tarif *= 1.2;
        }
        return tarif * 0.9;
    }
}
```

Les deux méthodes sont identiques à un coefficient près. Remontez ce qui est commun dans la classe abstraite et ne laissez descendre que ce qui varie :

PRÉFÉREZ
```
public abstract class Jeu {

    private static final int    SEUIL_GRAND_GROUPE     = 4;
    private static final double MAJORATION_GRAND_GROUPE = 1.2;

    public double calculerTarif() {
        double tarif = prixBase * duree;
        if (nombreDeJoueurs > SEUIL_GRAND_GROUPE) {
            tarif *= MAJORATION_GRAND_GROUPE;
        }
        return tarif * coefficient();
    }

    protected abstract double coefficient();
}

public class JeuDePlateau extends Jeu {
    protected double coefficient() {
        return 0.9;
    }
}
```

Le calcul du tarif n'existe plus qu'à un seul endroit, et chaque sous-classe se réduit à ce qui la distingue vraiment. Notez que les constantes de CC-NOM.7 sont venues d'elles-mêmes : dès qu'on rassemble, les nombres magiques deviennent visibles.

C'est une erreur très fréquente à l'examen. Devant une hiérarchie, le réflexe est de recopier la méthode dans chaque sous-classe et d'y changer un détail. Le bon réflexe est de chercher ce qui est commun d'abord.

CC-CLS.4 — Préférez le polymorphisme à une chaîne de tests sur un type

Quand une méthode demande à un objet de quel type il est pour décider quoi faire, c'est que le travail devrait être confié à l'objet lui-même.

À ÉVITER
```
public double calculerTarif(Jeu jeu) {
    if (jeu instanceof JeuDeCartes) {
        return jeu.getPrixBase() * 1.0;
    } else if (jeu instanceof JeuDePlateau) {
        return jeu.getPrixBase() * 0.9;
    } else if (jeu instanceof JeuVideo) {
        return jeu.getPrixBase() * 1.5;
    }
    throw new IllegalArgumentException("Type de jeu inconnu");
}
```

PRÉFÉREZ
```
// chaque sous-classe repond pour elle-meme
```

```
double tarif = jeu.calculerTarif();
```

La différence décisive apparaît le jour où vous ajoutez un type de jeu. Avec la chaîne de tests, il faut retrouver tous les endroits qui testent le type — et rien ne vous signale ceux que vous avez oubliés : le programme compile et se contente de lever son exception à l'exécution.

Avec le polymorphisme, la méthode `abstract` de la classe mère oblige le compilateur à réclamer l'implémentation manquante. L'oubli devient impossible.

Une exception à la règle. Un `switch` sur une énumération d'états ou de commandes est parfaitement légitime — ce n'est pas un test de type. La règle vise les cascades de `instanceof` et les `switch` sur un champ qui code le type d'un objet.

La règle du boy-scout

Trente-quatre règles, cela peut décourager. Elles ne se prennent pas d'un bloc, et personne n'écrit du code parfait du premier coup — pas même l'auteur du livre dont elles viennent.

Les scouts américains ont une consigne de camp : laissez l'emplacement plus propre que vous ne l'avez trouvé. Appliquée au code, elle donne une règle praticable : chaque fois que vous ouvrez un fichier, améliorez une chose. Un nom plus clair, un commentaire remplacé par une méthode nommée, une constante à la place d'un nombre magique.

Une seule chose, pas dix. C'est suffisant pour qu'un programme s'améliore avec le temps au lieu de se dégrader — ce qui est, en pratique, le seul objectif atteignable.

Pour aller plus loin

Clean Code: A Handbook of Agile Software Craftsmanship, Robert C. Martin, Prentice Hall, 2008. Les chapitres 2 (noms), 3 (fonctions), 4 (commentaires), 6 (objets), 7 (erreurs), 9 (tests) et 10 (classes) correspondent aux sections de ce mémento.

Le site cleancoders.com propose les vidéos de l'auteur. La conférence « Clean Code » d'Uncle Bob, disponible en ligne, en donne un aperçu d'une heure.

En français, Julien Lucas présente ces principes dans une vidéo : https://www.youtube.com/watch?v=VioMQpZMtnA.

Index

@

@BeforeEach 114
@DisplayName 113
@ParameterizedTest 114
@Test 109, 112

=

== (comparaison) 28, 45, 47, 51, 84, 110

A

À quoi sert Maven (TEC-MVN.1) 193
abstract 43, 54, 56
abstraction 14, 62
accesseur 12, 16, 20, 25, 27, 41, 49, 54, 63, 69, 75, 77, 78, 82, 90, 119, 141, 158
Accesseur n'est pas une obligation (CC-OBJ.2) 209
ActionEvent 136, 138
addEventHandler 137, 138, 140, 147, 148
Aller à la déclaration, et revenir (IDE.23) 175
Application 133, 134, 147, 148
ArrayDeque 95
ArrayIndexOutOfBoundsException 37, 104
ArrayList 84, 86, 95, 156
Arrays.copyOf 36
Arrays.toString 36
assertEquals 109, 116
assertion 107, 108, 109, 112, 113, 116
assertThrows 110, 111, 116, 129
autoboxing 24, 84, 121
Avertissements et leurs corrections (IDE.15) 169

B

binding 142, 147, 149
Booléen se retourne tel quel (CC-MET.9) 207
boucle
  for-each 37, 86
  imbriquée 37
  indexée 35, 37, 41
break 46
BufferedReader 154, 156
bytecode 14

C

Cannot resolve symbol (ERR.2) 180
catch 17, 98, 100, 101, 102, 105, 112, 153
Ce qu'est un gestionnaire de versions (TEC-GIT.1) 196
Ce que contient vraiment une variable (TEC-EXE.3) 192
champ
  d'objet 18
  de classe 18, 138
  final 20
Character.isWhitespace 24, 31
Chercher n'importe quoi (IDE.24) 175
ChronoUnit 26, 28, 29
classe
  anonyme 43, 126, 127, 128, 131
  exécutable 13
classe abstraite 7, 52, 55, 56, 57, 59, 60, 62, 63, 64, 65, 66, 68, 70, 73, 127, 128, 151
Classe n'a qu'une raison de changer (CC-CLS.1) 215
Classe reste petite (CC-CLS.2) 216
Classpath (TEC-CMP.3) 191
Code de test se soigne comme le code de production (CC-TST.1) 213
collection 7, 36, 56, 74, 83, 84, 85, 87, 90, 91, 93, 95, 96, 117, 119, 130
Collections 87, 92, 95, 96
Commentaire nécessaire signale un code à réécrire (CC-COM.1) 208
Comparable 27, 66, 92, 96, 97
Comparaison se protège de null (CC-ERR.7) 213
Comparator 74, 93, 96, 97, 125, 129
compareTo 44, 51, 92
compilation 14, 51, 56, 66, 78, 84, 118
Complétion et import automatique (IDE.11) 167
compteur d'instances 19
computeIfAbsent 90, 94, 96, 130
ConcurrentModificationException 91, 96
constante 16, 20, 28, 38, 42, 43, 44, 45, 47, 49, 51, 73, 87, 89, 91, 104, 127
constructeur 12, 15, 19, 25, 27, 45, 47, 49, 52, 53, 56, 67, 93, 101, 105, 125, 128, 134, 152, 153, 154
  compact 50, 105
  par défaut 15
  privé 21
consume 140, 147, 149
Consumer 123, 129, 131
conteneur 132, 134, 137, 139, 143, 146, 147, 148
contrôle (JavaFX) 16, 108, 114, 121, 132, 133, 135, 141, 143, 146, 147, 153
Cycle de vie (TEC-MVN.4) 195

D

DateTimeFormatter 27
DateTimeParseException 34
DayOfWeek 28
Débogueur (IDE.19) 172
débordement (int) 29, 32, 68, 92
Déclarez une variable près de l'endroit où elle sert (CC-NOM.8) 203
découplage 65, 68, 74
default 46, 51, 73, 128
Définissez vos exceptions selon ce que l'appelant doit distinguer (CC-ERR.5) 212
Demandez à l'objet d'agir, ne lui demandez pas ses entrailles (CC-OBJ.1) 209
Déplacer une classe (IDE.21) 174
Dépôt du fil rouge, et comment le parcourir (TEC-GIT.3) 197
Dépôt local (TEC-MVN.5) 195

Deque 83, 94, 95
Deux classes du même nom (ERR.5) 181
Deux paramètres au plus (CC-MET.6) 206
Dossier target (TEC-CMP.4) 191
doublure 115
Duration 26, 27, 33, 34

E

effacement de type 117, 120
encapsulation 12, 14, 16
enum 42, 43, 47
énumération 7, 13, 28, 42, 43, 44, 45, 47, 48, 49, 51, 89, 92, 138
  ordre de déclaration 44, 51, 95
EOFException 153, 156, 157
equals 28, 45, 49, 51, 84, 88, 89, 92, 96, 110, 118
Erreur se signale par une exception, pas par une valeur de retour (CC-ERR.1) 210
espace insécable 31
événement 8, 72, 116, 131, 132, 136, 137, 139, 146, 148
exception 7, 17, 22, 32, 34, 44, 45, 50, 84, 89, 95, 98, 99, 100, 102, 104, 105, 107, 110, 111, 113, 129, 142, 153, 154, 155
  contrôlée 98, 101, 105, 153
  non contrôlée 17, 98, 99, 101, 104
Exercice inachevé bloque tous les autres (ERR.9) 183
extends 53, 64, 67, 117, 122
Extraire une archive au bon endroit (TEC-ZIP.1) 195
Extraire une méthode (IDE.22) 175

F

F.I.R.S.T. 108
fabrique 15, 21, 27, 30, 32, 50, 99, 105, 117, 124, 131, 142
familles de cas 107, 112, 116
Fenêtre Maven (IDE.9) 166
Fenêtre Project, créer une classe (IDE.8) 165
Fichier disparaît de la fenêtre Project (ERR.10) 183
File 152, 154, 157
FileInputStream 151, 155, 157
FileNotFoundException 153, 155, 157
FileOutputStream 151, 155, 157
FileReader 151, 154
FileWriter 151, 153, 156
filtre d'événement 132, 140, 147, 149
final 16, 17, 20, 25, 40, 47, 50, 73
finally 98, 103, 105, 153
Fisher-Yates (mélange) 49
flux 130, 150, 151, 152, 154, 155, 157, 158
  d'octets 151
  de caractères 151
format de fichier 27, 33, 37, 41, 119, 154, 155, 158
Function 91, 94, 95, 125, 129, 131
FXML 132, 143, 146, 147

G

GameHub (fil rouge) ... 9, 11, 16, 22, 51, 63, 74, 83, 95, 103, 105, 125, 131, 140, 142, 156
Générer accesseurs et constructeurs (IDE.12) 168
gestionnaire d'événement ...74, 131, 136, 137, 139, 140, 142, 143, 147, 148
getOrDefault 90, 96
graphe de scène 134, 139

H

hashCode 49, 51, 88, 89, 96
HashMap 84, 88, 90, 95
HashSet 84, 88, 95
héritage 7, 14, 40, 52, 53, 59, 67, 77, 82, 102, 117, 124, 151

I

IllegalArgumentException 17, 105, 111
IllegalStateException 105
immuabilité 16, 20, 30, 34, 43, 50, 119
implements 63, 66, 67, 74, 158
import 56, 75, 76, 80, 104, 109, 147
Installer IntelliJ (IDE.2) 161
Installer Scene Builder (IDE.29) 179
Installer un JDK depuis IntelliJ (IDE.5) 162
instanceof 54, 60, 62, 71
Integer.parseInt 21, 25, 104, 154, 157
interface 7, 13, 53, 62, 65, 66, 67, 68, 70, 72, 73, 74, 82, 83, 84, 86, 89, 90, 92, 94, 115, 118, 125, 126, 127, 128, 129, 132, 135, 136, 141, 153, 155, 156
  de marquage 73, 156
  fonctionnelle 74, 91, 93, 94, 128, 129, 131, 141
IOException 145, 150, 153, 154, 157, 158
Iterable 86
Iterator 86, 91, 96

J

Javadoc 11, 12, 22, 25, 29, 33, 45, 87, 89, 135, 141, 144
Javadoc ne répète pas la signature (CC-COM.3) 208
JavaFX 7, 11, 70, 72, 126, 131, 132, 133, 134, 136, 138, 139, 140, 142, 143, 146
JavaFX
  rien à configurer, ou presque (IDE.28) 177
JavaFX runtime components are missing (ERR.7) 182
JDK isn't specified for module (ERR.1) 180
JDK ne vient pas avec IntelliJ (IDE.3) 161
joker 91, 117, 122, 125
JVM 11, 14, 59, 99, 104, 120, 133, 151
JVM, JRE, JDK
  qui fait quoi (TEC-PLA.1) 186

K

KeyEvent 138, 147

L

lambda 7, 74, 126, 127, 128, 130, 131, 136, 137, 141
Lancer les tests et lire le rapport (IDE.17) 170
Lancer malgré une erreur de compilation (ERR.6) 181
Lancer une classe (IDE.16) 170
LinkedList 86, 95
Lire la Javadoc sans quitter l'éditeur (IDE.14) 168
List 73, 83, 84, 85, 93, 95, 117, 118, 120, 121, 122, 123, 125, 155
listener 141, 142
LocalDate 17, 21, 26, 27, 30, 32, 33, 34, 156
LocalDateTime 26, 27, 32, 33, 34
locale 31, 34, 118, 129, 147
LocalTime 26, 27, 33, 34
Longueur du nom suit la portée (CC-NOM.6) 202

M

main..... 10, 13, 19, 24, 25, 31, 77, 100, 101, 109, 133, 145, 147
Map..... 83, 84, 89, 90, 93, 95, 117, 119, 131
Math..... 19, 76, 87
Message d'exception dit ce qui a échoué et avec quelle valeur (CC-ERR.4)..... 211
méthode abstraite...52, 57, 63, 64, 66, 68, 73, 90, 128, 130, 134
Méthode agit ou répond, jamais les deux (CC-MET.8) 207
méthode default 73, 90
Méthode ne fait qu'une seule chose (CC-MET.3)..... 205
Méthode reste courte (CC-MET.2) 204
mode strict (dates) 34
Modèles
psvm, sout, fori (IDE.13)..... 168
modificateur d'accès 75, 77
Modules Maven ont disparu
plus rien ne se lance (ERR.11)..... 184
mot-clé 15, 18, 43, 46, 49, 53, 56, 66, 77
MouseEvent 138, 147
MVC 69, 143, 147, 149

N

N'avalez pas une exception en silence (CC-ERR.6).... 212
name..... 44
Ne pas écraser son propre travail (TEC-ZIP.2)..... 196
Ne passez pas null en paramètre (CC-ERR.3)..... 211
Ne retournez pas null (CC-ERR.2) 210
Ne vous répétez pas (CC-CLS.3)..... 216
nœud 134, 139, 144, 147, 148
Nom communique l'intention (CC-NOM.1) 200
Nom de la méthode dit ce qu'elle fait (CC-MET.1)... 204
Nom n'encode pas le type (CC-NOM.4) 201
Nom ne dit pas autre chose que ce qu'il désigne (CC-NOM.2)..... 200
Nom se prononce (CC-NOM.3)..... 201
Noms pour les choses, verbes pour les actions, prédicats pour les booléens (CC-NOM.5)..... 201
null.. 15, 36, 38, 40, 43, 45, 84, 90, 94, 95, 96, 104, 110, 122, 154, 156
NullPointerException..... 36, 38, 90, 98, 104, 105
NumberFormatException..... 104, 157

O

ObjectInputStream..... 155, 157, 158
ObjectOutputStream..... 155, 157, 158
ordinal..... 44, 51
Ouvrir et fermer les fenêtres d'outils (IDE.10) 167
Ouvrir par le pom.xml, jamais par le dossier (IDE.4) 161

P

package... 9, 13, 21, 24, 26, 27, 54, 75, 76, 77, 79, 80, 81, 82, 84, 91, 109, 129, 138, 145, 151
paramètre
passage par valeur..... 23, 25, 113
paramètre de type..... 117, 118, 123
Pas d'effet de bord (CC-MET.7)..... 206
Pas de code laissé en commentaire (CC-COM.2) 208
Pas de drapeau booléen en paramètre (CC-MET.5).. 205
Pas de flèche verte sur une classe de test (ERR.4)..... 181
Pas de nombre magique nommez-le (CC-NOM.7)..... 202
Passer des arguments au programme (IDE.18)..... 172
patron Observateur..... 65, 71, 74, 136
Period..... 26, 27, 33, 34
Pile (TEC-EXE.1) 192
pile des appels 27, 34, 59, 70, 98, 99, 101, 105, 110, 112, 139
Plugins et apparence (IDE.27) 176
polymorphisme..... 14, 52, 59, 64, 67, 124
Pom.xml (TEC-MVN.2) 194
Pourquoi certains dossiers sont ignorés (TEC-GIT.2) 197
Pourquoi Java tourne partout (TEC-CMP.2)..... 191
Predicate..... 91, 93, 94, 95, 122, 125, 127, 128, 129, 131
Préférez le polymorphisme à une chaîne de tests sur un type (CC-CLS.4)..... 217
primitif (type)..... 22, 24, 28, 84, 130, 155
PrintWriter..... 153, 156, 158
Projet, plusieurs modules Maven (TEC-MVN.3)..... 194
propagation (capture, bouillonnement) 139
propriété (JavaFX)..... 17, 132, 140, 142, 149, 150
protected..... 53, 57, 61, 75, 77, 80, 81, 82

Q

Quatre zones (IDE.7)..... 164
Que contient ma machine (TEC-PLA.4)..... 187
Quelle édition d'IntelliJ (IDE.1)..... 161

R

Raccourcis à connaître (IDE.26)..... 176
ramasse-miettes..... 24
Ramasse-miettes (TEC-EXE.4)..... 193
Random..... 48, 87, 116
record..... 7, 34, 42, 43, 49, 61, 88, 93, 96, 105
redéfinition..... 16, 52, 53, 58, 64, 66, 73, 124
référence 6, 10, 22, 25, 35, 36, 38, 40, 45, 50, 54, 56, 81, 88, 89, 126, 130, 137
référence de méthode..... 126, 130, 131, 137
removeIf..... 90, 93, 94, 96, 97, 122, 127, 128, 130
Renommer (IDE.20)..... 174
RuntimeException 99, 101, 104, 105

S

Scanner..... 12, 21, 87, 151, 153, 154, 156, 158
nextInt..... 22, 48, 87
nextLine..... 22, 154, 157
Scene 134, 147
Scene Builder..... 143, 146
séparer calcul et dialogue..... 12, 24, 29, 37
sérialisation..... 66, 73, 155, 157, 158
Set 83, 84, 87, 95
setOnAction..... 136, 140, 145, 148
Source level should be in ... 21 (ERR.8) 182
split..... 31, 154, 156, 157, 158
Stage..... 134, 139, 147
static..... 12, 13, 16, 18, 25, 73, 109, 121, 124, 128
stream 130, 151
String.format..... 31, 34
StringJoiner 49
Structure d'un fichier (IDE.25)..... 175

super 54, 57, 60, 64, 91, 101, 117, 122, 124
Supplier 129, 131
surface publique 75, 79, 82
switch 42, 45, 51, 131, 138
  expression 46
  forme à flèches 46
  forme classique 46
System.out.print 30, 153
System.out.printf 30, 34, 153, 156
System.out.println 13, 16, 30, 36, 153, 156

T

tableau 7, 9, 27, 35, 36, 38, 40, 41, 43, 44, 48, 56, 59, 63, 67, 68, 72, 73, 79, 81, 83, 85, 95, 104, 108, 110, 113, 121, 129, 130, 142, 151
  de tableaux 35, 36, 38
  taille fixe 36, 40, 41, 72, 83, 85
Tas (TEC-EXE.2) 192
ternaire (opérateur) 35, 38, 41, 125
test (vérifier son code) ... 8, 9, 10, 13, 31, 63, 70, 77, 78, 79, 107, 108, 109, 115, 137, 146
Test est rapide, indépendant, répétable et auto-vérifiant (CC-TST.3) 214
Test se termine par une assertion (CC-TST.4) 215
test unitaire ... 7, 9, 107, 109, 112, 114, 129, 133, 137, 143, 147
Test vérifie une seule chose (CC-TST.2) 214
this 15, 19, 25, 125, 137
throw 17, 50, 98, 100, 101, 102, 105
throws 98, 101, 104, 105, 112, 157
toString 12, 16, 44, 49, 51, 118
Tout est rouge sans raison (ERR.3) 181
TreeMap 88, 89, 92, 95
TreeSet 88, 92, 95
Trois éditions (TEC-PLA.2) 187
Trois états de votre programme (TEC-CMP.1) 190
try 17, 98, 100, 101, 102, 104, 150, 153, 155, 157, 158
try avec ressources 150, 153, 155, 157, 158
type cible 126, 127, 128, 130, 131
type générique ... 7, 117, 118, 119, 120, 123, 124, 125, 130, 136, 155

U

UnsupportedOperationException 9, 105

V

valueOf 44, 130
values 44, 48, 51
Vérifier le SDK du projet (IDE.6) 163
Versions et le support long terme (TEC-PLA.3) 187
visibilité de package 77, 79, 81
Vocabulaire est cohérent d'une méthode à l'autre (CC-MET.4) 205

W

wrapper 24, 27

Y

yield 46